



42115 Network Optimization

David Pisinger

`pisinger@man.dtu.dk`

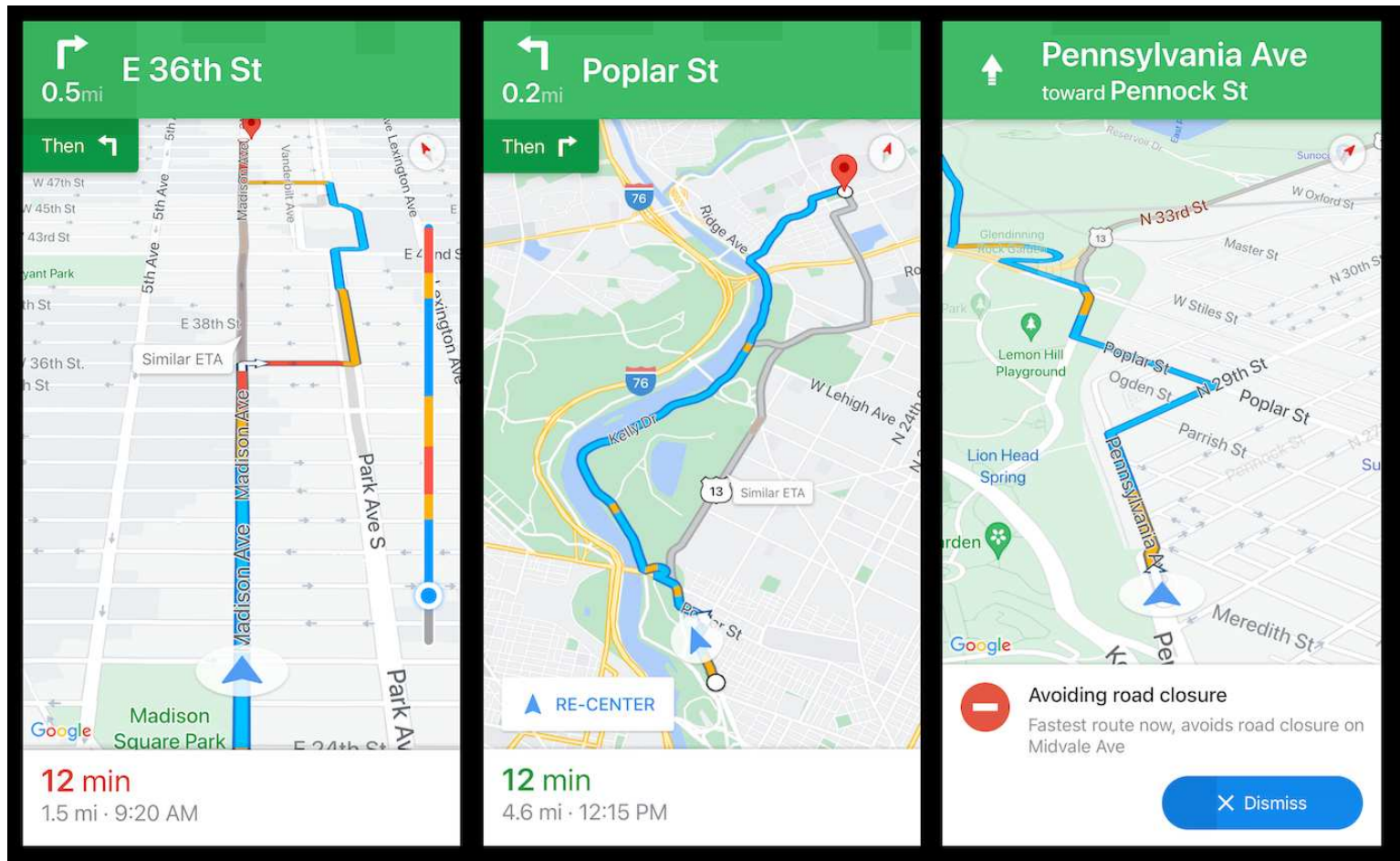
DTU Management

Technical University of Denmark

© Copyright David Pisinger. May not be distributed



Network Examples



Finding way (google maps)

shortest path, cheapest path time constraint, alternative path

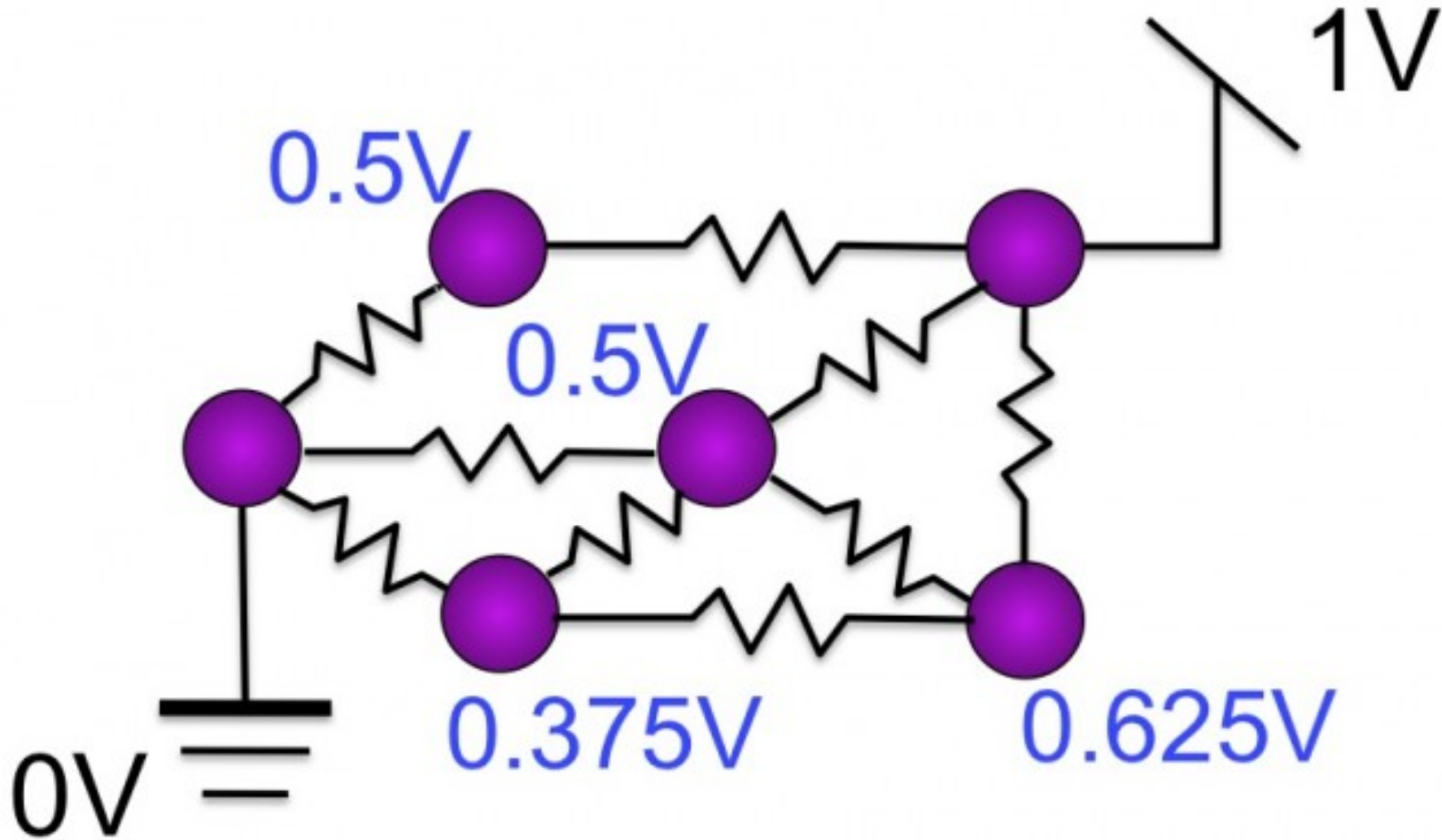
Network Examples



Telecommunication network

network design, routing data

Network Examples



Electrical network

current flows (quadratic MDF)

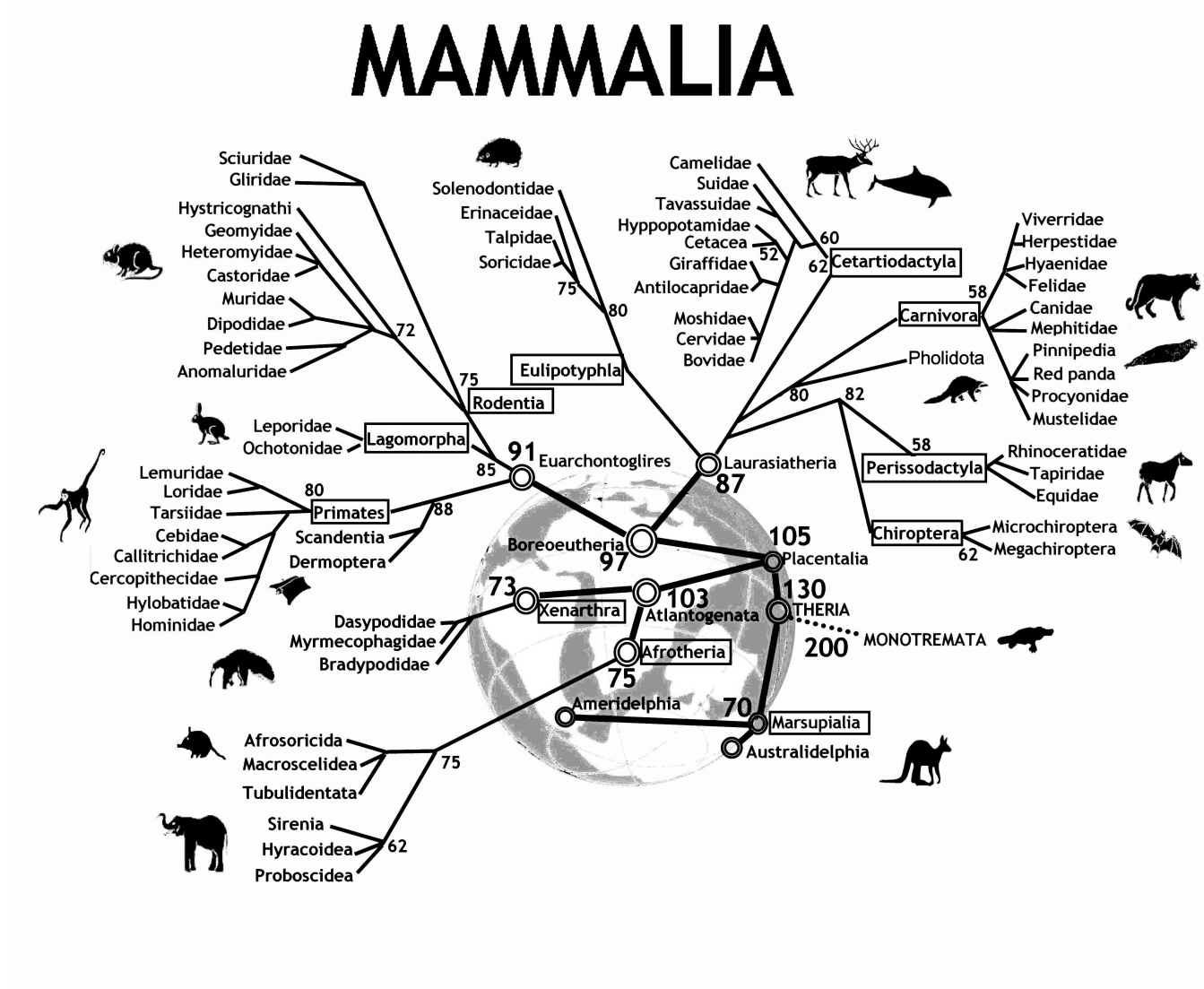
Network Examples



Relation network (e.g. linkedin)

distance to person, connectivity

Network Examples



Evolution network

network design, distance

Motivation

- Network problems some of the most frequently occurring optimization problems
- Network models easy to model, intuitive to understand
- Many network problems can be solved efficiently
- Network problems appear as subproblem (e.g. shortest path subproblem in manpower planning)

Motivation

Complements course 42114 Integer Programming

- Use networks to model problems instead of constr. $Ax = b$
- Develop efficient network algorithms where integer programming is hard to solve
- Use knowledge from LP to design algorithms
- Use knowledge from LP to prove that algorithms work correct

Course Plan

- **5 sept** Introduction, critical path, project planning
- **12 sept** Transportation problem, network simplex, linear programming
- **19 sept** Fixed-charge transportation problem, decomposition, column generation
- **26 sept** Shortest path problem, threshold partitioned shortest path algorithm
- **3 oct** Resource constrained shortest path problem
- **10 oct** Maximum flow problem I
- **17 oct — fall vacations —**
- **24 oct** Maximum flow problem II
- **31 oct** Minimum cost flow I
- **7 nov** Minimum cost flow II
- **14 nov** Multicommodity flow I
- **21 nov** Multicommodity flow II
- **28 nov** Project assignment
- **5 dec** Project assignment

Literature

- Chapters from Larsen, Clausen "Supplementary Notes"
- Chapters from Ahuja, Orlin, Magnanti "Network Flows"

Available on campusnet (DTU inside)

Exam

- Project assignment - split into two parts
- Project in groups 2-3 people
- Written exam, 17. december
- Individual written exam 4 hours, books/computer allowed, no internet
- Project must be passed to attend exam
- Passed project gives acces to 3 following exams (December, June, December)

Expectations

5 ECTS corresponds to **9 hours** of work per week

- At home: read text (briefly) before the lecture
- Attend lecture, exercises
- At home: solve last exercises and read text more carefully

You cannot expect to pass course if you only work 4 hours per week

Expectations

- Come to the lectures - video recordings are intended as a backup in case you are sick
- Come to the exercises - I do not have time to answer questions on email
- No plagiarism in project assignment - every year I report 1-2 groups



Modeling network problems

David Pisinger

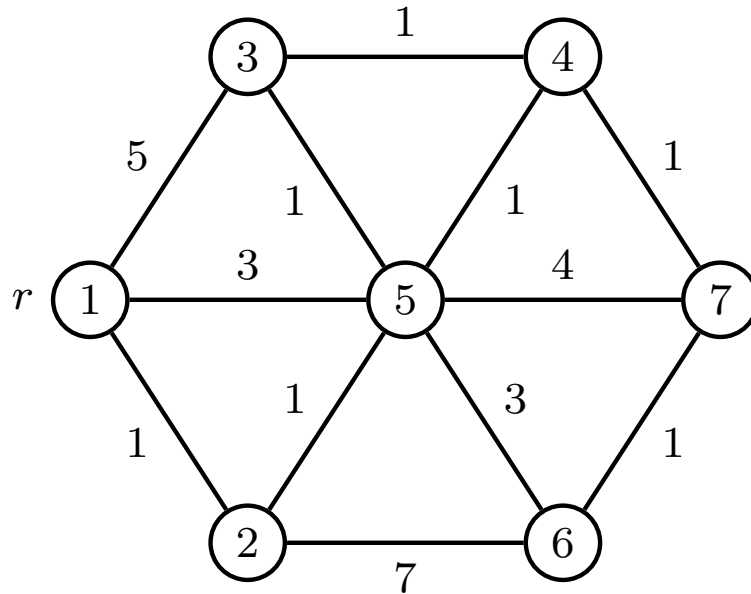
`pisinger@man.dtu.dk`

DTU Management

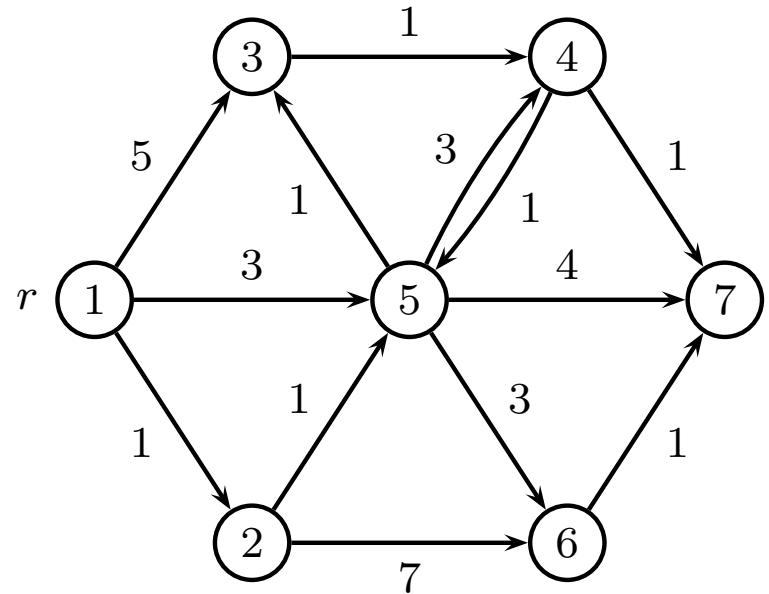
Technical University of Denmark

Modeling network problems

Graph (V, E)



Directed graph (V, E) *Digraph*



- Special graphs: path, cycle, tree
- Edge weights: cost, capacity, time
- Node weights: cost, time
- Special nodes: (root r , sink s), (source s , terminal t)

LP-models

Variable x_{ij} for each edge (i, j) .

For path problems, x_{ij} is decision variable

$$x_{ij} = \begin{cases} 1 & \text{if edge } (i, j) \text{ is chosen} \\ 0 & \text{otherwise} \end{cases}$$

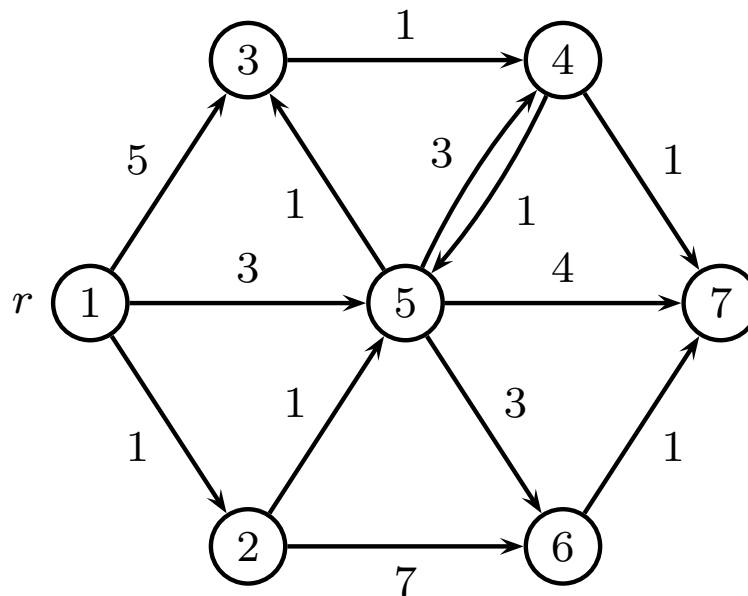
For flow problems, x_{ij} is flow on edge (i, j)

$$x_{ij} \geq 0$$

Capacity

If an edge (i, j) has a capacity c_{ij} then

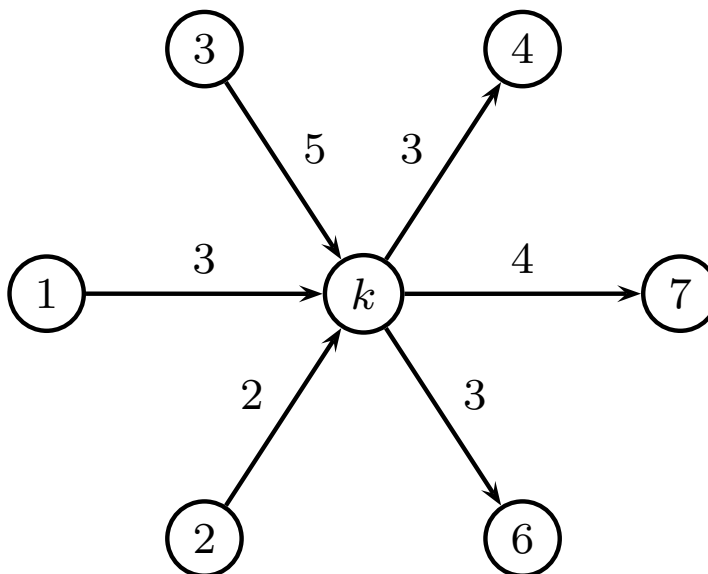
$$0 \leq x_{ij} \leq c_{ij}$$



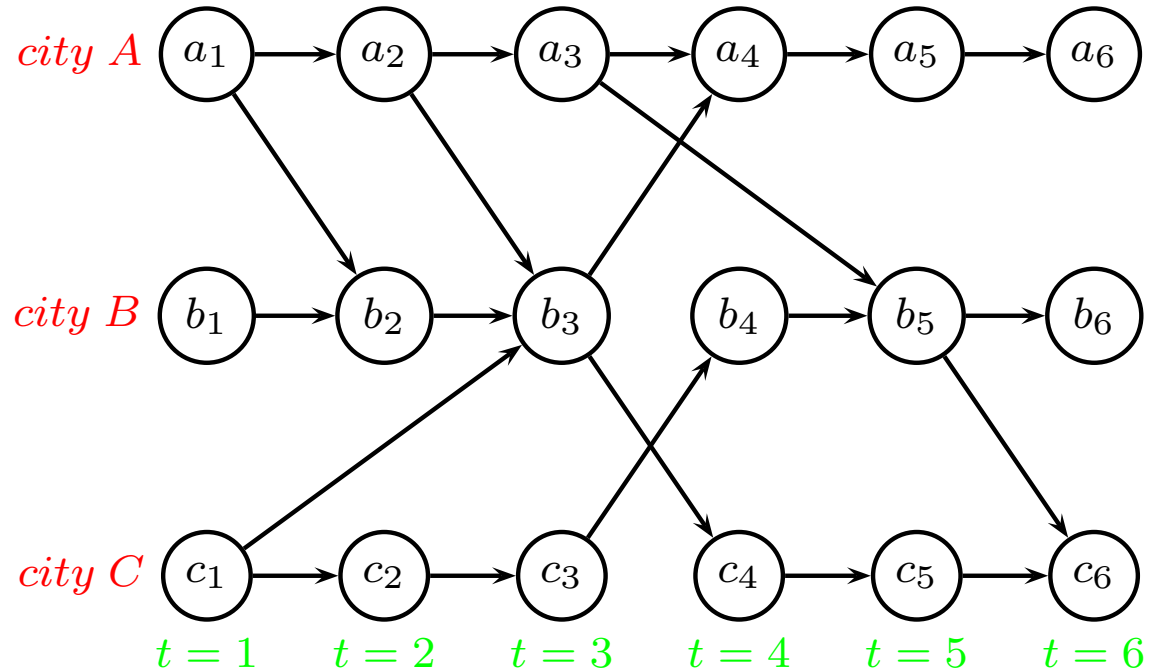
Flow conservation

In every node k : What flows in must flow out

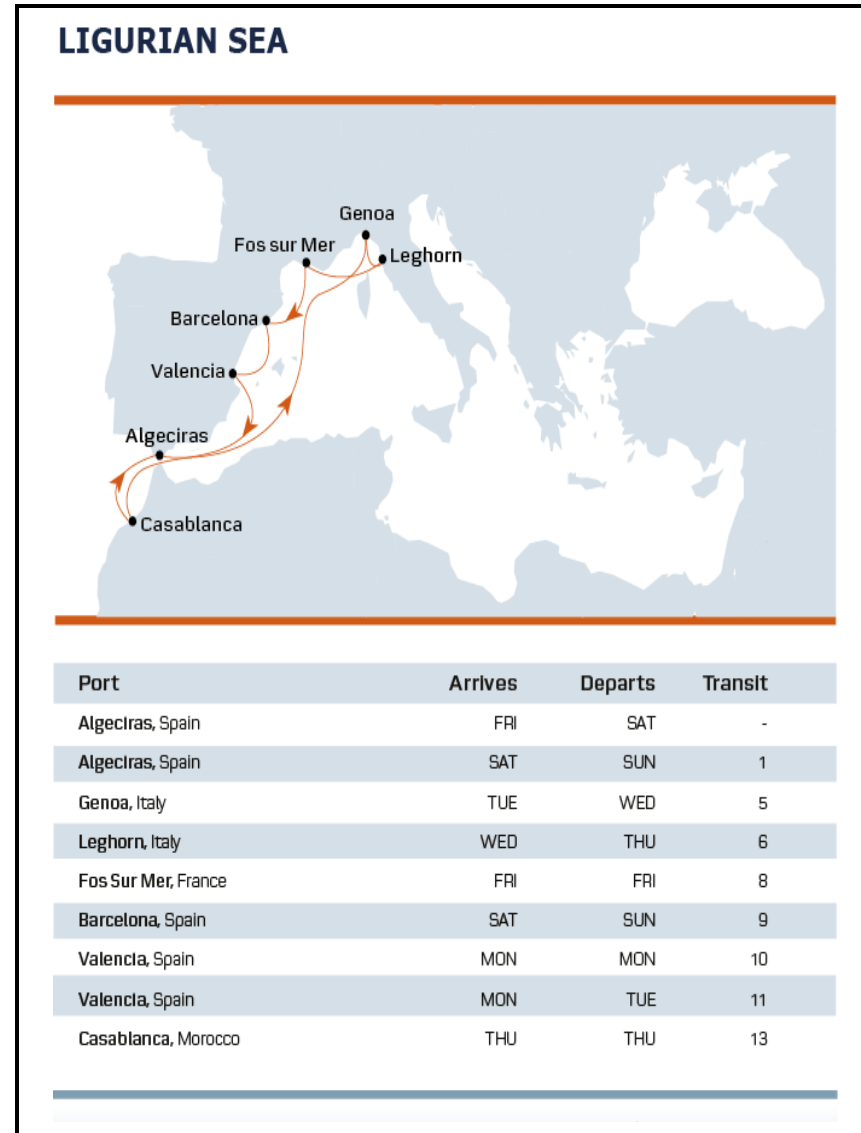
$$\sum_{i \in V} x_{ik} = \sum_{i \in V} x_{ki}$$



Time-space graph

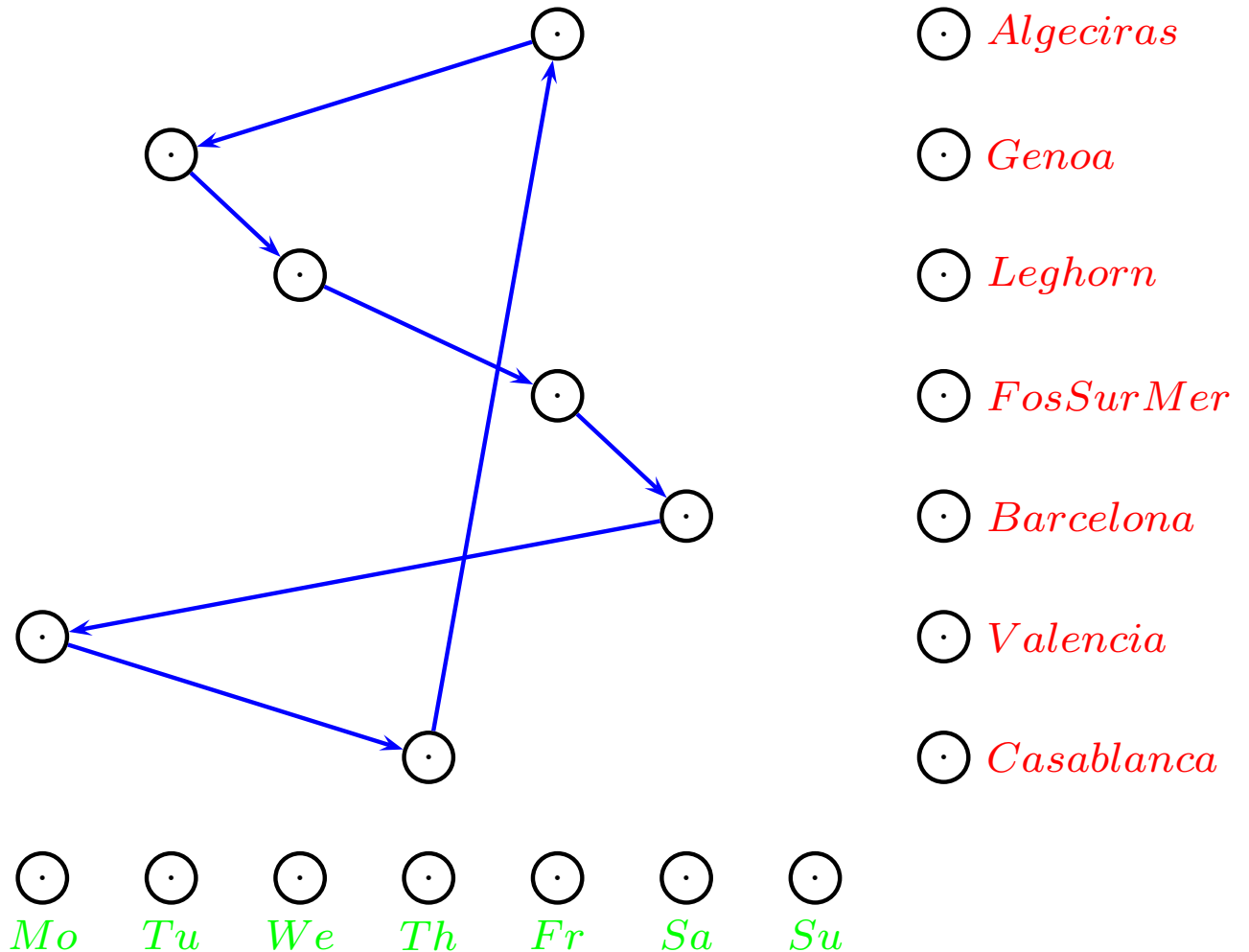


Cyclic time-space graph



Cyclic time-space graph

Arrival days (first day):

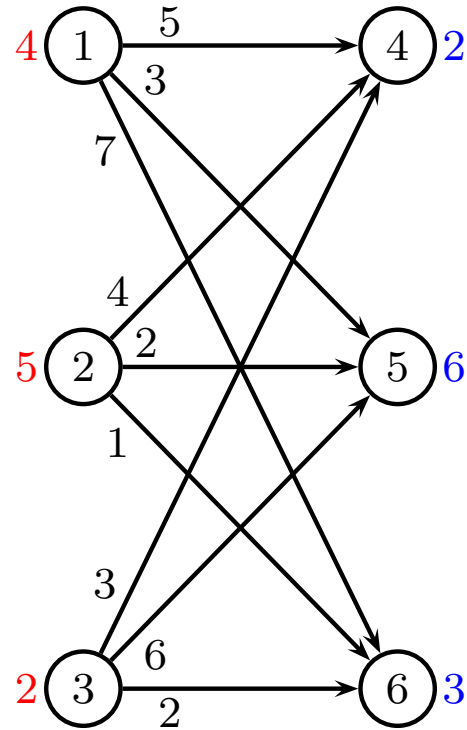


Problems

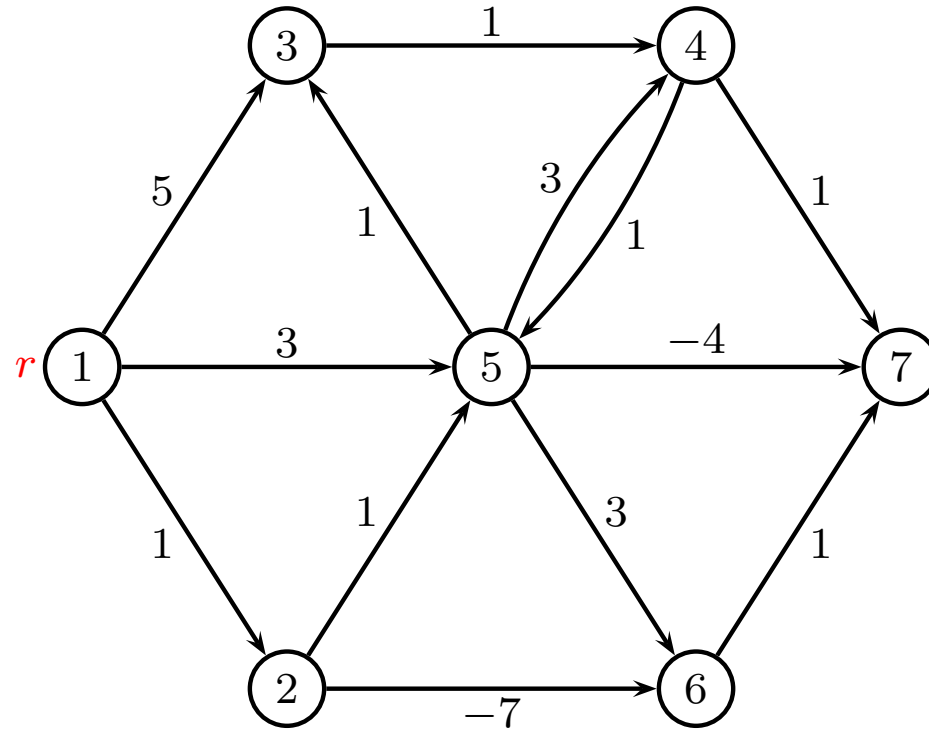
Problems we will consider in the course

- Transportation problem
- Shortest path
- Time constrained shortest path
- Critical path
- Maximum flow
- Minimum-cost flow
- Multicommodity-flow

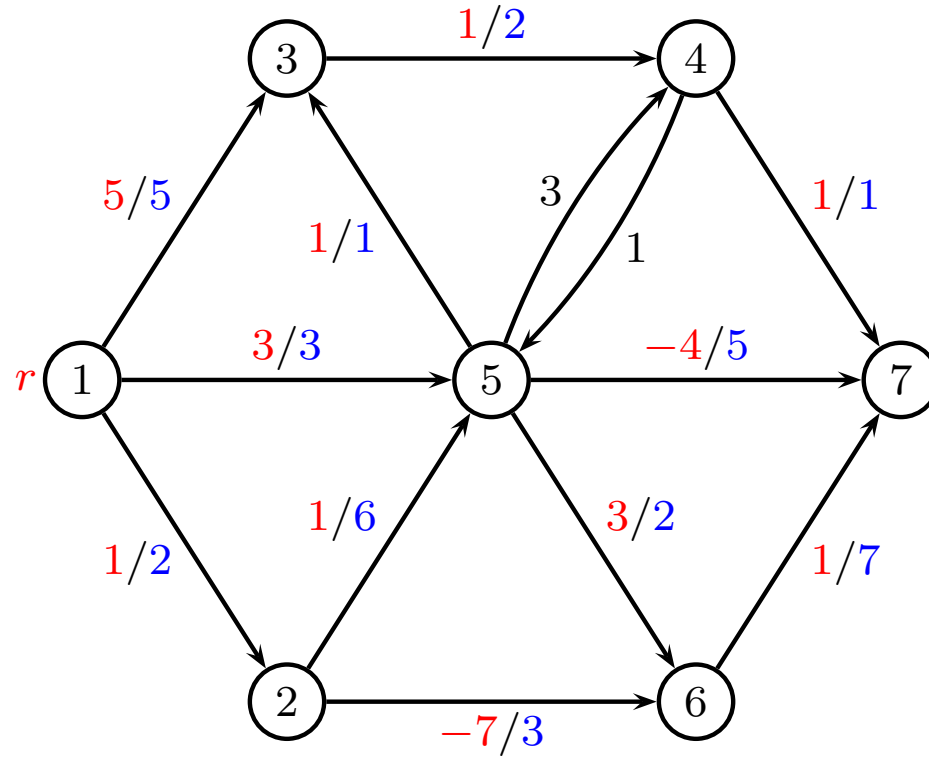
Transportation Problem



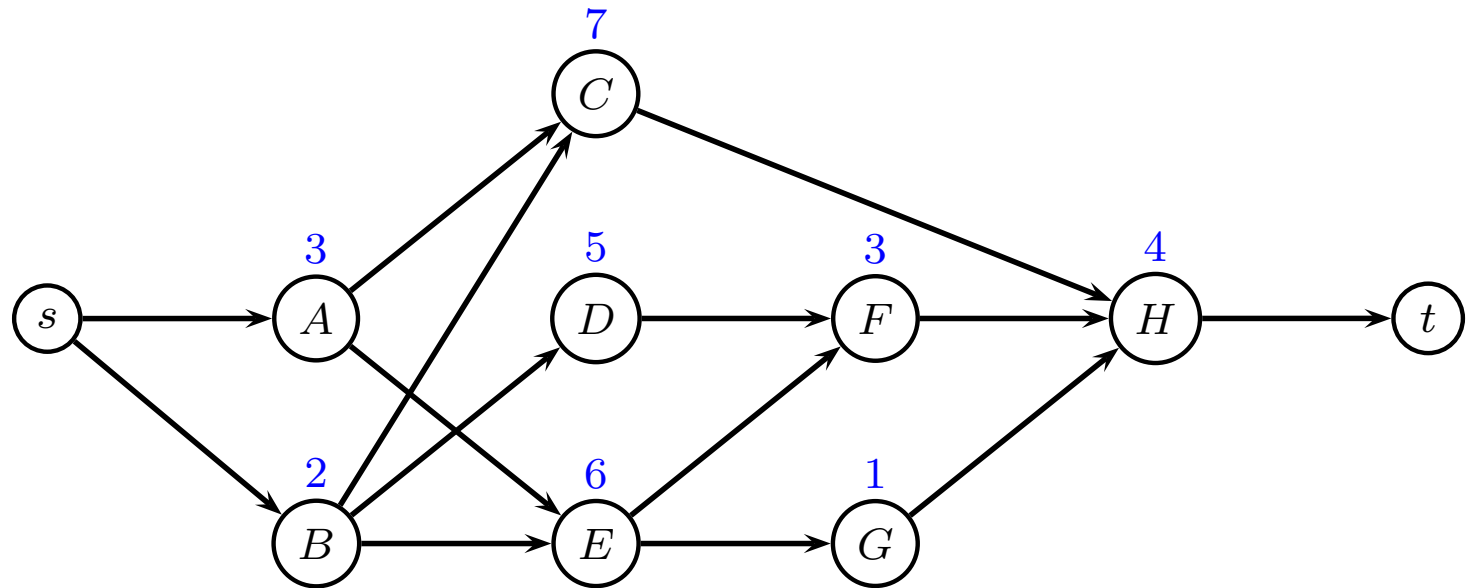
Shortest Path Problem



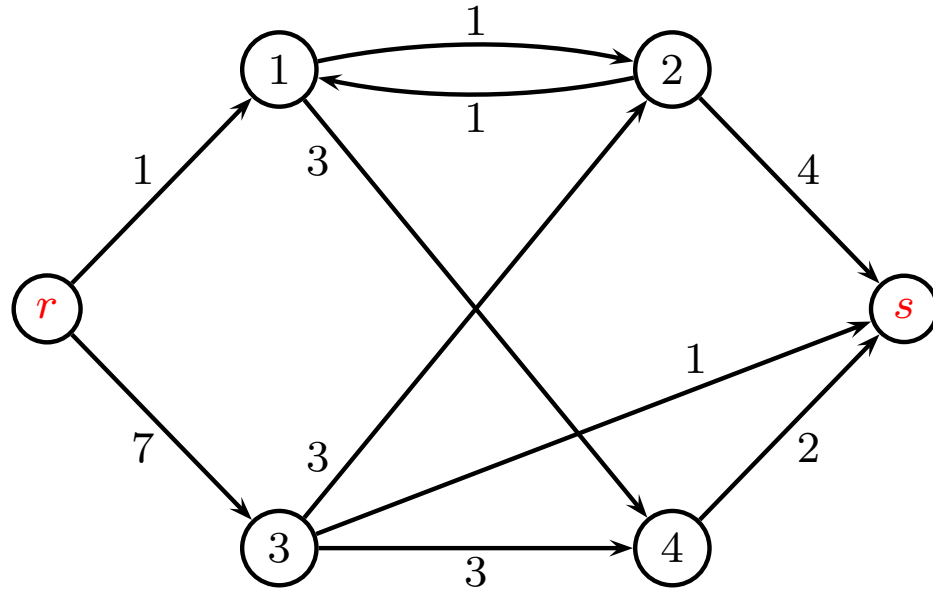
Time-constrained Shortest Path Problem



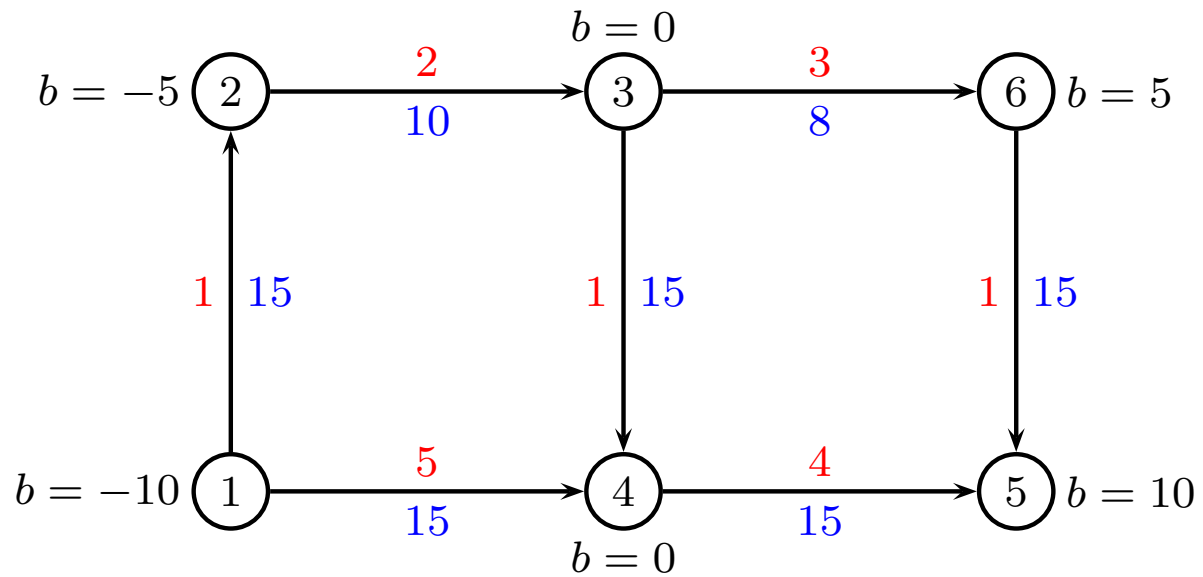
Critical Path Problem



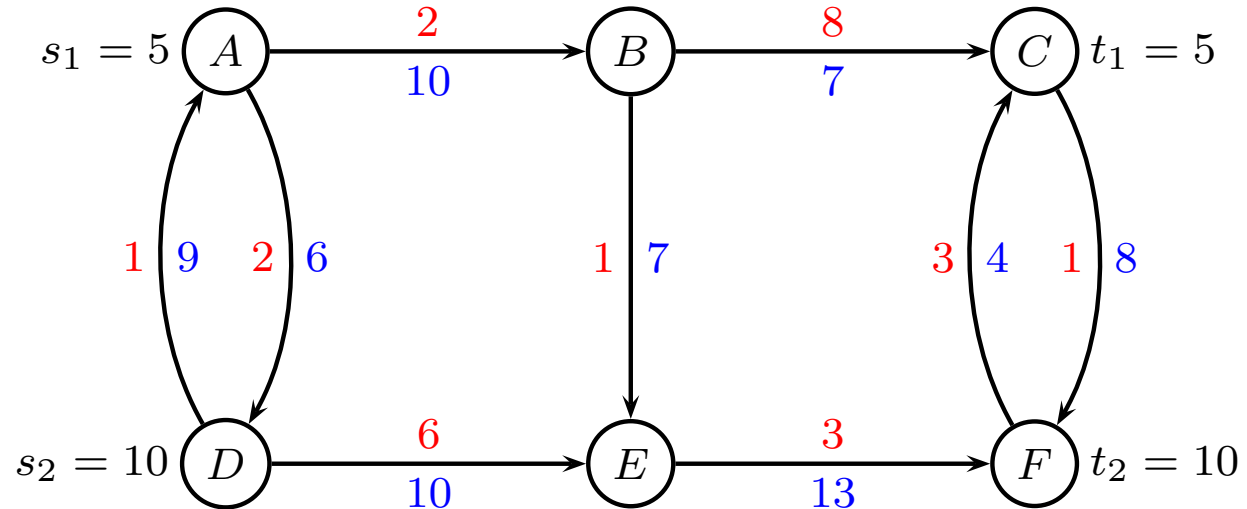
Max Flow Problem



Min Cost Flow



Multicommodity Cost Flow





Critical Path Methods

David Pisinger

`pisinger@man.dtu.dk`

DTU Management

Technical University of Denmark

Project Management

Consider building a house:

Step A	Prepare site (5 days)	
Step B	Build foundation (8 days)	A
Step C	Frame walls and roof (15 days)	B
Step D	Rough in Plumbing (12 days)	A
Step E	Rough in Electrical (10 days)	A, B
Step F	HVAC Venting (8 days)	A, B, C
Step G	Drywall (11 days)	B, C
Step H	Finish Electrical (5 days)	E, F, G
Step I	Finish Plumbing (4 days)	D, F, G
Step J	Finish HVAC (2 days)	F, G
Step K	Install Kitchen (8 days)	J
Step L	Install Baths (14 days)	I, J
Step M	Paint (5 days)	K, L
Step N	Landscape (5 days)	A, B, G, J

Project Management

What questions might project managers be interested in?

- How long will the project take?
- What tasks are on the critical path?
- Is the project on schedule?
- When should materials and personnel be in place to begin a task?
- Can I add manpower or tools to reduce the overall project length?
- To which tasks should I add manpower?
- Other?

Definitions

- set J of jobs/tasks
- each job $j \in J$ has a processing time p_j
- directed graph $G = (V, E)$. Nodes are jobs, edges state precedences
- no cycles in G
- WLOG, assume J is topologically sorted

Minimize completion time (makespan) T

Topological sorting

Topological sorting of nodes: All edges (i, j) satisfy $i < j$ (i.e. all edges are pointing forward)

Algorithm:

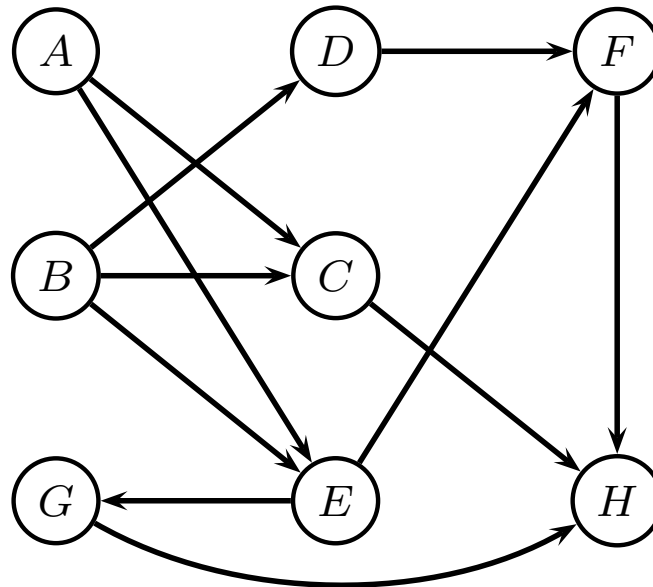
- Given directed acyclic graph $G = (V, E)$
- Repeat
 - Since no cycles, there must be a node with no ingoing edges
 - Remove this node i and corresponding edges
 - Add i to ordering of nodes

Can be done in $O(V + E)$

Small example

j	A	B	C	D	E	F	G	H
p_j	3	2	7	5	6	3	1	4

Precedences:

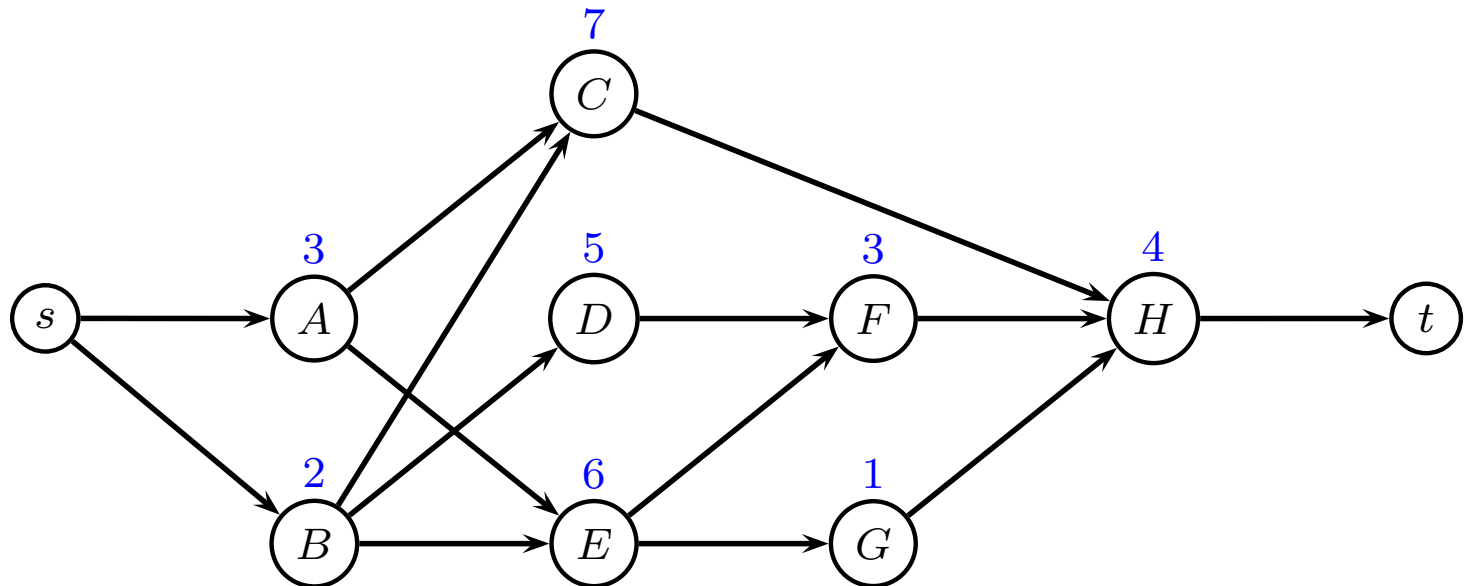


Find makespan, critical path

Small example

j	A	B	C	D	E	F	G	H
p_j	3	2	7	5	6	3	1	4

Topological ordering of precedences:



Find makespan, critical path

Critical Path method

Forward procedure:

Starting at time zero, calculate the earliest time U_j each job j can be started. The completion time of the last job is the makespan T

Backward procedure:

Starting at time equal to the makespan, calculate the latest time V_j each job j can be started so that this makespan is realized

Forward procedure

Dynamic programming method:

1 $U_s = 0$

2 for $j = 1$ to n

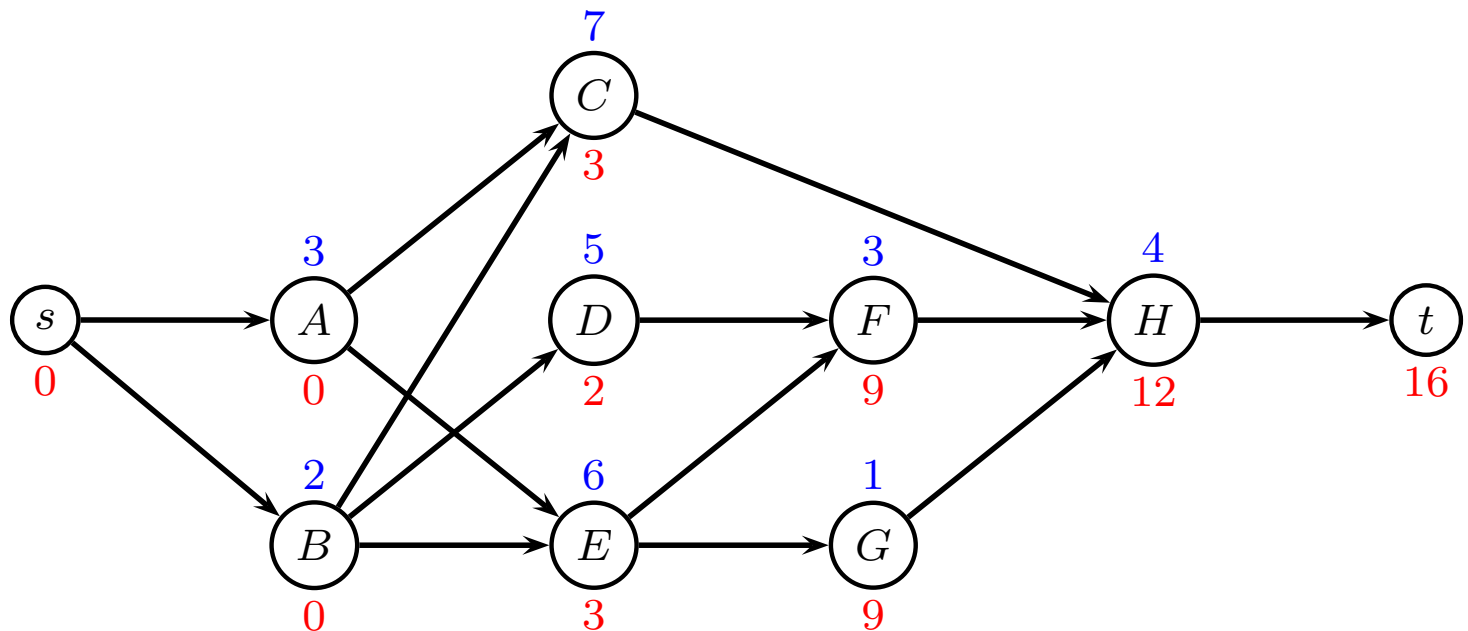
3 $U_j = \max_{(i,j) \in E} \{U_i + p_i\}$

4 makespan: $T = U_t$

U_j earliest starting time job j

Forward algorithm

j	A	B	C	D	E	F	G	H
p_j	3	2	7	5	6	3	1	4
U_j	0	0	3	2	3	9	9	12



Makespan: $T = 16$

Backward procedure

Dynamic programming method:

1 $V_t = T$

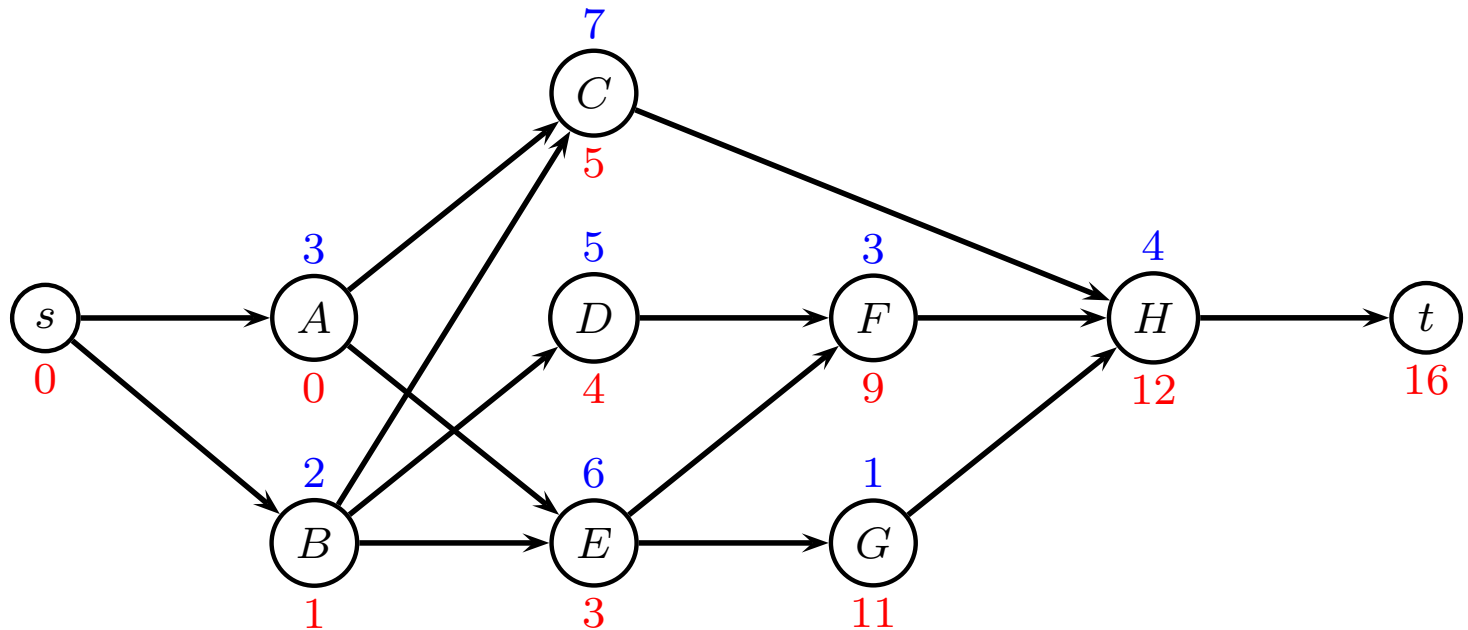
2 for $i = n$ to 1

3 $V_i = \min_{(i,j) \in E} \{V_j - p_i\}$

V_j latest starting time job j

Backward algorithm

j	A	B	C	D	E	F	G	H
p_j	3	2	7	5	6	3	1	4
U_j	0	0	3	2	3	9	9	12
V_j	0	1	5	4	3	9	11	12

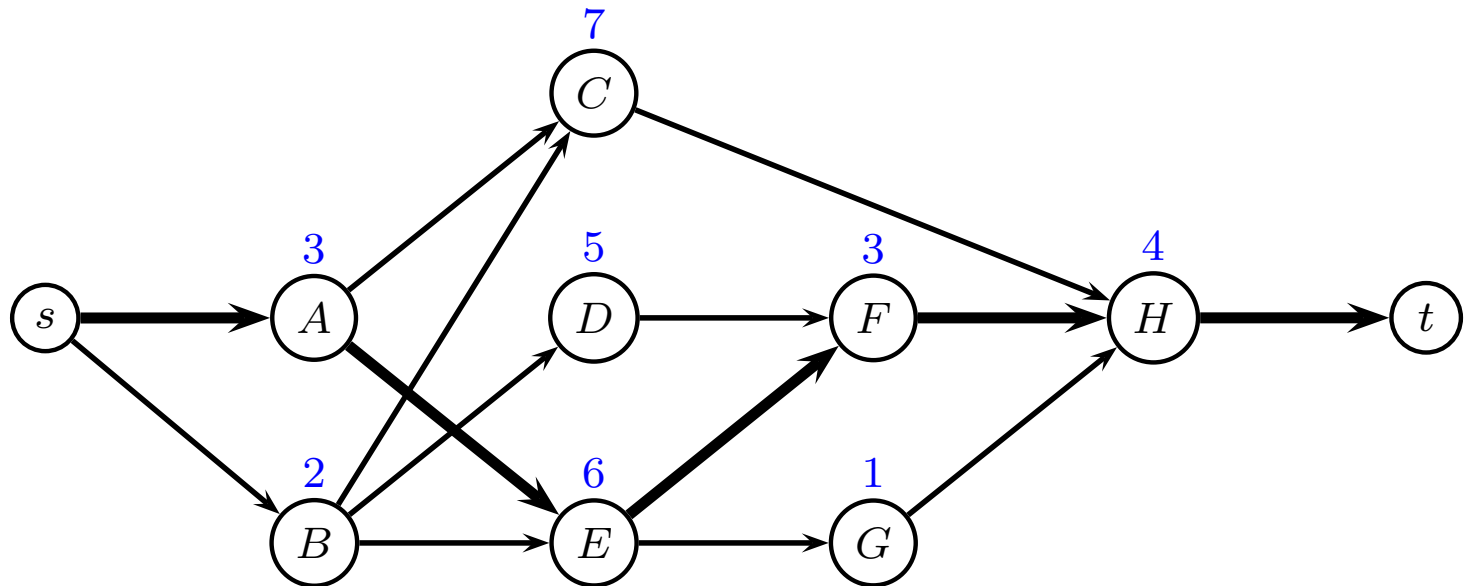


Critical Path

- The forward procedure gives the earliest possible starting time for each job
- The backwards procedures gives the latest possible starting time for each job
- If these are equal the job is a critical job.
- If these are different the job is a slack job, and the difference is the float.
- A critical path is a chain of jobs starting at time 0 and ending at T .

Critical Path

j	A	B	C	D	E	F	G	H
p_j	3	2	7	5	6	3	1	4
U_j	0	0	3	2	3	9	9	12
V_j	0	1	5	4	3	9	11	12



Nodes with no slack define critical path

LP formulation

Variable x_j is start time of job j

LP-model:

$$\begin{array}{ll}\min & T = x_t - x_s \\ \text{s.t.} & x_j \geq x_i + p_i \quad (i, j) \in E \\ & x_j \geq 0 \quad j \in J\end{array}$$

LP formulation

$$\begin{array}{ll}\min & T = x_t - x_s \\ \text{s.t.} & x_A \geq x_s + 0 \\ & x_B \geq x_s + 0 \\ & x_C \geq x_A + 3 \\ & x_C \geq x_B + 2 \\ & x_D \geq x_B + 2 \\ & x_E \geq x_A + 3 \\ & x_E \geq x_B + 2 \\ & x_F \geq x_D + 5 \\ & x_F \geq x_E + 6 \\ & x_G \geq x_E + 6 \\ & x_H \geq x_C + 7 \\ & x_H \geq x_F + 3 \\ & x_H \geq x_G + 1 \\ & x_t \geq x_H + 4 \\ & x_i \geq 0 \quad i \in J\end{array}$$

LP formulation

```
minimize
    xt - xs
subject to
sA:  xA - xs >= 0
sB:  xB - xs >= 0
AC:  xC - xA >= 3
BC:  xC - xB >= 2
BD:  xD - xB >= 2
AE:  xE - xA >= 3
BE:  xE - xB >= 2
DF:  xF - xD >= 5
EF:  xF - xE >= 6
EG:  xG - xE >= 6
CH:  xH - xC >= 7
FH:  xH - xF >= 3
GH:  xH - xG >= 1
Ht:  xt - xH >= 4
end
```

Running CPLEX

- If you use Windows, install “Thinlinc” so you can connect to another computer. You can download it from <http://gbar.dtu.dk/faq/43-thinlinc>
- Login to GBAR at DTU using Thinlinc. Username is your study number “s123456”, while host is “thinlinc.gbar.dtu.dk”
- If you use Linux, you can connect to DTU using `ssh username@thinlinc.gbar.dtu.dk`
- Use an editor, e.g. “gedit” to type in the model
- Save model to a file, e.g. “ex1.lp”
- `cplex` (to start CPLEX)

Output from CPLEX

```
CPLEX> read modell
Problem 'modell.lp' read.
Read time =      0.01 sec.
CPLEX> opt
Tried aggregator 1 time.
LP Presolve eliminated 3 rows and 1 columns.
Aggregator did 5 substitutions.
Reduced LP has 6 rows, 4 columns, and 12 nonzeros.
Presolve time =      0.00 sec.
Initializing dual steep norms . . .

Iteration log . . .
Iteration:      1    Dual infeasibility =          0.000000
Iteration:      2    Dual objective      =          14.000000

Dual simplex - Optimal:  Objective =  1.60000000000e+01
Solution time =      0.00 sec.  Iterations = 3 (1)
```

CPLEX solution

```
CPLEX> dis sol var -
```

Variable Name	Solution Value
xt	16.000000
xC	5.000000
xD	4.000000
xE	3.000000
xF	9.000000
xG	11.000000
xH	12.000000

All other variables in the range 1-10 are 0.

```
CPLEX> dis sol dua -
```

Constraint Name	Dual Price
sA	1.000000
AE	1.000000
EF	1.000000
FH	1.000000
Ht	1.000000

All other dual prices in the range 1-14 are 0.

If dual variable is positive, edge is on critical path

Hints on using CPLEX

- Use an editor to write the CPLEX input file, e.g. gedit
- Remember to move to the same directory as the file before starting CPLEX, this can be done with the command `cd directoryname`
- All variables must be on the left side of the inequality
- All constants must be on the right side of the inequality
- No constant is allowed in the objective
- No parentheses are allowed, simplify the equation by hand
- No multiplication sign is needed, just write 2×1
- All the output from the screen is also written to a file `cplex.log` which is found in the same directory. You can copy-paste output from this file to your assignments.

Project Management

- Can normal task times be reduced?
- Is there an increase in direct costs? Additional manpower, Additional machines, Overtime, etc
- Can there be a reduction in indirect costs? Less overhead costs, Less daily rental charges, Bonus for early completion, Avoid penalties for running late, Avoid cost of late startup

CPM addresses these cost trade-offs.

Crashing project

j	A	B	C	D	E	F	G	H
p_j	3	2	7	5	6	3	1	4
m_j	2	1	4	4	6	2	1	3
c_j	4	3	1	2	7	5	3	6

- p_j normal process time
- m_j minimal process time
- c_j cost of decreasing time by one unit

LP formulation, model 1

Variable x_i is start time of task i

Variable y_i is duration of task i

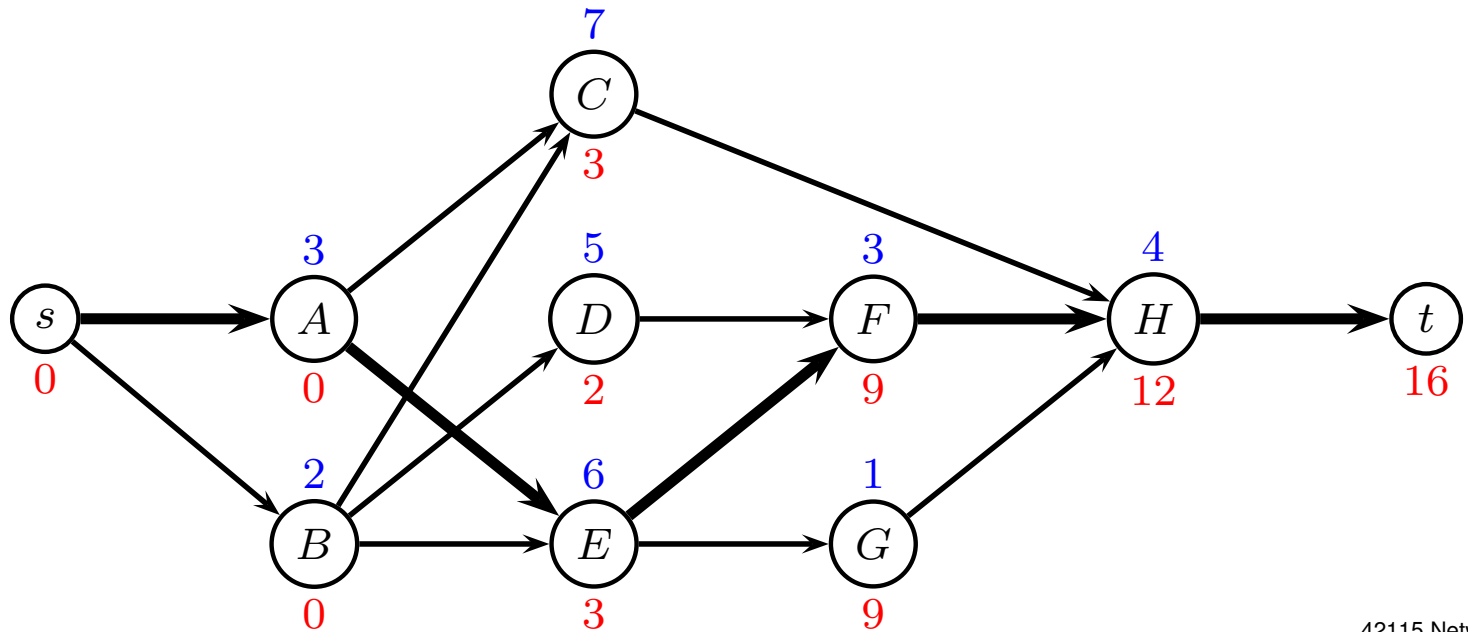
Model 1: cheapest way of finishing within time T

$$\begin{aligned} \min \quad & \sum_{j \in J} c_j (p_j - y_j) \\ & x_j \geq x_i + y_i & (i, j) \in E \\ & x_t - x_s \leq T \\ & m_j \leq y_j \leq p_j & j \in J \\ & x_j \geq 0 & j \in J \end{aligned}$$

LP solution, T=16

j	A	B	C	D	E	F	G	H
p_j	3	2	7	5	6	3	1	4
m_j	2	1	4	4	6	2	1	3
c_j	4	3	1	2	7	5	3	6
x_j	0	0	5	4	3	9	11	12
y_j	3	2	7	5	6	3	1	4

cost: 0



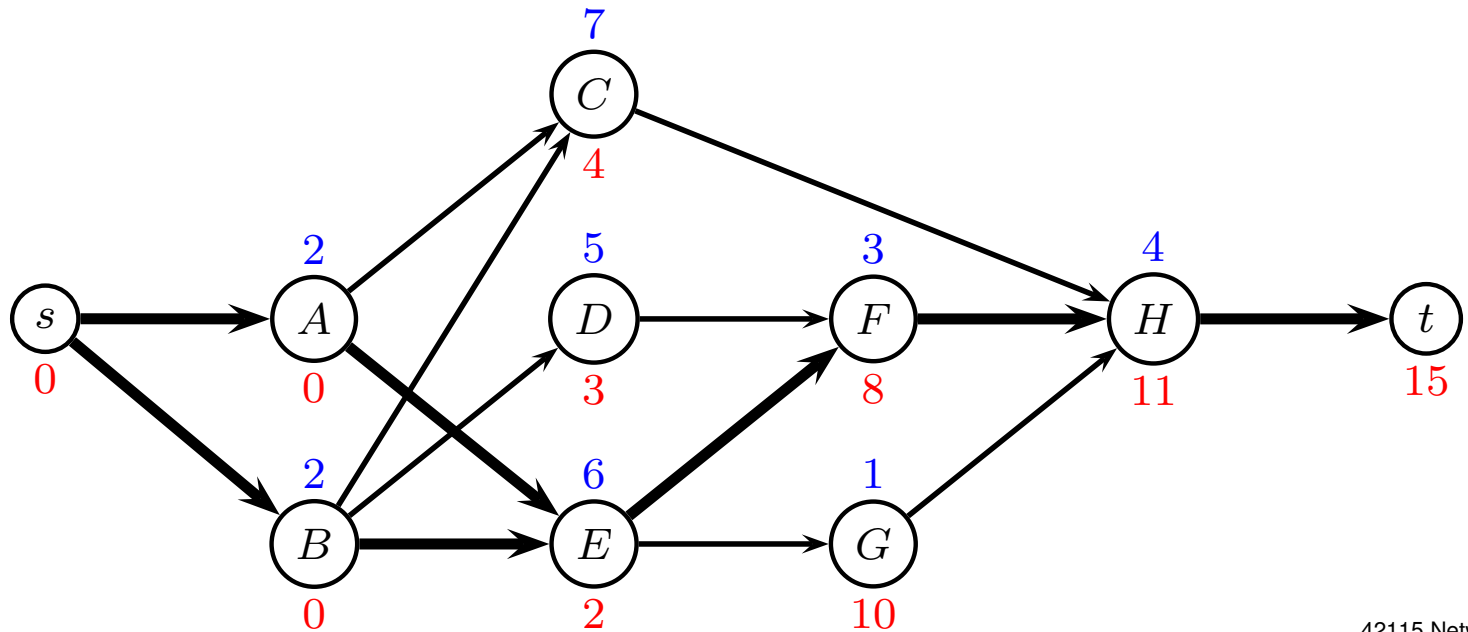
LP solution, T=16

Variable Name	Solution Value
yA	3.000000
yB	2.000000
yC	7.000000
yD	5.000000
yE	6.000000
yF	3.000000
yG	1.000000
yH	4.000000
xC	5.000000
xD	4.000000
xE	3.000000
xF	9.000000
xG	11.000000
xH	12.000000
xt	16.000000

LP solution, T=15

j	A	B	C	D	E	F	G	H
p_j	3	2	7	5	6	3	1	4
m_j	2	1	4	4	6	2	1	3
c_j	4	3	1	2	7	5	3	6
x_j	0	0	4	3	2	8	10	11
y_j	2	2	7	5	6	3	1	4

cost: 4



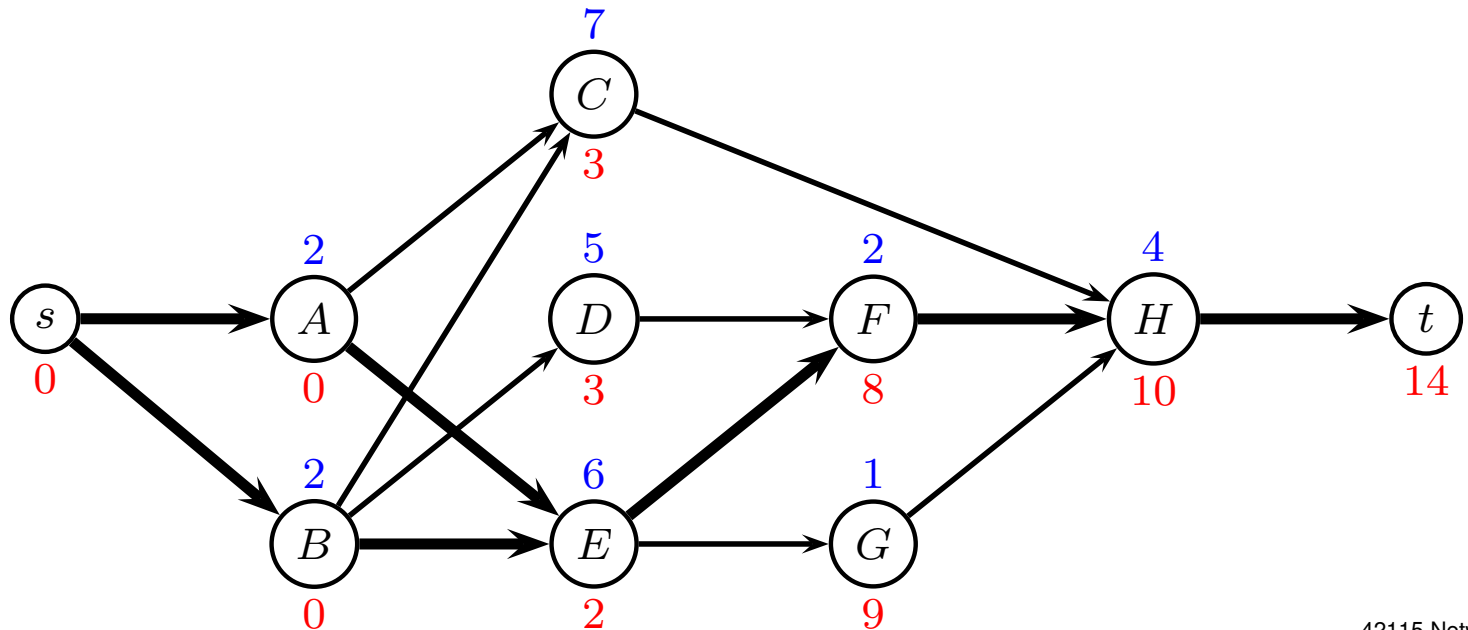
LP solution, T=15

yA	2.000000
yB	2.000000
yC	7.000000
yD	5.000000
yE	6.000000
yF	3.000000
yG	1.000000
yH	4.000000
xC	4.000000
xD	3.000000
xE	2.000000
xF	8.000000
xG	10.000000
xH	11.000000
xt	15.000000

LP solution, T=14

j	A	B	C	D	E	F	G	H
p_j	3	2	7	5	6	3	1	4
m_j	2	1	4	4	6	2	1	3
c_j	4	3	1	2	7	5	3	6
x_j	0	0	3	3	2	8	9	10
y_j	2	2	7	5	6	2	1	4

cost: 9



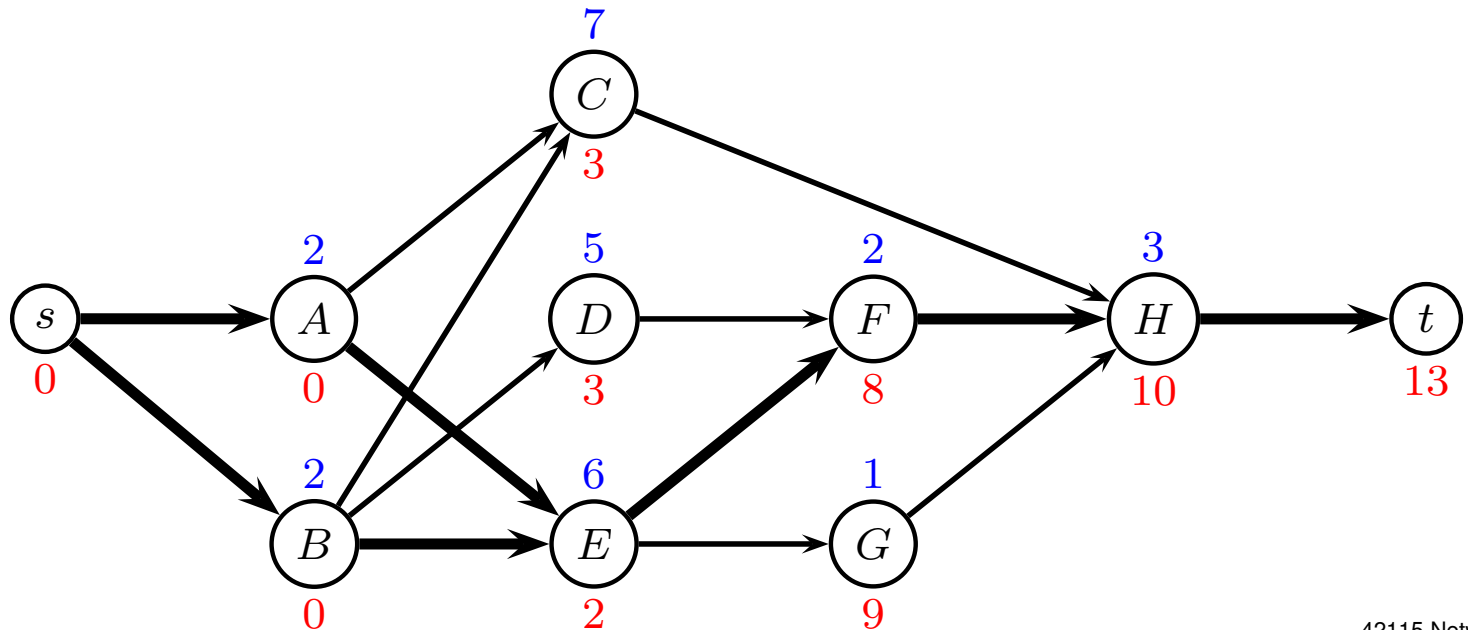
LP solution, T=14

yA	2.000000
yB	2.000000
yC	7.000000
yD	5.000000
yE	6.000000
yF	2.000000
yG	1.000000
yH	4.000000
xC	3.000000
xD	3.000000
xE	2.000000
xF	8.000000
xG	9.000000
xH	10.000000
xt	14.000000

LP solution, T=13

j	A	B	C	D	E	F	G	H
p_j	3	2	7	5	6	3	1	4
m_j	2	1	4	4	6	2	1	3
c_j	4	3	1	2	7	5	3	6
x_j	0	0	3	3	2	8	9	10
y_j	2	2	7	5	6	2	1	3

cost: 15



LP solution, T=13

cost: 15

yA	2.000000
yB	2.000000
yC	7.000000
yD	5.000000
yE	6.000000
yF	2.000000
yG	1.000000
yH	3.000000
xC	3.000000
xD	3.000000
xE	2.000000
xF	8.000000
xG	9.000000
xH	10.000000
xt	13.000000

LP solution, $T=12$

Impossible



All tasks on critical path have minimum processing time

LP formulation, model 2

Variable x_i is start time of task i

Variable y_i is duration of task i

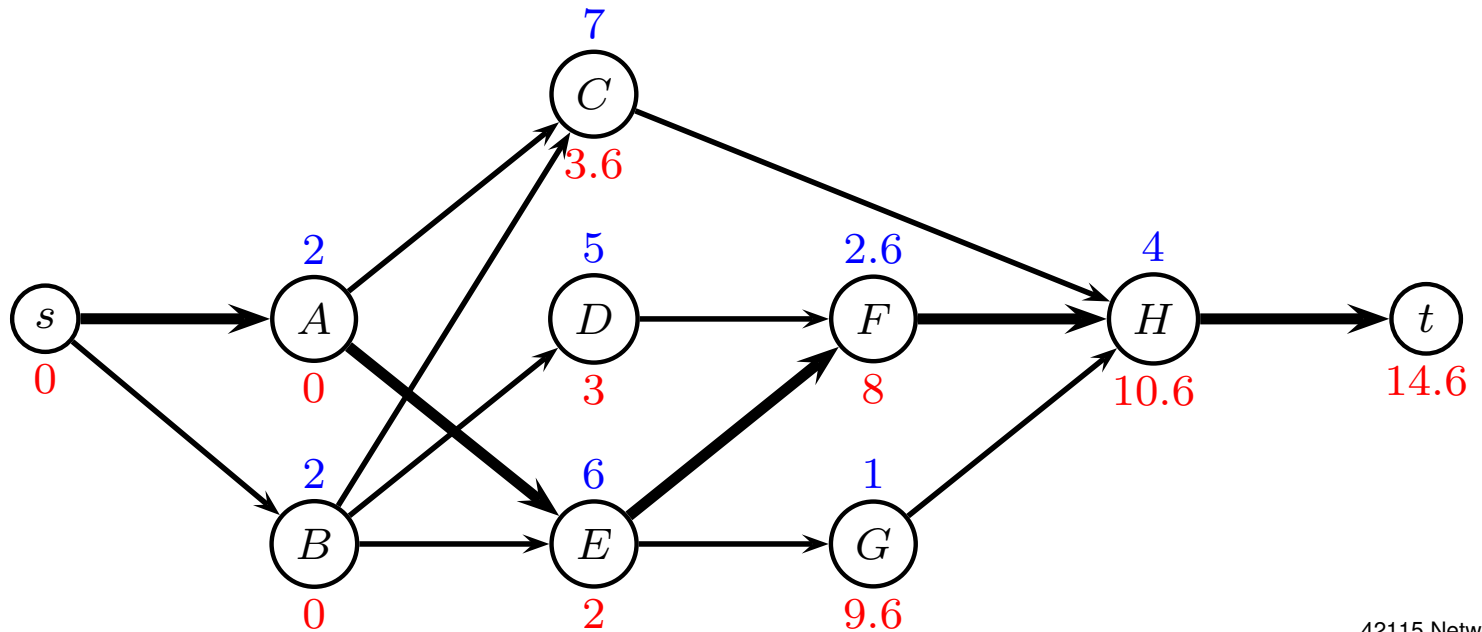
Model 2: shortest completion time, given budget B

$$\begin{aligned} \min \quad & x_t - x_s \\ & x_j \geq x_i + y_i & (i, j) \in E \\ & \sum_{j \in J} c_j (p_j - y_j) \leq B \\ & m_j \leq y_j \leq p_j & j \in J \\ & x_j \geq 0 & j \in J \end{aligned}$$

LP solution, B=6

j	A	B	C	D	E	F	G	H
p_j	3	2	7	5	6	3	1	4
m_j	2	1	4	4	6	2	1	3
c_j	4	3	1	2	7	5	3	6
x_j	0	0	3.6	3	2	8	9.6	10.6
y_j	2	2	7	5	6	2.6	1	4

T: 14.6



LP solution, B=6

T: 14.6

xt	14.600000
xC	3.600000
yA	2.000000
yB	2.000000
xD	3.000000
xE	2.000000
xF	8.000000
yD	5.000000
yE	6.000000
xG	9.600000
xH	10.600000
yC	7.000000
yF	2.600000
yG	1.000000
yH	4.000000

Resource constraints

Renewable resources are available on a period-by-period basis, i.e. the available amount is renewed from period to period.

Typical examples are manpower, machines, tools, equipment and space.

Let R'_i be the amount of resource i

Let R_{ij} be the amount of resource i used by job j

Resource constraints

- No LP formulation
- Can develop an IP
- Hard to solve

Let job t be dummy job (sink) and

$$x_{j\tau} = \left\{ \begin{array}{ll} 1 & \text{if job } j \text{ is completed at time } \tau \\ 0 & \text{otherwise} \end{array} \right\}$$

Resource constraints

Let H be an upper bound on the makespan, e.g.

$$H = \sum_{j=1}^n p_j$$

Completion time of job j is

$$\sum_{\tau=1}^H \tau x_{j\tau}$$

Makespan is

$$T = \sum_{\tau=1}^H \tau x_{t,\tau}$$

Project Management

$$\begin{array}{ll} \min & \sum_{\tau=1}^H \tau x_{t,\tau} \\ \text{s.t.} & \sum_{\tau=1}^H \tau x_{j\tau} + p_k - \sum_{\tau=1}^H \tau x_{k\tau} \leq 0 \quad (j, k) \in E \\ & \sum_{\tau=1}^H \left(R_{ij} \sum_{u=\tau}^{\tau+p_j-1} x_{ju} \right) \leq R'_i \quad \text{each resource used} \\ & \sum_{\tau=1}^H x_{j\tau} = 1 \quad j \in J \end{array}$$

- minimize completion time of last job
- precedence constraints
- resource constraint constraints
- job must finish at exactly one time

Transportation Problems

(and linear programming)

David Pisinger

`pisinger@man.dtu.dk`

DTU Management

Technical University of Denmark

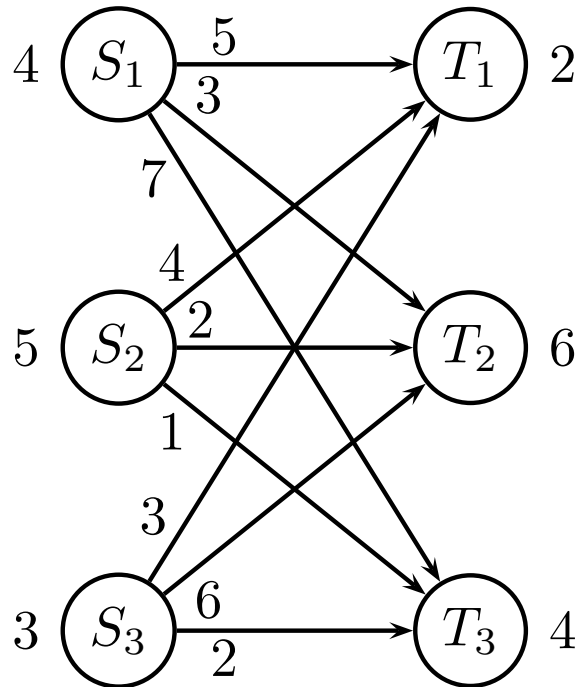
© Copyright David Pisinger. May not be distributed

Transportation Problem

- Bi-partite graph ($m + n$ nodes)
- Sources $S_1, \dots, S_m$, terminals $T_1, \dots, T_n$
- Supply s_i in S_i , demand d_j in T_j
- One commodity, various demands
- Edges have cost $c_{ij} \in \mathbb{R}$, infinite capacity
- Specialized network simplex
- **Next week:** Fixed charge
- Dantzig-Wolfe decomposition
- Column generation

Transportation Problem

Sources S_1, S_2, S_3 , terminals T_1, T_2, T_3



transportation costs:

c_{ij}	T_1	T_2	T_3
S_1	5	3	7
S_2	4	2	1
S_3	3	6	2

Transportation Problem, LP-model

Variable x_{ij} denotes flow from S_i to T_j

$$\min 5x_{11} + 3x_{12} + 7x_{13} + 4x_{21} + 2x_{22} + 1x_{23} + 3x_{31} + 6x_{32} + 2x_{33}$$

$$\text{s.t. } -x_{11} \quad -x_{12} \quad -x_{13} \quad \quad \quad = -4 \quad (u_1)$$

$$\quad \quad -x_{21} \quad -x_{22} \quad -x_{23} \quad \quad \quad = -5 \quad (u_2)$$

$$\quad \quad \quad -x_{31} \quad -x_{32} \quad -x_{33} = -3 \quad (u_3)$$

$$x_{11} \quad \quad \quad +x_{21} \quad \quad \quad +x_{31} \quad \quad = 2 \quad (v_1)$$

$$\quad \quad x_{12} \quad \quad \quad +x_{22} \quad \quad \quad +x_{32} \quad \quad = 6 \quad (v_2)$$

$$\quad \quad \quad x_{13} \quad \quad \quad +x_{23} \quad \quad \quad +x_{33} = 4 \quad (v_3)$$

$$x_{11}, x_{12}, x_{13}, x_{21}, x_{22}, x_{23}, x_{31}, x_{32}, x_{33} \geq 0$$

Solution of Trans. Problem

Solution method

- we can use LP to solve the problem
- network simplex (for transportation model) more interesting

Integer solutions

- constraint matrix is TU (Totally Unimodular)
- if demands are integers the LP-solution will always be integral

Solution of Transportation model

Negative demand for sources, positive for terminals. Model:

$$\begin{aligned} \min \quad & \sum_i \sum_j c_{ij} x_{ij} \\ \text{s.t.} \quad & - \sum_j x_{ij} = -s_i \quad \forall i = 1, \dots, m & u_i \\ & \sum_i x_{ij} = d_j \quad \forall j = 1, \dots, n & v_j \\ & x_{ij} \geq 0 \quad \forall i, j \end{aligned}$$

Assume $\sum_i s_i = \sum_j d_j$

Last terminal takes the rest.

One of the constraints is redundant

so we have only $m + n - 1$ basis variables

Dual

$$\begin{aligned} \max \quad & - \sum_i s_i u_i + \sum_j d_j v_j \\ \text{s.t.} \quad & -u_i + v_j \leq c_{ij} \quad \forall (i, j) \in E \\ & u_i, v_j \in \mathbb{R} \end{aligned}$$

Dual variables are not well-defined (several solutions)
Reduced cost for x_{ij} is found in row (0) in simplex

$$c_{ij} + u_i - v_j$$

Complementary slack

$$\begin{aligned}
 \min \quad & \sum_i \sum_j c_{ij} x_{ij} \\
 \text{s.t.} \quad & -\sum_j x_{ij} = -s_i \quad \forall i \quad \textcolor{red}{u_i} \\
 & \sum_i x_{ij} = d_j \quad \forall j \quad \textcolor{red}{v_j} \\
 & x_{ij} \geq 0 \quad \forall ij
 \end{aligned}$$

$$\begin{aligned}
 \max \quad & \sum_i -s_i u_i + \sum_j d_j v_j \\
 \text{s.t.} \quad & -u_i + v_j \leq c_{ij} \quad \forall ij \quad \textcolor{red}{x_{ij}} \\
 & u_i, v_j \text{ real}
 \end{aligned}$$

Complementary slack: $x_{ij} = 0 \quad \vee \quad -u_i + v_j = c_{ij}$

We demand: $x_{ij} > 0 \quad \Rightarrow \quad c_{ij} + u_i - v_j = 0$

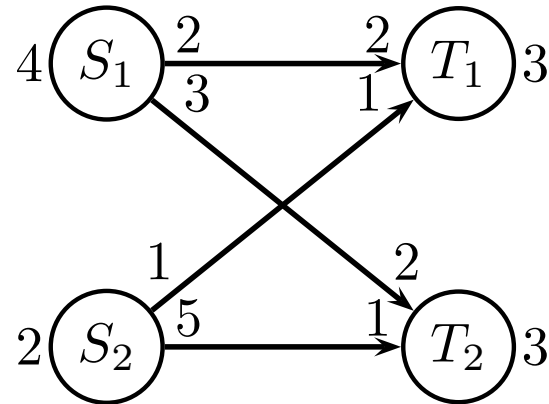
Solution method: Maintain primal feasible solution x_{ij} .

Improve solutions by considering **cycles**

Guess dual solution u_i, v_j satisfying complementary slack

When dual solution is legal, optimality is reached

Improving cycles



- Demands are still satisfied
- When traversing edge backward, we “cancel” flow
- We can only cancel as much as we currently send

Simplex

Z	x	x_s	
1	c	0	0
	A	I	b
1	$c - y^* A$	y^*	$y^* b$
	$S^* A$	S^*	$S^* b$

first tableau

last tableau

● $c - y^* A \geq 0$

● $y^* \geq 0$

● $S^* b = x_B^* \geq 0$

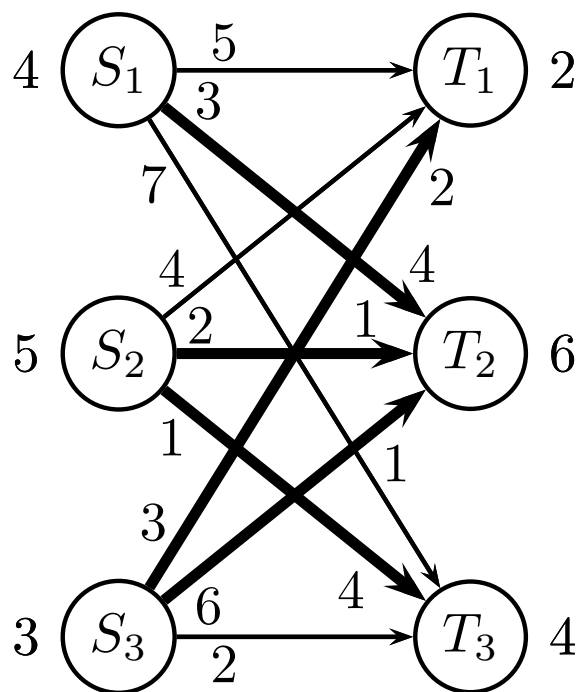
● canonic form: $c - y^* A = 0$ for basis variables

Network Simplex, Greedy heuristics

Heuristic: not optimal

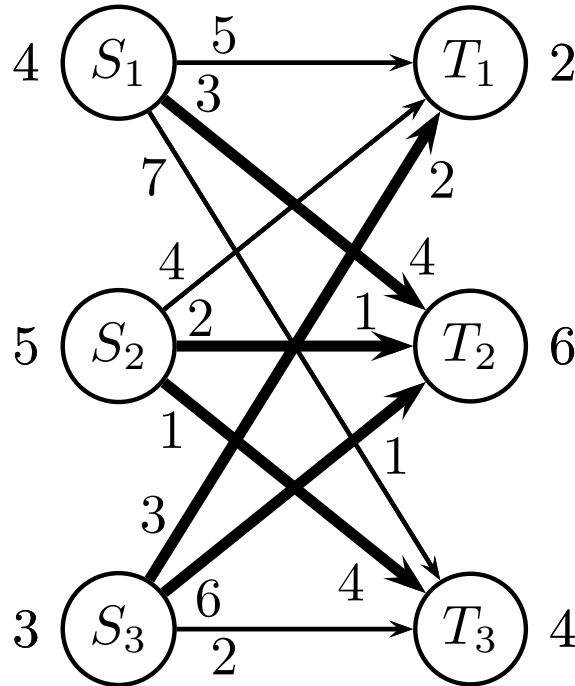
Greedy: chooses the most promising in current step

Consider sources S_1, S_2, S_3 one by one
transport items as cheap as possible to terminals



cost: $4 \cdot 3 + 1 \cdot 2 + 4 \cdot 1 + 2 \cdot 3 + 1 \cdot 6 = 30$

Network Simplex, Greedy heuristic



Thick edges if flow > 0 on edge

5 thick edges ($m + n - 1$) form **basis**

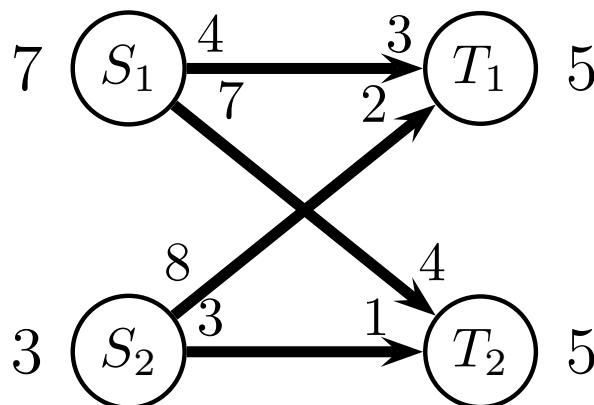
Basis is a **spanning tree** by construction

Algorithm will ensure tree in every iteration

If less than $m + n - 1$ edges have flow > 0 then choose extra edges with flow 0

Network Simplex, optimal solution is tree

Assume optimal solution is not a tree



E.g. cycle $C = \{S_1, T_1, S_2, T_2\}$

- $\text{cost}(C) > 0$: send some units backward
- $\text{cost}(C) < 0$: send some units forward
- $\text{cost}(C) = 0$: send some units forward or backward

So, an optimal solution exists without cycles

Network Simplex, iterations

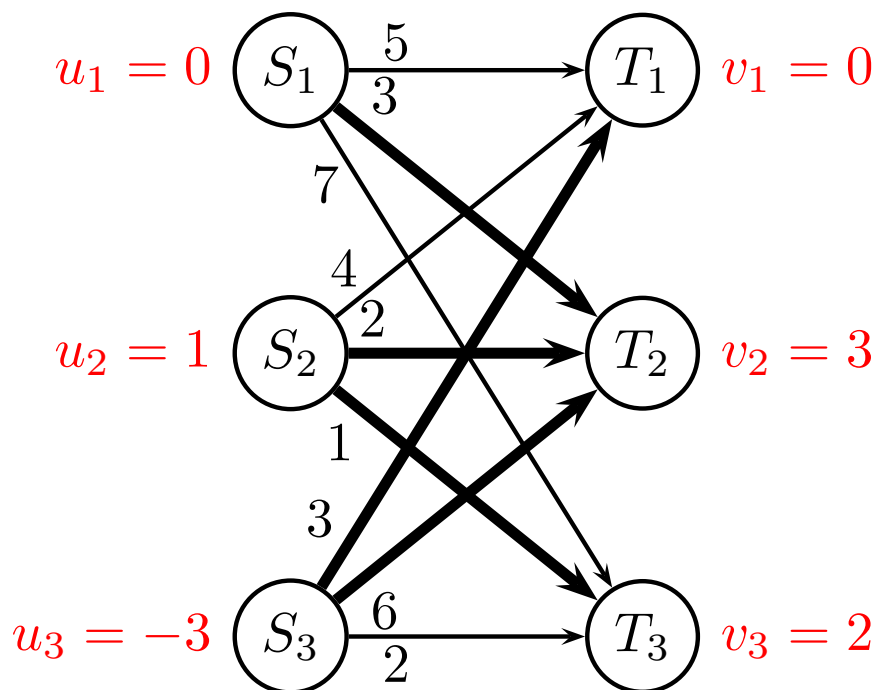
Choose node S_1 as root, calculate distance to nodes

Only thick edges may be used

→ cost positive

← cost negative

u_i = distance to S_i -nodes, v_j = distance to T_j -nodes



Network Simplex, iterations

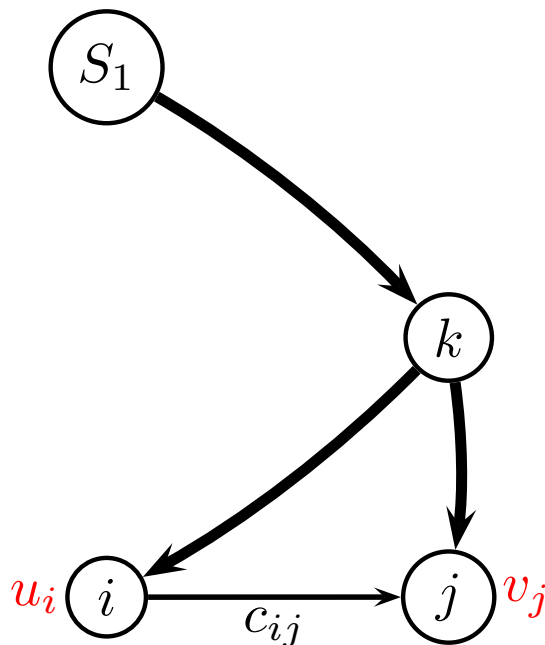
Find cycle in graph, saving transportation costs

- (S_1, T_1) impose cycle $(S_1, T_1, S_3, T_2, S_1)$, cost
 $5 - 3 + 6 - 3 = 5$
reduced cost $\bar{c}_{11} = c_{11} + u_1 - v_1 = 5 + 0 - 0 = 5$
- (S_1, T_3) impose cycle $(S_1, T_3, S_2, T_2, S_1)$, cost
 $7 - 1 + 2 - 3 = 5$
reduced cost $\bar{c}_{13} = c_{13} + u_1 - v_3 = 7 + 0 - 2 = 5$
- (S_2, T_1) impose cycle $(S_2, T_1, S_3, T_2, S_2)$, cost
 $4 - 3 + 6 - 2 = 5$
reduced cost $\bar{c}_{21} = c_{21} + u_2 - v_1 = 4 + 1 - 0 = 5$
- (S_3, T_3) impose cycle $(S_3, T_3, S_2, T_2, S_3)$, cost
 $2 - 1 + 2 - 6 = -3$
reduced cost $\bar{c}_{33} = c_{33} + u_3 - v_3 = 2 - 3 - 2 = -3$

Choose cycle with most negative reduced cost: edge (S_3, T_3)

Network Simplex, cost of cycle

Cost of cycle C imposed by edge (i, j) is equal to reduced cost $\bar{c}_{ij}$



$$\begin{aligned}\bar{c}_{ij} &= c_{ij} + u_i - v_j \\ &= c_{ij} + \text{dist}(S_1, k) + \text{dist}(k, i) - \text{dist}(S_1, k) - \text{dist}(k, j) \\ &= c_{ij} + \text{dist}(j, k) + \text{dist}(k, i) \\ &= \text{cost}(C)\end{aligned}$$

Network Simplex, iterations

How much can flow along cycle?

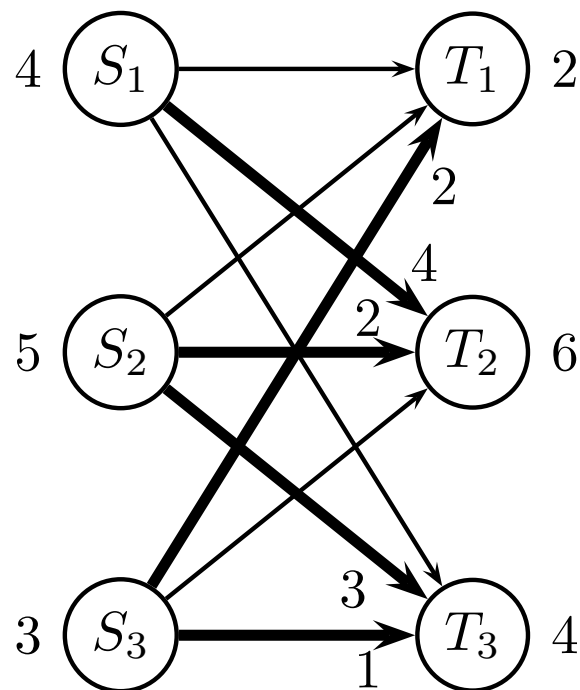
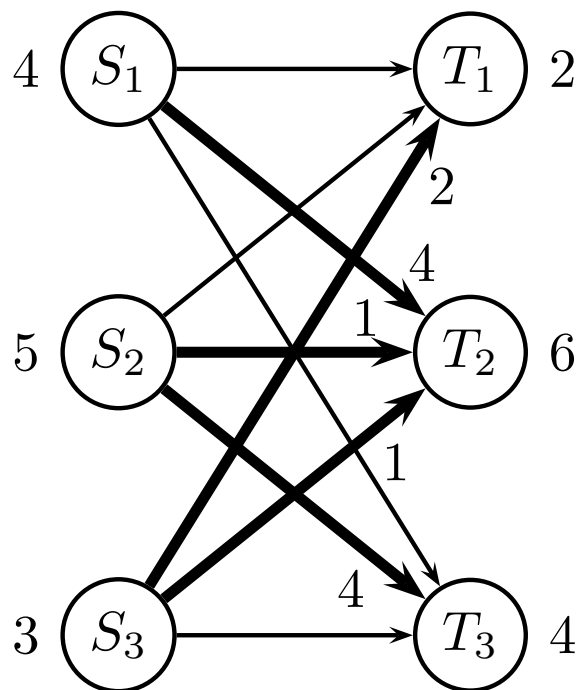
Forward edges: ∞ capacity

Backward edges: as much as current flow

Cycle $(S_3, T_3, S_2, T_2, S_3)$ has backward edges:

(T_3, S_2) flow 4
 (T_2, S_3) flow 1

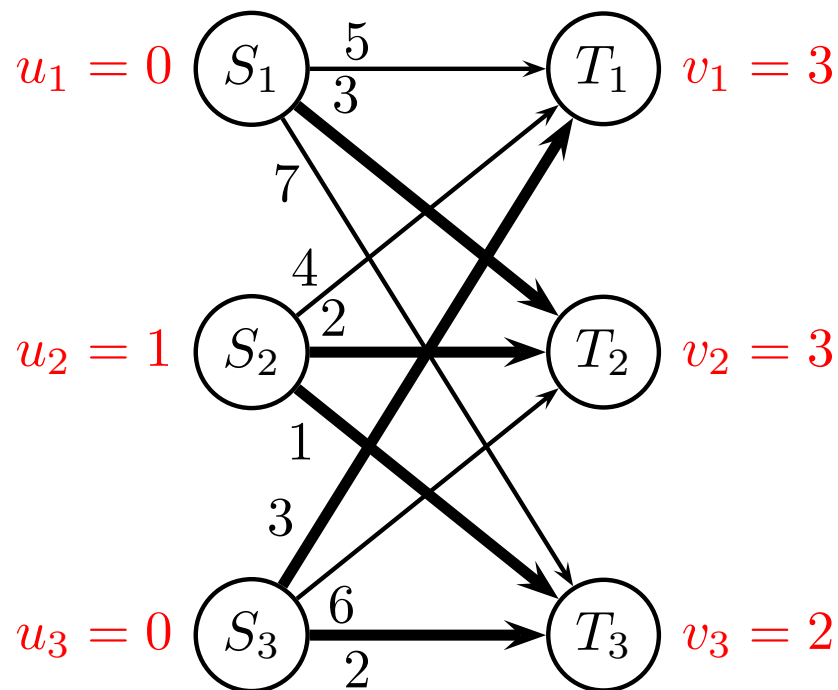
} send $\theta = 1$ unit



Cost: $4 \cdot 3 + 2 \cdot 2 + 3 \cdot 1 + 2 \cdot 3 + 1 \cdot 2 = 27$.

Network Simplex, iterations

Distances from S_1 to all other nodes in the tree:



Network Simplex, stop criteria

Find cycle in graph, saving transportation costs

- Edge (S_1, T_1) has $\bar{c}_{11} = c_{11} + u_1 - v_1 = 5 + 0 - 3 = 2$
- Edge (S_1, T_3) has $\bar{c}_{13} = c_{13} + u_1 - v_3 = 7 + 0 - 2 = 5$
- Edge (S_2, T_1) has $\bar{c}_{21} = c_{21} + u_2 - v_1 = 4 + 1 - 3 = 2$
- Edge (S_3, T_2) has $\bar{c}_{32} = c_{32} + u_3 - v_2 = 6 + 0 - 3 = 3$

Since no negative reduced cost, simplex terminates.

Reduced costs are:

- $\bar{c}_{ij} = 0$ for thick edges.
- $\bar{c}_{ij} \geq 0$ for thin edges.

This shows u, v is an optimal dual solution

Network Simplex, pseudocode

find greedy solution

let basis (fat edges) be those edges where flow > 0

if basis has less than $n + m - 1$ edges, add additional (thin) edges

repeat

- calculate dual costs u_i, v_j as distance (using fat edges) from node S_1
- for every thin edge calculate reduced cost as $\bar{c}_{ij} = c_{ij} + u_i - v_j$
- choose the thin edge (i', j') with most negative reduced cost
- define cycle C as (i', j') and fat edges
- the cycle C should move in the direction of (i', j')
- calculate the “bottleneck” θ as minimum of flow on backward edges in C
- send θ units along C
- edge (i', j') becomes fat, and the bottleneck edge becomes thin

until all reduced costs $\bar{c}_{ij} \geq 0$

Tips-and-tricks

Network simplex

- check that problem is balanced, otherwise add surplus node to make problem balanced
- if some edges are missing, you can add them with big cost M , this will ensure that you can find a greedy solution
- if $\theta = 0$ then just send zero items along the cycle, this will give a new basis

Network Simplex, running time

Let U be an upper bound on the objective, e.g.

$$U = \sum_i s_i \max_j c_{ij}$$

If demands and costs are integers, each iteration will decrease objective by at least 1

Each iteration takes:

- Find dual costs $O(E)$
- Find reduced costs for all thin edges $O(E)$
- Pivot operation $O(E)$

In total: $O(EU)$

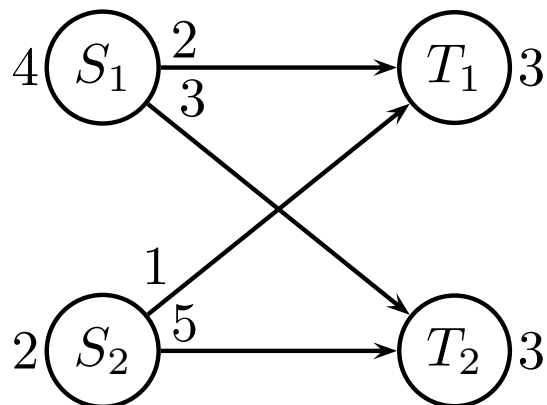
Relation to Simplex Algorithm

David Pisinger

`pisinger@man.dtu.dk`

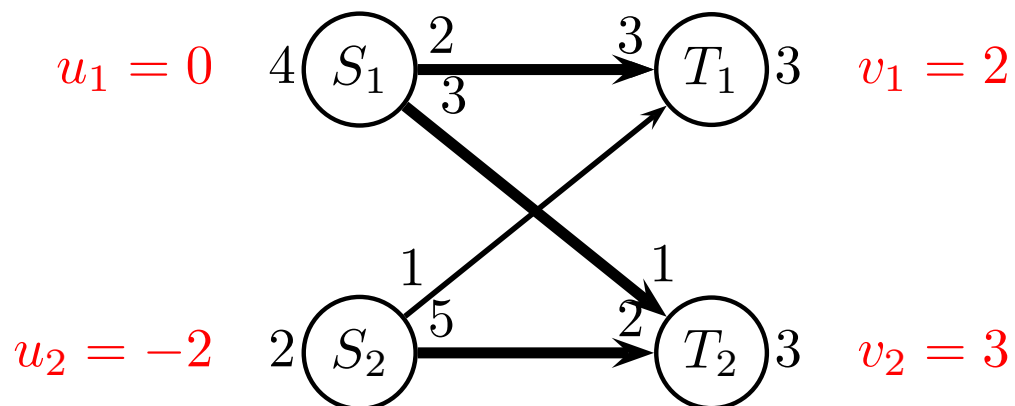
DTU Management
Technical University of Denmark

A Small Example



$$\begin{array}{llllllll}
 \min & 2 & x_{11} & + & 3 & x_{12} & + & 1 & x_{21} & + & 5 & x_{22} \\
 \text{s.t.} & -x_{11} & - & x_{12} & & & & & & & & = & -4 \\
 & & & & & & - & x_{21} & - & x_{22} & & = & -2 \\
 & x_{11} & & & & & + & x_{21} & & & & = & 3 \\
 & & x_{12} & & & & & & + & x_{22} & & = & 3
 \end{array}$$

A Small Example



Cost:

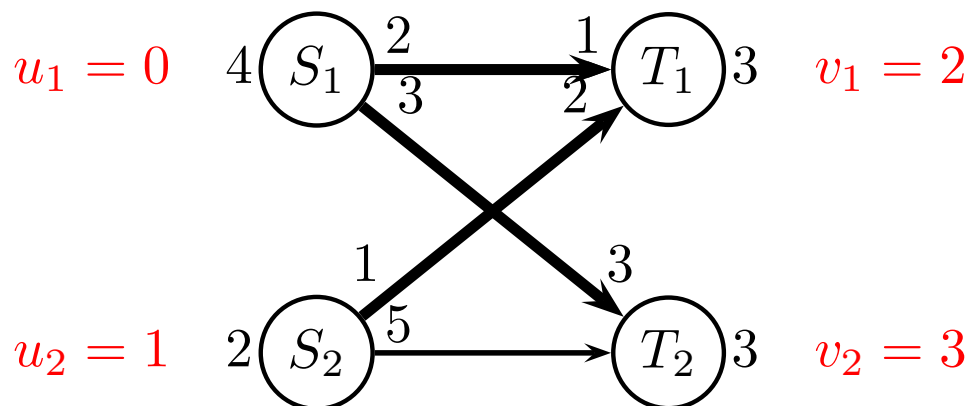
$$2 \cdot 3 + 3 \cdot 1 + 5 \cdot 2 = 19$$

Reduced cost:

$$\bar{c}_{21} = c_{21} + u_2 - v_1 = 1 - 2 - 2 = -3$$

Cycle S_2, T_1, S_1, T_2 , capacity $\min\{3, 2\} = 2$.

A Small Example



Cost:

$$2 \cdot 1 + 3 \cdot 3 + 1 \cdot 2 = 13$$

Reduced cost:

$$\bar{c}_{22} = c_{22} + u_2 - v_2 = 5 + 1 - 3 = 3$$

Stop

A Small Example

$$\begin{array}{rcll}
 -Z + 2x_{11} + 3x_{12} + 1x_{21} + 5x_{22} & = & 0 & (Z) \\
 -x_{11} - x_{12} & = & -4 & (A) \\
 & -x_{21} - x_{22} & = & -2 \quad (B) \\
 & x_{11} + x_{21} & = & 3 \quad (C) \\
 & x_{12} + x_{22} & = & 3 \quad (D)
 \end{array}$$

Since $-(A)-(B)-(C) = (D)$, last row is redundant, can be deleted

$$\begin{array}{rcll}
 -Z + 2x_{11} + 3x_{12} + 1x_{21} + 5x_{22} & = & 0 & (Z) \\
 -x_{11} - x_{12} & = & -4 & (A) \\
 & -x_{21} - x_{22} & = & -2 \quad (B) \\
 & x_{11} + x_{21} & = & 3 \quad (C)
 \end{array}$$

A Small Example

Should run two-phase algorithm, but in this case easy to guess solution with greedy algorithm.

Want basis x_{11}, x_{12}, x_{22}

$$\begin{array}{rclcl}
 -Z + 2x_{11} + 3x_{12} + 1x_{21} + 5x_{22} & = & 0 & & (Z) \\
 x_{11} + x_{12} & = & 4 & & (A) \\
 & & x_{21} + x_{22} & = & 2 & (B) \\
 x_{11} & + & x_{21} & = & 3 & (C)
 \end{array}$$

Add $-(C)$ to (A)

$$\begin{array}{rclcl}
 -Z + 2x_{11} + 3x_{12} + 1x_{21} + 5x_{22} & = & 0 & & (Z) \\
 & & x_{12} - x_{21} & = & 1 & (A) \\
 & & x_{21} + x_{22} & = & 2 & (B) \\
 x_{11} & + & x_{21} & = & 3 & (C)
 \end{array}$$

A Small Example

Must ensure that basis variables have 0 in (Z)

$$\begin{array}{rcll}
 -Z + 2x_{11} + 3x_{12} + 1x_{21} + 5x_{22} & = & 0 & (Z) \\
 & x_{12} - & x_{21} & = 1 \quad (A) \\
 & & x_{21} + & x_{22} = 2 \quad (B) \\
 & x_{11} & + & x_{21} = 3 \quad (C)
 \end{array}$$

Add -2(C) to (Z), add -3(A) to (Z), and add -5(B) to (Z)

$$\begin{array}{rcll}
 -Z + & & -3x_{21} & = -19 \quad (Z) \\
 & x_{12} - & x_{21} & = 1 \quad (A) \\
 & & x_{21} + & x_{22} = 2 \quad (B) \\
 & x_{11} & + & x_{21} = 3 \quad (C)
 \end{array}$$

Read basis solution: $x_{11} = 3, x_{12} = 1, x_{22} = 2$ and $z = 19$.

A Small Example

Reduced cost of x_{21} is -3, so x_{21} enters basis

$$\begin{array}{rclcl}
 -Z+ & & -3 & x_{21} & = -19 & (Z) \\
 & x_{12} & - & x_{21} & = 1 & (A) \\
 & & & x_{21} + x_{22} & = 2 & (B) \\
 & x_{11} & + & x_{21} & = 3 & (C)
 \end{array}$$

Coefficient test shows that (B) is most binding

Add (B) to (A), add -(B) to (C), and add 3(B) to (Z)

$$\begin{array}{rclcl}
 -Z+ & & +3 & x_{22} & = -13 & (Z) \\
 & x_{12} & + & x_{22} & = 3 & (A) \\
 & & x_{21} + & x_{22} & = 2 & (B) \\
 & x_{11} & - & x_{22} & = 1 & (C)
 \end{array}$$

No negative reduced cost, so stop.

Read basis solution: $x_{11} = 1, x_{12} = 3, x_{21} = 2$ and $z = 13$.

Relation to Simplex Algorithm

Similarities

- Basis (thick edges)
- Reduced costs
- Entering variable
- Pivot operation

Advantage

- Easy to construct initial solution
- More intuitive representation
- More compact

Complementary slack theorem (proof)

Assume that x is feasible to primal, and y is feasible to dual

$$\begin{array}{ll} \max & cx \\ \text{s.t.} & Ax \leq b \\ & x \geq 0 \end{array} \qquad \begin{array}{ll} \min & yb \\ \text{s.t.} & yA \geq c \\ & y \geq 0 \end{array}$$

If complementary slack is satisfied

$$(yA - c)x = 0 \quad \text{and} \quad y(b - Ax) = 0$$

then the solutions x, y are optimal.

Proof write out the above constraints

$$cx = yAx \quad \text{and} \quad yAx = yb$$

hence, $cx = yb$, implying that x and y are optimal ■

Totally Unimodular

Definition

An $m \times n$ integral matrix A is called *totally unimodular* (TU) if the determinant of each square submatrix of A is equal to 0, 1 or -1.

Obviously $a_{ij} \in \{0, 1, -1\}$

Recognising whether A is TU demands an exponential number of steps

Totally Unimodular

If A is TU then A^{-1} is integral

From Cramer's rule $A_{ij}^{-1} = C_{ji} / \det(A)$ where C_{ji} is the adjoint matrix

$$C_{ji} = (-1)^{i+j} \det(A_{\text{row } i, \text{ column } j \text{ removed}})$$

If A is TU and b integral, then every basis solution $x_B = A_B^{-1}b$ is integral

Totally Unimodular, sufficient condition

A matrix $A = (a_{ij})$ is TU if

1. $a_{ij} \in \{+1, -1, 0\}$ for all i, j .
2. Each column contains at most two nonzero coefficients
3. There exists a partition (P_1, P_2) of the set M of rows such that each column j containing two nonzeros satisfies:
 - coefficients have same sign $\Leftrightarrow$ rows belong to different sets
 - coefficients have different sign $\Leftrightarrow$ rows belong to same set

Totally Unimodular, examples

$$\begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$$

$$\begin{pmatrix} 0 & 1 & -1 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\begin{pmatrix} 1 & -1 & -1 & 0 \\ -1 & 0 & 0 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

$$\begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & -1 & 1 & 0 \\ -1 & 0 & 1 & 0 & -1 \\ 0 & 0 & 0 & 0 & -1 \\ 0 & -1 & 0 & 0 & 0 \end{pmatrix}$$

Upper matrices are not TU, lower matrices are TU.

Big-O notation

for Network Problems

David Pisinger

`pisinger@man.dtu.dk`

DTU Management

Technical University of Denmark

Complexity of Network Algorithm

A good algorithm should

- be fast
- use little space

Larger instances use more time/space

- n is size of instance (how many nodes/edges)
- measure running time and space as function of n

Independent of computer

- we count number of additions/multiplications/moves
- we do not care about constants

Complexity of Algorithm

Algorithm 1

n	time (s)	space (Mb)
10	1	1
20	2	2
30	4	4
40	8	8
50	16	16
60	32	32
70	64	64
80	128	128
90	256	256
100	512	512

$2^{n/10}$

$2^{n/10}$

Algorithm 2

n	time (s)	space (Mb)
10	110	11
20	120	12
30	130	13
40	140	14
50	150	15
60	160	16
70	170	17
80	180	18
90	190	19
100	200	20

$100 + n$

$10 + n/10$

Algorithm 3

n	time (s)	space (Mb)
10	23	10000
20	30	10000
30	34	10000
40	37	10000
50	39	10000
60	41	10000
70	42	10000
80	44	10000
90	45	10000
100	46	10000

$10 \log n$

10000

Complexity of Algorithm

n	10	50	100	300	1000
$5n$	50	250	100	1500	5000
n^2	100	2500	10.000	90.000	10^7
n^3	1000	125.000	10^7	10^8	10^{10}
2^n	1024	10^{16}	10^{31}	10^{91}	10^{302}
$n!$	10^7	10^{65}	10^{161}	10^{623}	∞
n^n	10^{11}	10^{85}	10^{201}	10^{744}	∞

Atoms in the universe: 10^{78} – 10^{82}

Time since big-bang: 13.73 billion years = 10^{18} seconds

Complexity of Algorithm

Asymptotic growth of functions
Big-O notation

$$f \text{ is } O(g) \Leftrightarrow \exists k, c : f(n) \leq kg(n) \text{ for } n > c$$

Ordering of complexity

$O(1)$	$O(\log n)$	$O(\log^2 n)$	$O(\log^3 n)$	$O(\sqrt{n})$
$O(n)$	$O(n \log n)$	$O(n \log^2 n)$	$O(n \log^3 n)$	$O(n\sqrt{n})$
$O(n^2)$	$O(n^2 \log n)$	$O(n^2 \log^2 n)$	$O(n^2 \log^3 n)$	$O(n^2 \sqrt{n})$
$O(n^3)$	$O(n^4)$	$O(2^n)$	$O(n!)$	$O(n^n)$

Notice

$$O(100n^3) = O(n^3)$$

$$O(2n^4 + 100n^2 + n \log n) = O(n^4)$$

Exercises

Exercise 1 Write in big-O notation the asymptotic growth of the following functions

a $4n^3 + n^4 + n^3(\log n)^2$

b $100n + n \log n + \sqrt{n}$

c $5n^7 + 100n^6 + n^8$

d $n\sqrt{n} + n \log n + n \log(n^2)$

Fixed Charge Transportation Problem

(and decomposition)

David Pisinger

`pisinger@man.dtu.dk`

DTU Management

Technical University of Denmark

Fixed charge transportation problem

Transport problem

- Bi-partite graph
- Sources $S_1, \dots, S_m$, terminals $T_1, \dots, T_n$
- Production s_i in S_i , demand d_j in T_j
- Edges have cost $c_{ij} \in \mathbb{R}$, infinite capacity

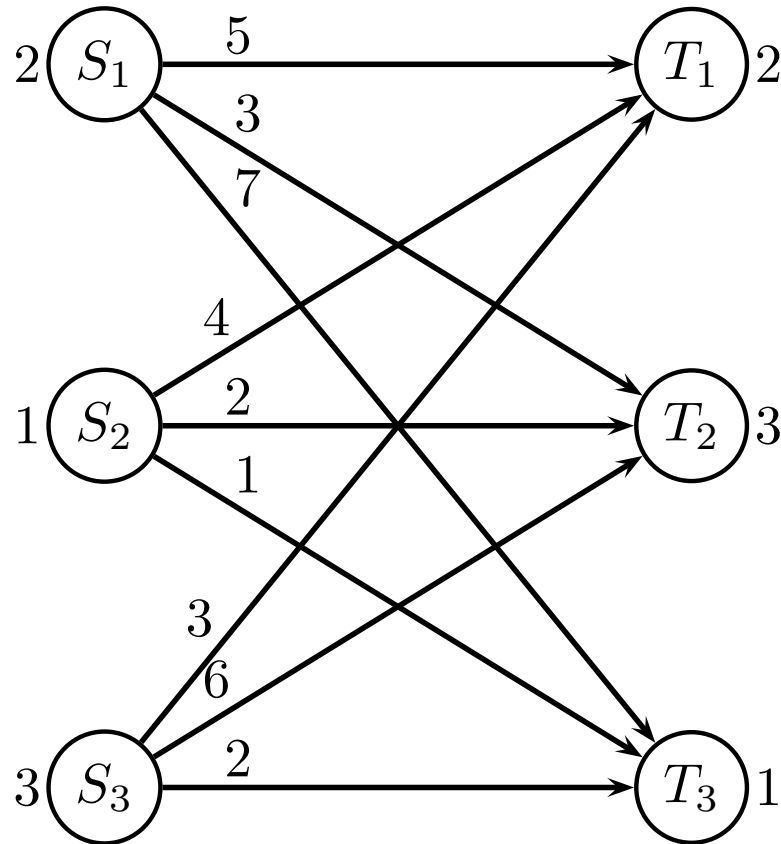
Now assume, **fixed charge** F for using an edge

- Cost for building connection (cable, pipeline)
- Startup cost for vehicle

If no flow on an edge, no fixed charge is paid

Fixed charge transportation problem

Example:



Fixed charge for using edge (i, j) is here $F = 8$
(could be different F_{ij} for each edge)

Formal definition

Without fixed cost

$$\begin{array}{ll}\min & \sum_i \sum_j c_{ij} x_{ij} \\ \text{s.t.} & - \sum_j x_{ij} = -s_i \quad \forall i = 1, \dots, m \\ & \sum_i x_{ij} = d_j \quad \forall j = 1, \dots, n \\ & x_{ij} \geq 0 \quad \forall i, j\end{array}$$

variable x_{ij} denotes flow from S_i to T_j

Formal definition

With fixed cost

$$\begin{array}{ll}\min & \sum_i \sum_j (c_{ij}x_{ij} + Fy_{ij}) \\ \text{s.t.} & -\sum_j x_{ij} = -s_i \quad \forall i = 1, \dots, m \\ & \sum_i x_{ij} = d_j \quad \forall j = 1, \dots, n \\ & x_{ij} \leq My_{ij} \quad \forall j = 1, \dots, n \\ & x_{ij} \geq 0 \quad \forall i, j \\ & y_{ij} \in \{0, 1\} \quad \forall i, j\end{array}$$

variable x_{ij} denotes flow from S_i to T_j

variable y_{ij} denotes whether edge from S_i to T_j is used

M is upper bound on flow

Fixed charge transportation problem

Variable x_{ij} denotes flow from S_i to T_j

Edge formulation *without* setup costs

$$\min 5x_{11} + 3x_{12} + 7x_{13} + 4x_{21} + 2x_{22} + 1x_{23} + 3x_{31} + 6x_{32} + 2x_{33}$$

$$\text{s.t. } -x_{11} \quad -x_{12} \quad -x_{13} \quad \quad \quad = -2 \quad (u_1)$$

$$\quad \quad \quad -x_{21} \quad -x_{22} \quad -x_{23} \quad \quad \quad = -1 \quad (u_2)$$

$$\quad \quad \quad \quad \quad -x_{31} \quad -x_{32} \quad -x_{33} = -3 \quad (u_3)$$

$$x_{11} \quad \quad \quad +x_{21} \quad \quad \quad +x_{31} \quad \quad \quad = 2 \quad (v_1)$$

$$\quad \quad x_{12} \quad \quad \quad +x_{22} \quad \quad \quad +x_{32} \quad \quad \quad = 3 \quad (v_2)$$

$$\quad \quad \quad x_{13} \quad \quad \quad +x_{23} \quad \quad \quad +x_{33} = 1 \quad (v_3)$$

$$x_{11}, x_{12}, x_{13}, x_{21}, x_{22}, x_{23}, x_{31}, x_{32}, x_{33} \geq 0$$

Fixed charge transportation problem

Edge formulation *without* setup costs

```
min
  5 x11 + 3 x12 + 7 x13 + 4 x21 + 2 x22 + 1 x23 + 3 x31 + 6 x32 + 2 x33
subject to
- x11 - x12 - x13                                     = -2
               - x21 - x22 - x23                       = -1
                           - x31 - x32 - x33           = -3
x11               + x21               + x31           = 2
      x12               + x22               + x32       = 3
            x13               + x23               + x33 = 1
end
```

Fixed charge transportation problem

Edge formulation *with* setup costs (*)

```

min
  5 x11 + 3 x12 + 7 x13 + 4 x21 + 2 x22 + 1 x23 + 3 x31 + 6 x32 + 2 x33
+ 8 y11 + 8 y12 + 8 y13 + 8 y21 + 8 y22 + 8 y23 + 8 y31 + 8 y32 + 8 y33
subject to
- x11 - x12 - x13                                     = -2
               - x21 - x22 - x23                       = -1
                           - x31 - x32 - x33             = -3
x11               + x21               + x31             = 2
      x12               + x22               + x32       = 3
            x13               + x23               + x33  = 1

x11 - 3 y11 <= 0
x12 - 3 y12 <= 0
x13 - 3 y13 <= 0
x21 - 3 y21 <= 0
x22 - 3 y22 <= 0
x23 - 3 y23 <= 0
x31 - 3 y31 <= 0
x32 - 3 y32 <= 0
x33 - 3 y33 <= 0

binary
  y11 y12 y13 y21 y22 y23 y31 y32 y33
end

```

Much harder to solve (NP-hard)

Fixed charge transportation problem

IP-solution

MIP - Integer optimal solution: Objective = 4.8000000000e+01
Solution time = 0.00 sec. Iterations = 2 Nodes = 0

Variable Name	Solution Value
x12	2.000000
x22	1.000000
x31	2.000000
x33	1.000000
y12	1.000000
y22	1.000000
y31	1.000000
y33	1.000000

All other variables in the range 1-18 are 0.

Fixed charge transportation problem

LP-relaxed problem

Tried aggregator 1 time.

LP Presolve eliminated 9 rows and 0 columns.

Aggregator did 9 substitutions.

Reduced LP has 6 rows, 9 columns, and 18 nonzeros.

Presolve time = 0.00 sec.

Iteration log . . .

Iteration: 1 Dual objective = 14.000000

Dual simplex - Optimal: Objective = 3.2000000000e+01

Solution time = 0.00 sec. Iterations = 5 (0)

CPLEX> dis sol var -

Variable Name	Solution Value
x12	2.000000
x22	1.000000
x31	2.000000
x33	1.000000
y12	0.666667
y22	0.333333
y31	0.666667
y33	0.333333

All other variables in the range 1-18 are 0.

(bad bound, many fractional variables)

Fixed charge transportation problem

Problem

- Bound from LP problem is horribly bad. Often 50% from optimum
- Branch-and-bound algorithm will have huge running times
- Larger problems cannot be solved to optimality

Solution

- Reformulate problem (Dantzig-Wolfe decomposition)

Two-stage problems

Consider problem

$$\begin{array}{ll}\min & cx \\ \text{s.t.} & Ax = b\end{array}$$

Columns in A are result of a subproblem

$$(c_j, A_j) = \min\{dx : x \in S\}$$

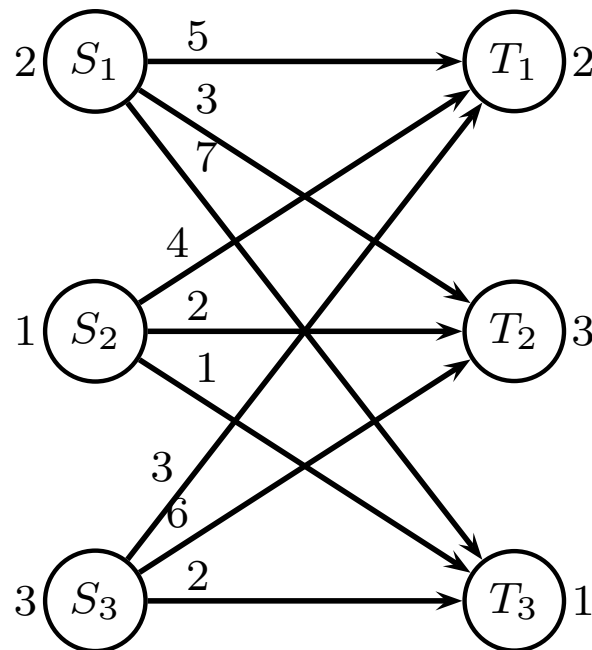
For instance:

- Columns are work plans, combine work plans to satisfy demand
- Columns are cutting patterns, combine patterns to satisfy demand
- Columns are cheapest paths, combine to flow through graph

In our case

- Stage 1: For each source S_i choose how the production should be flown
- Stage 2: Combine patterns, so that demand at T_j is satisfied and only one pattern per S_i is chosen.

Dantzig-Wolfe decomposition of problem



Fixed charge transportation problem

1 item sent from a S -node can be split in 3 ways (in our example, all legal if sent from S_2)

$$\begin{bmatrix} 1 & & \\ & 1 & \\ & & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} 1 & & \\ & 1 & \\ & & 1 \end{bmatrix}$$

2 items from a S -node can be split in 6 ways (in our example 5 legal ways if sent from S_1)

$$\begin{bmatrix} 2 & & 1 & 1 & \\ & 2 & 1 & & 1 \\ & & 2 & 1 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} 2 & & 1 & 1 & \\ & 2 & 1 & & 1 \\ & & & 1 & 1 \end{bmatrix}$$

Fixed charge transportation problem

3 items sent from a S -node can be split in 10 ways (in our example 6 legal ways if sent from S_3)

$$\begin{bmatrix} 3 & & 2 & 2 & 1 & & 1 & & 1 \\ & 3 & & 1 & & 2 & 2 & & 1 & 1 \\ & & 3 & & 1 & & 1 & 2 & 2 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} & 2 & 2 & 1 & & 1 \\ 3 & 1 & & 2 & 2 & 1 \\ & & 1 & & 1 & 1 \end{bmatrix}$$

Fixed charge transportation problem

Variable $p_{ij} \in \{0, 1\}$ denotes if we choose pattern j for node S_i

Cost and pattern:

$$\begin{array}{l}
 \begin{array}{cccccc}
 +18 & p_{11} & +14 & p_{12} & +24 & p_{13} & +28 & p_{14} & +26 & p_{15} \\
 \hline
 \end{array} \\
 S1: \quad \begin{array}{l}
 + 2 p_{11} + 0 p_{12} + 1 p_{13} + 1 p_{14} + 0 p_{15} \\
 + 0 p_{11} + 2 p_{12} + 1 p_{13} + 0 p_{14} + 1 p_{15} \\
 + 0 p_{11} + 0 p_{12} + 0 p_{13} + 1 p_{14} + 1 p_{15}
 \end{array}
 \end{array}$$

$$\begin{array}{l}
 \begin{array}{cccc}
 +12 & p_{21} & +10 & p_{22} & + 9 & p_{23} \\
 \hline
 \end{array} \\
 S2: \quad \begin{array}{l}
 + 1 p_{21} + 0 p_{22} + 0 p_{23} \\
 + 0 p_{21} + 1 p_{22} + 0 p_{23} \\
 + 0 p_{21} + 0 p_{22} + 1 p_{23}
 \end{array}
 \end{array}$$

$$\begin{array}{l}
 \begin{array}{ccccccccc}
 +26 & p_{31} & +28 & p_{32} & +24 & p_{33} & +31 & p_{34} & +30 & p_{35} & +35 & p_{36} \\
 \hline
 \end{array} \\
 S3: \quad \begin{array}{l}
 + 0 p_{31} + 2 p_{32} + 2 p_{33} + 1 p_{34} + 0 p_{35} + 1 p_{36} \\
 + 3 p_{31} + 1 p_{32} + 0 p_{33} + 2 p_{34} + 2 p_{35} + 1 p_{36} \\
 + 0 p_{31} + 0 p_{32} + 1 p_{33} + 0 p_{34} + 1 p_{35} + 1 p_{36}
 \end{array}
 \end{array}$$

Fixed charge transportation problem

Dantzig-Wolfe decomposed model, LP relaxed (**)

```
minimize
18p11+14p12+24p13+28p14+26p15+12p21+10p22+ 9p23+26p31+28p32+24p33+31p34+30p35+35p36
subject to
 2p11+ 0p12+ 1p13+ 1p14+ 0p15+ 1p21+ 0p22+ 0p23+ 0p31+ 2p32+ 2p33+ 1p34+ 0p35+ 1p36 = 2
 0p11+ 2p12+ 1p13+ 0p14+ 1p15+ 0p21+ 1p22+ 0p23+ 3p31+ 1p32+ 0p33+ 2p34+ 2p35+ 1p36 = 3
 0p11+ 0p12+ 0p13+ 1p14+ 1p15+ 0p21+ 0p22+ 1p23+ 0p31+ 0p32+ 1p33+ 0p34+ 1p35+ 1p36 = 1
 1p11+ 1p12+ 1p13+ 1p14+ 1p15                                     = 1
                                     + 1p21+ 1p22+ 1p23                                     = 1
                                     + 1p31+ 1p32+ 1p33+ 1p34+ 1p35+ 1p36 = 1
end
```

First 3 constraints: Meet demand at nodes T_j

Last 3 constraints: Only one patten for each S_i

Fixed charge transportation problem

Dantzig-Wolfe decomposed model, LP relaxed

```
CPLEX> opt
Dual simplex - Optimal:  Objective =  4.80000000000e+01
Solution time =      0.01 sec.  Iterations = 6 (0)
CPLEX> dis sol var -
Variable Name           Solution Value
p12                      1.000000
p22                      1.000000
p33                      1.000000
All other variables in the range 1-14 are 0.
```

) (not always an integer solution)

Fixed charge transportation problem

- Lower bound from Dantzig-Wolfe decomposed problem is much better
- Can reverse role of S and T to calculate other bound
- Choose best of bounds $S \rightarrow T$ and $T \rightarrow S$

Solving large problems

- If demands are large, impossible to write up Dantzig-Wolfe model
- *Delayed column generation*

Solving large LP problems

Frequently we have

$$\max cx$$

$$Ax = b$$

$$x \in \mathbb{R}_+^n$$

with thousands of constraints, and millions of variables.
However, columns in A follow some structure

Although Simplex in theory can solve these problems

- Problems can be too large to write up
- Problems cannot be stored in RAM on computer
- Numerical instability can make Simplex cycle

Solving large LP problems

For given basis B problem can be rewritten

$$-z + c_B x_B + c_N x_N = 0$$

$$A_B x_B + A_N x_N = b$$

Basis solution

$$-z + (c_N - c_B A_B^{-1} A_N) x_N = -c_B A_B^{-1} b$$

$$I x_B + A_B^{-1} A_N x_N = A_B^{-1} b$$

$$x_B, x_N \geq 0$$

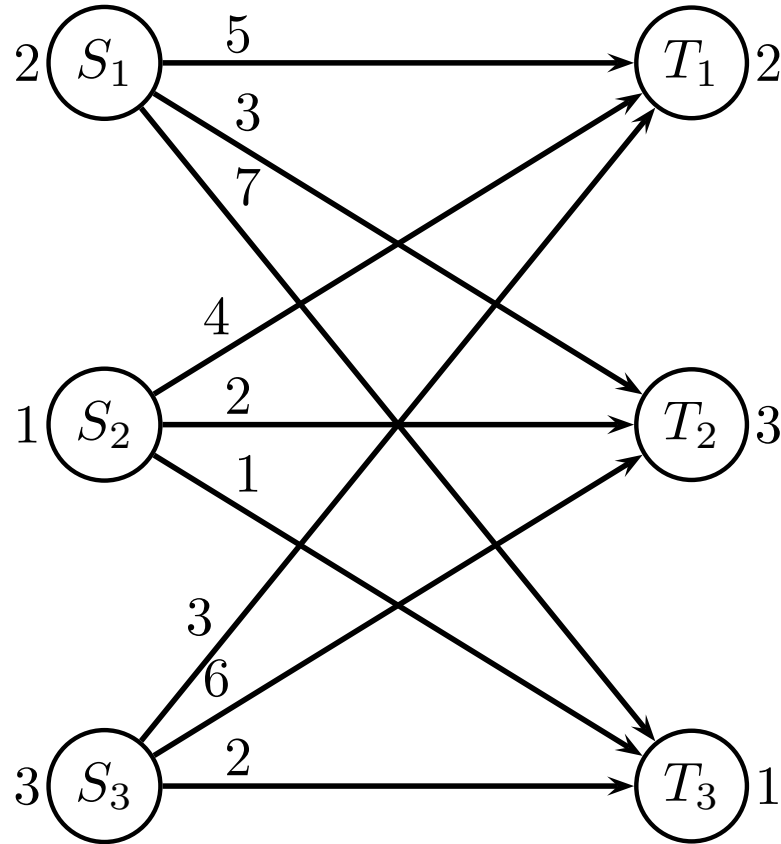
0				$c_N - y A_N$	$-y b$
1	0	...	0	$A_B^{-1} A_N$	$A_B^{-1} b$
0	1		0		
⋮	⋮	⋱	⋮		
0	0	...	1		

Solving large LP problems

- Assume number of columns much larger than number of constraints
- Simplex algorithm only needs basis B , columns A_B , reduced costs $c - yA$
- if we can provide this information, simplex algorithm does not need to know the full problem

Fixed charge transportation problem

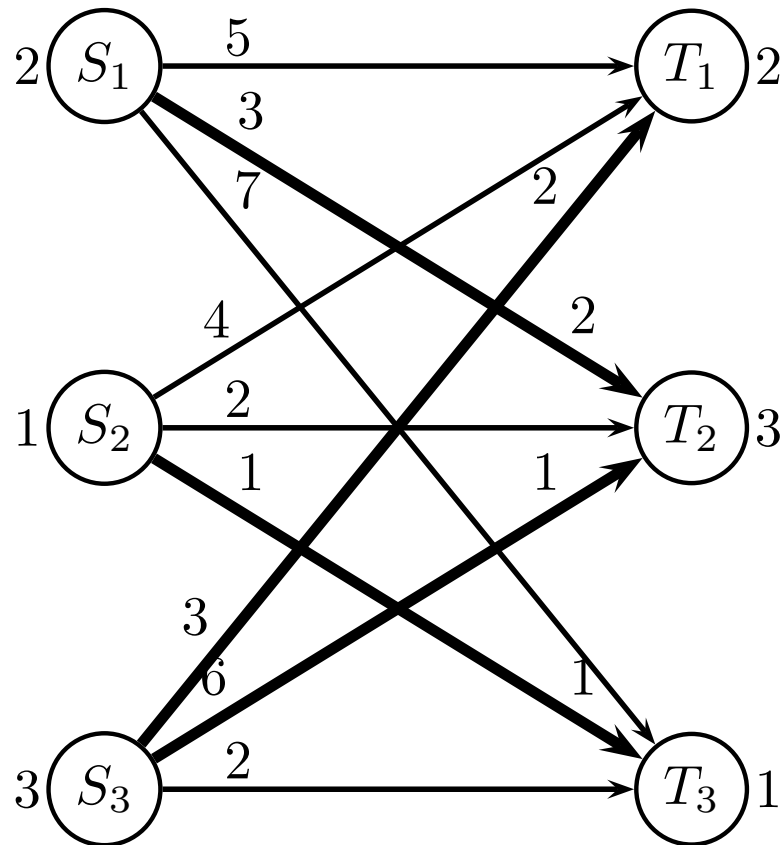
Greedy solution is found as last week (ignoring fixed charge)



(Normally fixed charge for using edge (i, j) is $F = 8$)

Fixed charge transportation problem

Greedy solution



Corresponds to patterns p_{12} , p_{23} and p_{32} .

Restricted master problem

Iteration 0: Master problem

```
minimize
    14p12+ 9p23+28p32
subject to
U1: 0p12+ 0p23+ 2p32 = 2
U2: 2p12+ 0p23+ 1p32 = 3
U3: 0p12+ 1p23+ 0p32 = 1
V1: 1p12                = 1
V2:      + 1p23          = 1
V3:      + 1p32          = 1
end
```

Solution: $z = 51$

Dual: $u_1 = 10.5, u_2 = 7, u_3 = 9$ and $v_1 = 0, v_2 = 0, v_3 = 0$

Reduced cost of a column

$$\bar{c} = c - yA$$

```

minimize
18p11+14p12+24p13+28p14+26p15+12p21+10p22+ 9p23+26p31+28p32+24p33+31p34+30p35+35p36
subject to
  2p11+ 0p12+ 1p13+ 1p14+ 0p15+ 1p21+ 0p22+ 0p23+ 0p31+ 2p32+ 2p33+ 1p34+ 0p35+ 1p36 = 2   (u1)
  0p11+ 2p12+ 1p13+ 0p14+ 1p15+ 0p21+ 1p22+ 0p23+ 3p31+ 1p32+ 0p33+ 2p34+ 2p35+ 1p36 = 3   (u2)
  0p11+ 0p12+ 0p13+ 1p14+ 1p15+ 0p21+ 0p22+ 1p23+ 0p31+ 0p32+ 1p33+ 0p34+ 1p35+ 1p36 = 1   (u3)
  1p11+ 1p12+ 1p13+ 1p14+ 1p15                                     = 1   (v1)
                                   + 1p21+ 1p22+ 1p23                                     = 1   (v2)
                                   + 1p31+ 1p32+ 1p33+ 1p34+ 1p35+ 1p36 = 1   (v3)
end
  
```

in our case

$$\bar{c}_{ij} = \left(\sum_j c_{ij} x_j + \sum_j F y_j \right) - \sum_j u_j x_j - v_i$$

Pricing problem

For each S_i we want the best splitting of the s_i items.
Let x_j denote how many items are sent from i to j .

$$\begin{array}{ll}\min & \left(\sum_j c_{ij} x_j + \sum_j F y_j \right) - \sum_j u_j x_j - v_i \\ \text{s.t.} & \sum_j x_j = s_i \\ & x_j \leq M y_j \\ & x_j \geq 0, \text{ integer} \\ & y_j \in \{0, 1\}\end{array}$$

The problem is a **multiple-choice knapsack problem**,
solvable with dynamic programming

Solution x will define column in A matrix

Pricing problem S_1

For S_1 we get the model

```
minimize
    5 x1 + 3 x2 + 7 x3          (c_ij x_j)
    + 8 y1 + 8 y2 + 8 y3       (      F y_j)
    - 10.5 x1 - 7 x2 - 9 x3     ( u_j x_j)
    - 0                         ( v_i      )
subject to
    x1 + x2 + x3 = 2
    x1 <= 3 y1
    x2 <= 3 y2
    x3 <= 3 y3
binary
    y1 y2 y3
end
```

Solution:

$x_1 = 2, x_2 = 0, x_3 = 0$ with $z = -3$

Can be recognized as p_{11} .

Notice: for fixed y , constraint matrix is TU, so x integer

Pricing problem S_2

For S_2 we get the model

```
minimize
    4 x1 + 2 x2 + 1 x3
    + 8 y1 + 8 y2 + 8 y3
    - 10.5 x1 - 7 x2 - 9 x3
    - 0
subject to
    x1 + x2 + x3 = 1
    x1 <= 3 y1
    x2 <= 3 y2
    x3 <= 3 y3
binary
    y1 y2 y3
end
```

Solution:

$x_1 = 0, x_2 = 0, x_3 = 1$ with $z = 0$

Can be recognized as p_{23} .

Pricing problem S_3

For S_3 we get the model

```
minimize
    3 x1 + 6 x2 + 2 x3
    + 8 y1 + 8 y2 + 8 y3
    - 10.5 x1 - 7 x2 - 9 x3
    - 0
subject to
    x1 + x2 + x3 = 3
    x1 <= 3 y1
    x2 <= 3 y2
    x3 <= 3 y3
binary
    y1 y2 y3
end
```

Solution:

$x_1 = 2, x_2 = 0, x_3 = 1$ with $z = -6$

Can be recognized as p_{33} .

Pricing problem if columns known, S_1

2 items sent from S_1

	pattern x_j	$\sum_j c_{1j}x_j + \sum_j Fy_j$	$\sum_j u_jx_j$	v_1	red.cost
p_{11}	2 0 0	18	21	0	-3
p_{12}	0 2 0	14	14	0	0
p_{13}	1 1 0	24	17.5	0	6.5
p_{14}	1 0 1	28	20.5	0	7.5
p_{15}	0 1 1	26	16	0	10

$$u_1 = 10.5, u_2 = 7, u_3 = 9 \text{ and } v_1 = 0, v_2 = 0, v_3 = 0$$

● Notice that $\sum_j c_{1j}x_j + \sum_j Fy_j$ is “old” cost of pattern

Pricing problem if columns known, S_2

1 item sent from S_2

	pattern x_j	$\sum_j c_{2j}x_j + \sum_j Fy_j$	$\sum_j u_jx_j$	v_2	red.cost
p_{21}	1 0 0	12	10.5	0	1.5
p_{22}	0 1 0	10	7	0	3
p_{23}	0 0 1	9	9	0	0

$$u_1 = 10.5, u_2 = 7, u_3 = 9 \text{ and } v_1 = 0, v_2 = 0, v_3 = 0$$

Pricing problem if columns known, S_3

3 items sent from S_3

	pattern x_j	$\sum_j c_{3j}x_j + \sum_j Fy_j$	$\sum_j u_jx_j$	v_3	red.cost
p_{31}	0 3 0	26	21	0	5
p_{32}	2 1 0	28	28	0	0
p_{33}	2 0 1	24	30	0	-6
p_{34}	1 2 0	31	24.5	0	6.5
p_{35}	0 2 1	30	23	0	7
p_{36}	1 1 1	35	26.5	0	8.5

$$u_1 = 10.5, u_2 = 7, u_3 = 9 \text{ and } v_1 = 0, v_2 = 0, v_3 = 0$$

Restricted master problem

Iteration 1: Adding pattern p_{33} to master problem

```
minimize
    14p12+ 9p23+28p32+24p33
subject to
U1: 0p12+ 0p23+ 2p32+ 2p33 = 2
U2: 2p12+ 0p23+ 1p32+ 0p33 = 3
U3: 0p12+ 1p23+ 0p32+ 1p33 = 1
V1: 1p12                      = 1
V2:      + 1p23                = 1
V3:      + 1p32+ 1p33          = 1
end
```

Solution: $z = 51$

Dual: $u_1 = 14, u_2 = 0, u_3 = -4$ and $v_1 = 14, v_2 = 13, v_3 = 0$

Pricing problem S_1

For S_1 we get the model

```
minimize
    5 x1 + 3 x2 + 7 x3
    + 8 y1 + 8 y2 + 8 y3
    - 14 x1 - 0 x2 + 4 x3
    - 14
subject to
    x1 + x2 + x3 = 2
    x1 <= 3 y1
    x2 <= 3 y2
    x3 <= 3 y3
binary
    y1 y2 y3
end
```

Solution:

$x_1 = 2, x_2 = 0, x_3 = 0$ with $z = -24$

Can be recognized as p_{11} .

Pricing problem S_2

For S_2 we get the model

```
minimize
    4 x1 + 2 x2 + 1 x3
    + 8 y1 + 8 y2 + 8 y3
    - 14 x1 - 0 x2 + 4 x3
    - 13
subject to
    x1 + x2 + x3 = 1
    x1 <= 3 y1
    x2 <= 3 y2
    x3 <= 3 y3
binary
    y1 y2 y3
end
```

Solution:

$x_1 = 1, x_2 = 0, x_3 = 0$ with $z = -15$

Can be recognized as p_{21} .

Pricing problem S_3

For S_3 we get the model

```
minimize
    3 x1 + 6 x2 + 2 x3
    + 8 y1 + 8 y2 + 8 y3
    - 14 x1 - 0 x2 + 4 x3
    - 0
subject to
    x1 + x2 + x3 = 3
    x1 <= 3 y1
    x2 <= 3 y2
    x3 <= 3 y3
binary
    y1 y2 y3
end
```

Solution:

$x_1 = 2, x_2 = 0, x_3 = 1$ with $z = 0$

Can be recognized as p_{33} .

Restricted master problem

Iteration 2: Adding pattern p_{11} to master problem

```
minimize
    18p11+14p12+ 9p23+28p32+24p33
subject to
U1: 2p11+ 0p12+ 0p23+ 2p32+ 2p33 = 2
U2: 0p11+ 2p12+ 0p23+ 1p32+ 0p33 = 3
U3: 0p11+ 0p12+ 1p23+ 0p32+ 1p33 = 1
V1: 1p11+ 1p12                        = 1
V2:                + 1p23                = 1
V3:                + 1p32+ 1p33 = 1
end
```

Solution: $z = 51$

Dual: $u_1 = 0, u_2 = 7, u_3 = 0$ and $v_1 = 0, v_2 = 9, v_3 = 21$

Pricing problem, S_1, S_2, S_3

Iteration 2: Pricing problem

For S_1 : $z \geq 0$

For S_2 : $x_1 = 0, x_2 = 1, x_3 = 0$ with $z = -6$ (pattern p_{22})

For S_3 : $x_1 = 2, x_2 = 0, x_3 = 1$ with $z = -16$ (pattern p_{31})

Restricted master problem

Iteration 3: Adding pattern p_{31} to master problem

```
minimize
    18p11+14p12+ 9p23+26p31+28p32+24p33
subject to
U1: 2p11+ 0p12+ 0p23+ 0p31+ 2p32+ 2p33 = 2
U2: 0p11+ 2p12+ 0p23+ 3p31+ 1p32+ 0p33 = 3
U3: 0p11+ 0p12+ 1p23+ 0p31+ 0p32+ 1p33 = 1
V1: 1p11+ 1p12                                = 1
V2:                + 1p23                        = 1
V3:                + 1p31+ 1p32+ 1p33 = 1
end
```

Solution: $z = 51$

Dual: $u_1 = 1, u_2 = 0, u_3 = -4$ and $v_1 = 14, v_2 = 13, v_3 = 26$

Pricing problem, S_1, S_2, S_3

Iteration 3: Pricing problem

For S_1 : $z \geq 0$

For S_2 : $x_1 = 0, x_2 = 1, x_3 = 0$ with $z = -3$ (pattern p_{22})

For S_3 : $z \geq 0$

Restricted master problem

Iteration 4: Adding pattern p_{22} to master problem

```
minimize
    18p11+14p12+10p22+ 9p23+26p31+28p32+24p33
subject to
U1: 2p11+ 0p12+ 0p22+ 0p23+ 0p31+ 2p32+ 2p33 = 2
U2: 0p11+ 2p12+ 1p22+ 0p23+ 3p31+ 1p32+ 0p33 = 3
U3: 0p11+ 0p12+ 0p22+ 1p23+ 0p31+ 0p32+ 1p33 = 1
V1: 1p11+ 1p12                                     = 1
V2:           + 1p22+ 1p23                             = 1
V3:           + 1p31+ 1p32+ 1p33 = 1
end
```

Solution: $z = 48, p_{12} = p_{22} = p_{33} = 1$

Dual: $u_1 = 8.16, u_2 = 8.66, u_3 = 7.66$ and
 $v_1 = -3.33, v_2 = 1.33, v_3 = 0$

Pricing problem, S_1, S_2, S_3

Iteration 4: Pricing problem

For S_1 : $z \geq 0$

For S_2 : $z \geq 0$

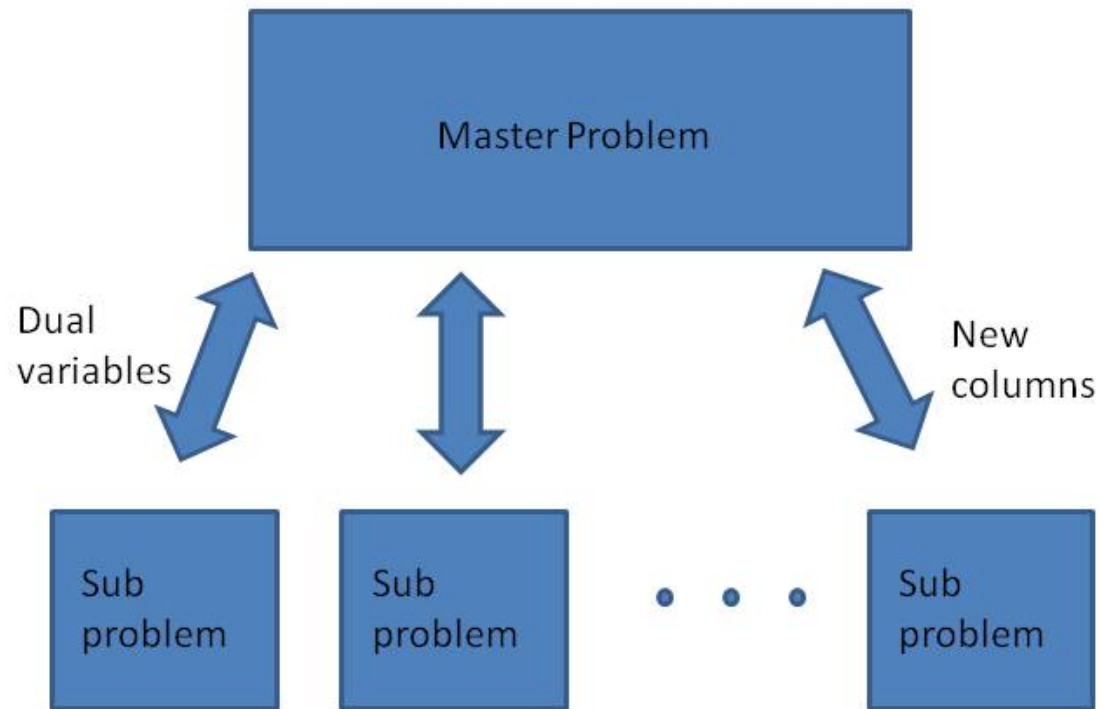
For S_3 : $z \geq 0$

Reduced costs are positive, so **stop**

Fixed charge transportation problem

- Only 4 iterations (7 columns out of 14 generated)
- Much better lower bounds than simple formulation
- Pricing problem quite easy to solve
- Objective function z moves in large “jumps”
- Using “ $\geq$ ” constraints in master may give more smooth development of objective
($z = 51, 51, 51, 49.7, 48$, paths added $p_{21}, p_{33}, p_{31}, p_{22}$)
Dual variables are restricted to ≥ 0 , so cannot fluctuate so much (deep dual inequalities)

Column generation



Column generation

- Find initial columns
- Repeat
 - Solve restricted master problem (LP-problem)
 - Find dual prices
 - Solve pricing problem(s)
 - Add new column(s)
- Until no columns with negative reduced cost

This solves the LP-problem to optimality
(but often IP problem also needs to be solved)

Tips-and-tricks

Column generation for fixed-charge transport

- check that problem is balanced, otherwise add surplus node to make problem balanced
- if some edges are missing, you can add them with big cost M . This will ensure that you can find a greedy solution
- You need to use an LP-solver to find dual variables. You cannot use the method from last week (it is a different LP-model)
- if you generate the same column twice, then there is something wrong
- avoid upper bounds on variables $0 \leq x_j \leq 1$ since they have an associated dual variable

Appendix: Pricing problem solved fast

Consider node S_i with production s_i (or just s)

Let x_{jk} indicate if we send k units to node T_j

multiple choice knapsack, capacity s

$$\min \left\{ \sum_{j=1}^n \sum_{k=1}^s c_{jk} x_{jk} \mid \sum_{j=1}^n \sum_{k=1}^s k \cdot x_{jk} = s \right\}$$

Subproblem for $0 \leq a \leq s$

$$f_i(a) = \min \left\{ \sum_{j=1}^i \sum_{k=1}^a c_{jk} x_{jk} \mid \sum_{j=1}^i \sum_{k=1}^a k \cdot x_{jk} = a \right\}$$

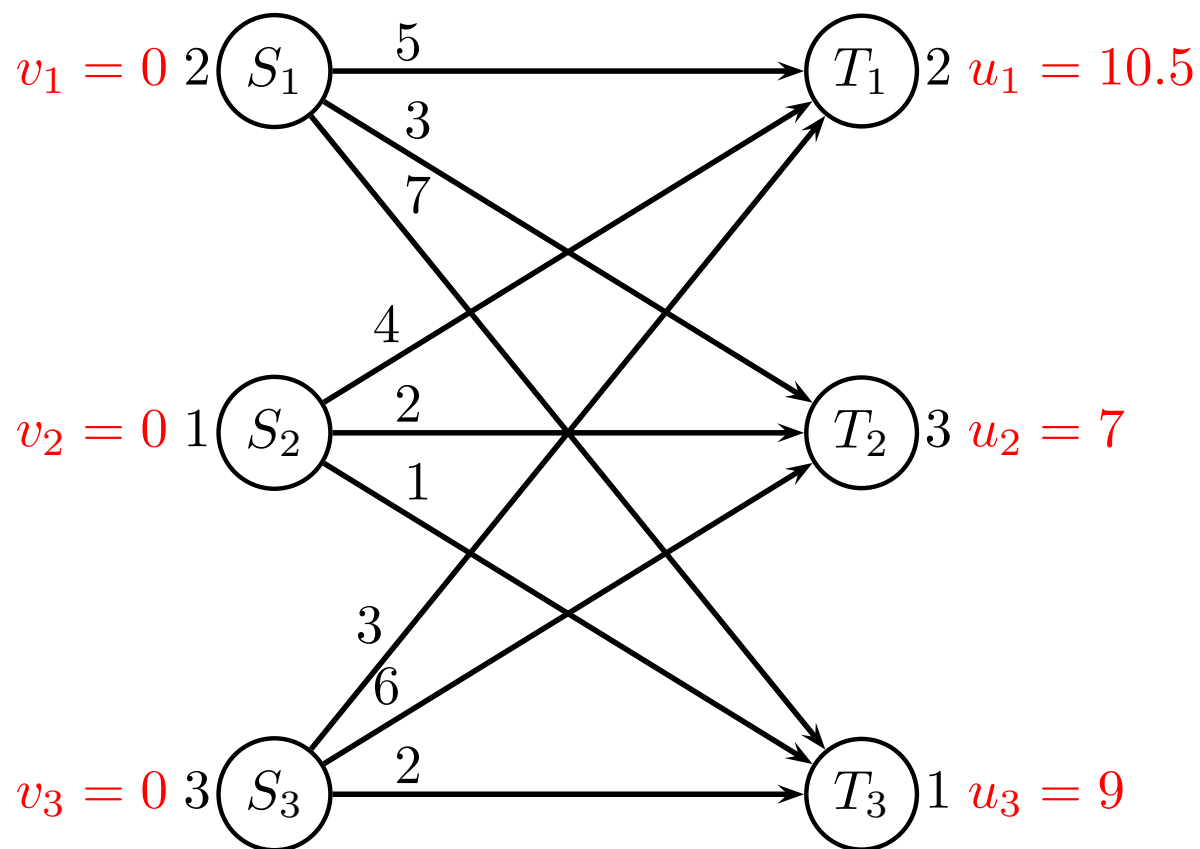
Recursion

$$f_i(a) = \min_{k=0, \dots, a} \{ f_{i-1}(a - k) + c_{ik} \}$$

Start condition

$$f_1(a) = c_{1a} \quad \forall a$$

Pricing problem, multiple-choice knapsack



For node S_1 we have pairs (c_{jk}, k) for $k = 0, 1, 2$

$$T_1 \begin{pmatrix} 0 + 0 & 0 \\ -5.5 + 8 & 1 \\ -11 + 8 & 2 \end{pmatrix} \quad T_2 \begin{pmatrix} 0 + 0 & 0 \\ -4 + 8 & 1 \\ -8 + 8 & 2 \end{pmatrix} \quad T_3 \begin{pmatrix} 0 + 0 & 0 \\ -2 + 8 & 1 \\ \infty & 2 \end{pmatrix}$$

Pricing problem, multiple-choice knapsack

For node S_1 we have costs c_{jk}

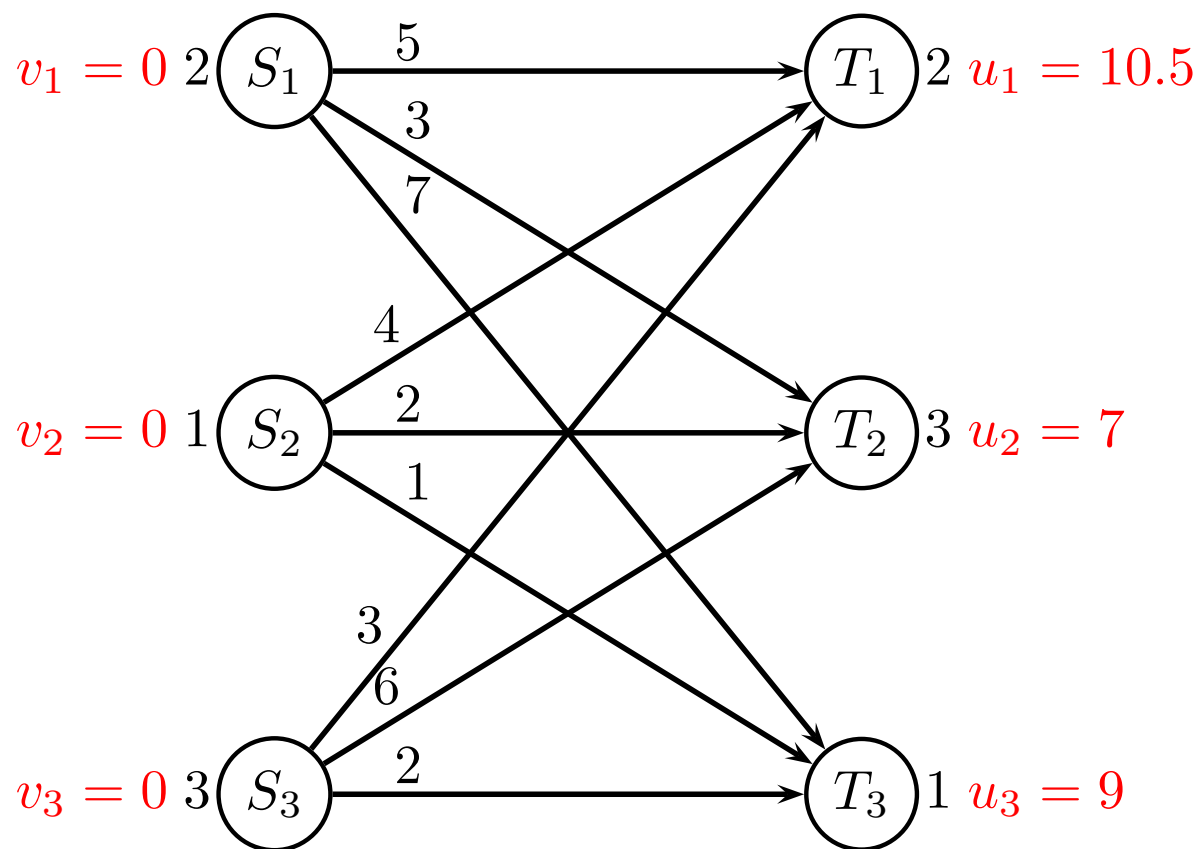
$k \backslash j$	1	2	3
0	0	0	0
1	2.5	4	6
2	-3	0	∞

Solving dynamic programming

$a \backslash i$	1	2	3
0	0	0	0
1	2.5	2.5	2.5
2	-3	-3	-3

Optimal solution $z = -3$ pattern 2,0,0

Pricing problem, multiple-choice knapsack



For node S_2 we have pairs (c_{jk}, k) for $k = 0, 1$

$$T_1 \begin{pmatrix} 0 + 0 & 0 \\ -6.5 + 8 & 1 \end{pmatrix} \quad T_2 \begin{pmatrix} 0 + 0 & 0 \\ -5 + 8 & 1 \end{pmatrix} \quad T_3 \begin{pmatrix} 0 + 0 & 0 \\ -8 + 8 & 1 \end{pmatrix}$$

Pricing problem, multiple-choice knapsack

For node S_2 we have costs c_{jk}

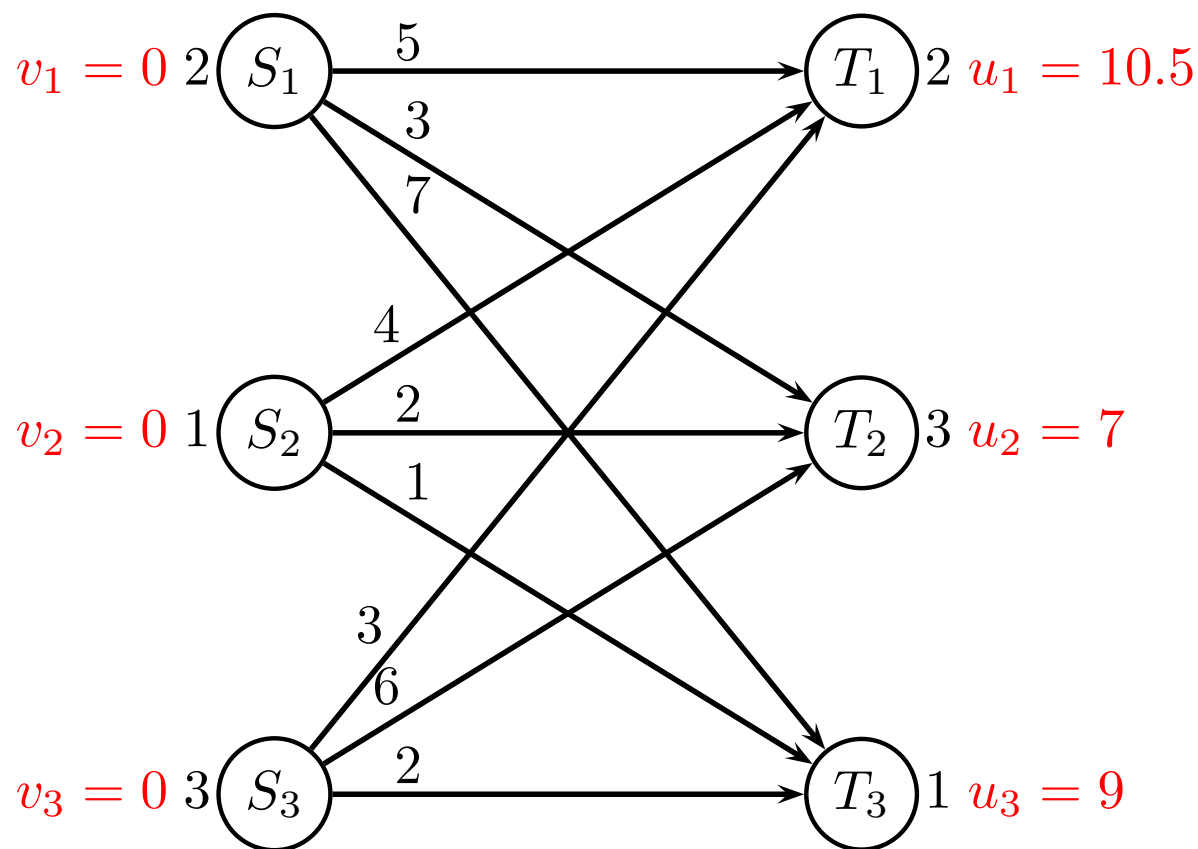
$k \backslash j$	1	2	3
0	0	0	0
1	1.5	3	0

Solving dynamic programming

$a \backslash i$	1	2	3
0	0	0	0
1	1.5	1.5	0

Optimal solution $z = 0$ pattern 0,0,1

Pricing problem, multiple-choice knapsack



For node S_3 we have pairs (c_{jk}, k) for $k = 0, 1, 2, 3$

$$T_1 \begin{pmatrix} 0 + 0 & 0 \\ -7.5 + 8 & 1 \\ -15 + 8 & 2 \\ \infty & 3 \end{pmatrix} \quad T_2 \begin{pmatrix} 0 + 0 & 0 \\ -1 + 8 & 1 \\ -2 + 8 & 2 \\ -3 + 8 & 3 \end{pmatrix} \quad T_3 \begin{pmatrix} 0 + 0 & 0 \\ -7 + 8 & 1 \\ \infty & 2 \\ \infty & 3 \end{pmatrix}$$

Pricing problem, multiple-choice knapsack

For node S_3 we have costs c_{jk}

$k \backslash j$	1	2	3
0	0	0	0
1	0.5	7	1
2	-7	6	∞
3	∞	5	∞

Solving dynamic programming

$a \backslash i$	1	2	3
0	0	0	0
1	0.5	0.5	0.5
2	-7	-7	-7
3	∞	0	-6

Optimal solution $z = -6$ pattern 2,0,1

The Shortest Path Problem

Bellman-Ford, Dijkstra, generalizations

David Pisinger

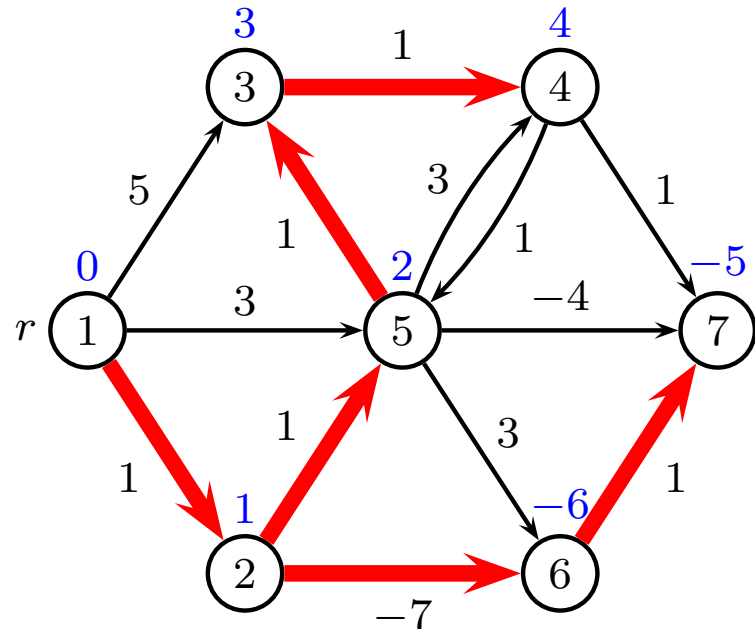
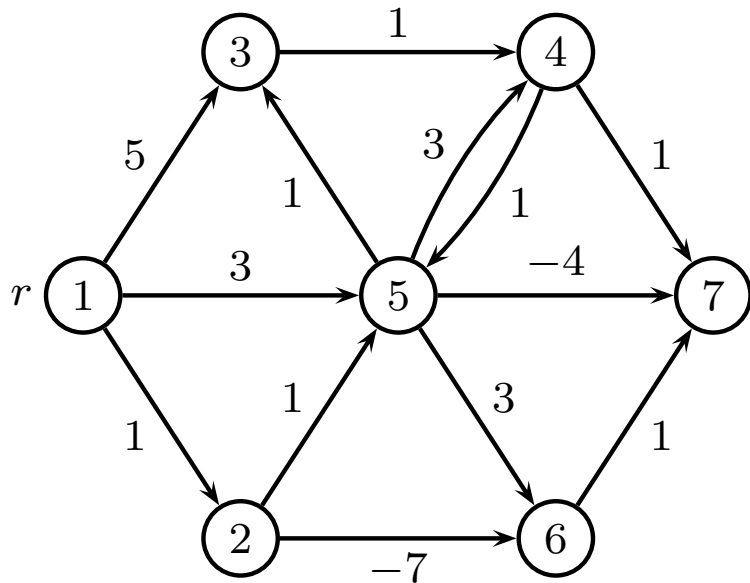
`pisinger@man.dtu.dk`

DTU Management
Technical University of Denmark

© Copyright David Pisinger. May not be distributed

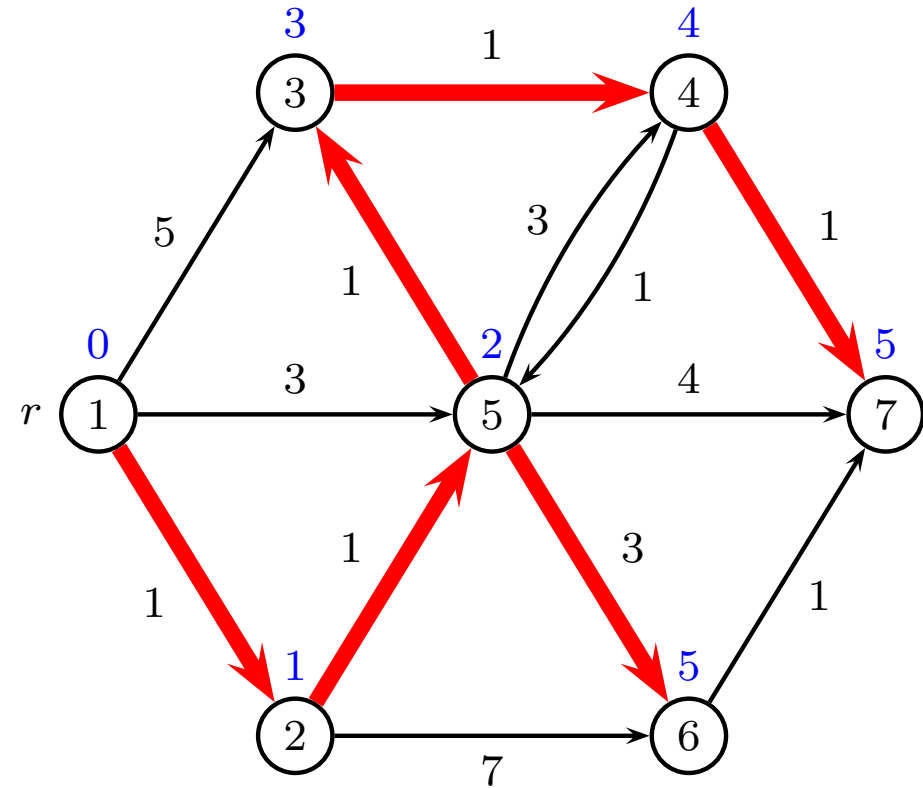
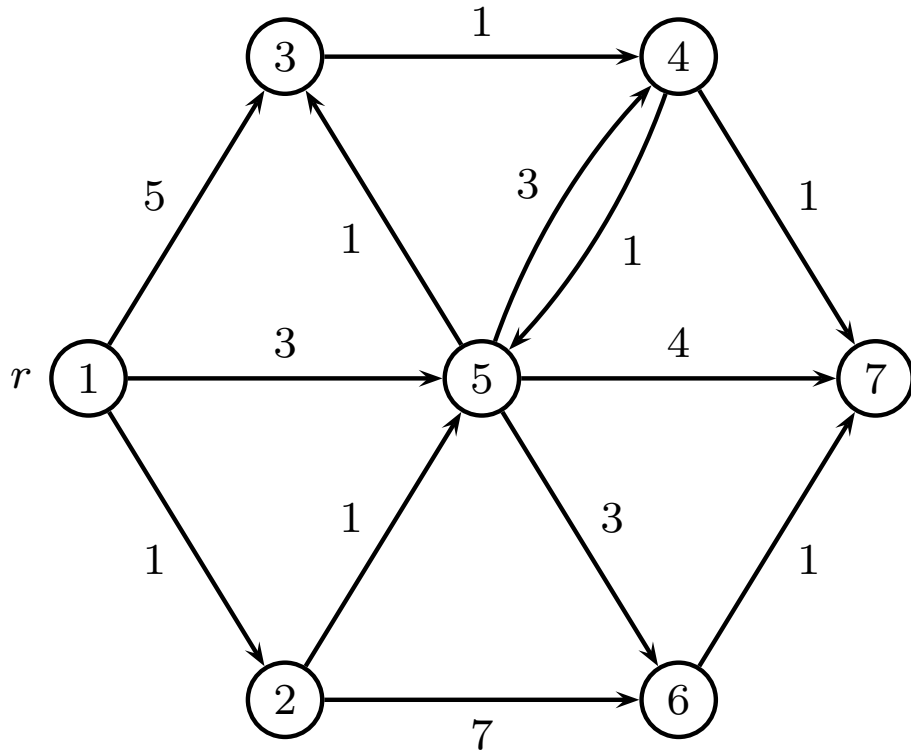
The Shortest Path Problem

- Given a **directed network** $\mathcal{G} = (V, E, w)$ where $w \in \mathbb{R}$ (n nodes, m edges)
- Furthermore, a **source vertex** r is given.
- Objective:** Find for each $v \in V$ a shortest (with respect to w) directed path from r to v (if such one exists).
- No negative cycles** reachable from r



Example

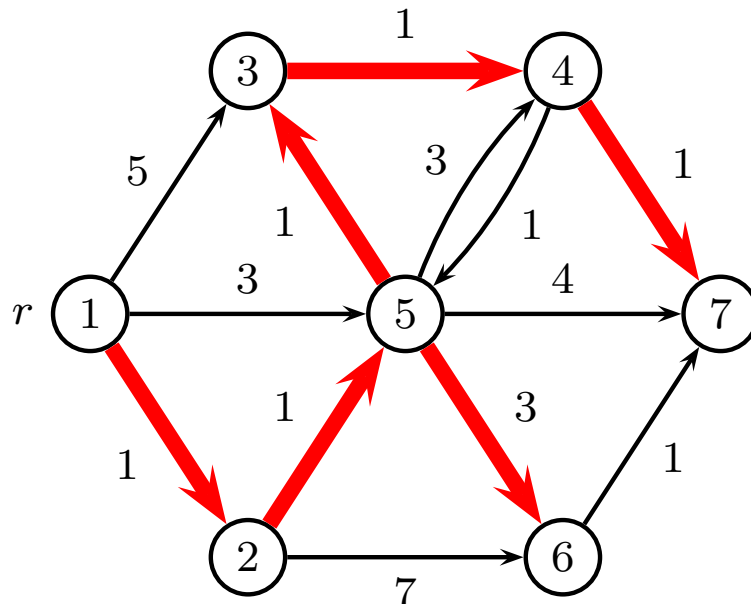
(another example with positive edges)



Shortest path tree rooted at r

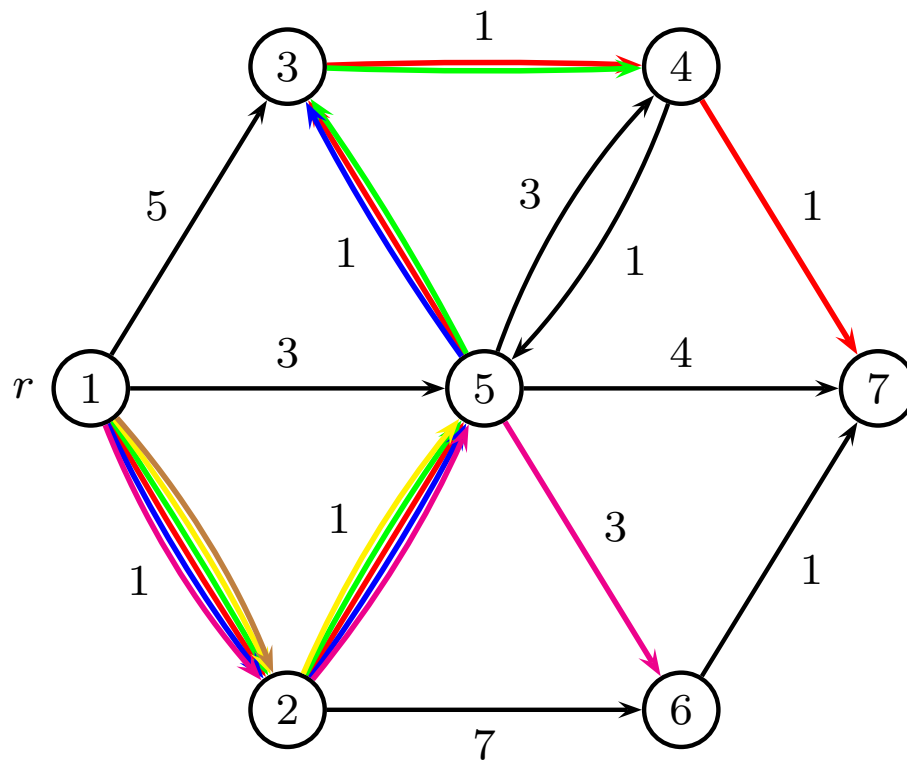
Integer Programming Formulation

- Tree rooted at r . Look at the **number** of paths leaving a vertex vs. the **number** of paths entering a vertex.
 - Vertex r must have $n - 1$ paths leaving the vertex.
 - For any other vertex, the number of paths entering the vertex must be exactly 1 larger than the number of paths leaving the vertex.
- Let x_e denote the **number** of paths using edge $e \in E$.



Example

x_e denotes number of paths using edge e



Mathematical Model

Shortest path from r to all nodes v

IP model:

$$\begin{aligned} \min \quad & \sum_{(i,j) \in E} w_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{i \in V} x_{ri} - \sum_{i \in V} x_{ir} = n - 1 \\ & \sum_{i \in V} x_{ji} - \sum_{i \in V} x_{ij} = -1 \quad j \neq r \\ & x_{ij} \in \mathbb{Z}_+ \quad (i,j) \in E \end{aligned}$$

LP-problem gives IP-solution, since matrix is TU

Can be solved with CPLEX.

Will show more efficient algorithm

Mathematical model for $r \rightarrow t$

Shortest path from r to t :

$$\begin{aligned} \min \quad & \sum_{(i,j) \in E} w_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{i \in V} x_{ri} - \sum_{i \in V} x_{ir} = 1 \\ & \sum_{i \in V} x_{ti} - \sum_{i \in V} x_{it} = -1 \\ & \sum_{i \in V} x_{ji} - \sum_{i \in V} x_{ij} = 0 \quad j \neq \{r, t\} \\ & x_{ij} \in \{0, 1\} \quad (i, j) \in E \end{aligned}$$

LP-problem gives IP-solution, since matrix is TU

Dual problem for $r \rightarrow t$

Objective $\max y_r - y_t$ equivalent to $\min y_t - y_r$

$$\begin{array}{ll}\min & y_t - y_r \\ \text{s.t.} & y_j - y_i \leq w_{ij} \quad (i, j) \in E \\ & y_i \in \mathbb{R} \quad i \in V\end{array}$$

Many equally good solutions, so fix $y_r = 0$

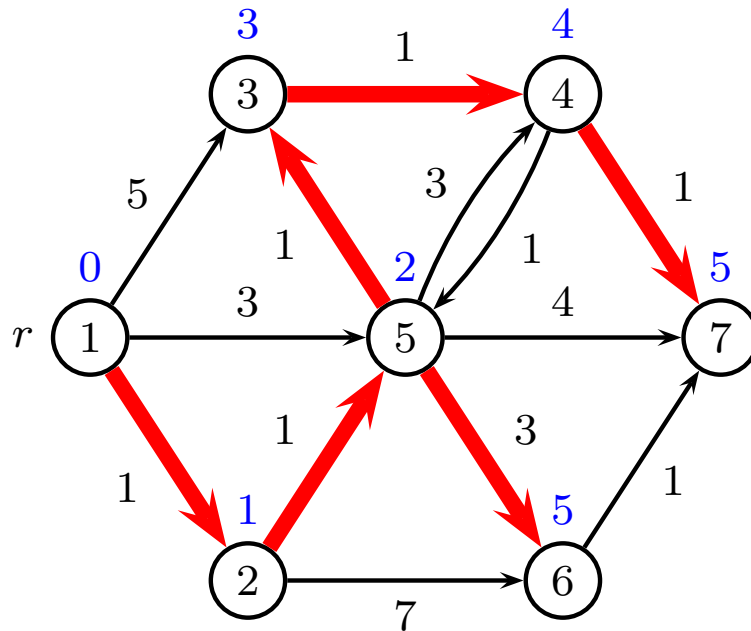
Feasible potentials

- Consider an n -vector $y = y_1, \dots, y_n$ (**potentials**)
- If y satisfies that $y_r = 0$ and

$$\forall (i, j) \in E : y_i + w_{ij} \geq y_j \quad (\text{no shortcuts})$$

then y is called a **feasible** potential.

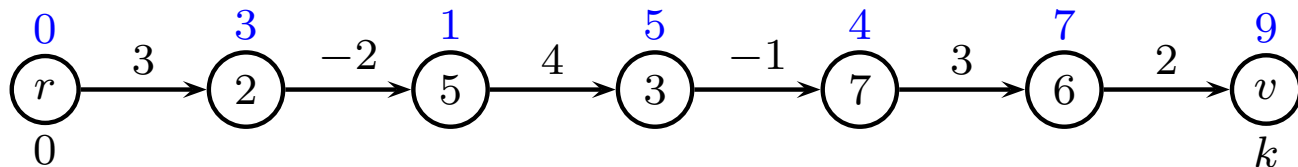
- Informally: potentials y_v are possible distances from r to v . Potentials are feasible when no more shortcuts are possible



Feasible potentials II

- If P is a path from r to $v \in V$, and, if y is a feasible potential, then $w_P \geq y_v$:

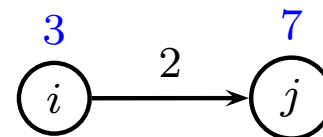
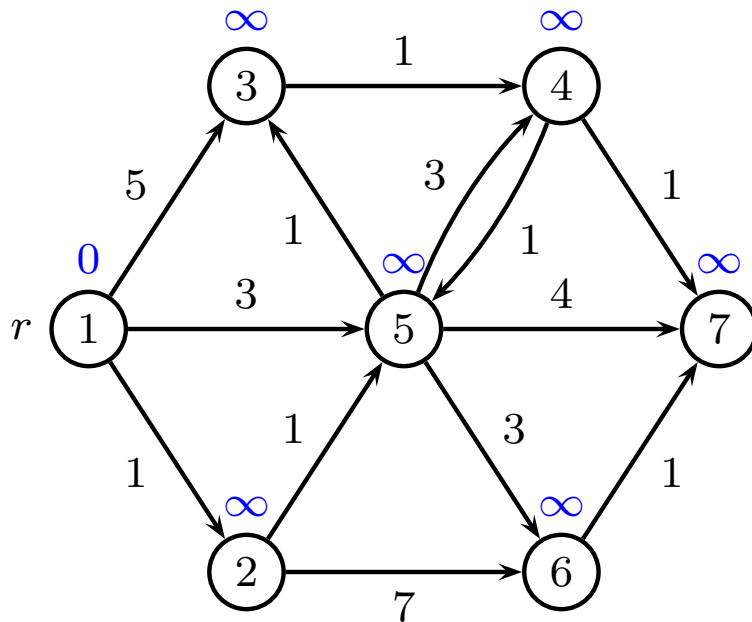
$$\begin{aligned}w_P &= \sum_{i=1}^k w_{e_i} \\&\geq \sum_{i=1}^k (y_{v_i} - y_{v_{i-1}}) \\&= y_{v_k} - y_{v_0} \\&= y_v\end{aligned}$$



- y_v is smaller or equal to length of any path r to v

Basic algorithmic idea

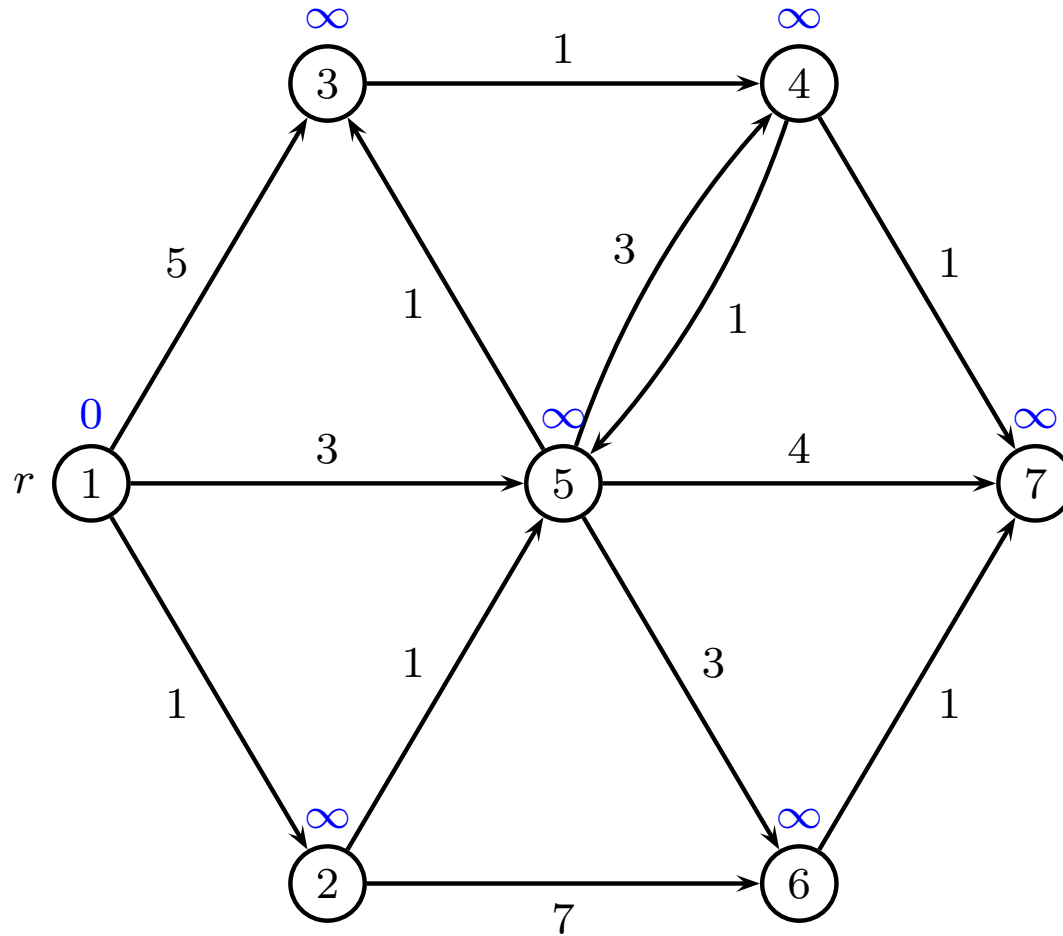
- Start with the potential $y_r = 0$ and $y_i = \infty$, $i \in V \setminus \{r\}$
- Check for each edge if the potential is feasible
- If YES - Stop - the potentials identify shortest paths
- If an edge (i, j) violates feasibility condition, update y_j . This is sometimes called “correct (i, j) ” or “relax (i, j) ”



Bellman-Ford's Shortest Path Algorithm

```
1  $y_r \leftarrow 0; y_i \leftarrow \infty$  for all other  $i$ ;  
2  $p_r \leftarrow 0; p_i \leftarrow \text{NIL}$  for all other  $i$ ;  
3 for  $k = 1$  to  $n - 1$  do  
4   for  $(i, j) \in E$  do  
5     if  $y_i + w_{ij} < y_j$  then                                /* correct  $(i, j)$  */  
6        $y_j \leftarrow y_i + w_{ij};$   
7        $p_j \leftarrow i;$   
8 for  $(i, j) \in E$  do    /* check if negative cycle */  
9   if  $y_i + w_{ij} < y_j$  then  
10   $\text{negative cycle found, follow } p_i \text{ to find the cycle}$ 
```


Example



Complexity of Bellman-Ford's Algorithm

A worst-case time complexity analysis leads to the following conclusions:

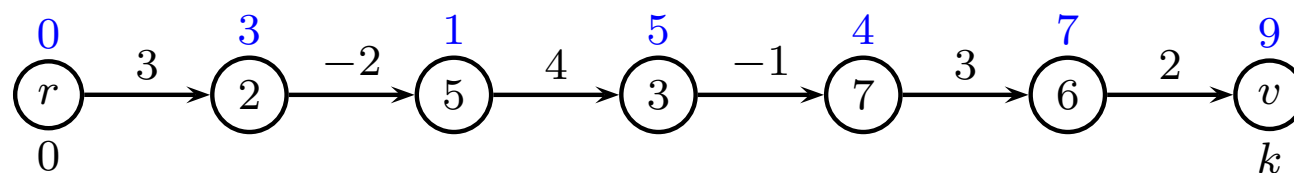
- Initialization: $O(n)$.
- Outer loop: $(n - 1)$ times.
- In the loop: each edge is considered one time: $O(m)$.

In total: $O(nm)$.

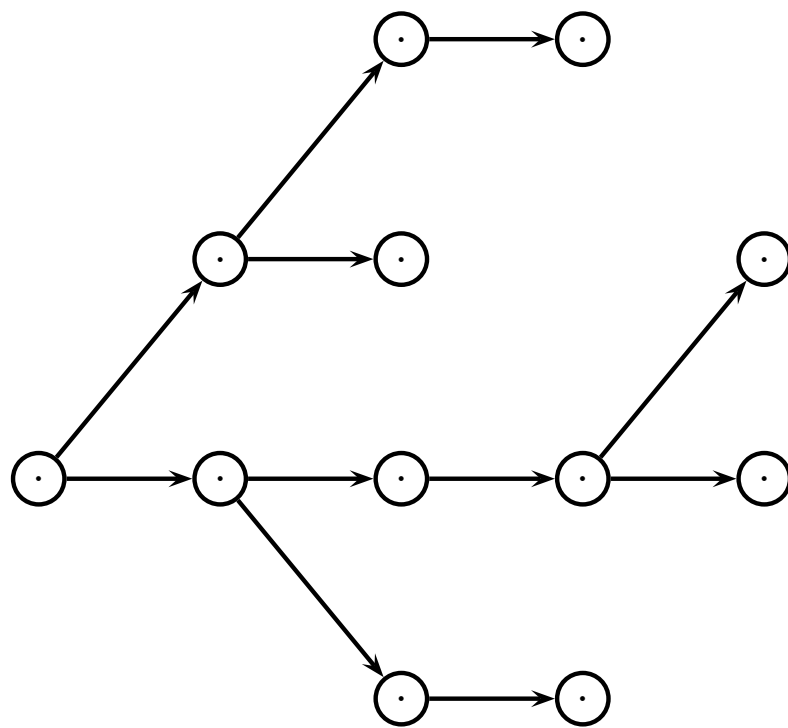
Correctness of Bellman-Ford's Algorithm

Proof by induction

- The induction hypothesis is: After iteration k of the main loop, y_i contains the length of any shortest path with *at most* k edges from 1 to i for any $i \in V$.
- For the **base case** $k = 0$ the induction hypothesis is trivially fulfilled. ($y_r = 0 =$ shortest path from r to r).
- For the **inductive step**, we assume that $y_{v_{k-1}} =$ shortest path from r to v_{k-1} after the $(k - 1)$ 'st pass. Then (v_{k-1}, v_k) is corrected in the k 'th iteration. So $y_{v_k} =$ shortest path from r to v_k .



Correctness of Bellman-Ford's Algorithm



$0 \quad \dots \quad k-1 \quad k$

Correctness of Bellman-Ford's Algorithm

- If **no negative length cycles**, a shortest path containing at most $(n - 1)$ edges exists for each $v \in V$
- A **negative length cycle** can be discovered by an extra iteration in the main loop. If at least one y_i changes, there is a negative length circuit.

Introduction to Dijkstra's Algorithm

If we assume all edge weights $w_e \geq 0$ are non-negative we can derive a more efficient algorithm.

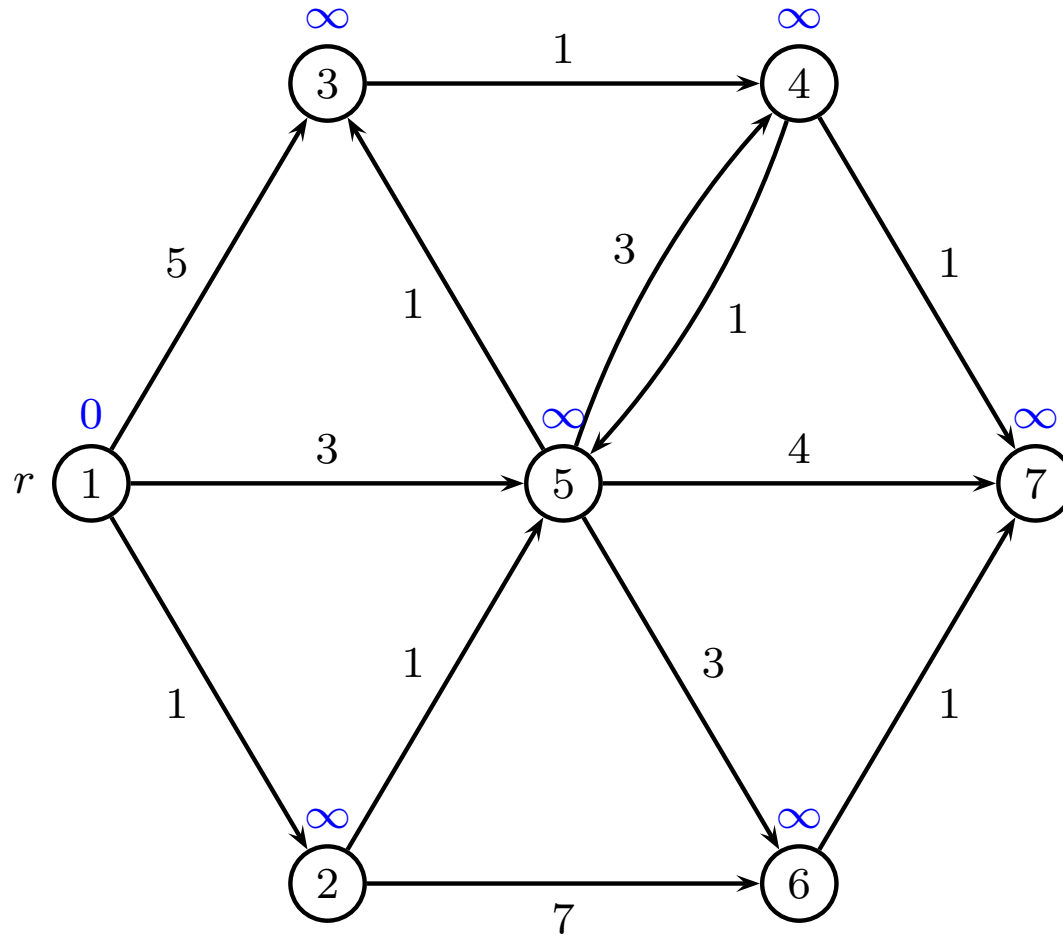
- Only consider each node once
- Add node v to set T when considered
- Always select the node $v \notin T$ with smallest potential
- Correct all outgoing edges from v

Greedy method

Dijkstra's Algorithm for Shortest Path

```
1  $T \leftarrow \emptyset;$ 
2  $y_r \leftarrow 0; y_i \leftarrow \infty$  for all other  $i$ ;
3 while  $V \setminus T \neq \emptyset$  do
4   select an  $i \in V \setminus T$  minimizing  $y_i$ ;
5   for  $\{(i, j) \in E : j \notin T\}$  do
6     if  $y_i + w_{ij} < y_j$  then
7        $y_j \leftarrow y_i + w_{ij};$ 
8        $p_j \leftarrow i;$ 
9    $T \leftarrow T \cup \{i\};$ 
```

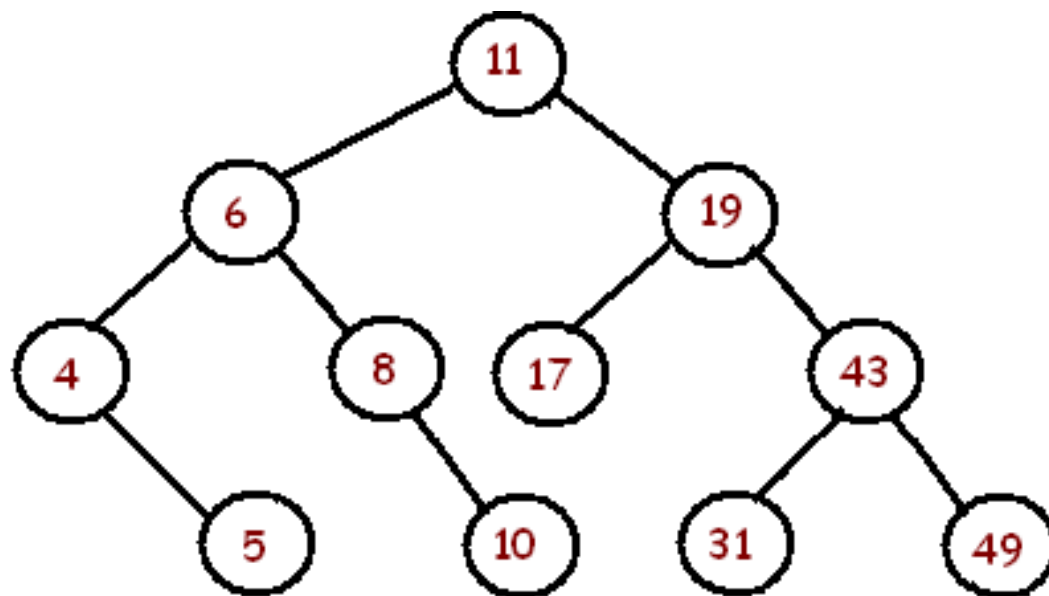
Example



Complexity of Dijkstras Algorithm

- n numbers $\{y_i\}$ in data structure
- n times find minimum in time $O(\log n)$
- m times correct edge (i, j) in time $O(1)$
- m times update y_j by delete/insert in time $O(\log n)$

In total: $O(m \log n)$



Correctness of Dijkstras Algorithm

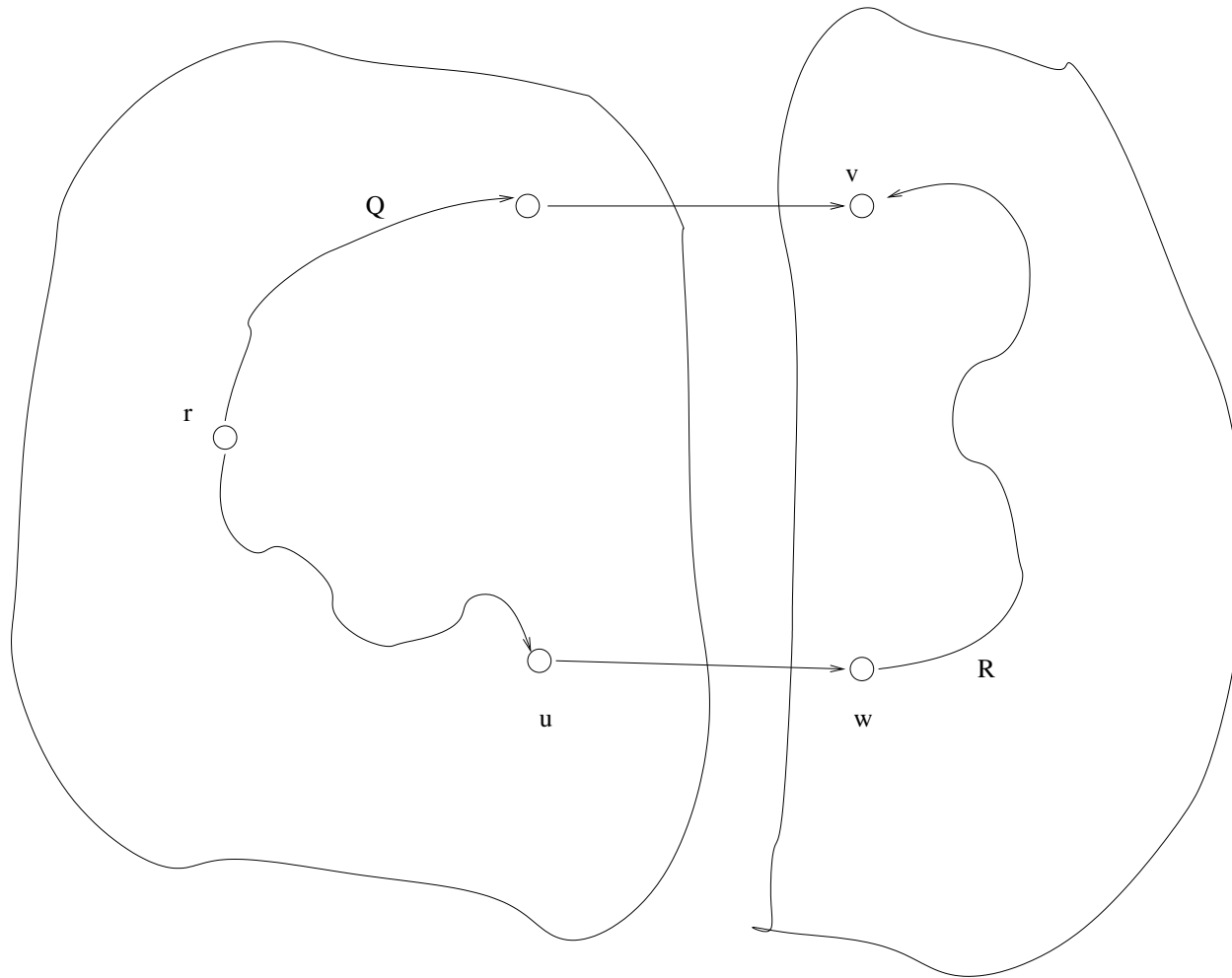
When v is added to T then $y_v = d_v$

- y_j is upper bound on distance to j
- d_j is correct shortest distance to j

Proof by contradiction

- Let v be first node where $y_v \neq d_v$ when v is added to T
- For the root $y_r = d_r = 0$ so $v \neq r$
- Another path R from r to v must be shorter than y_v

Correctness of Dijkstras Algorithm III



Correctness of Dijkstras Algorithm

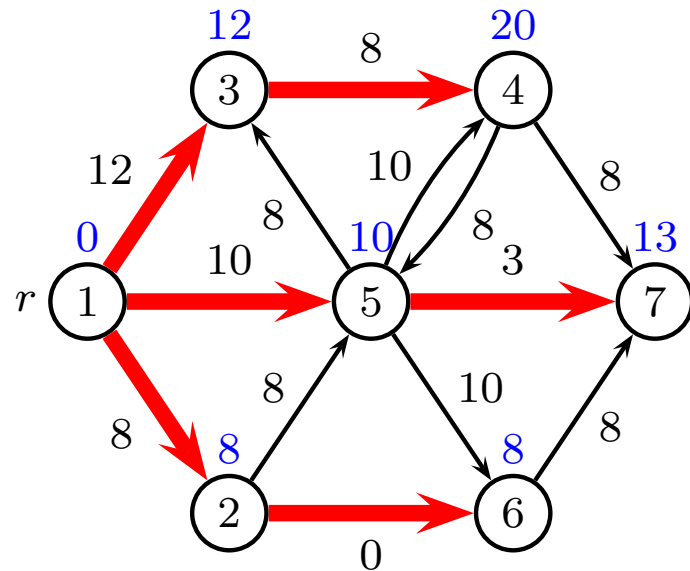
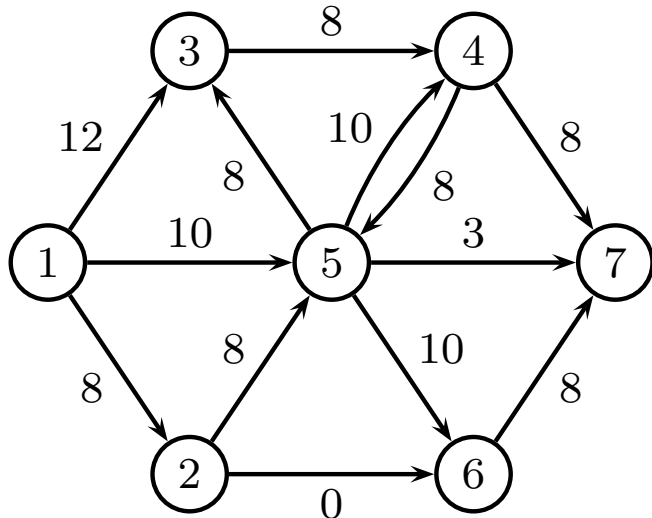
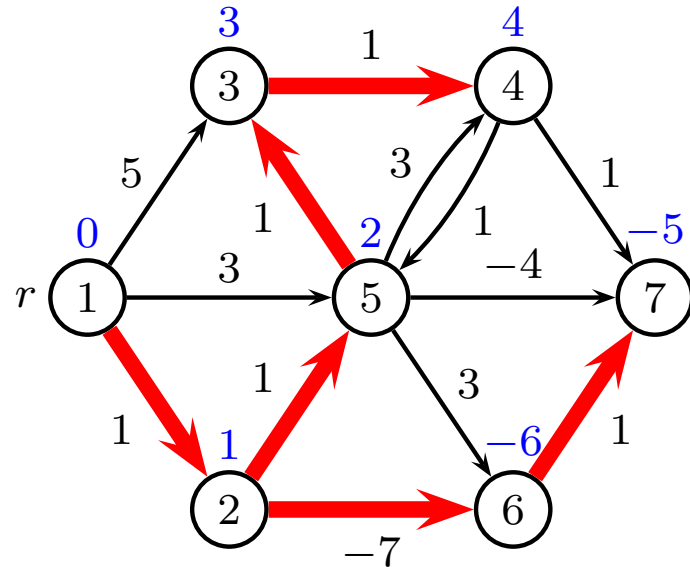
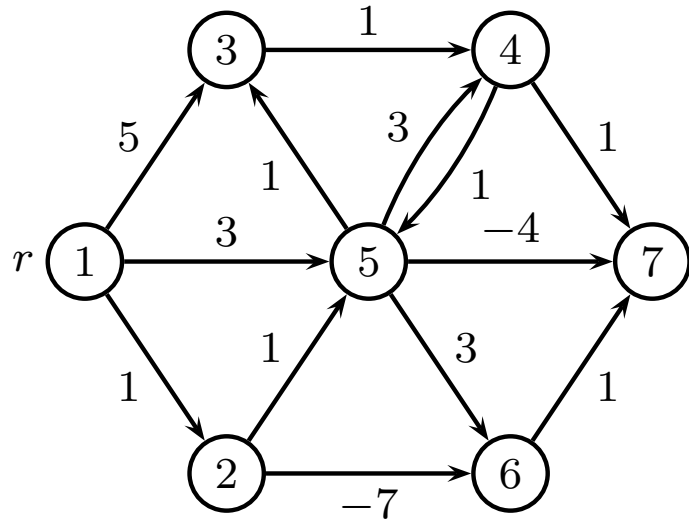
- $y_v > d_v$ since algorithm failed
- $y_v \leq y_w$ since v was selected before w
- $y_u = d_u$ since $u \in T$
- $y_w = d_w$ since when u was considered, edge (u, w) was updated
- $d_v \geq d_w$ since edge weights are positive

Contradiction

$$d_v < y_v \leq y_w = d_w \leq d_v$$

Dijkstra's Algorithm, negative edges ??

Find the most negative edge weight w_m and add $|w_m|$ to all edge weights. Now all $w_e \geq 0$. Does it work?

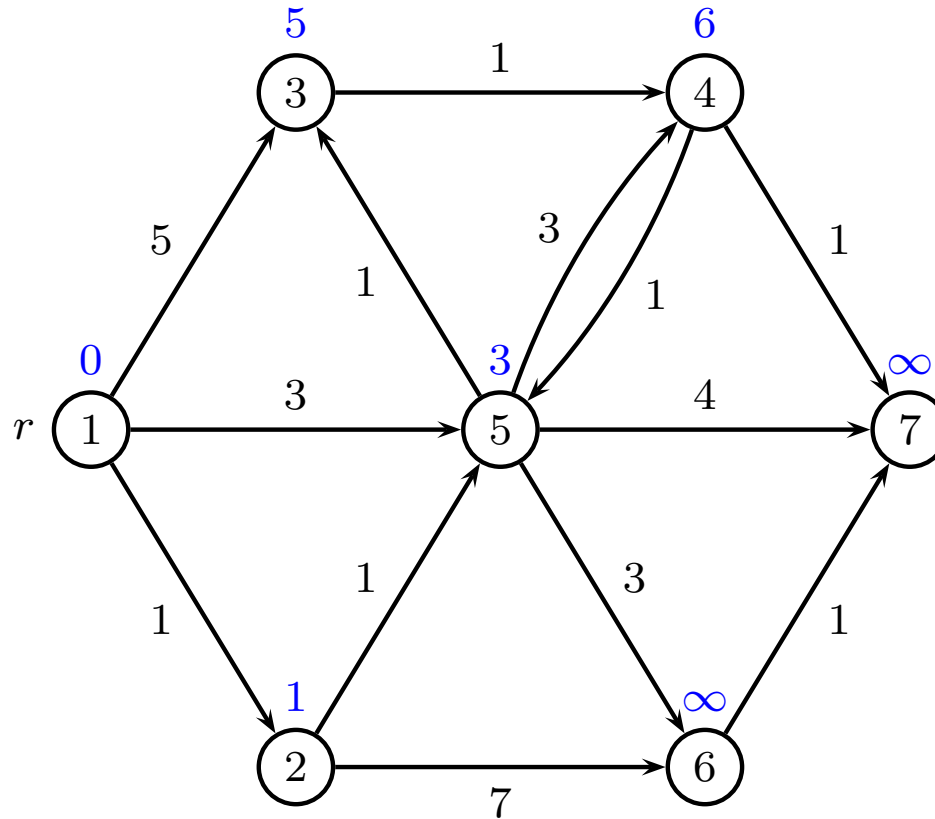


Generic shortest path

```
1  $y_i \leftarrow \infty$  for all  $i \neq r$ ;  
2  $y_r \leftarrow 0$ ;  
3  $Q \leftarrow \{r\}$ ;  
4 while  $Q \neq \emptyset$  do  
5   select a  $i \in Q$ ;  
6   for  $\{(i, j) \in E\}$  do  
7     if  $y_i + w_{ij} < y_j$  then  
8        $y_j \leftarrow y_i + w_{ij}$ ;  
9       add  $j$  to  $Q$  if not already present;
```

Q is called *candidate list* or *queue*

Example



Candidate list: $Q = [2, 5, 4]$

Label setting or label correcting

Label setting methods

- Each node is considered once
- Remove $i \in Q$ with minimum label
- Dijkstra methods
- Differ by which data structure is used to find minimum
- If arc lengths positive, only n iterations

Label correcting methods

- Each node is considered many times
- Queue Q (i.e. insert once, always remove at top)
- Many schemes for inserting $j \in Q$

Bellman-Ford

Label correcting

- Remove $i \in Q$ from top
- Insert new node j at bottom
- At most n^2 iterations

Bellman-Ford methods can be superior because of the smaller overhead per iteration!

For acyclic graphs, Bellman-Ford uses n iterations, same as Dijkstra

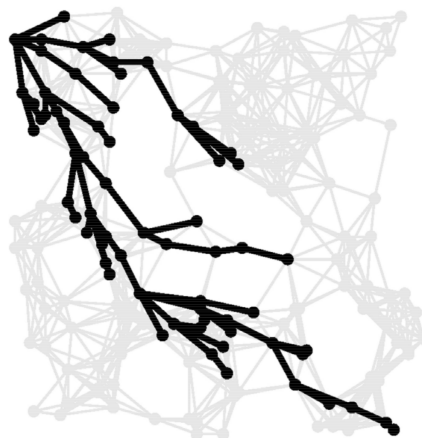
Shortest path Euclidian graph (map)



A^* algorithm finding path from r to t

Shortest path Euclidian graph (map)

A^* **algorithm** finding path from r to t



Assume that h_i is heuristic distance from i to t .
(we can use Euclidean or Manhattan distance).

- Run Dijkstra, but selecting node with lowest $y_i + h_i$
- Stop when t is added to T
- Dijkstra's proof of correctness still holds
- In practice only $O(\sqrt{V})$ vertices examined

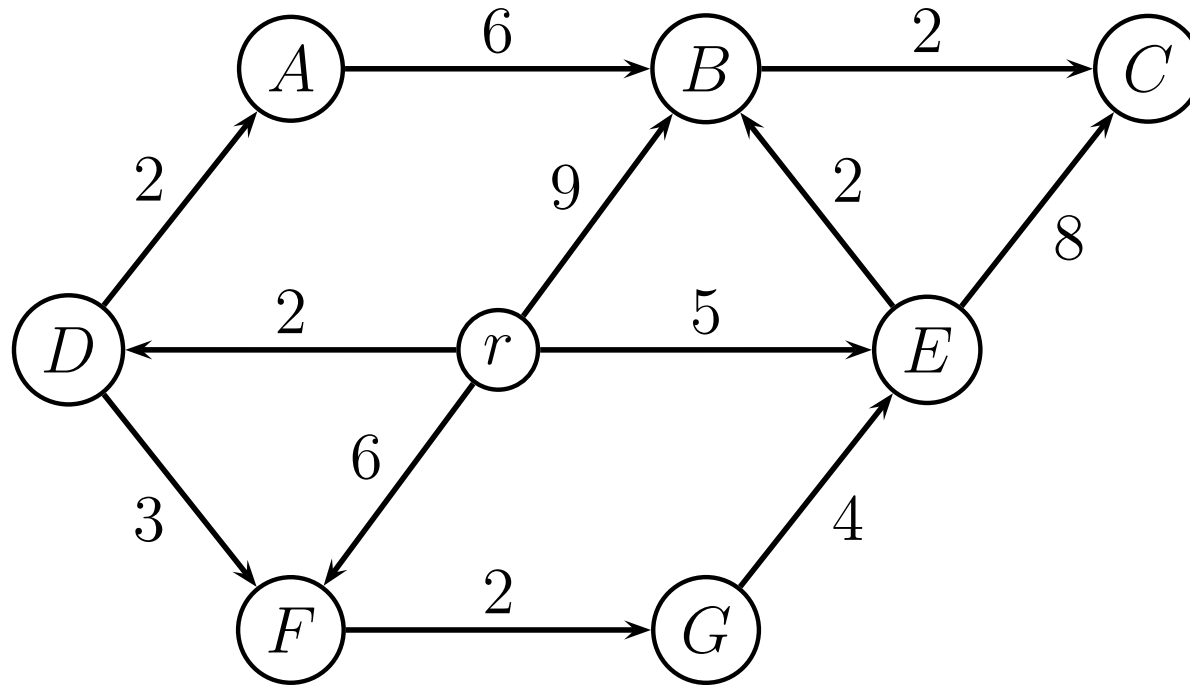
A* Algorithm for Shortest Path

Let h_i be a heuristic distance from i to t

Run Dijkstra, but selecting node i with lowest $y_i + h_i$

```
1  $T \leftarrow \emptyset$ ;  
2  $y_r \leftarrow 0$ ;  $y_i \leftarrow \infty$  for all other  $i$ ;  
3 while  $V \setminus T \neq \emptyset$  do  
4   select an  $i \in V \setminus T$  minimizing  $y_i + h_i$ ;  
5   for  $\{(i, j) \in E : j \notin T\}$  do  
6     if  $y_i + w_{ij} < y_j$  then  
7        $y_j \leftarrow y_i + w_{ij}$ ;  
8    $T \leftarrow T \cup \{i\}$ ;  
9   if  $i = t$  then  
10    stop
```

A^* Algorithm for Shortest Path, example



A^* Algorithm for Shortest Path, example

Normal Dijkstra: y_i values

node	r	A	B	C	D	E	F	G
init	0	∞	∞	∞	∞	∞	∞	∞
r			9		2	5	6	
D		4					5	
A								
F								7
E			7	13				
G								
B				9				
C								

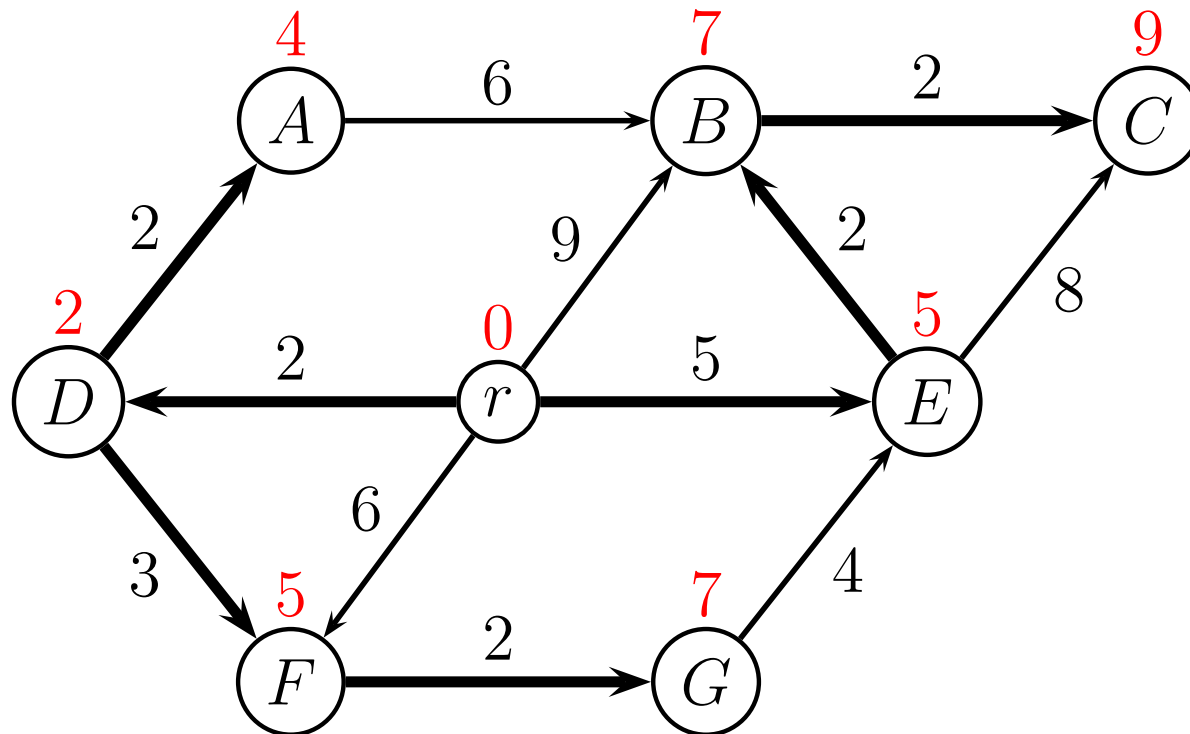
Shortest distances: marked with bold

Shortest path tree: bold numbers find when was updated

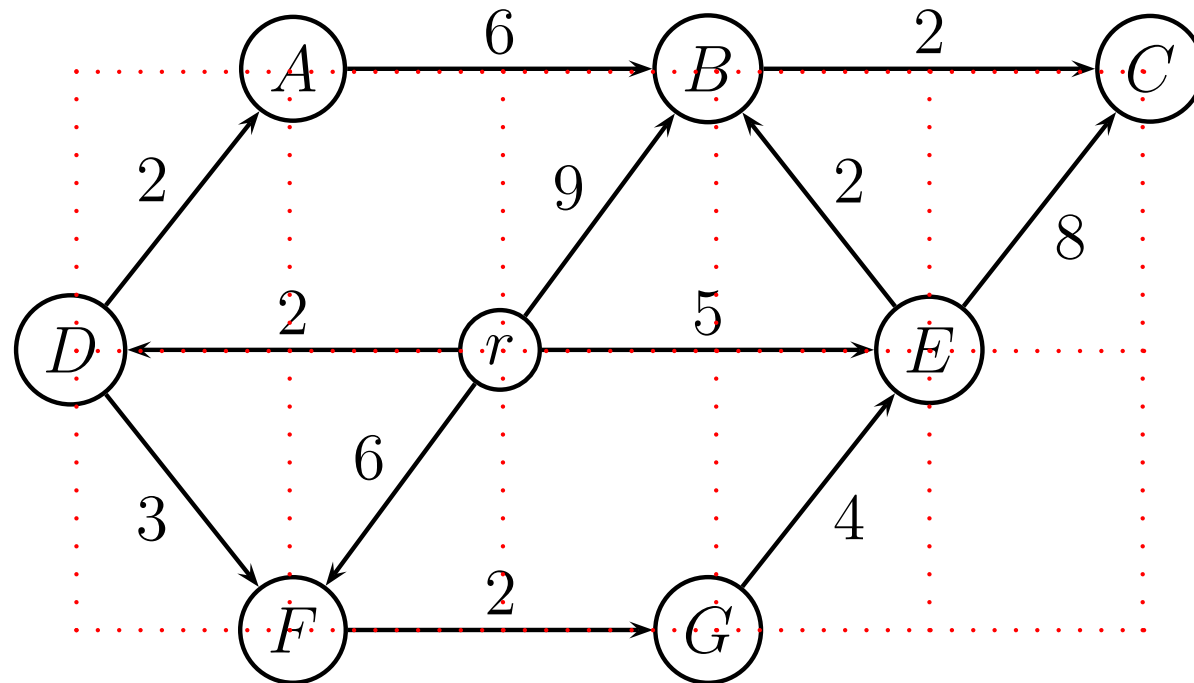
- e.g. node C (distance 9) was updated from B

A^* Algorithm for Shortest Path, example

Normal Dijkstra: shortest path tree



A^* Algorithm for Shortest Path, example



Manhattan distances to $t = C$

i	r	A	B	C	D	E	F	G
h_i	4	4	2	0	6	2	6	4

A^* Algorithm for Shortest Path, example

A^* algorithm: y_i and $y_i + h_i$ values

node	r	A		B		C		D		E		F		G	
h_i	4	4		2		0		6		2		6		4	
init	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
r				9	11			2	8	5	7	6	12		
E				7	9	13	13								
D		4	8									5	11		
A															
B						9	9								
C															

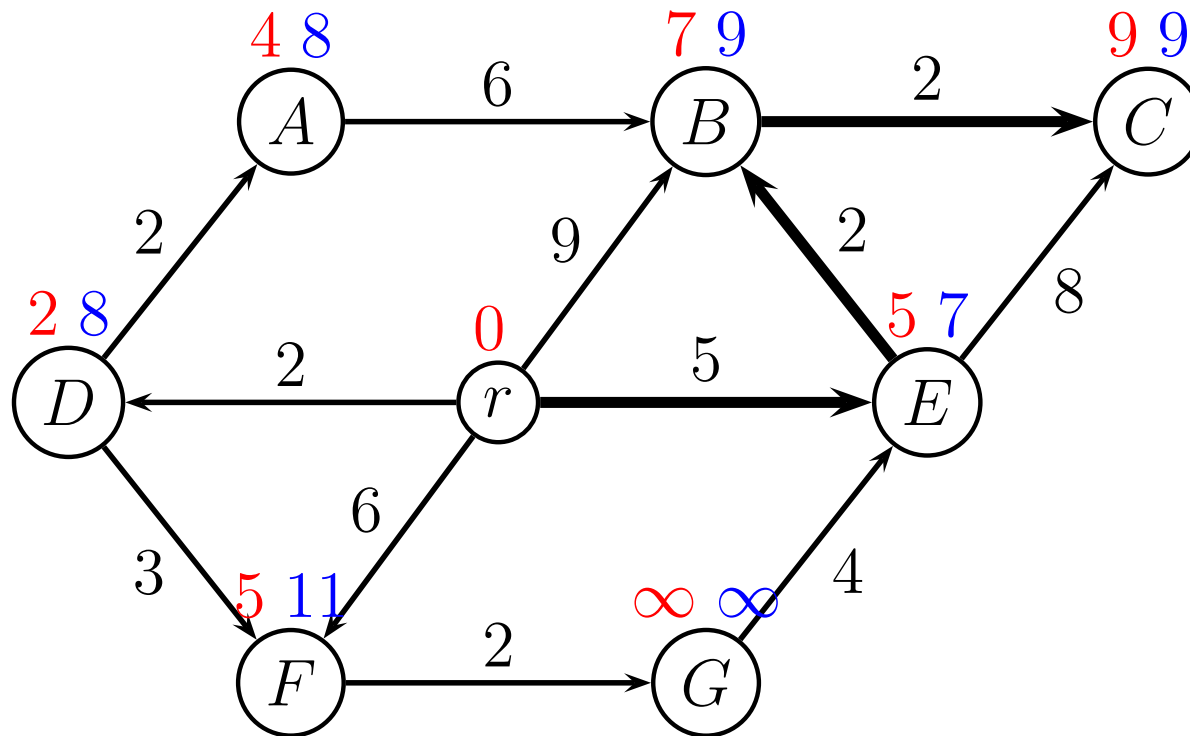
Shortest path tree: Predecessor of C is B .

Predecessor of B is E . Predecessor of E is r .

The shortest path is $r \rightarrow E \rightarrow B \rightarrow C$, cost $5 + 2 + 2 = 9$.

A^* Algorithm for Shortest Path, example

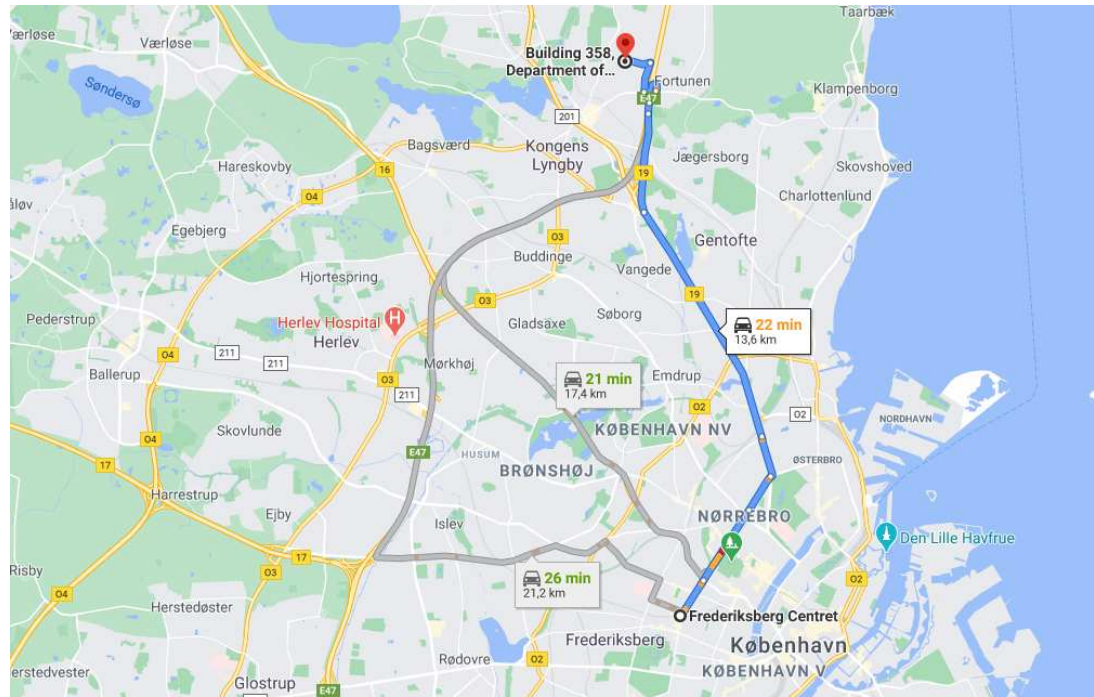
A^* algorithm: shortest path



i	r	A	B	C	D	E	F	G
h_i	4	4	2	0	6	2	6	4

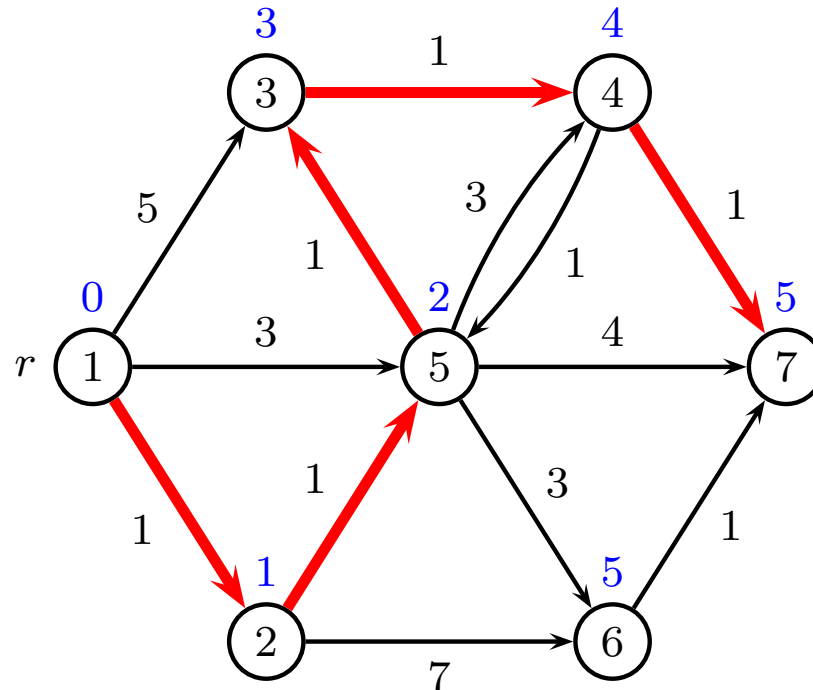
Finding the second-best path

Google maps: find alternatives



Column generation, forbidden column

Finding the second-best path

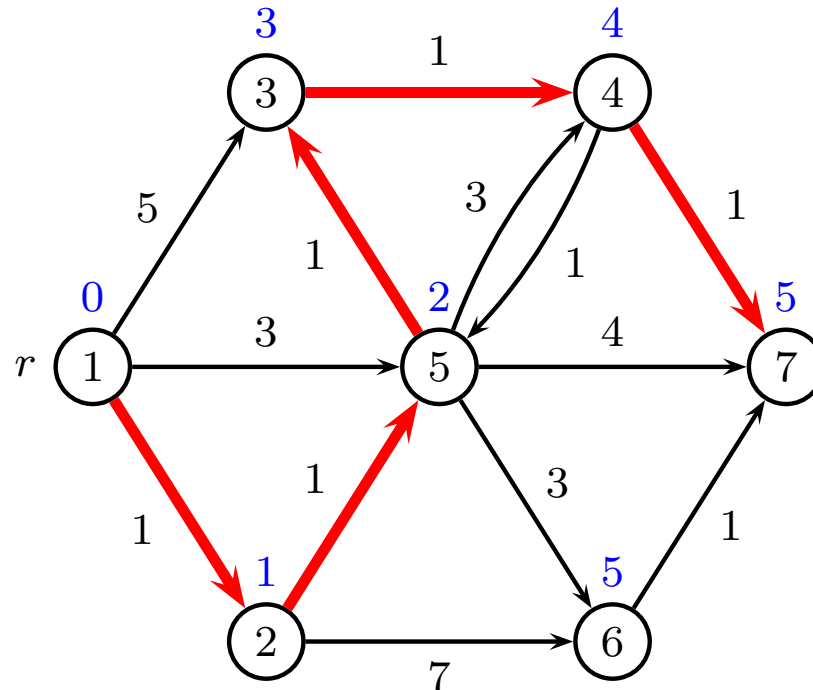


Repeat for all edges (i, j) on shortest path

- set $w_{ij} = \infty$
- solve shortest-path problem
- set w_{ij} to old value

Select shortest path among all candidates

Finding the k best paths

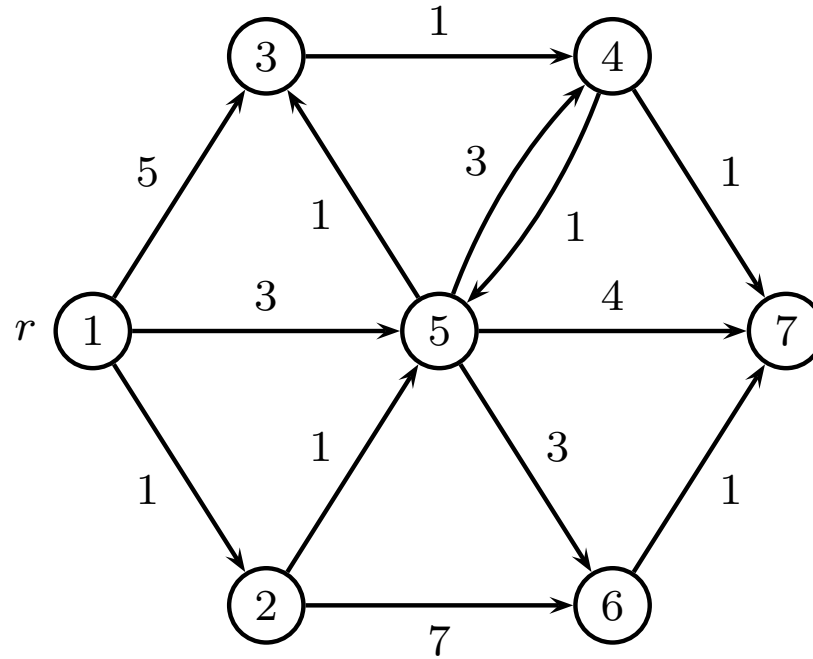


Same principle, but now multiple edges may be forbidden
Explore a branch-and-bound tree of forbidden edges

However, paths will often be very similar

Shortest-path diversification

Edge weight randomization



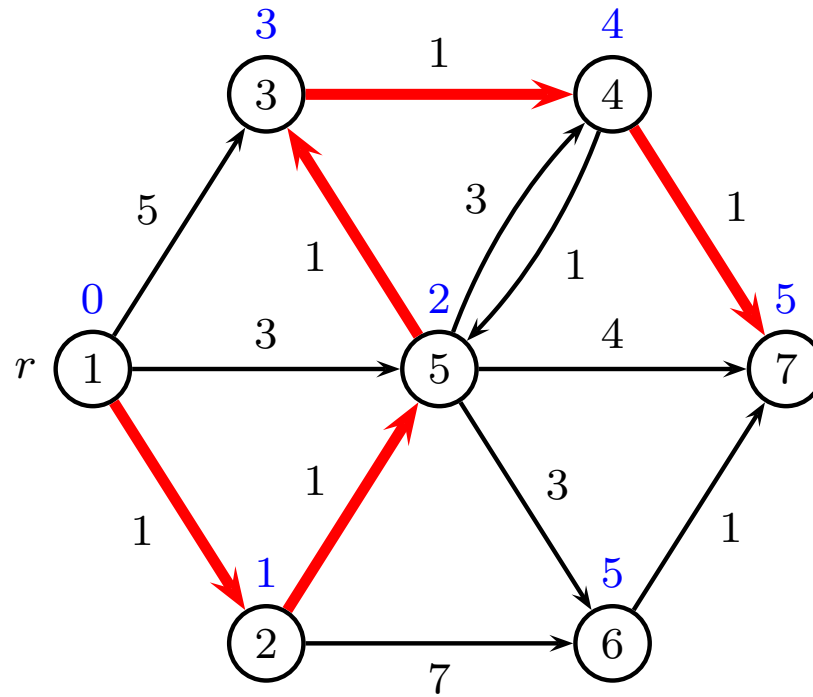
Repeat k times

- for all edges set $w_{ij} := w_{ij} + \text{noise}$
- solve shortest-path problem

Return k candidates

Shortest-path diversification

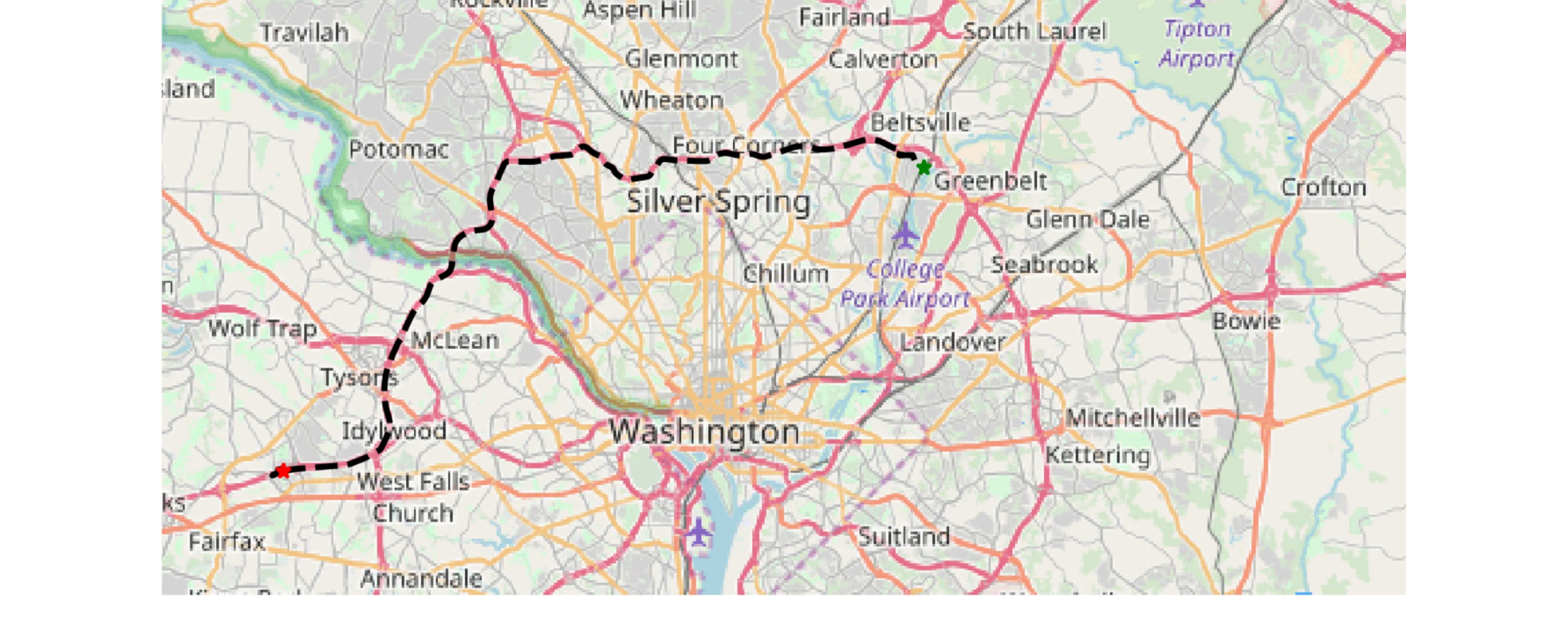
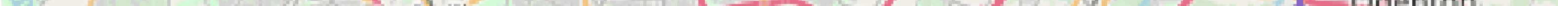
Edge weight penalization



Penalize edges on shortest path

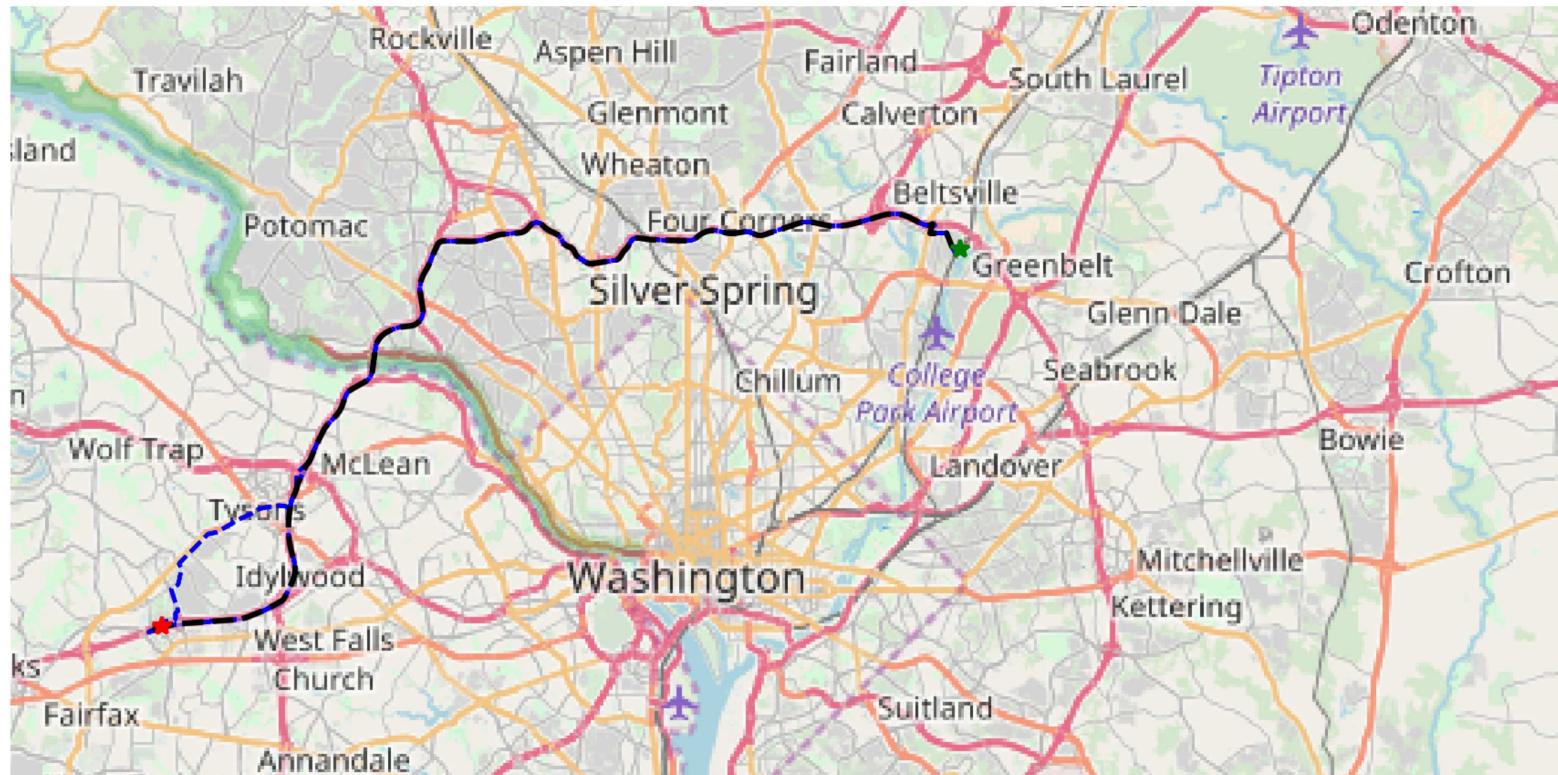
Shortest path using Dijkstra

Shortest path using Dijkstra



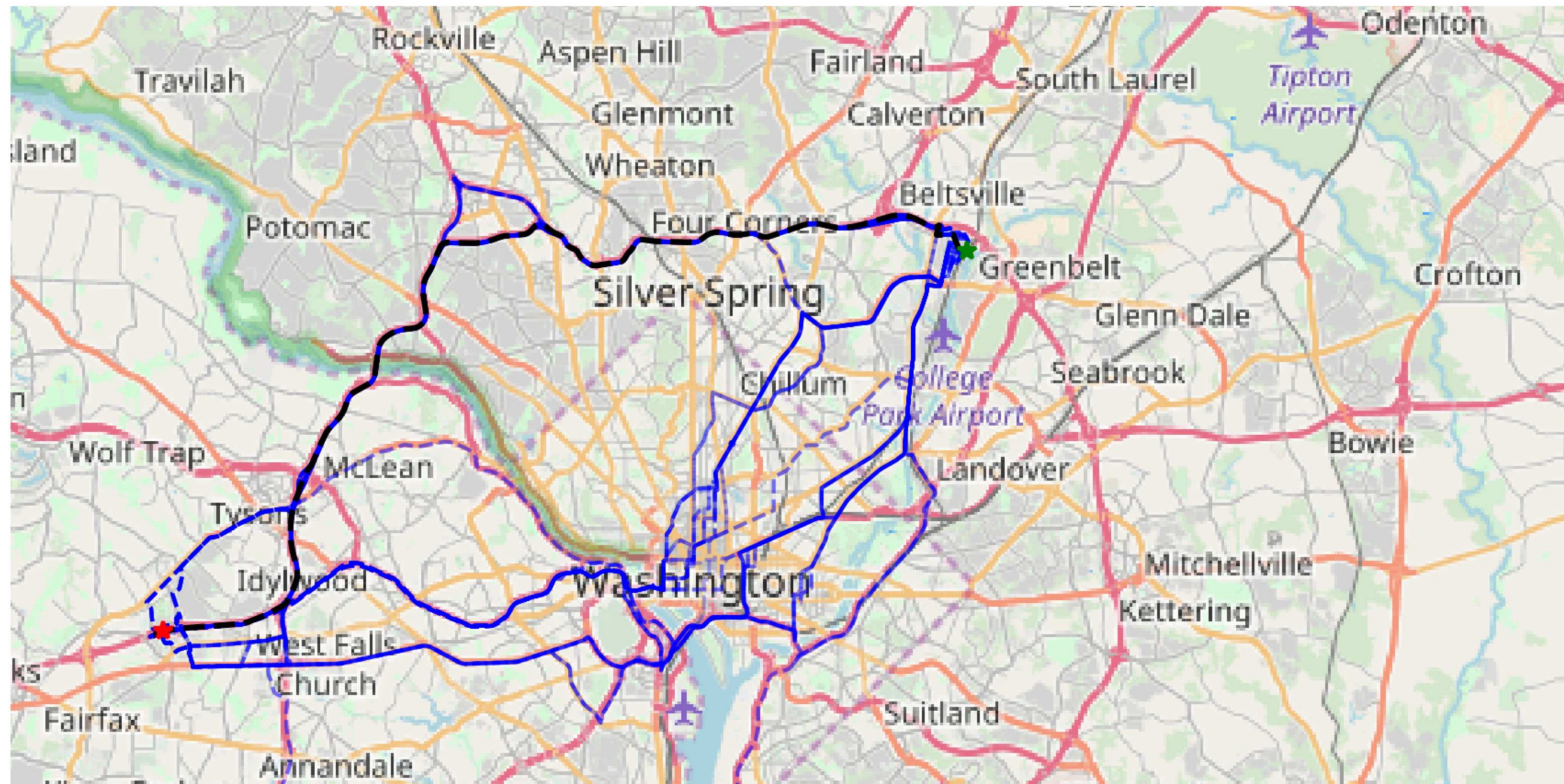
Shortest path diversification

100 path using graph randomization ($\delta = 0.1$)



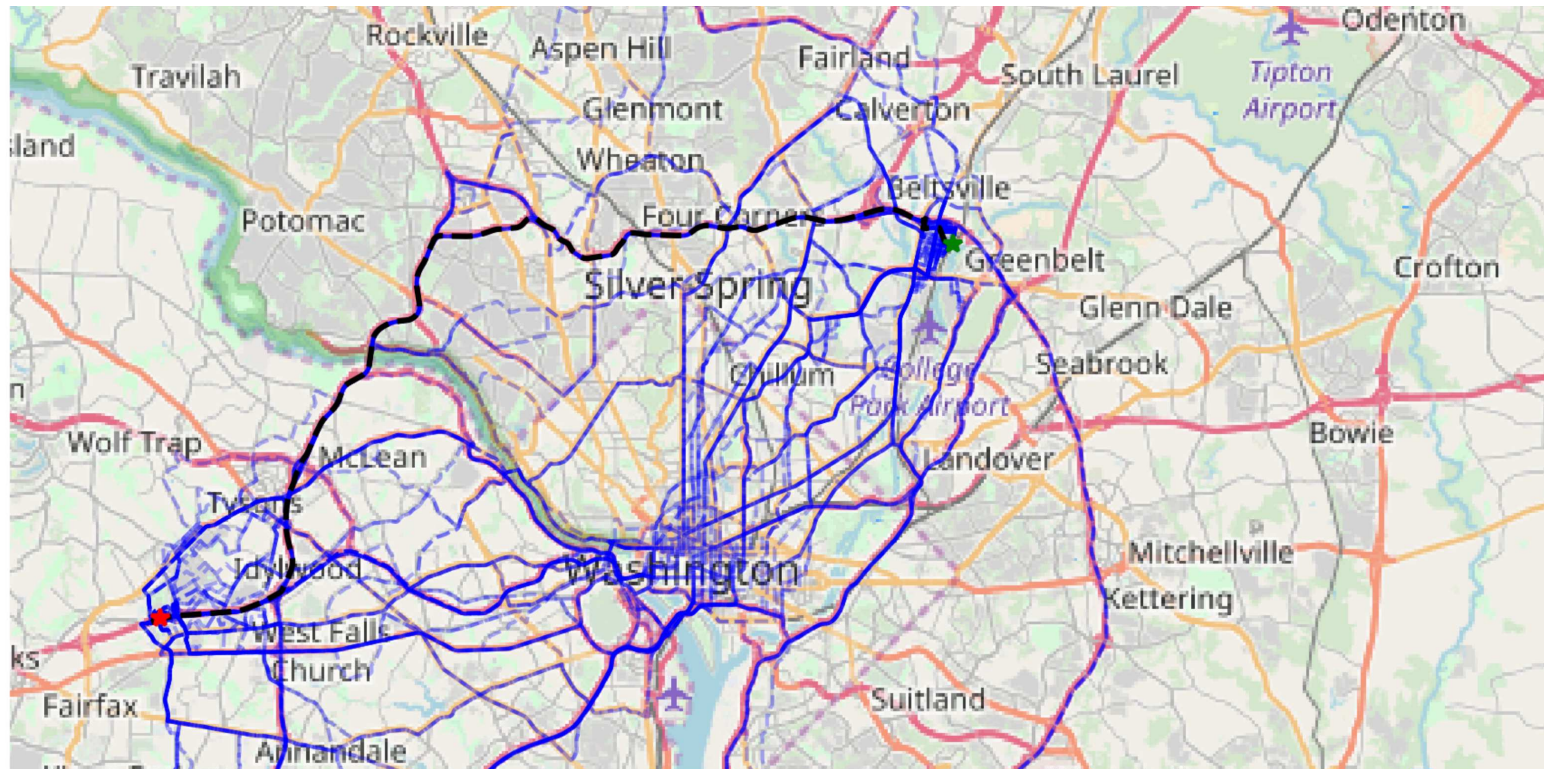
Shortest path diversification

100 path using penalization ($\rho = 0.01$)



Shortest path diversification

100 path using penalization ($\rho = 0.1$)



Resource Constrained Shortest Path Problem

David Pisinger

`pisinger@man.dtu.dk`

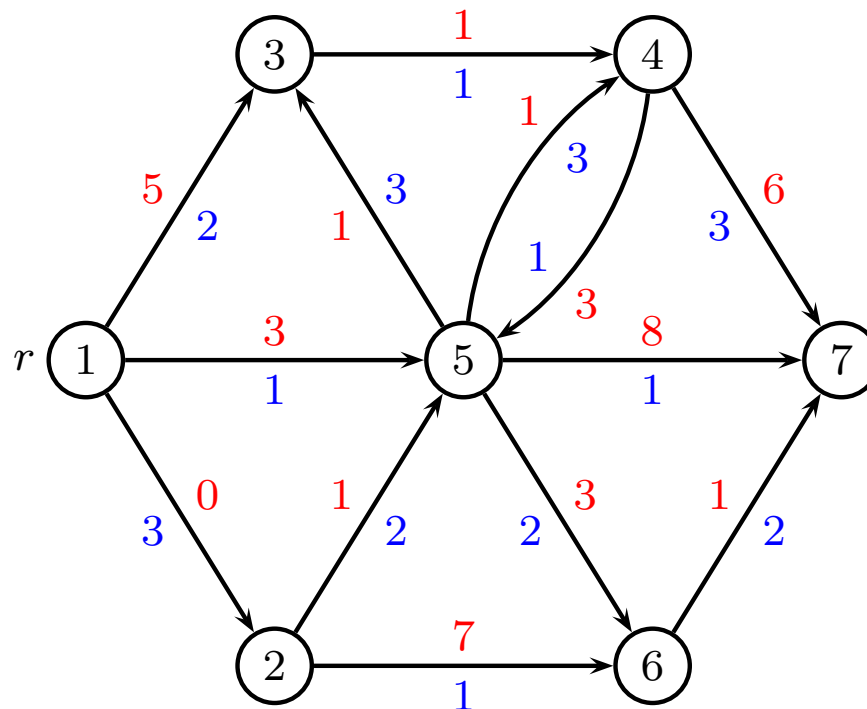
DTU Management

Technical University of Denmark

© Copyright David Pisinger. May not be distributed

Resource Constrained Shortest Path Pr.

- Given a **directed** network $\mathcal{G} = (V, E, w, t)$ where $w, t \in \mathbb{R}$
- A **source vertex** r is given.
- An upper bound T on time.
- Objective:** Find for each $v \in V$ a shortest (with respect to w) directed path from r to v (if such one exists), respecting upper bound T .



cost **red**
time **blue**

Applications

- Time constrained shortest path (i). Find cheapest path, respecting an upper limit on time consumption T .
- Cost constrained shortest path (ii). Find fastest path, respecting an upper limit on cost T .
- Vehicle brings out goods to customers (nodes). Each customer j has a demand t_j and vehicle has a capacity T .



Variants

Resource constrained shortest path

- One resource (e.g. min time, subject to cost)
- Many resources (e.g. time, cost, capacity)

Time constrained shortest path

- Must arrive in node in given time window

Elementary shortest path (each customer visited at most once)

- Appears when an “award” is received for visiting a customer
- Award is put on ingoing edges (edges may become negative)
- Handled by having one resource for each node

Mathematical Model for $r \rightarrow s$

Shortest path from r to s

$$\begin{aligned} \min \quad & \sum_{(i,j) \in E} w_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{(i,j) \in E} t_{ij} x_{ij} \leq T \\ & \sum_{i \in V} x_{ri} - \sum_{i \in V} x_{ir} = 1 \\ & \sum_{i \in V} x_{si} - \sum_{i \in V} x_{is} = -1 \\ & \sum_{i \in V} x_{ij} - \sum_{i \in V} x_{ji} = 0 \quad j \neq \{r, s\} \\ & x_{ij} \in \{0, 1\} \quad (i, j) \in E \end{aligned}$$

LP-problem does not return IP-solution. Problem is NP-hard (difficult!)

Label correcting algorithm, normal SPP

```
1  $y_i \leftarrow \infty$  for all  $i \neq r$ ;  
2  $y_r \leftarrow 0$ ;  
3  $Q \leftarrow \{r\}$ ;  
4 while  $Q \neq \emptyset$  do  
5   extract an  $i \in Q$ ;  
6   for  $\{(i, j) \in E\}$  do  
7     if  $y_i + w_{ij} < y_j$  then  
8        $y_j \leftarrow y_i + w_{ij}$ ;  
9       add  $j$  to  $Q$  if not already present;
```

Q is called candidate list

Resource constrained shortest path

List $L_i = \{\overline{w}_k, \overline{t}_k\}$ of labels for each node i

cost	time
4	5
3	6
4	3
2	7
1	6

Label correcting algorithm, resource SPP

```
1  $L_i \leftarrow \emptyset$  for all  $i \neq r$ ;  
2  $L_r \leftarrow \{(0, 0)\}$ ;  
3  $Q \leftarrow \{r\}$ ;  
4 while  $Q \neq \emptyset$  do  
5   extract an  $i \in Q$ ;  
6   for  $\{(i, j) \in E\}$  do  
7     for  $(\bar{w}_k, \bar{t}_k) \in L_i$  do  
8        $\lfloor$  insert  $(\bar{w}_k + w_{ij}, \bar{t}_k + t_{ij})$  into  $L_j$ ;  
9        $\lfloor$  add  $j$  to  $Q$  if not already present;
```

Note: We only need to add j to Q if some labels were changed in L_j

Dominance

If a label can reach a node cheaper and faster than another label, then delete dominated label.

Assume that at node i we have labels

cost	time
w_1	t_1
w_2	t_2

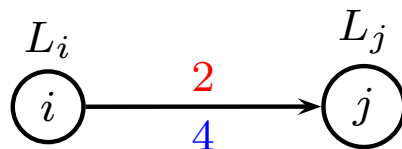
If we have

$$w_1 \leq w_2 \text{ and } t_1 \leq t_2$$

then (w_1, t_1) *dominates* (w_2, t_2)

Dominated labels can be deleted

Merging lists



Update L_j

$L_i + (w_{ij}, t_{ij})$ merged with L_j

L_i		$L_i + (2, 4)$		L_j	
cost	time	cost	time	cost	time
2	9	4	13	2	12
3	6	5	10	4	11
6	4	8	8	7	9
7	3	9	7	9	3
8	2	10	6	11	1

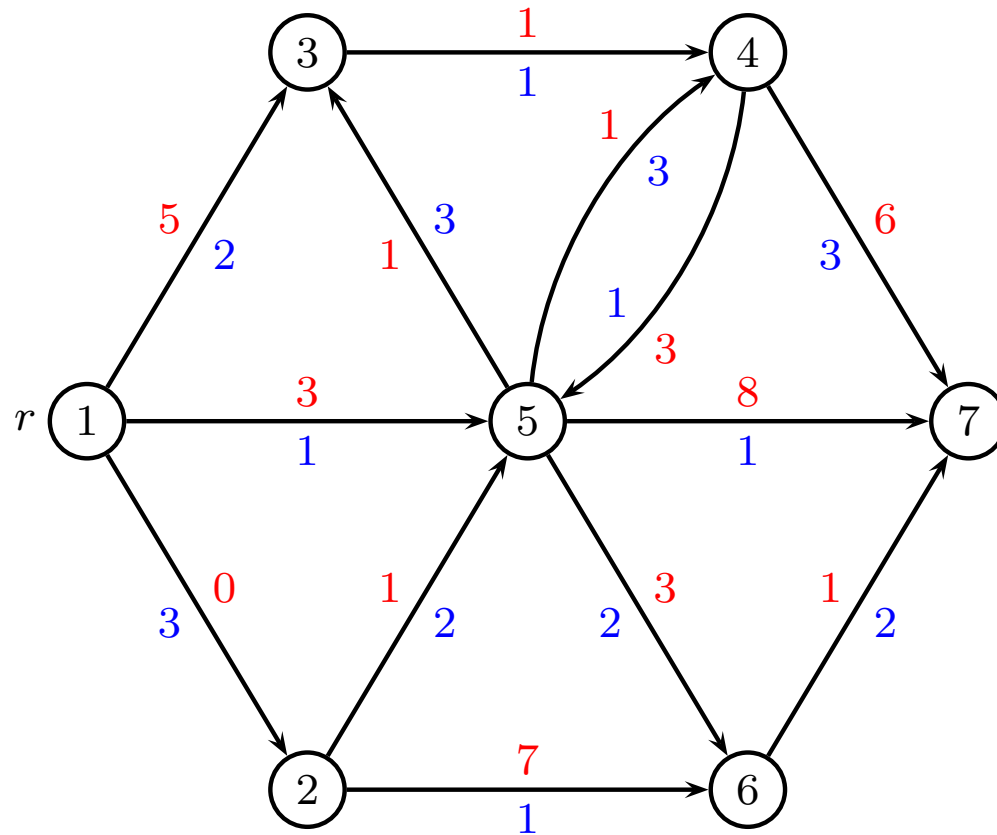
Merging lists

Resulting list in node j

L_j	
cost	time
2	12
4	11
5	10
7	9
8	8
9	3
11	1

Merging can be done in linear time $O(|L|)$
Only add node j to Q if L_j has changed

Example $T = 8$ (init)



node 1	
cost	time
0	0

node 2	
cost	time

node 3	
cost	time

node 4	
cost	time

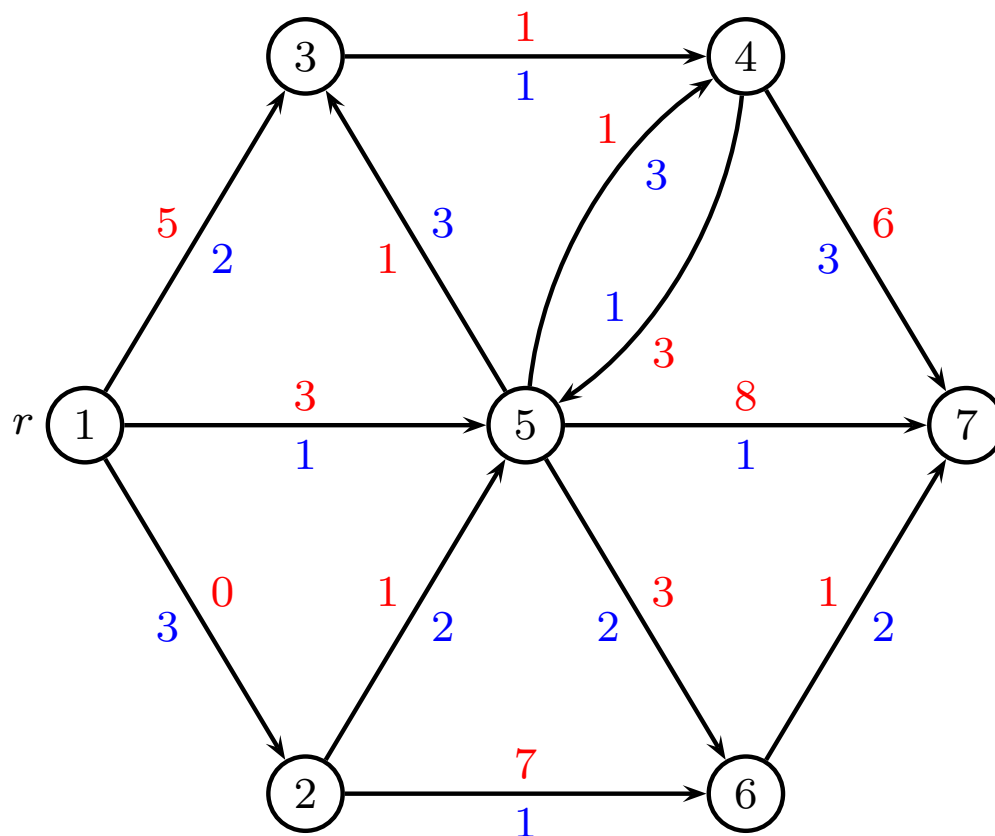
node 5	
cost	time

node 6	
cost	time

node 7	
cost	time

$$Q = \{1\}$$

Example $T = 8$ (node 1)



node 1	
cost	time
0	0

node 2	
cost	time
0	3

node 3	
cost	time
5	2

node 4	
cost	time

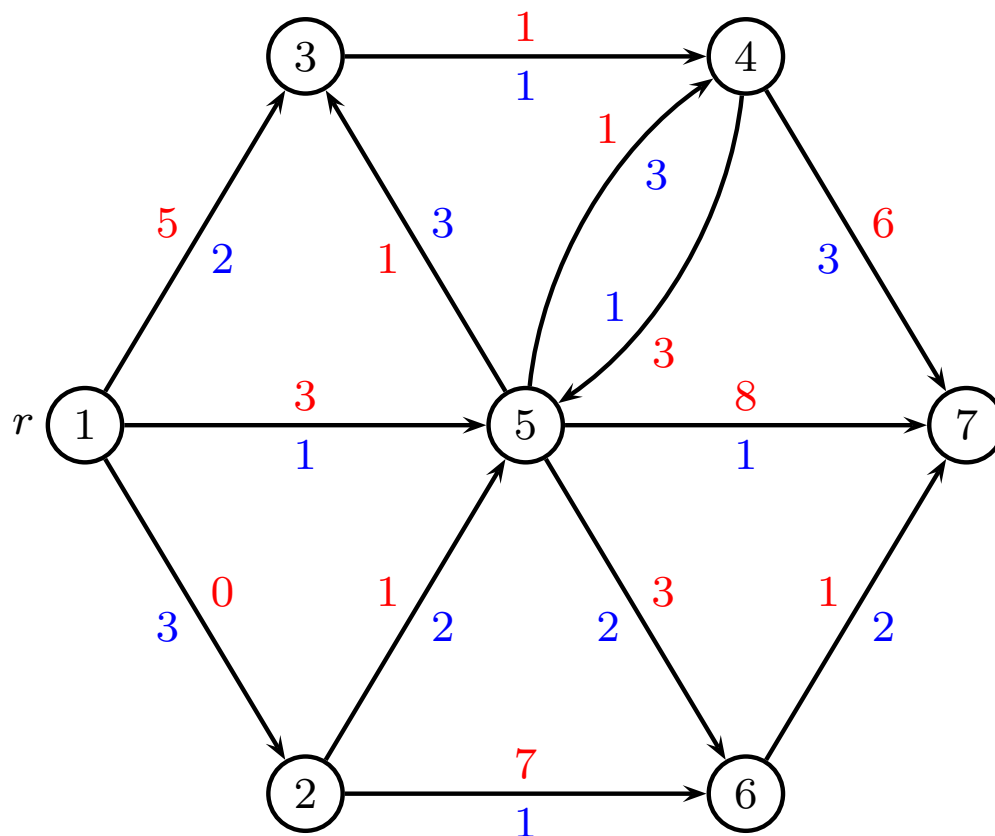
node 5	
cost	time
3	1

node 6	
cost	time

node 7	
cost	time

After considering node 1: $Q = \{2, 3, 5\}$

Example $T = 8$ (node 2)



node 1	
cost	time
0	0

node 2	
cost	time
0	3

node 3	
cost	time
5	2

node 4	
cost	time

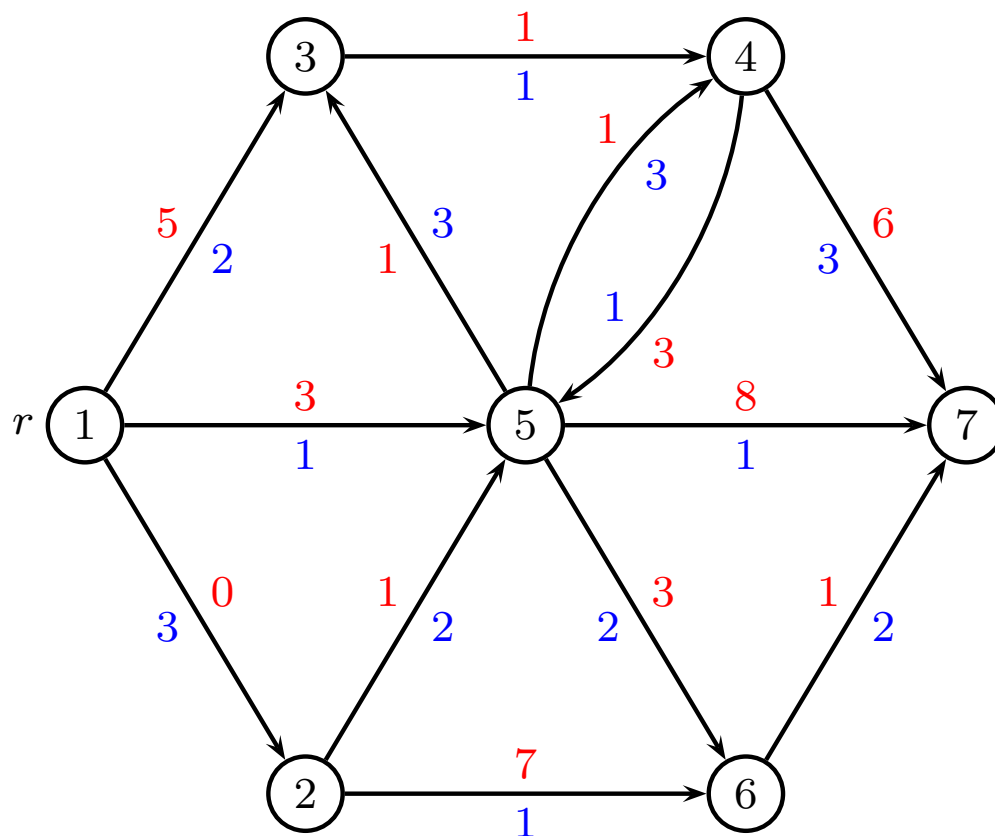
node 5	
cost	time
1	5
3	1

node 6	
cost	time
7	4

node 7	
cost	time

After considering node 2: $Q = \{3, 5, 6\}$

Example $T = 8$ (node 3)



node 1	
cost	time
0	0

node 2	
cost	time
0	3

node 3	
cost	time
5	2

node 4	
cost	time
6	3

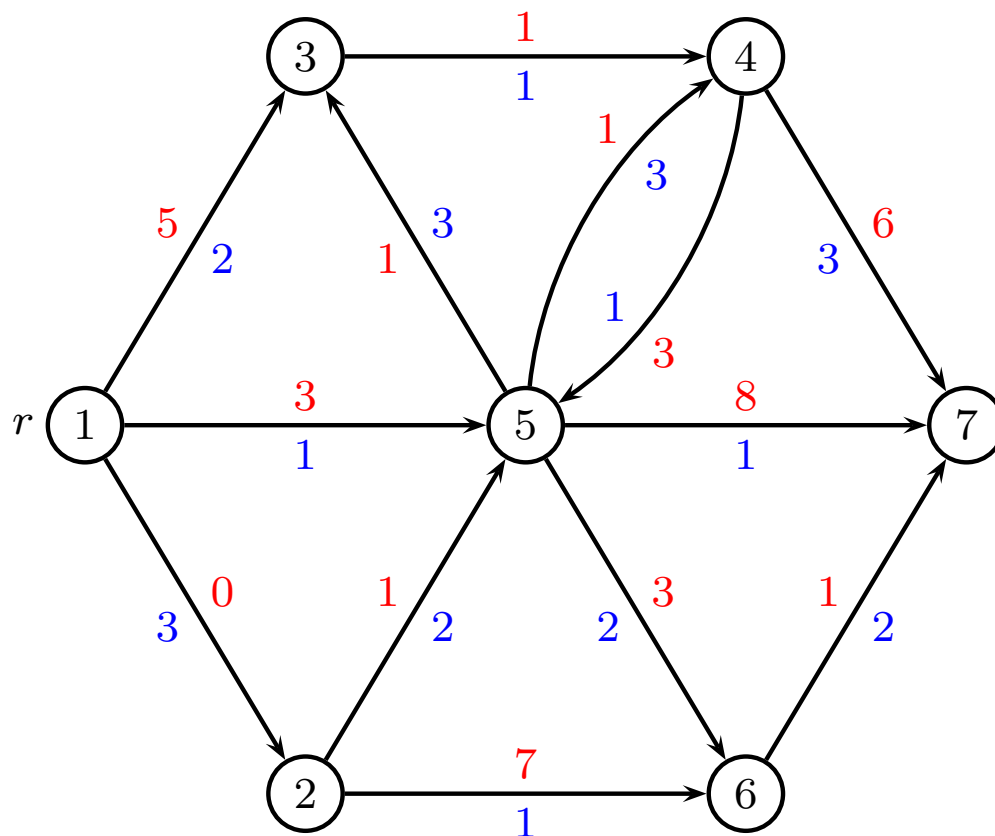
node 5	
cost	time
1	5
3	1

node 6	
cost	time
7	4

node 7	
cost	time

After considering node 3: $Q = \{5, 6, 4\}$

Example $T = 8$ (node 5)



node 1	
cost	time
0	0

node 2	
cost	time
0	3

node 3	
cost	time
2	8
4	4
5	2

node 4	
cost	time
2	8
4	4
6	3

node 5	
cost	time
1	5
3	1

node 6	
cost	time
4	7
6	3
(7	4)

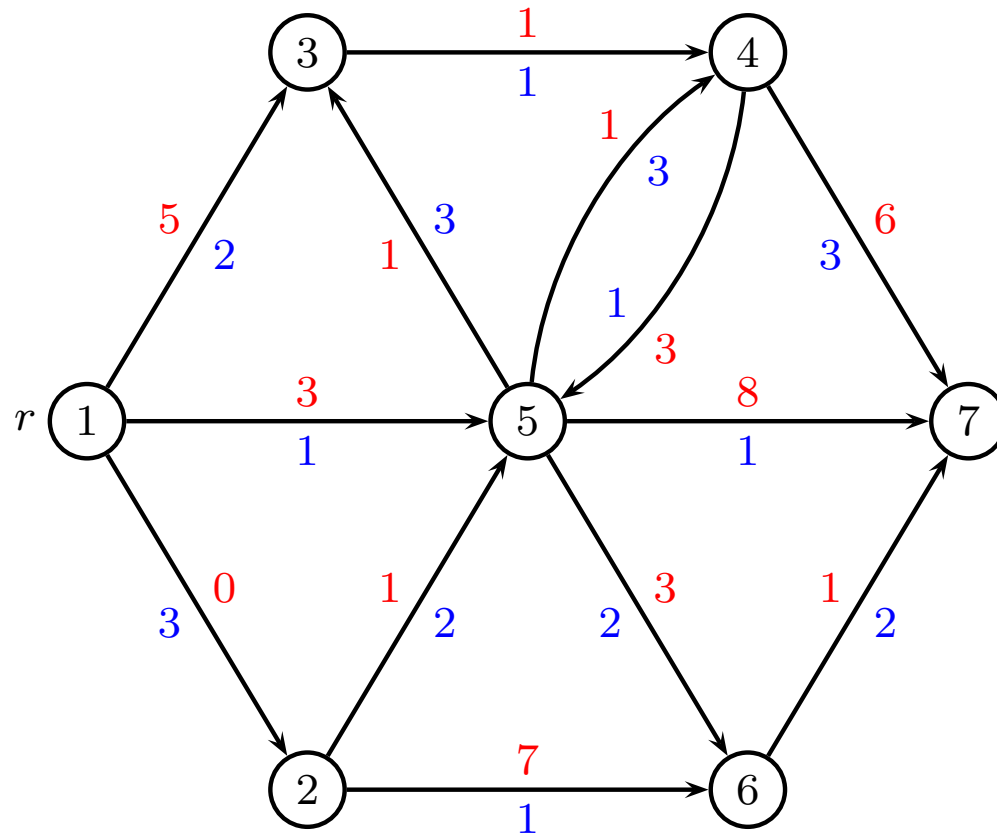
node 7	
cost	time
9	6
11	2

After considering node 5: $Q = \{6, 4, 3, 7\}$

Example (node 6 ...)

... Continue ...

Example $T = 8$ (end)



node 1	
cost	time
0	0

node 2	
cost	time
0	3

node 3	
cost	time
2	8
4	4
5	2

node 4	
cost	time
2	8
4	4
6	3

node 5	
cost	time
1	5
3	1

node 6	
cost	time
4	7
6	3

node 7	
cost	time
5	9
7	5
11	2

End: $Q = \{\}$

Pareto optimal solutions

For every node we get a set of solutions

- All undominated pairs of cost and time
- These are called *Pareto optimal solutions*
- Many travel planners are based on this idea

Time complexity

If all times t_{ij} are integers

- At most n iterations
- Each time extend m edges
- Each list of labels L_i is $O(T)$

Total: $O(nmT)$

where $n = |V|$ and $m = |E|$

If all costs w_{ij} are integers

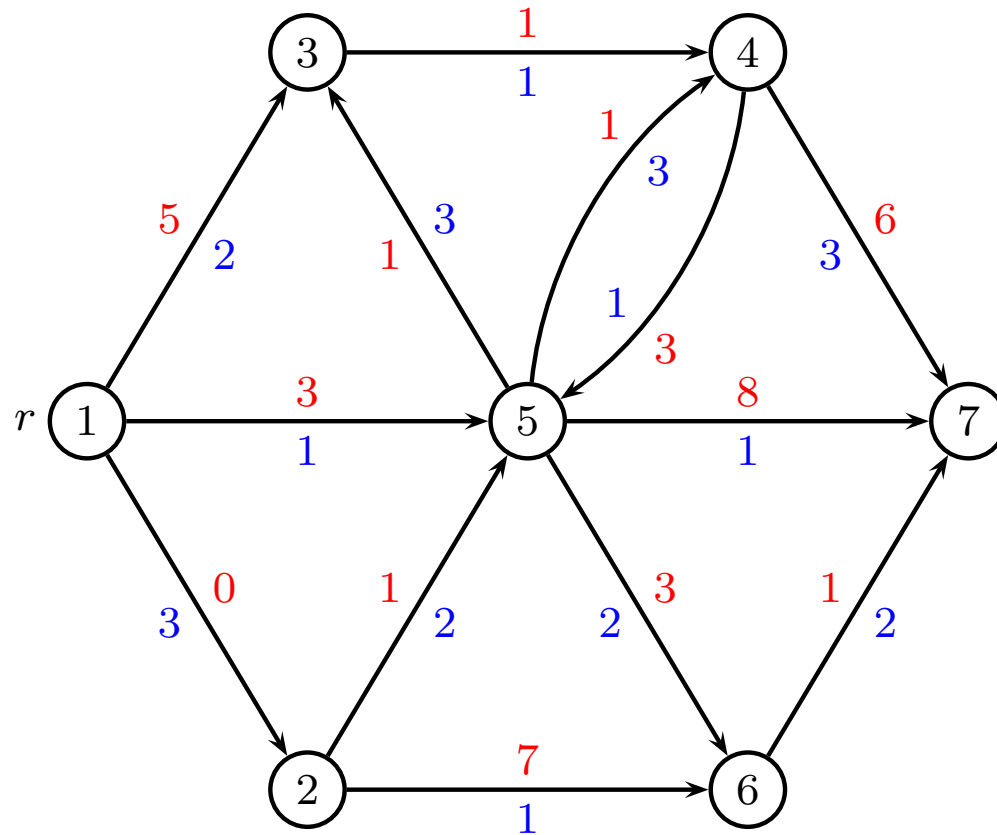
Total: $O(nmW)$

where W is upper bound on objective

If all costs, times real

Can be exponential

Finding the optimal path



node 1		
cost	time	pred
0	0	⊥

node 2		
cost	time	pred
0	3	1

node 3		
cost	time	pred
2	8	5
4	4	5
5	2	1

node 4		
cost	time	pred
2	8	5
4	4	5
6	3	3

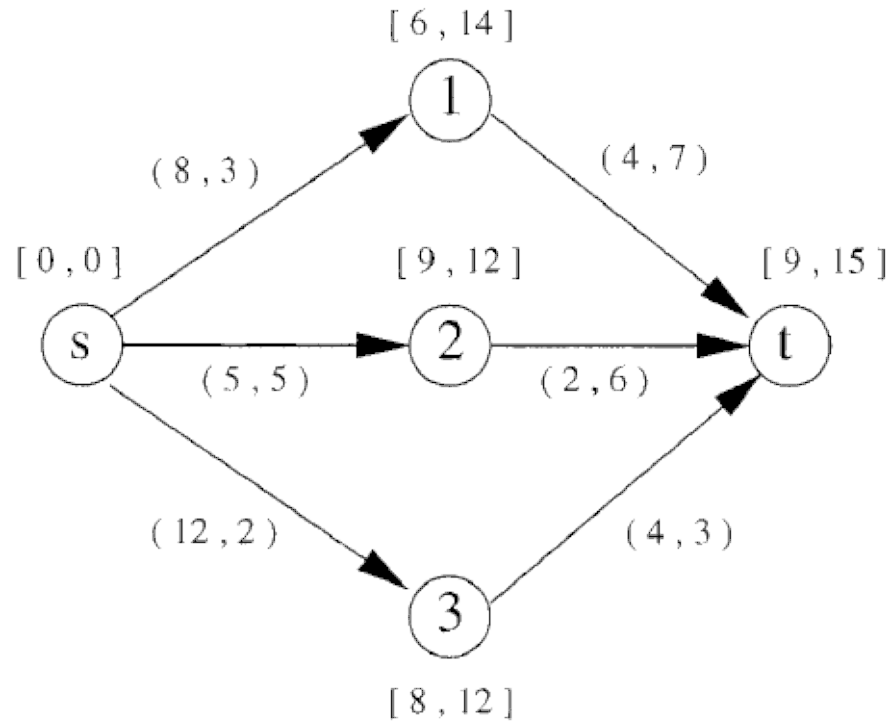
node 5		
cost	time	pred
1	5	2
3	1	1

node 6		
cost	time	pred
4	7	5
6	3	5

node 7		
cost	time	pred
5	9	6
7	5	6
11	2	5

Keep track of predecessor for each label, follow back to r

Time windows on nodes

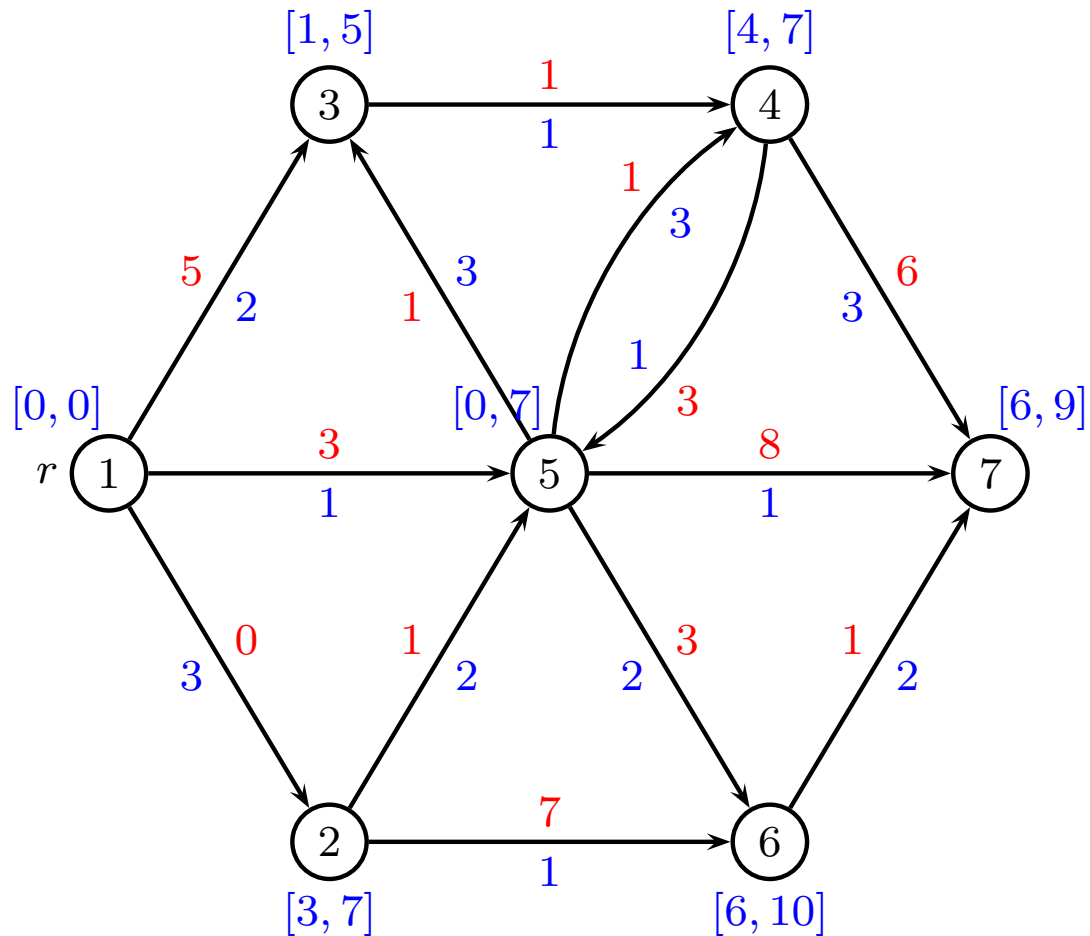


Examples

- Arrive at customer at time [8, 10]
- Green light for cars/trains [4, 6]
- Bridge opens [13, 14]

Time windows on nodes

Must arrive at node in time window $[u_i, v_i]$



Time windows on nodes

Assume node i has list L_i and time window $[4, 10]$.

cost	time
2	12
4	11
5	10
7	9
8	8
9	3

Remove labels not satisfying time window

cost	time
5	10
7	9
8	8

Arrive too early: road (wait), train (no), plane (no)

Elementary shortest path

- Every node can be visited at most once
- Run label correcting algorithm with a resource indicating which nodes have been visited
- Every node has labels

cost	time	visited
4	5	{13425}
3	6	{3451}
4	3	{2163}
2	7	{34}
1	6	{612}

Dominance

If a label can reach a node cheaper and faster than another label, then delete dominated label.

Assume that in a node i we have labels

cost	time	visited
w_1	t_1	V_1
w_2	t_2	V_2

If we have

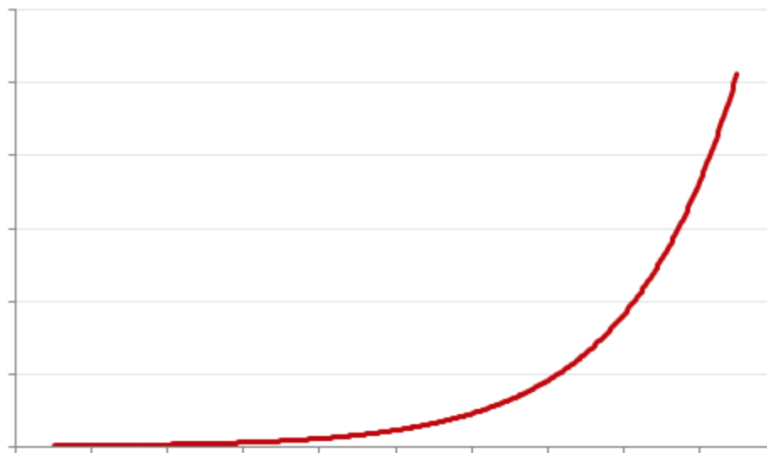
$$w_1 \leq w_2 \text{ and } t_1 \leq t_2 \text{ and } V_1 \subseteq V_2$$

then (w_1, t_1, V_1) dominates (w_2, t_2, V_2)

Bidirectional method

Motivation

- Number of labels $|L_j|$ at each node often grow exponentially with j 's distance from r



- If we can limit the search to “half” the distance, we get large saving

How do we know what is “half” the distance?

Bidirectional method

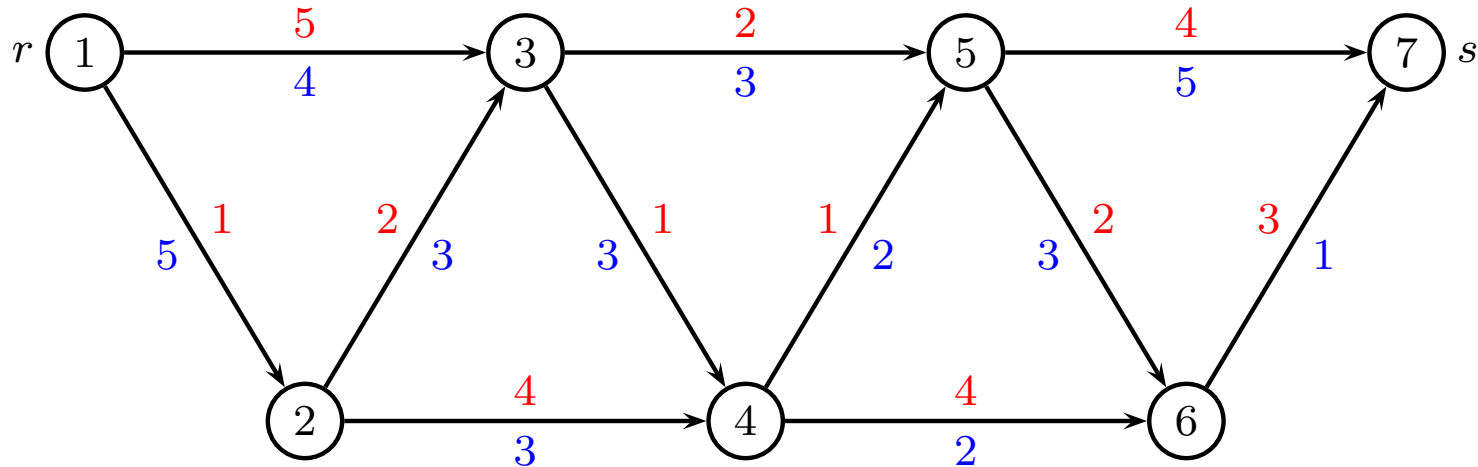
- Choose a monotone resource (e.g. time or capacity)
- Run label correcting algorithm “as before” starting from root
- When more than half of the resource is used in a node, do not extend any more
- Run label correcting algorithm “as before” starting from destination
- When more than half of the resource is used in a node, do not extend any more
- When label correcting algorithms stop, merge forward and backwards lists in all nodes

Label correcting algorithm, limit $T/2$

```
1  $L_i \leftarrow \emptyset$  for all  $i \neq r$ ;  
2  $L_r \leftarrow \{(0, 0)\}$ ;  
3  $Q \leftarrow \{r\}$ ;  
4 while  $Q \neq \emptyset$  do  
5   extract an  $i \in Q$ ;  
6   for  $\{(i, j) \in E\}$  do  
7     for  $(\bar{w}_k, \bar{t}_k) \in L_i$  do  
8       if  $\bar{t}_k < T/2$  then  
9         insert  $(\bar{w}_k + w_{ij}, \bar{t}_k + t_{ij})$  into  $L_j$ ;  
10    add  $j$  to  $Q$  if not already present;
```

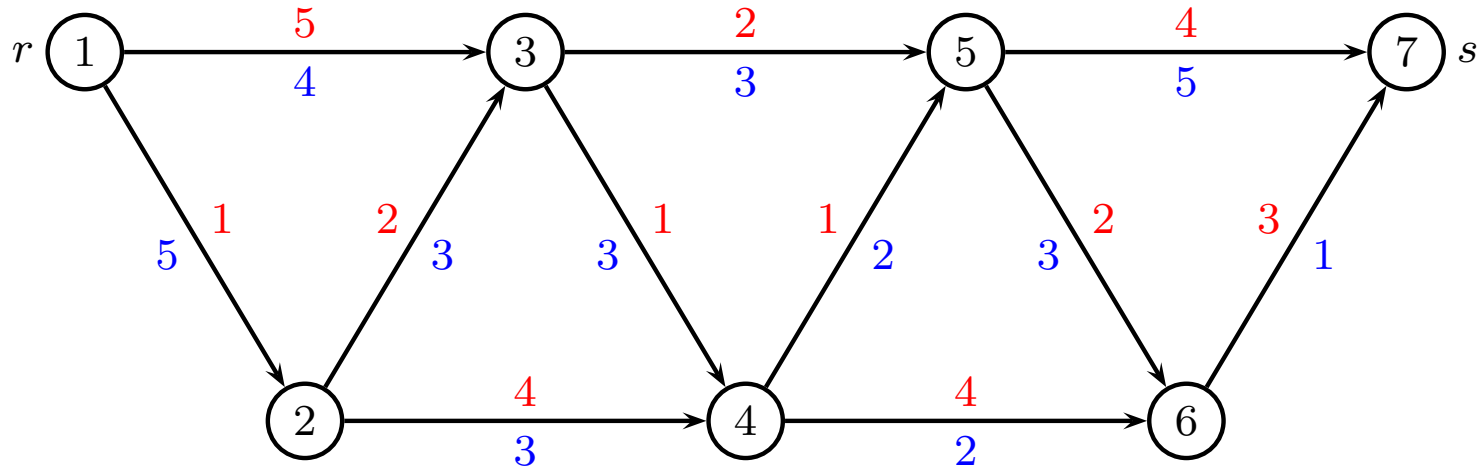
Note: We only need to add j to Q if some labels were changed in L_j

Bidirectional method, example



Shortest path from r to node 7
Time constraint $T = 12$

Bidirectional method, forward



node 1	
cost	time
0	0

node 2	
cost	time
1	5

node 3	
cost	time
3	8
5	4

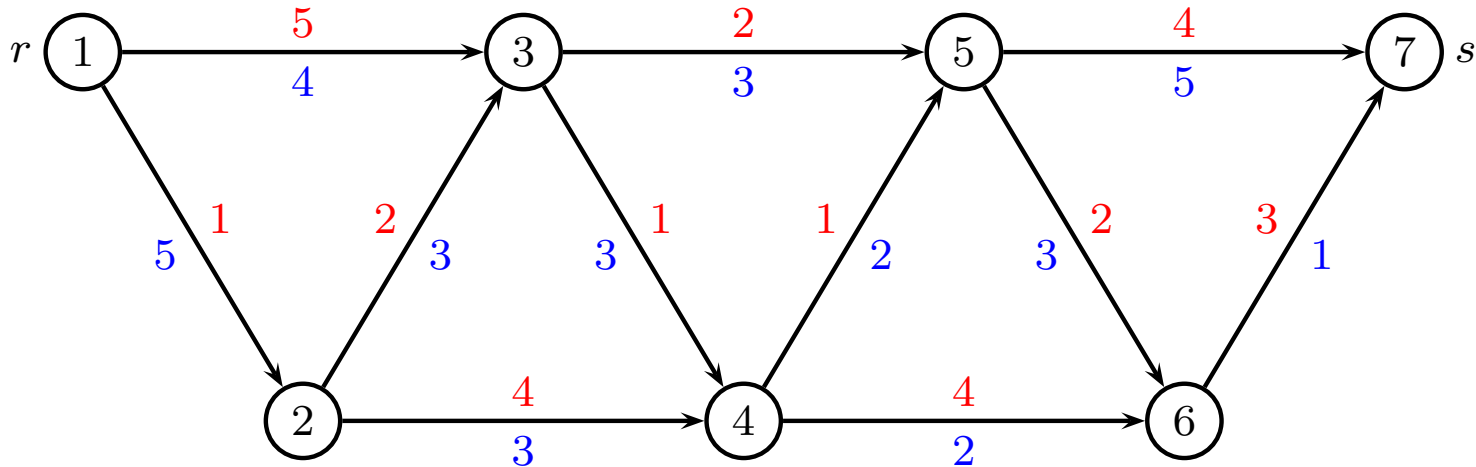
node 4	
cost	time
4	11
5	8
6	7

node 5	
cost	time
5	11
(5	13)
6	10
7	7
(7	9)

node 6	
cost	time
7	14
8	13
(8	13)
9	10
(9	10)
10	9

node 7	
cost	time
9	16
10	15
(10	15)
11	12
(11	14)
12	11
13	10

Bidirectional method, backward



node 1	
cost	time
9	16
10	15
11	12
(11	14)
12	11
(12	11)
13	10

node 2	
cost	time
8	11
9	10
(9	10)
10	9
(10	9)
11	6

node 3	
cost	time
6	8
(6	10)
7	7
(7	9)
8	6

node 4	
cost	time
5	7
6	6
7	3

node 5	
cost	time
4	5
5	4

node 6	
cost	time
3	1

node 7	
cost	time
0	0

Bidirectional method, truncated

Stop dynamic programming when resource $\geq T/2$

Forward:

node 1	node 2	node 3	node 4	node 5	node 6	node 7
cost time	cost time	cost time	cost time	cost time	cost time	cost time
0 0	1 5	3 8	6 7	7 7		
		5 4				

Backward:

node 1	node 2	node 3	node 4	node 5	node 6	node 7
cost time	cost time	cost time	cost time	cost time	cost time	cost time
		7 7	5 7	4 5	3 1	0 0
		8 6	6 6	5 4		
			7 3			

Merge in all nodes.

Choose best merged solution satisfying time limit T

Bidirectional method, merge

For each node, all feasible combinations with time limit T

node 1	node 2	node 3	node 4	node 5	node 6	node 7
<u>cost</u> <u>time</u>	<u>cost</u> <u>time</u>	<u>cost</u> <u>time</u>	<u>cost</u> <u>time</u>	<u>cost</u> <u>time</u>	<u>cost</u> <u>time</u>	<u>cost</u> <u>time</u>
		12 11	13 10	11 12		

Note: $(\text{cost}, \text{time}) = (11, 12)$ is cheapest choice within T

Bidirectional method, summing up

- A good approach if the number of labels L_i at every node i grows exponentially (elementary shortest path, or times t_{ij} are not integers)
- Too complex to use if graph is DAG, or times t_{ij} are integers and T is small
- Can only be used for a specific root r and destination s , where generic algorithm finds shortest path to all nodes
- We only find the optimal solution with time limit T , but not all pareto-optimal solutions for $t \leq T$.
- The optimal path can be found by keeping track of predecessors (both for forward and backward algorithm)

The Max Flow Problem

Augmenting path method

David Pisinger

`pisinger@man.dtu.dk`

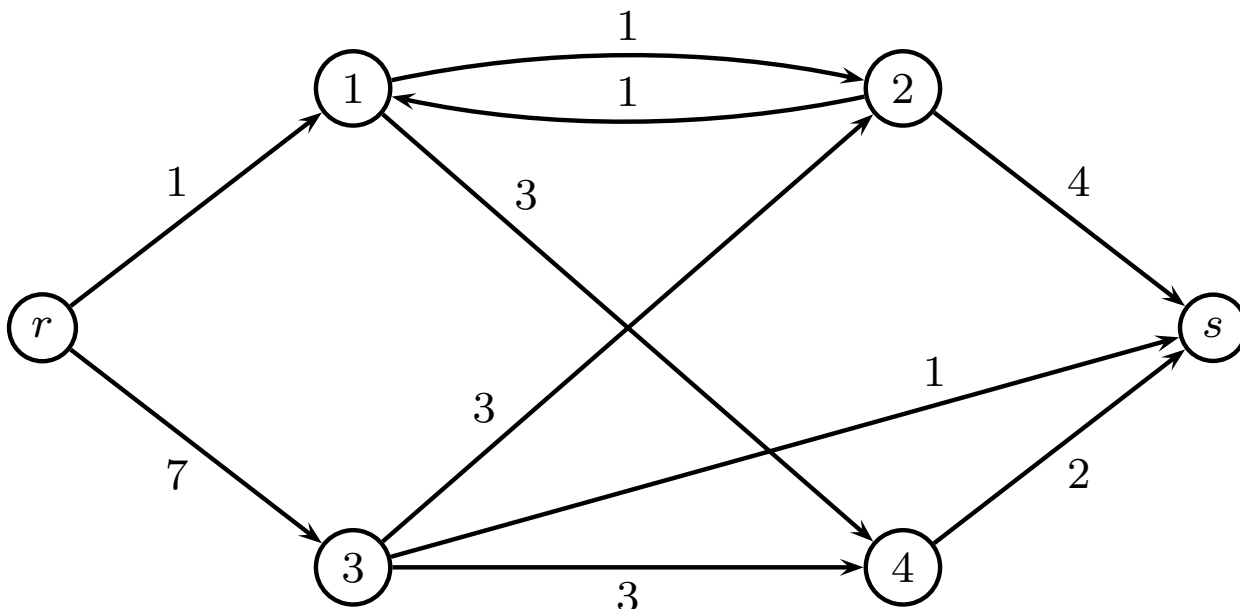
DTU Management

Technical University of Denmark

© Copyright David Pisinger. May not be distributed

The Max Flow Problem

- We consider a **directed graph** $G = (V, E)$ which for each e has a **capacity** $u_e \in \mathbb{R}_+$.
- Furthermore two “special” vertices r and s are given; these are called resp. the **source** and the **sink**.
- Aim is to find the maximum flow from the source to the sink in the graph while respecting the capacities of the edges.



Flow

● A **flow** in G is a function $x : E \rightarrow \mathbb{R}_+$ satisfying:

$$(1) \quad \forall i \in V \setminus \{r, s\} : \sum_{(j,i) \in E} x_{ji} - \sum_{(i,j) \in E} x_{ij} = 0$$

$$(2) \quad \forall (i, j) \in E : 0 \leq x_{ij} \leq u_{ij}$$

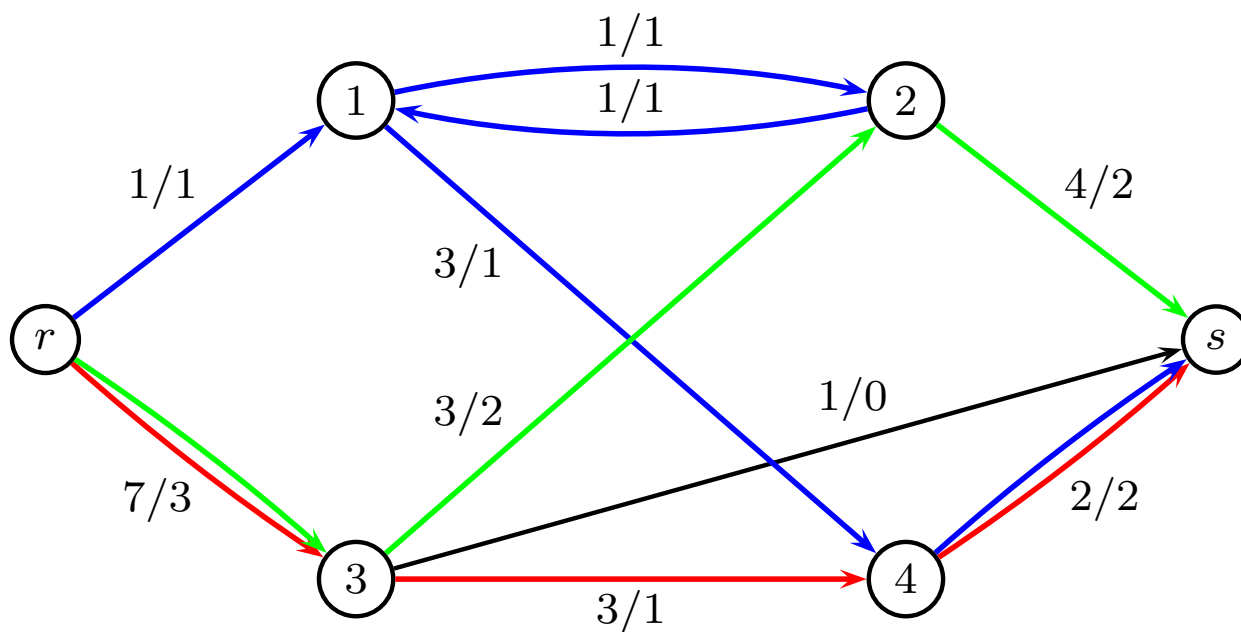
● For a flow we define: $f_x(i) = \sum_{(j,i) \in E} x_{ji} - \sum_{(i,j) \in E} x_{ij}$

● $f_x(i)$ is the **net flow into** i or the **excess for** x in i .

● $f_x(s)$ is called the **value** of x .

Example with a flow

Here we have added a flow of value 4 to our graph.



Notation: u_{ij}/x_{ij}

MIP Model of Max Flow

$$\begin{array}{ll}\max & f_x(s) \\ \text{s.t.} & f_x(i) = 0 \quad i \in V \setminus \{r, s\} \\ & 0 \leq x_e \leq u_e \quad e \in E \\ & x_e \in \mathbb{Z}_+ \quad e \in E\end{array}$$

where $f_x(i) = \sum_{(j,i) \in E} x_{ji} - \sum_{(i,j) \in E} x_{ij}$

Can be solved with LP-solver

Totally unimodular

	x_{e_1}	x_{e_2}	...	...	x_{e_m}		
1	-1		1	...		=	0
2		-1		...		=	0
.	1			...	1	=	.
.		1		...		=	.
n			-1	...	-1	=	0
e_1	1					≤	u_1
e_2		1				≤	u_2
.			1			≤	.
.						≤	.
e_m					1	≤	u_m

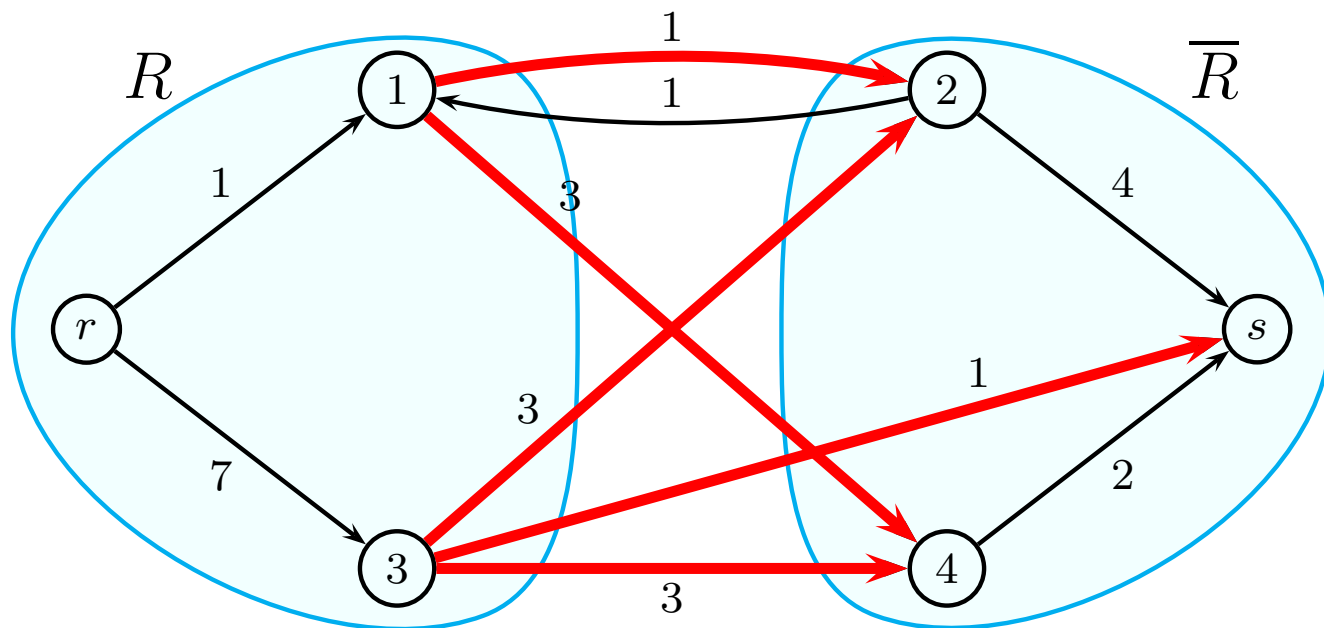
(No flow conservation constraints for node r and s)

Max Flow and Cuts I

- We now consider $R \subset V$. $\delta(R)$ is the set of edges incident from a vertex in R to a vertex in $\bar{R}$. $\delta(R)$ is also called the cut **generated** by R :

$$\delta(R) = \{(i, j) \in E : i \in R, j \in \bar{R}\}$$

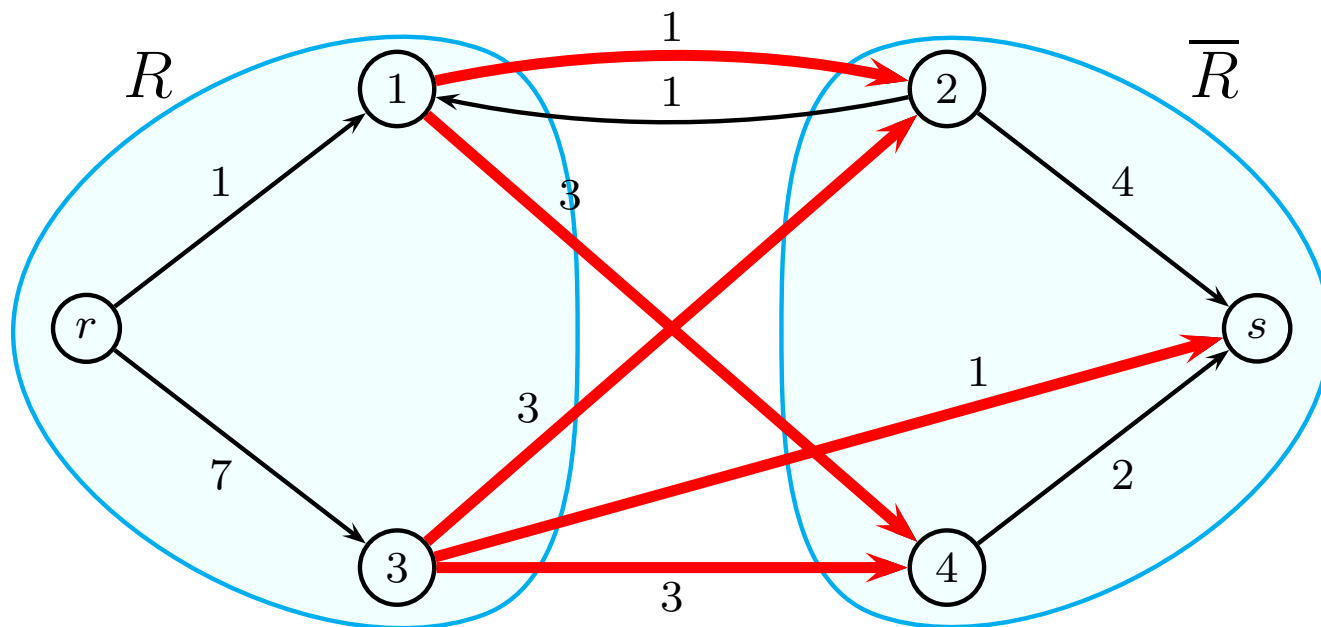
- Note the difference to the cut in an undirected graph. In a directed graph the cut only consists of arcs going from R to $\bar{R}$ and not the opposite direction.



Max Flow and Cuts II

- In an undirected graph we have $\delta(R) = \delta(\overline{R})$, this does not hold here.
- R is an **r,s-cut** if $r \in R$ and $s \notin R$.
- The **capacity** of the cut R is defined as:

$$u(\delta(R)) = \sum_{(i,j) \in E, i \in R, j \in \overline{R}} u_{ij}$$



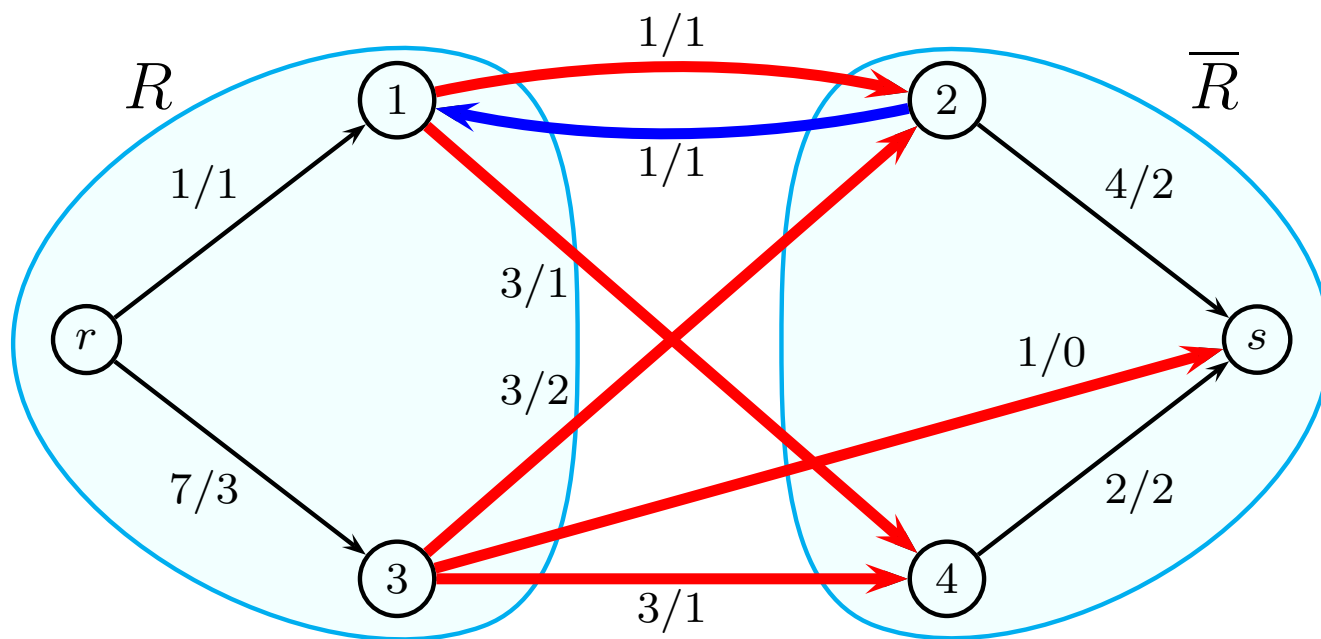
Relationship between flows and cuts

- **Proposition:** For any r, s -cut $\delta(R)$ and any flow x we have:

$$x(\delta(R)) - x(\delta(\bar{R})) = f_x(s)$$

- **Corollary:** For any feasible flow x and any r, s -cut $\delta(R)$, we have:

$$f_x(s) = x(\delta(R)) - x(\delta(\bar{R})) \leq x(\delta(R)) \leq u(\delta(R))$$



The Residual graph

- Suppose that x is a flow in G . The **residual graph** shows *how flow excess can be moved* in G given that the flow x is already present.
- The **residual graph** G_x for G wrt. x is defined by:

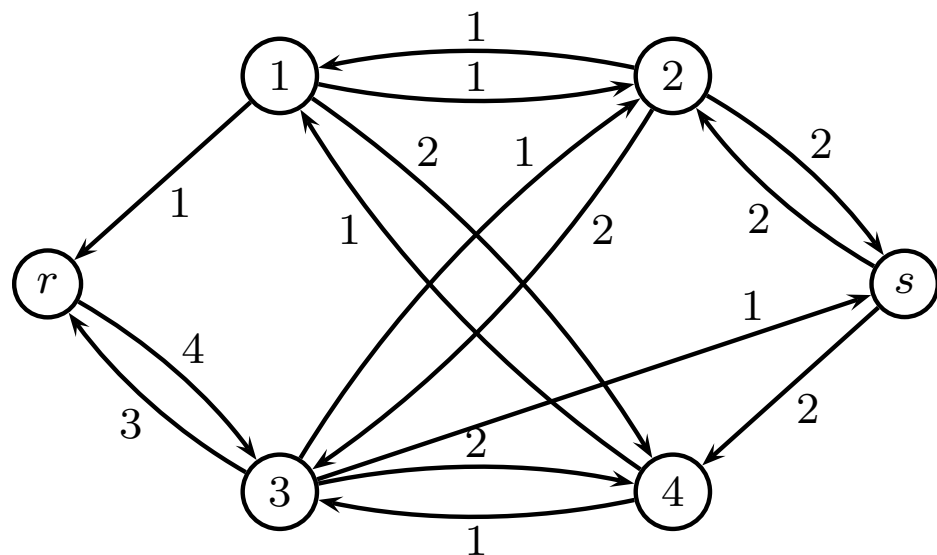
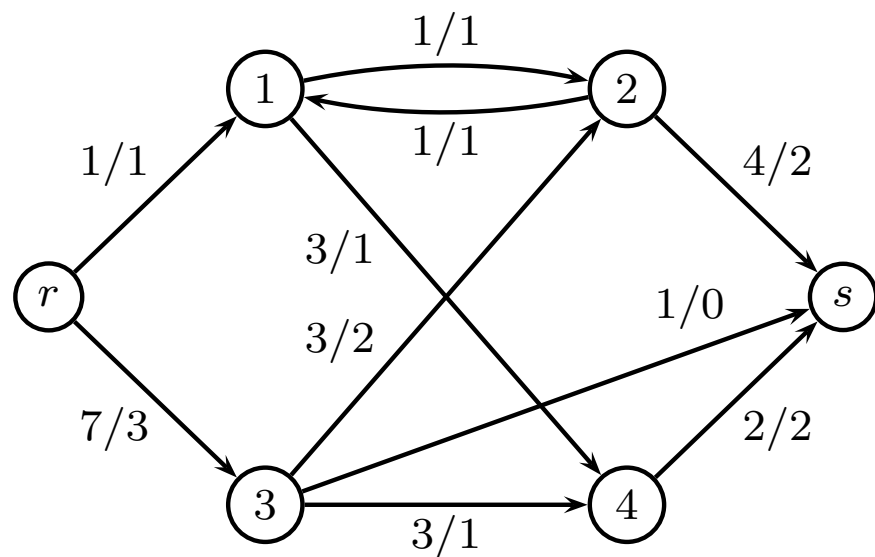
$$\begin{aligned} V(G_x) &= V \\ E(G_x) &= \{(i, j) : (i, j) \in E \wedge x_{ij} < u_{ij}\} \cup \\ &\quad \{(j, i) : (i, j) \in E \wedge x_{ij} > 0\} \end{aligned}$$

Short we write $V_x = V(G_x)$ and $E_x = E(G_x)$

- Some textbooks use the word “auxiliary graph” instead of residual graph.

The Residual graph

Graph with flow of 4, and corresponding residual graph

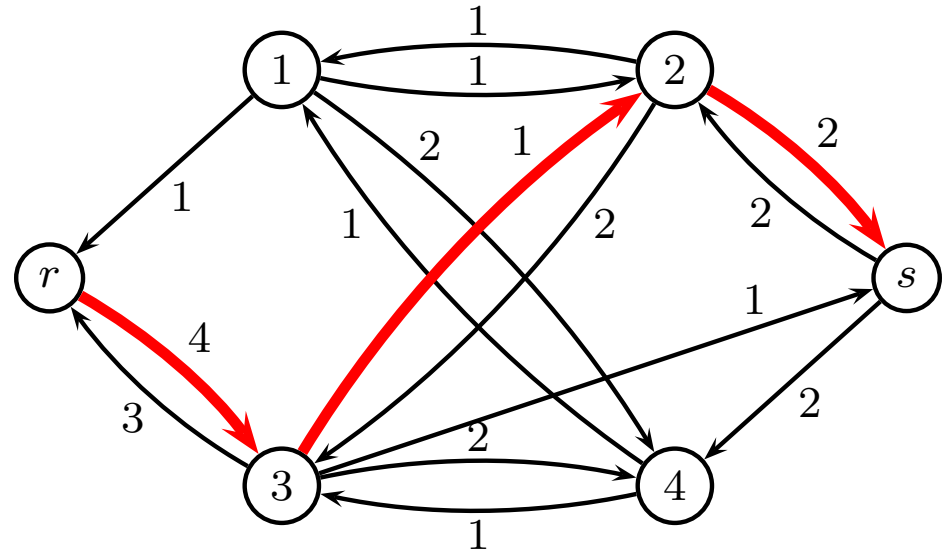
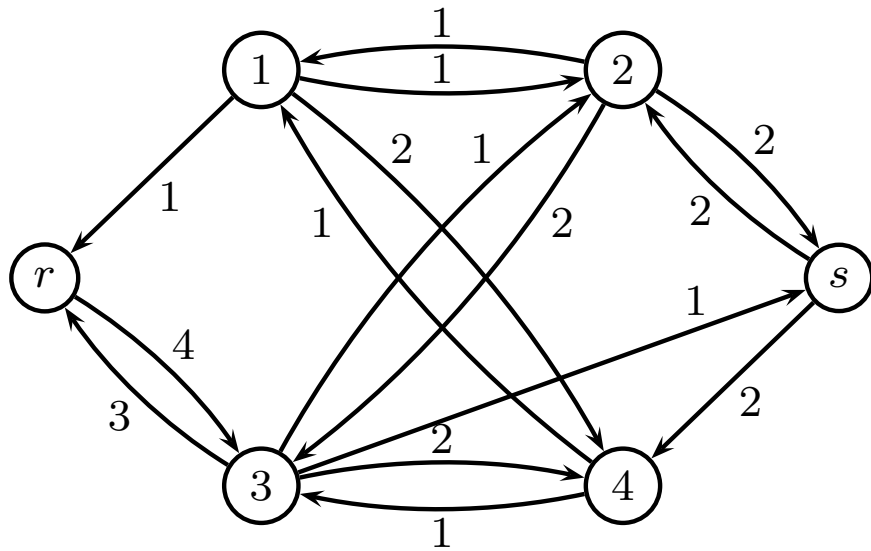


Incrementing and augmenting path

- A path is called *x*-**incrementing** if for every forward arc e $x_e < u_e$ and for every backward arc $x_e > 0$.
- A *x*-incrementing path from r to s is called *x*-**augmenting**.
- Note: this relates to the original graph. In the Residual graph all arcs in a path are forward arcs.

Augmenting path

Residual graph and augmenting path of capacity 1



Basic Approach

1. Build residual graph
2. Find augmenting path
3. If no augmenting path **stop**
4. Find capacity of augmenting path
5. Increase flow along the augmenting path.
6. Go to step 1

The Augmenting Path Algorithm

```
1  $x_{ij} \leftarrow 0$  for all  $(i, j) \in E$ ;  
2 repeat  
3   | construct residual graph  $G_x$ ;  
4   | find an augmenting path  $P$ ;  
5   | if  $s$  is reached then  
6   |   | determine max amount  $K$  of augmentation;  
7   |   | augment  $x$  along the path  $P$  by  $K$ ;  
8 until  $s$  is not reachable from  $r$  in  $G_x$ ;
```

Finding an augmenting path I

- **Input:** The network $G = (V, E, u)$ and the current feasible flow x .
- **Output:** An augmenting path from r to s and the capacity for the flow augmentation.

Initialize: all vertices are unlabeled;

$Q \leftarrow \{r\}, S \leftarrow \emptyset, p[\cdot] \leftarrow 0;$

$u_{\max}[i] \leftarrow +\infty$ for $i \in G$

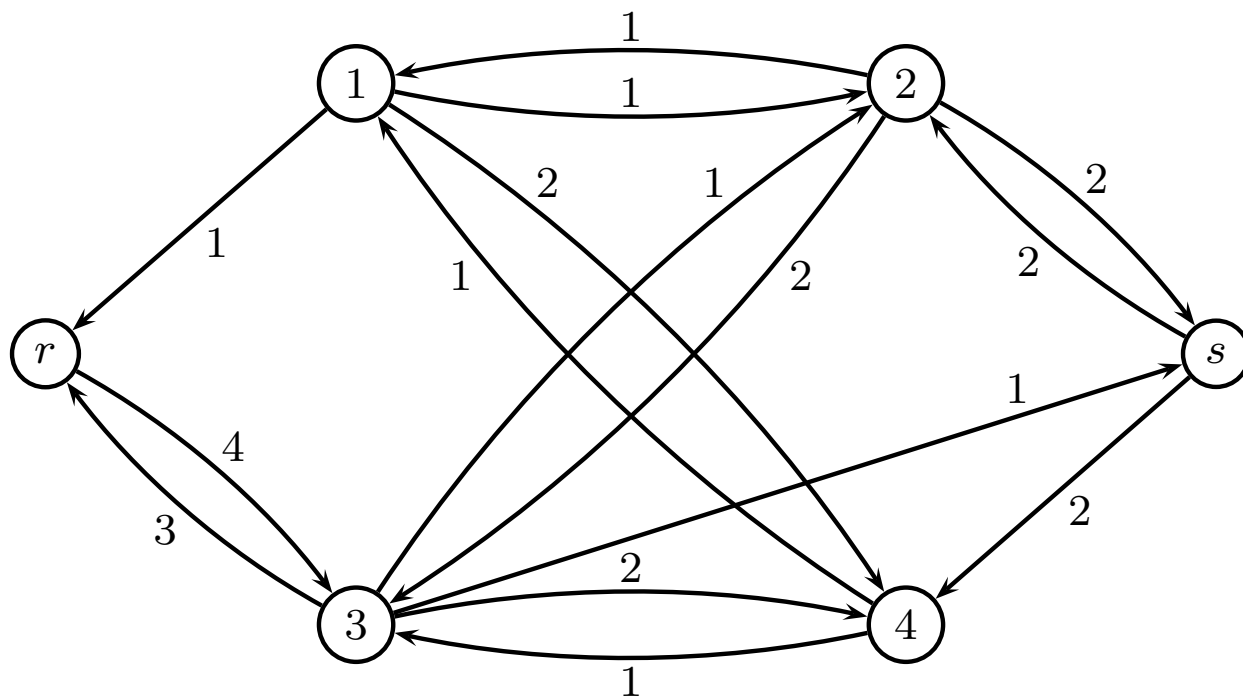
Finding an augmenting path II

```
1 while  $Q \neq \emptyset$  and  $s$  is not labeled yet do  
2   select a  $i \in Q$  and scan  $(i, j) \in E_x$ ;  
3   if  $(i, j) \in E$  and  $j$  unlabeled then  
4     label  $j$ ;  $Q \leftarrow Q \cup \{j\}$ ;  $p[j] \leftarrow i$ ;  
5      $u_{\max}[j] \leftarrow \min\{u_{\max}[i], u_{ij} - x_{ij}\}$ ;  
6   if  $(j, i) \in E$  and  $j$  unlabeled then  
7     label  $j$ ;  $Q \leftarrow Q \cup \{j\}$ ;  $p[j] \leftarrow i$ ;  
8      $u_{\max}[j] \leftarrow \min\{u_{\max}[i], x_{ji}\}$ ;  
9    $Q \leftarrow Q \setminus \{i\}$ ;  $S \leftarrow S \cup \{i\}$ ;
```

If s is labeled then the augmenting path is given by the $p[\cdot]$.
Otherwise no augmenting path exists.

Example

Residual graph

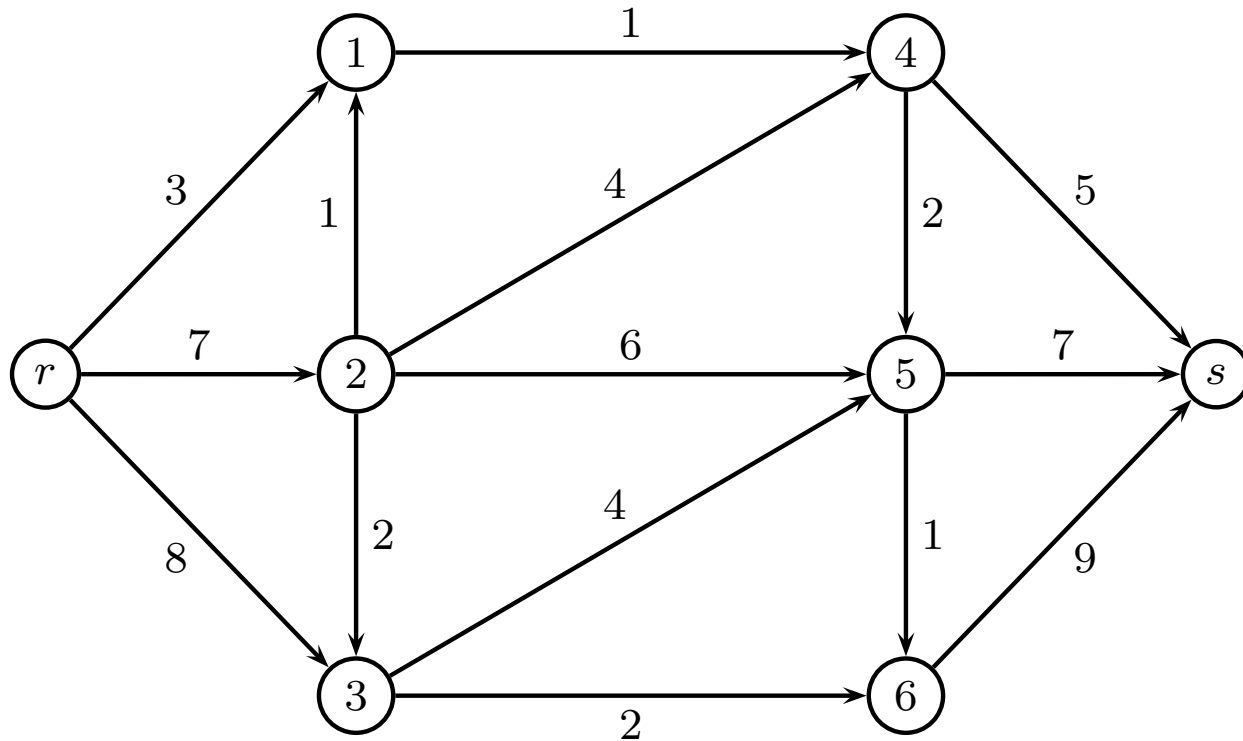


Augmenting a feasible flow x

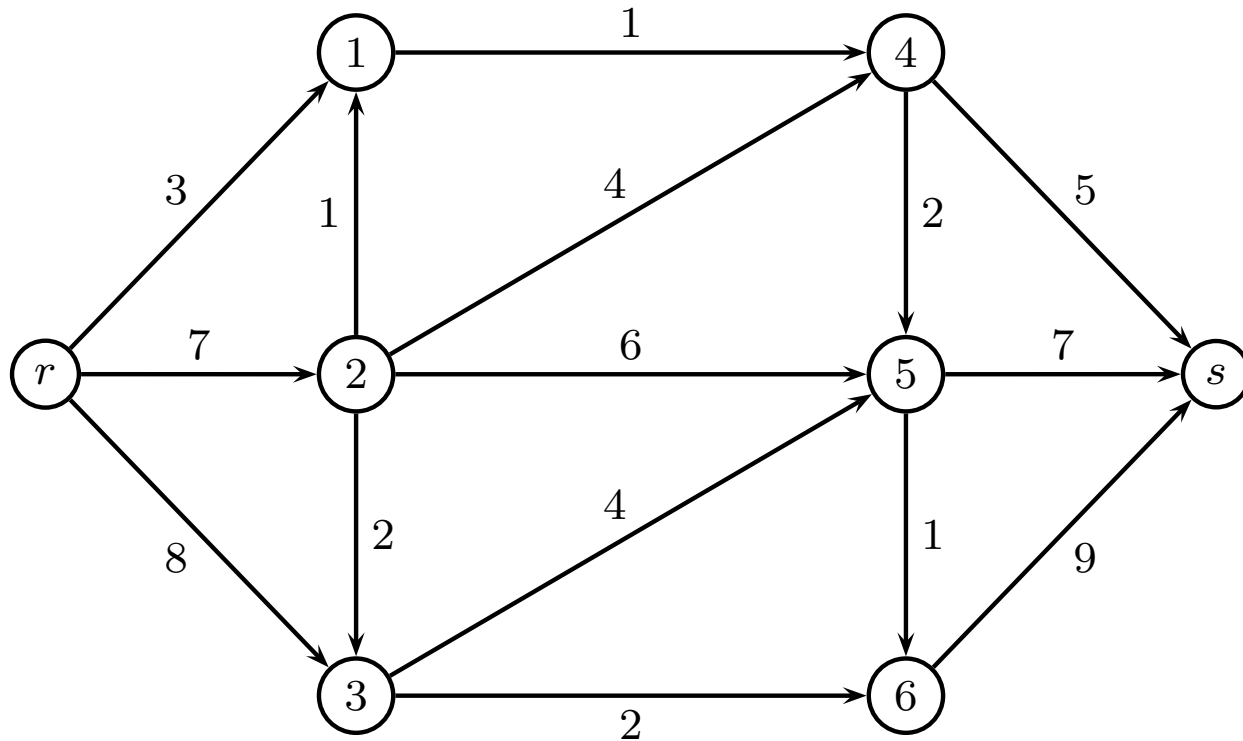
- **Input:** The network $G = (V, E, u)$, the current feasible flow x , an augmenting path P from r to s in G_x and the x -width of P .
- **Output:** An “updated” feasible flow x' .
- An updated flow is achieved by:

for $(i, j) \in P$ $(i, j) \in E : x'_{ij} \leftarrow x_{ij} + u_{\max}[s]$
for $(i, j) \in P$ $(j, i) \in E : x'_{ji} \leftarrow x_{ji} - u_{\max}[s]$
for all other $(i, j) \in E : x'_{ij} \leftarrow x_{ij}$

Example



Example



path $r25s$, capacity 6

path $r3524s$, capacity 4

path $r36s$, capacity 2

path $r14s$, capacity 1

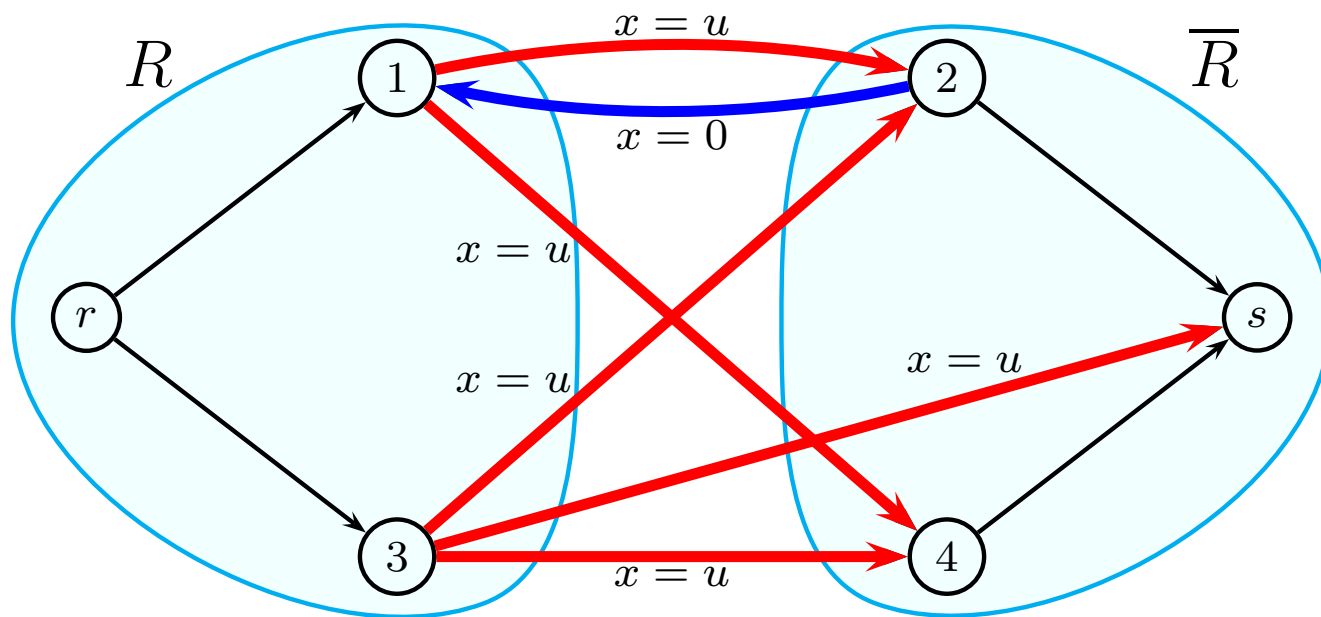
path $r25s$, capacity 1

cut $R = \{r, 1, 3\}$

Max Flow - Min Cut Theorem

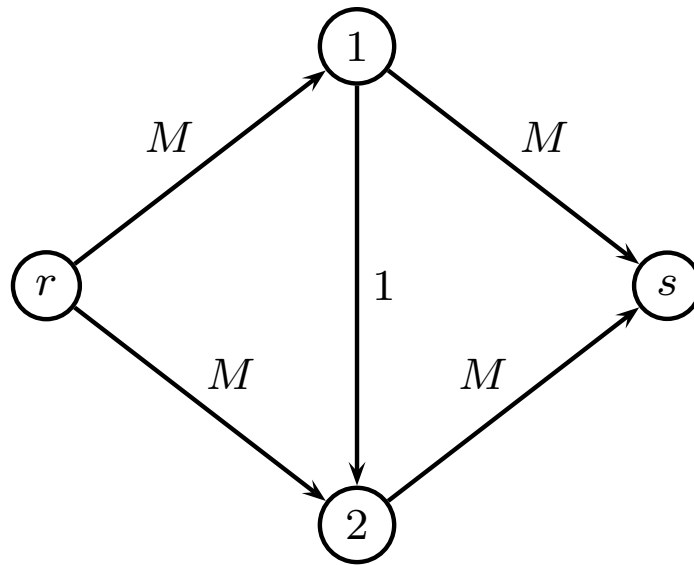
Given a network $G = (V, E, u)$ and a current feasible flow x . The following 3 statements are equivalent:

1. x is a maximum flow in G .
2. A flow augmenting path does not exist (r - s -path in G_x).
3. An (r, s) -cut R exists with capacity equal to the value of x , i.e. $u(R) = f_x(s)$.



Time complexity

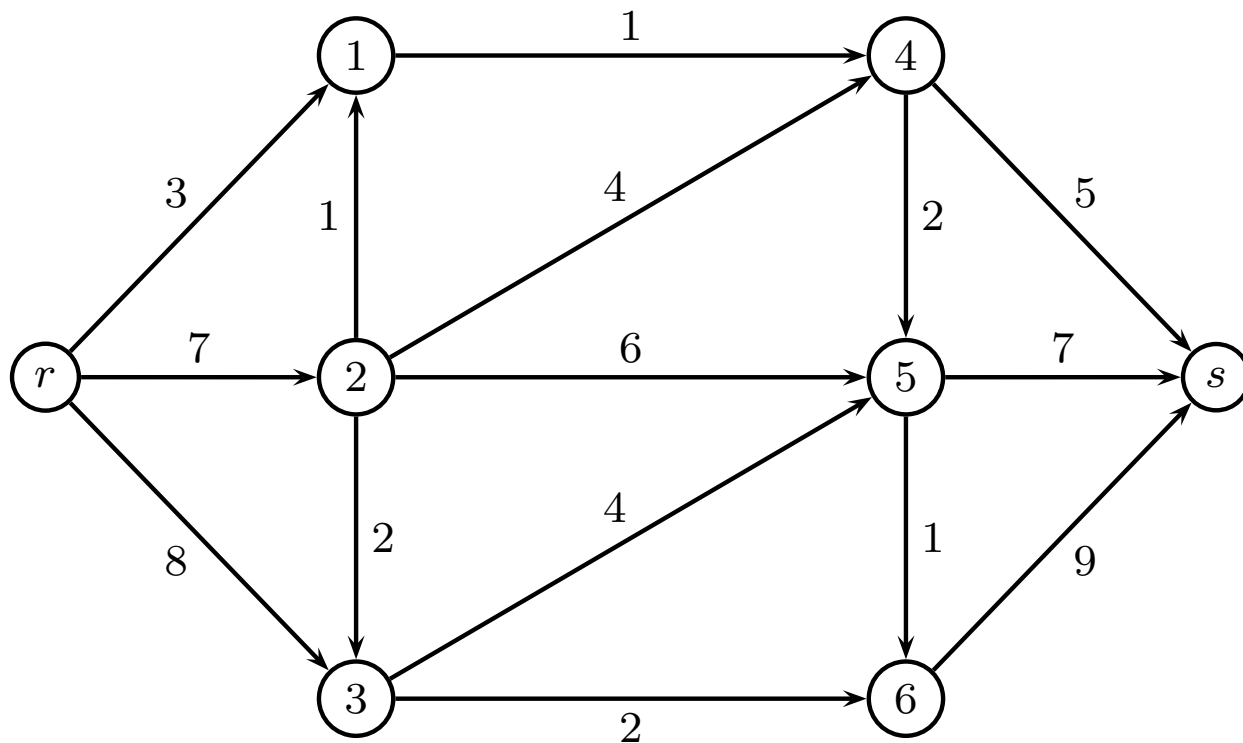
If we choose edge with capacity 1 in each iteration, we use $O(M)$ iterations.



Greedy variant

- In each iteration find the augmenting path with maximum residual capacity
- It can be shown that after $2m$ iterations the algorithm has found a maximum flow, or reduced residual capacity of max capacity augmenting path by a factor at least 2.
- At most $O(m \log U)$ iterations
- How do we find maximum capacity augmenting path?
- Each iteration $O(m)$
- Total running time $O(m^2 \log U)$.

Example



Scaling algorithm

Start with $\Delta = 2^{\log U}$. Repeat

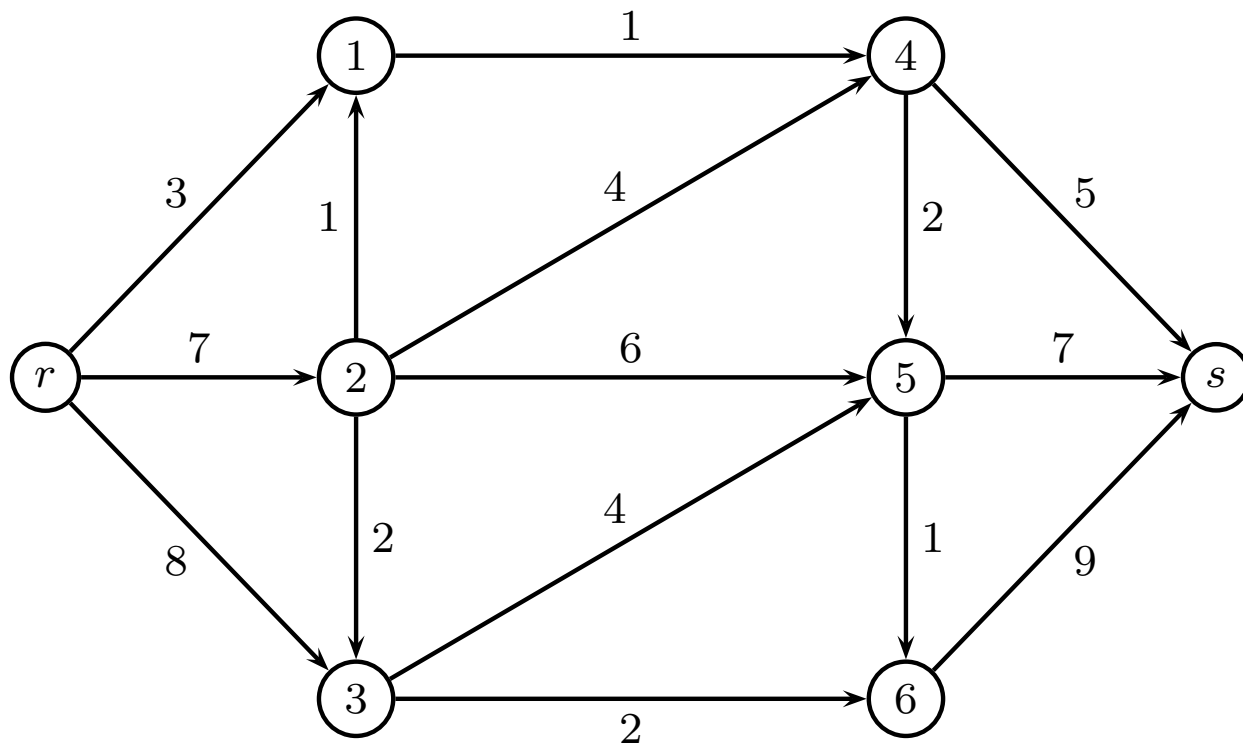
- Scaling phase: Remove all edges with residual capacity below Δ
- Solve maximum flow problem
- $\Delta := \Delta/2$

Since $\Delta = 1$ in last iteration, we find optimal solution

- It can be shown that at most $2m$ iterations for each Δ
- $O(\log U)$ scaling phases
- Running time $O(m^2 \log U)$

Example

Scaling with $\Delta = 4$



Edmonds-Karp variant

- We call an augmenting path from r to s **shortest** if it has the minimum possible number of arcs.
- The augmenting path algorithm with a breadth-first search solves the maximum flow problem in $O(nm^2)$ (Edmonds-Karp, Dinic)
- Breadth-first can easily be established using a queue. (A label correcting algorithm).

Algorithms

Linear programming	??
Ford-Fulkerson algorithm	$O(E \max f)$
Edmonds-Karp algorithm	$O(V E^2)$
Dinitz blocking flow algorithm	$O(V^2 E)$
General push-relabel maximum flow algorithm	$O(V^2 E)$
Push-relabel algorithm with FIFO vertex selection rule	$O(V^3)$
Dinitz blocking flow algorithm with dynamic trees	$O(V E \log(V))$
Push-relabel algorithm with dynamic trees	$O(V E \log(V^2 / E))$
MPM (Malhotra, Pramodh-Kumar and Maheshwari) algorithm	$O(V^3)$
Jim Orlin's + KRT (King, Rao, Tarjan)'s algorithm	$O(V E)$

Modeling

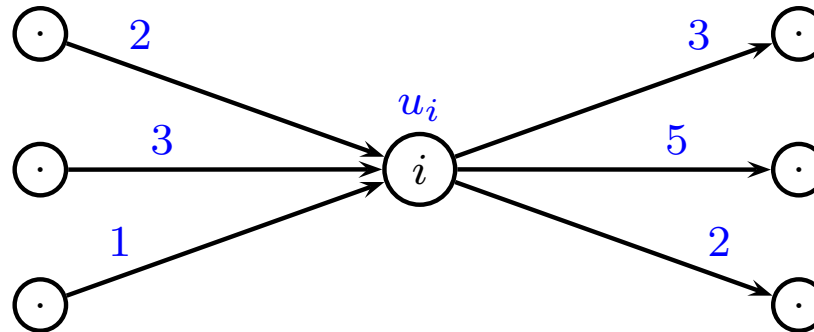
- Capacity on nodes
- Multiple roots and sinks
- Lower bound on edges (important in B&B)
- Circulation problem
- Circulation problem with upper/lower bounds on edges
- Maximum flow with upper/lower bound on edges

Max-flow, capacity on nodes

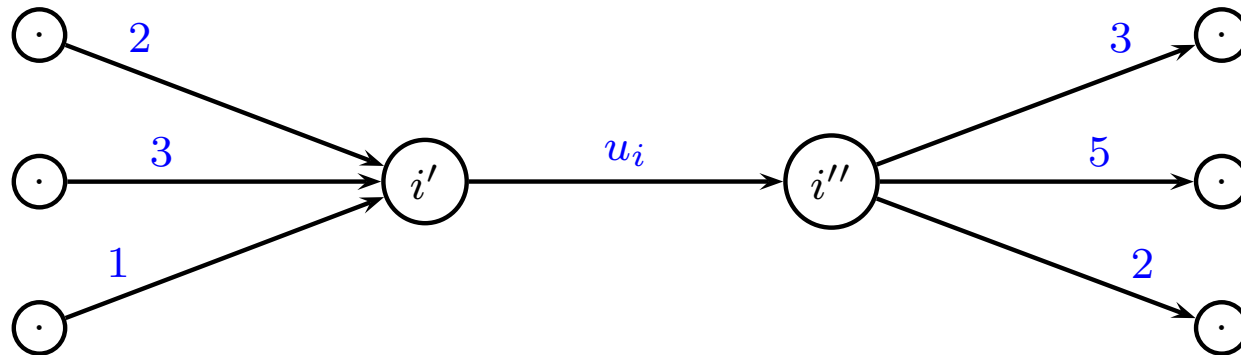
Assume that there is a capacity u_i on node i

- Replace node i with two nodes i' and i''
- Create an arc from i' to i'' with capacity u_i
- Replace every ingoing arc (j, i) with an arc (j, i')
- Replace every outgoing arc (i, j) with an arc (i'', j)

Max-flow, capacity on nodes



becomes



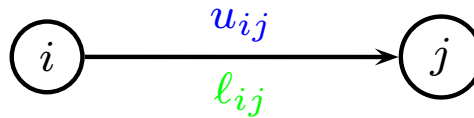
Max-flow, multiple roots and sinks

Max-flow must have *one* root and sink.
(otherwise augmenting path not well-defined)

- Create super-root r^*
- Create super-sink s^*
- Connect super-root r^* to roots with infinite capacity
- Connect sinks to super-sink s^* with infinite capacity

Max-flow, lower bound on edges

Assume we have lower bound ℓ_{ij} on an edge (i, j)



Linear programming can handle lower bounds

Difficult to ensure bounds using augmenting path method
But we can do it by first studying circulation problem

Circulation problem

No objective function, just seeking feasible solution
Multiple roots and sinks, with demand

If a node has positive demand $d_i > 0$ it is a *sink*

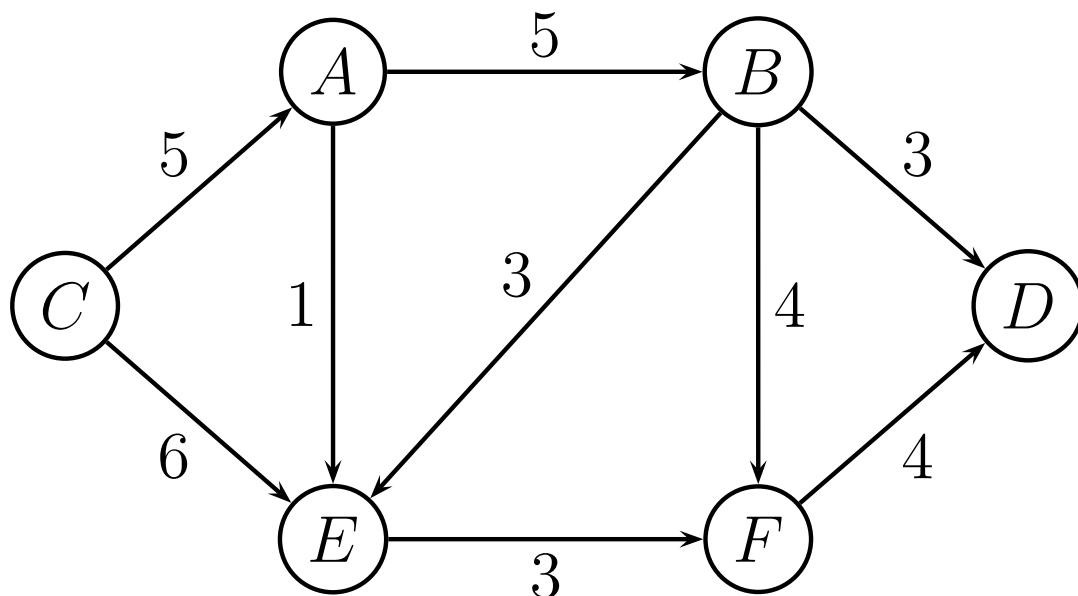
If a node has negative demand $d_i < 0$ it is a *source*

Obviously, must have $\sum_{i \in V} d_i = 0$

Transformation to ordinary max-flow

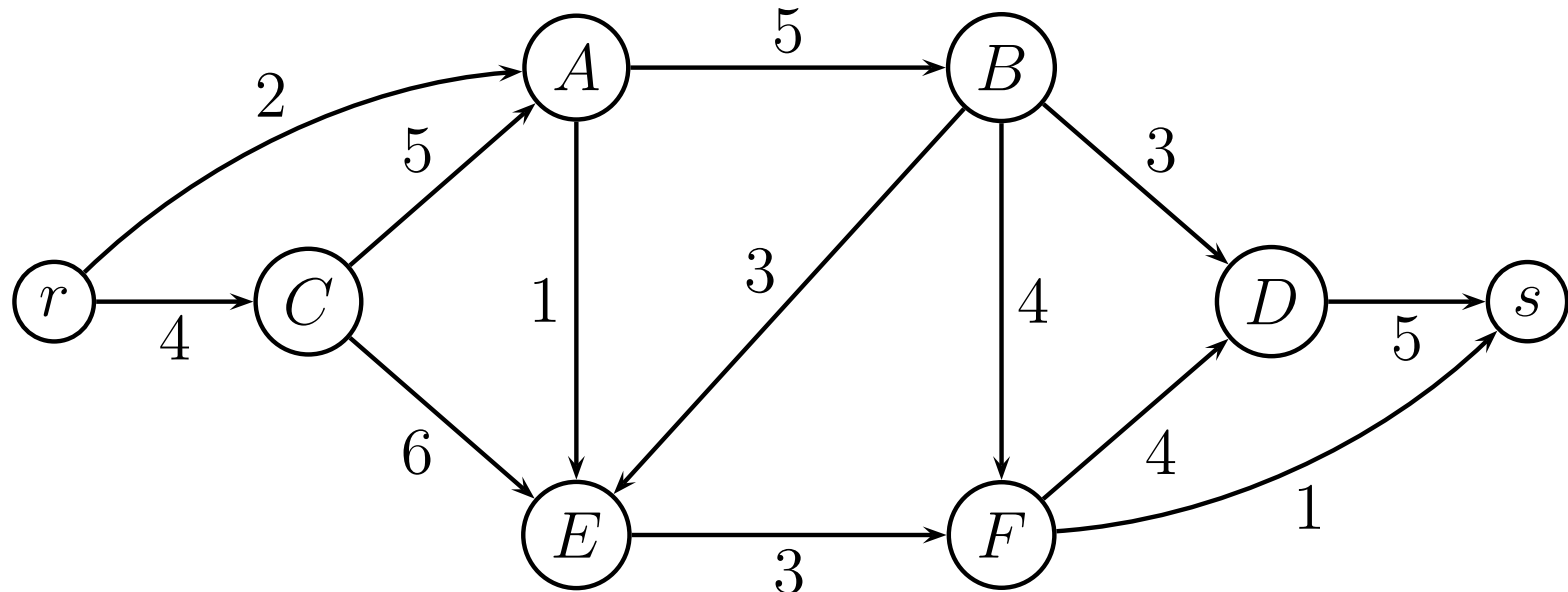
- Create super-root r^*
- Create super-sink s^*
- Connect super-root r^* to sources i with capacity $-d_i$
- Connect sinks i to super-sink s^* with capacity d_i

Circulation problem, example



node i	A	B	C	D	E	F
demand d_i	-2	0	-4	5	0	1

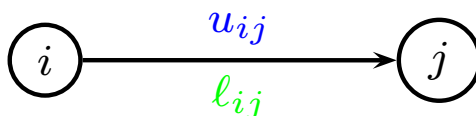
Circulation problem, example



If a max-flow solution of value $2 + 4$ exists, circulation problem is *feasible*
(all edges from r need to be filled)

Circulation, lower bound on edges

Assume lower bound ℓ_{ij} , capacity u_{ij} on edge (i, j)



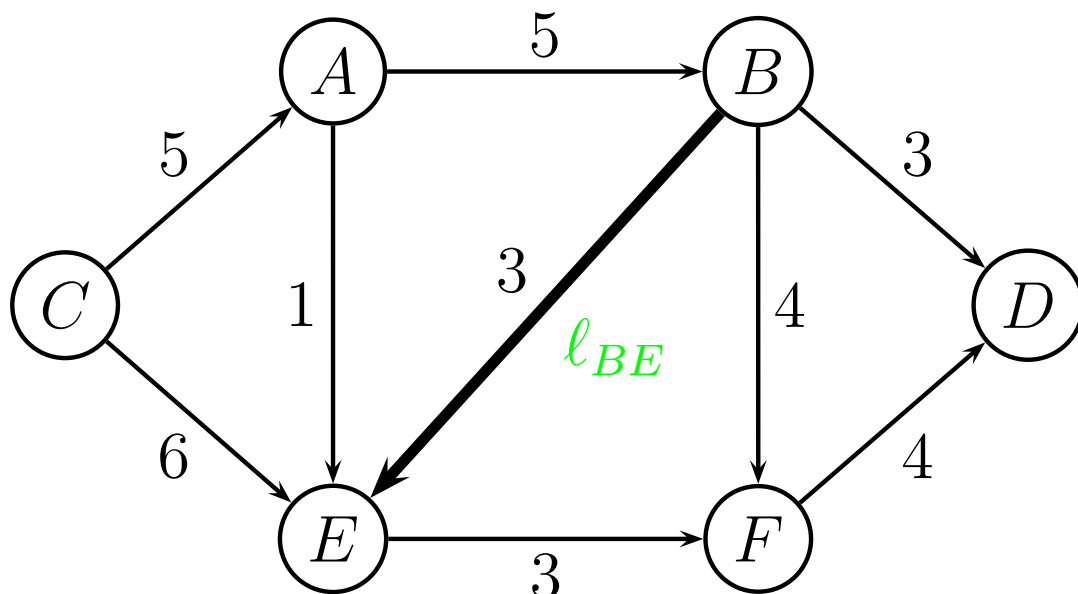
- Decrease capacity $u_{ij} := u_{ij} - \ell_{ij}$
- Increase demand $d_i := d_i + \ell_{ij}$
- Decrease demand $d_j := d_j - \ell_{ij}$

Next, use previous slide for circulation problem

After the circulation problem is solved, add ℓ_{ij} to x_{ij}

Circulation, lower bound, example

Lower bound on edge (B, E) of value $\ell_{BE} = 2$.



node i	A	B	C	D	E	F
demand d_i	-2	0	-4	5	0	1

Circulation, lower bound, example

Lower bound on edge (B, E) of value $\ell_{BE} = 2$.

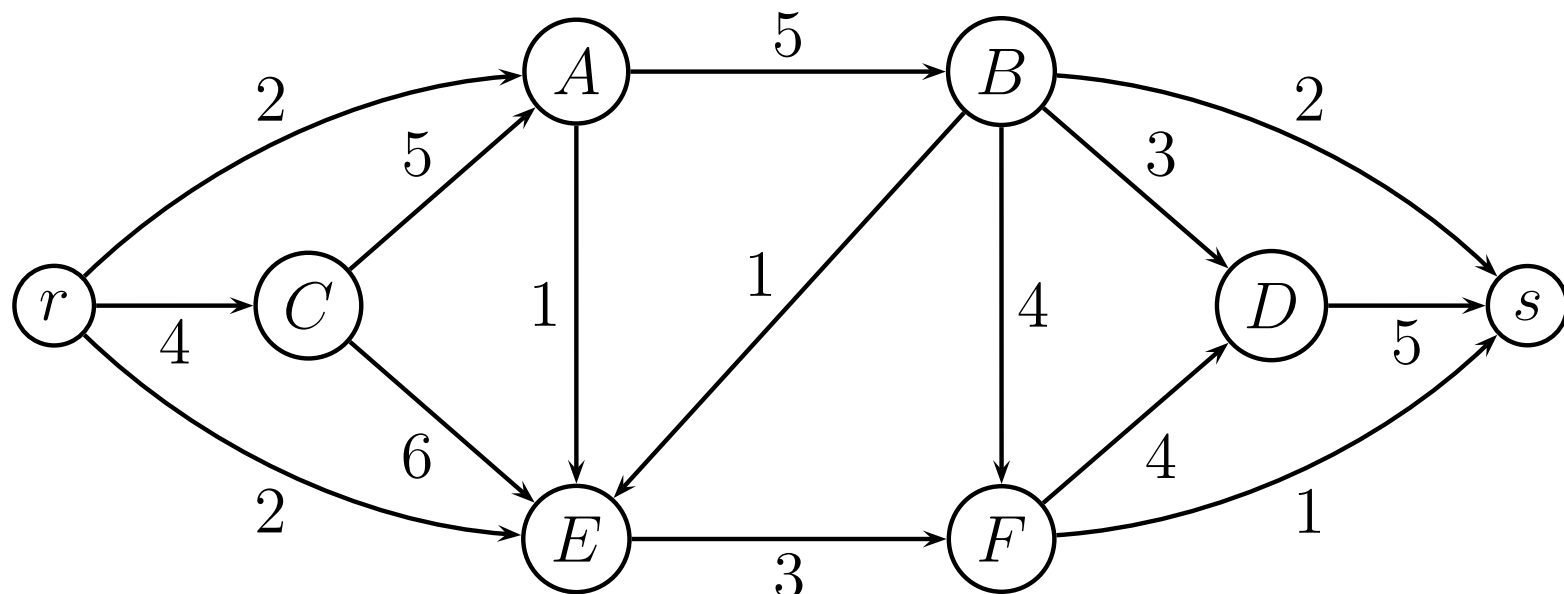
- Reduce the capacity of edge (B, E) from 3 to 1.
- Increase demand at node B from 0 to 2.
- Decrease demand at node E from 0 to -2.

node i	A	B	C	D	E	F
demand d_i	-2	2	-4	5	-2	1

Next, formulate as ordinary max-flow problem

After the circulation problem is solved, add ℓ_{BE} to x_{BE}

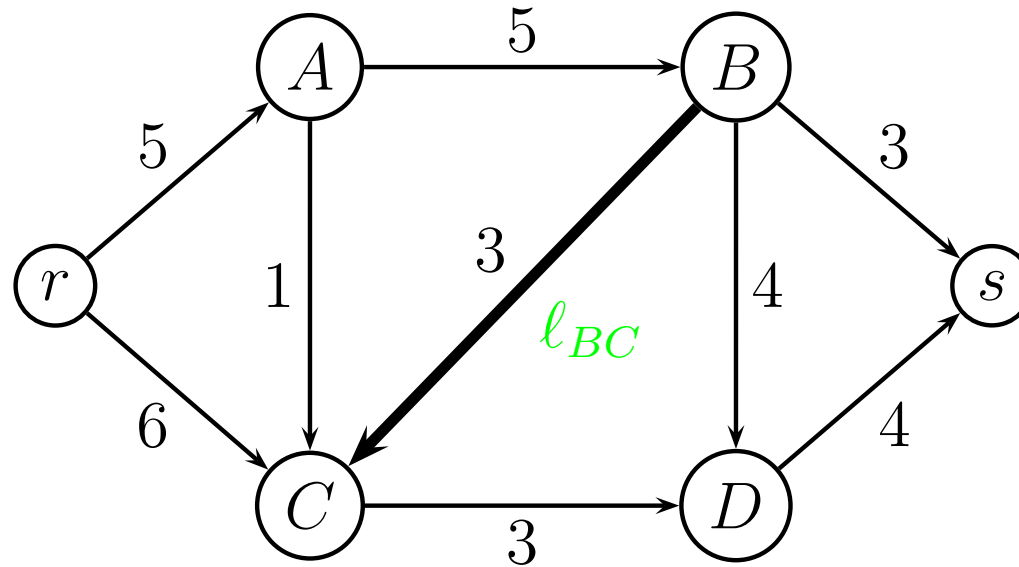
Circulation, lower bound, example



If a max-flow solution of value $2 + 4 + 2$ exists, circulation problem is *feasible*
(all edges from r need to be filled)

Maximum flow with lower bound(s)

Example: Lower bound on edge (B, C)



- Guess optimal flow z . Then we get circulation problem

node i	r	s	A	B	C	D
demand d_i	$-z$	z	0	0	0	0

- Transform circulation problem with lower bounds to ordinary max-flow problem
- Use binary search to find correct value of z

Maximum flow with lower bound(s)

Running time

- Let u be an upper bound on maximum flow problem (e.g. sum of capacities from r)
- We need to solve a maximum-flow problem $\log u$ times
- This is a polynomial algorithm

The Max Flow Problem

Push-relabel (preflow-push)

David Pisinger

`pisinger@man.dtu.dk`

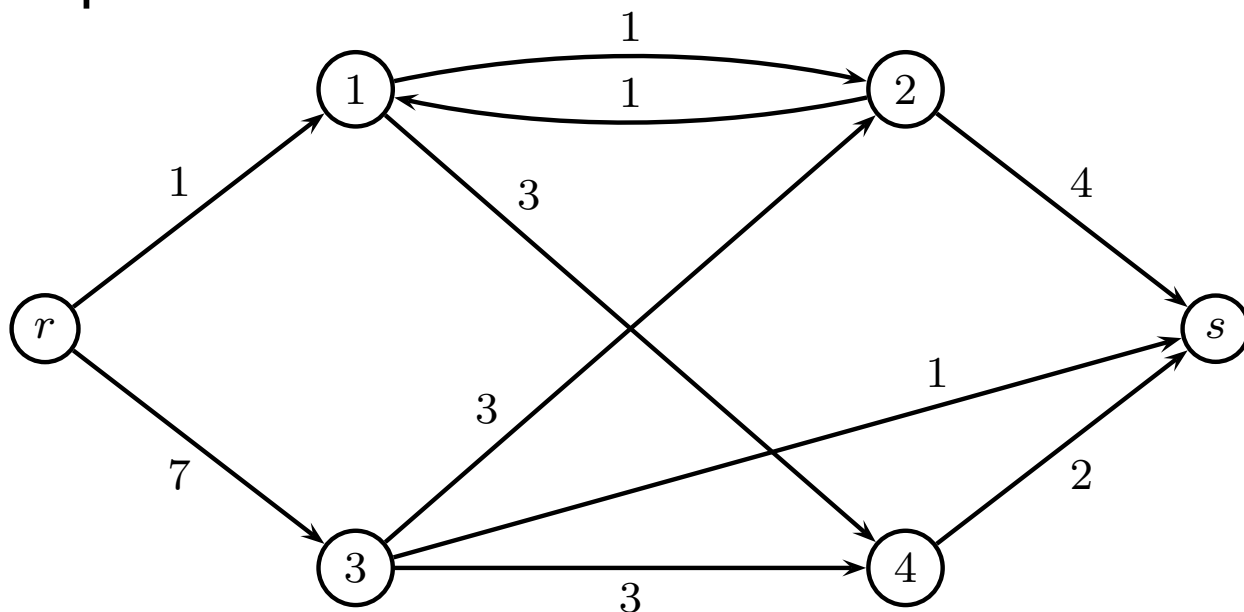
DTU Management

Technical University of Denmark

© Copyright David Pisinger. May not be distributed

The Push-relabel method (preflow push)

- Consider again a directed graph $G = (V, E)$, in which each edge e has a capacity $u_e \in \mathbb{R}_+$.
- Two “special” vertices r and s are given; these are called resp. the **source** and the **sink**.



"Dual approach". Start from better-than optimal solution. Search for feasible solution.

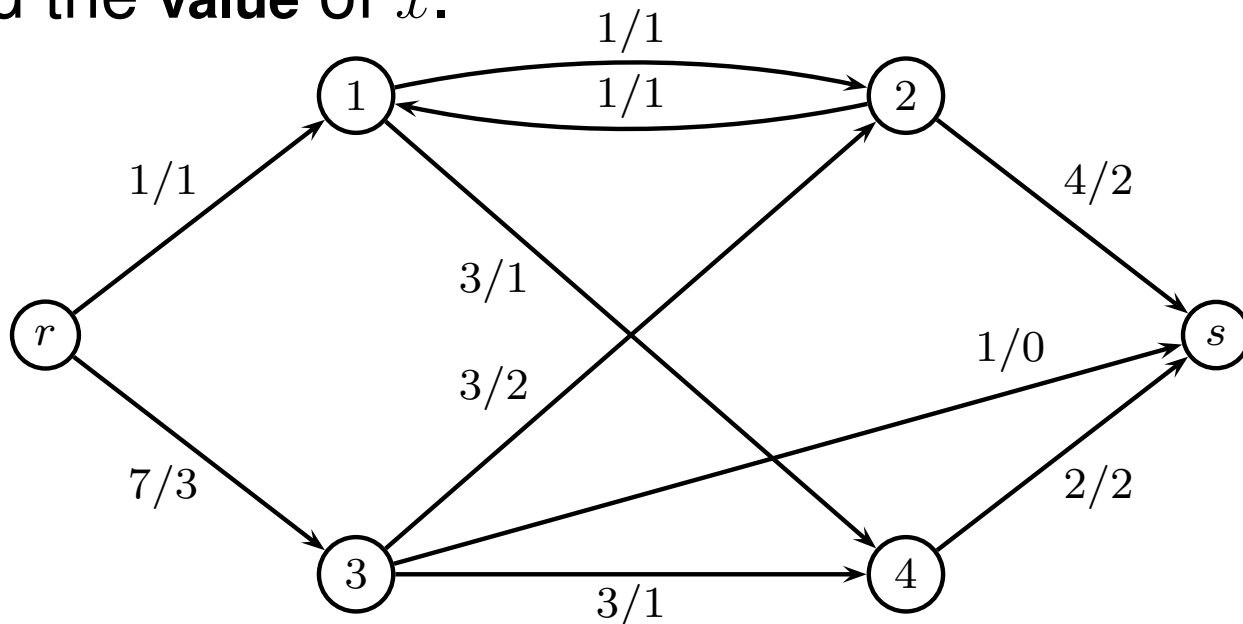
Flow

- A **flow** in G is a function $x : E \rightarrow \mathbb{R}_+$ satisfying:

$$\forall i \in V \setminus \{r, s\} : \sum_{(j,i) \in E} x_{ji} - \sum_{(i,j) \in E} x_{ij} = 0$$

$$\forall (i, j) \in E : 0 \leq x_{ij} \leq u_{ij}$$

- For a flow we define: $f_x(i) = \sum_{(j,i) \in E} x_{ji} - \sum_{(i,j) \in E} x_{ij}$
- $f_x(i)$ is the **net flow into** i or the **excess for** x **in** i . $f_x(s)$ is called the **value of** x .



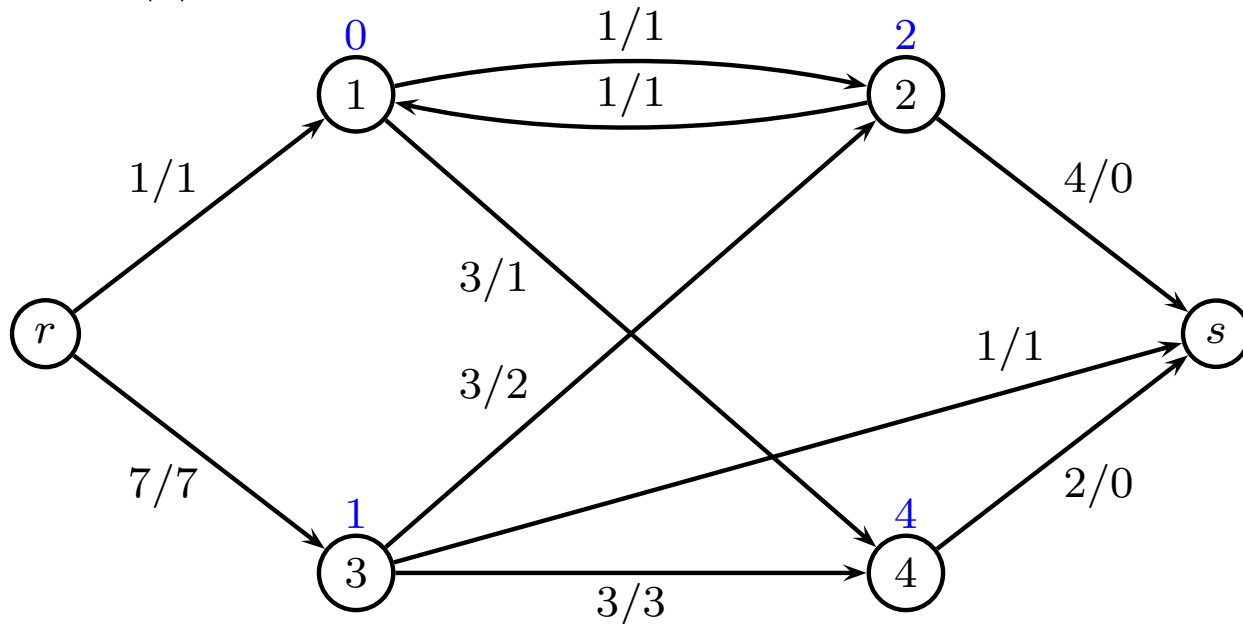
Preflow

- A preflow in G is a function $x : E \rightarrow \mathbb{R}_+$ satisfying:

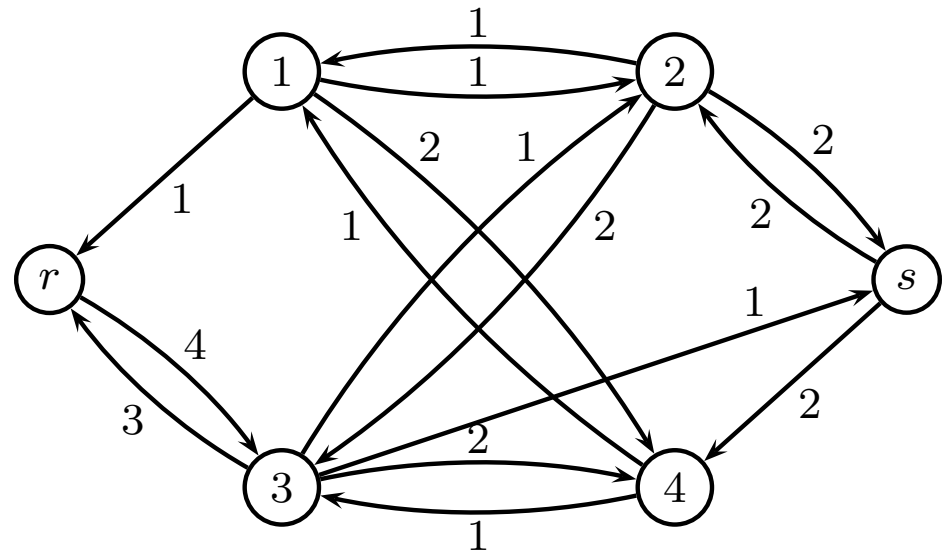
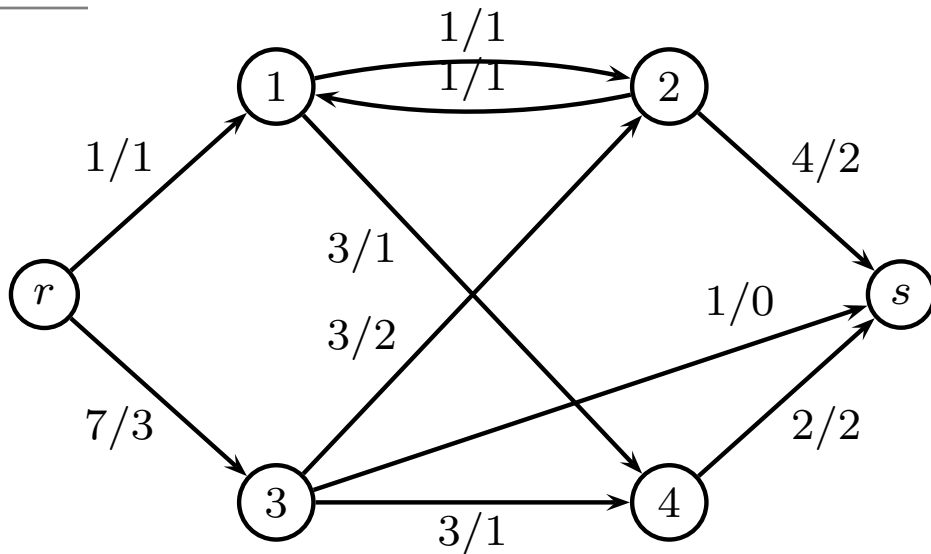
$$\forall i \in V \setminus \{r, s\} : \sum_{(j,i) \in E} x_{ji} - \sum_{(i,j) \in E} x_{ij} \geq 0$$

$$\forall (i,j) \in E : 0 \leq x_{ij} \leq u_{ij}$$

- In other words: for any vertex i except r and s , the flow excess $f_x(i)$ is non-negative - “more runs into i than out of i ”. If $f_x(i) > 0$ the i is called **active**.

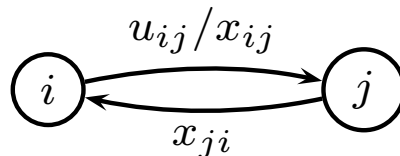


The residual graph



- The Residual graph G_x remains defined as previously.
- Let $\tilde{u}_{ij}$ be the “residual capacity” of the arc (i, j) , that is, the total flow we can push from i to j is

$$\tilde{u}_{ij} = u_{ij} - x_{ij} + x_{ji}.$$

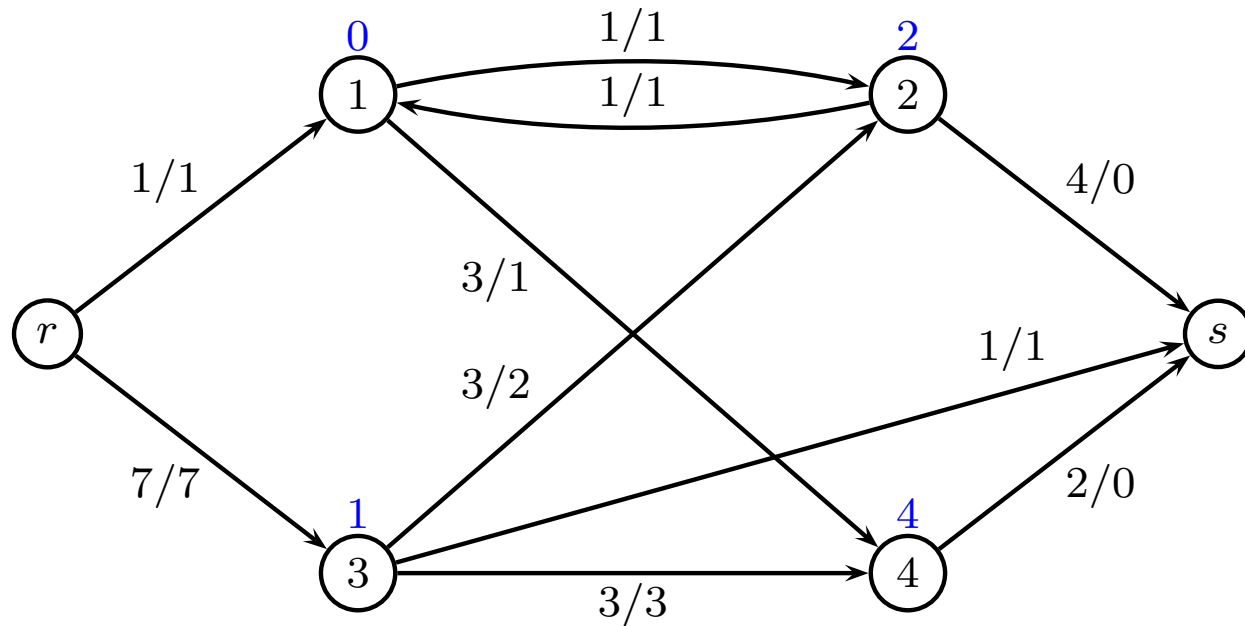


Push operation

- Looking at a
 1. feasible preflow,
 2. an edge (i, j) with $\tilde{u}_{ij} > 0$ and $f_x(i) > 0$pushing up to $\epsilon = \min\{\tilde{u}_{ij}, f_x(i)\}$ will result in a new feasible preflow.
- Notice: A preflow with no active nodes is a flow.

Preflow:
$$f_x(i) = \sum x_{ji} - \sum x_{ij} \geq 0 \quad \forall i \in V \setminus \{r, s\}$$
$$0 \leq x_{ij} \leq u_{ij} \quad \forall (i, j) \in E$$

A Push Operation



- node 2: push 2 units forward – limited by $f_x(i)$
- node 4: push 2 units forward – limited by $\tilde{u}_{ij}$

General idea

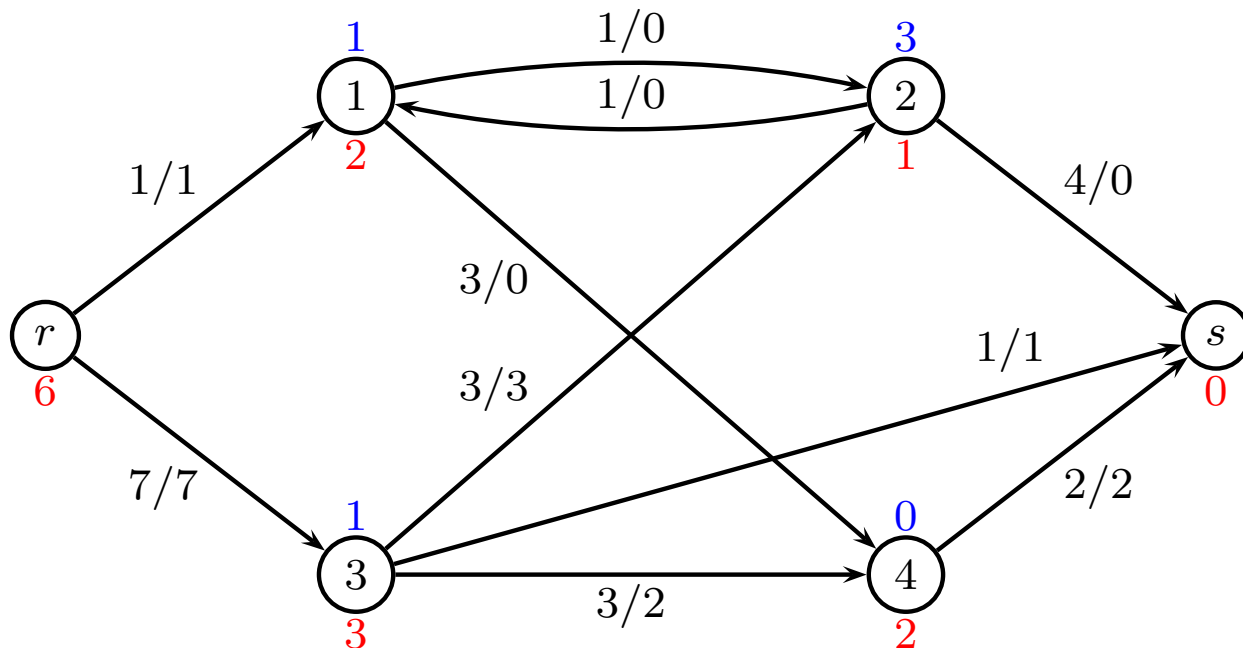
- The general idea is to push as much flow as possible from the source towards the sink.
- However when no more flow can be pushed towards the sink s , **and** there are still active nodes we push the excess back toward the source r . This restores the balance of flow.
- The challenge is to “control” the push operations.

Labelling

- In order to control the direction of the pushes we introduce estimates on the distances in G_x .
- Valid labelling** $d[i] \in \mathbb{Z}$ of the vertices in V wrt. a preflow x

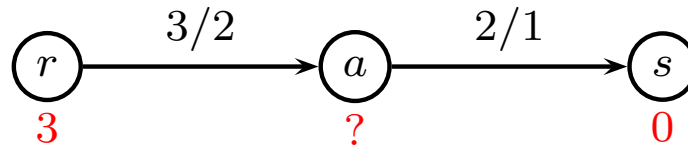
$$d[r] = n, \quad d[s] = 0, \quad \forall (i, j) \in E_x : d[i] \leq d[j] + 1$$

- $d[i]$ is a lower bound on the number of edges in a path from i to s in G_x , if such a path exists.



Preflows vs. valid labellings

- Not all feasible preflows admits a valid labelling.

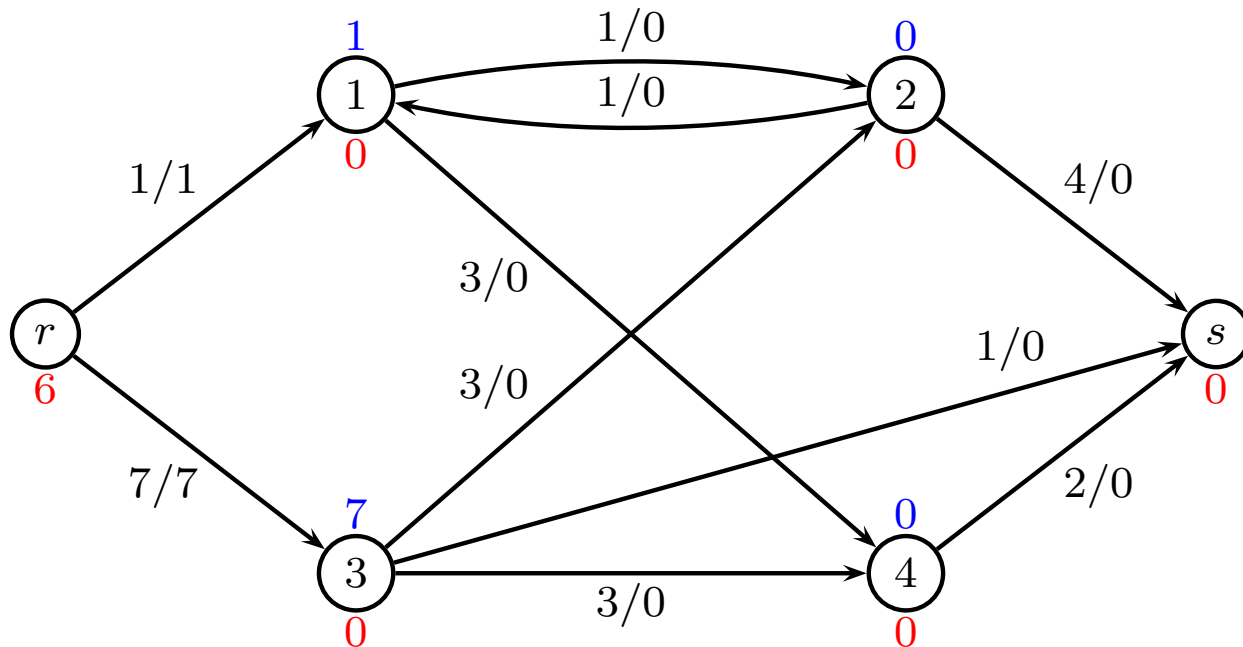


$$d[r] \leq d[a] + 1 \quad d[a] \leq d[s] + 1 \quad \Rightarrow \quad d[r] \leq d[s] + 2$$

Initialize

- It is easy to construct a feasible preflow and a valid labelling for it: (called **initialize** x, d)

1. set $x_e \leftarrow u_e$ for all outgoing arcs of r
2. set $x_e \leftarrow 0$ for all remaining arcs.
3. set $d[r] \leftarrow n$ and $d[i] \leftarrow 0$ otherwise



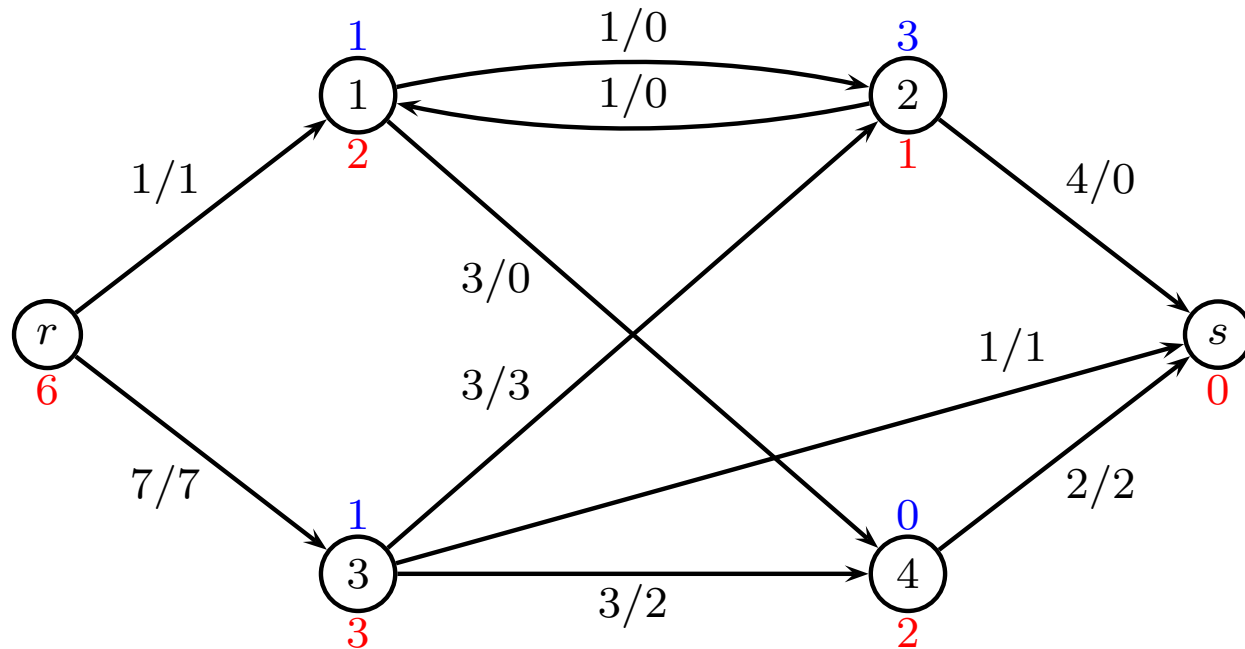
Valid labeling and saturated cut

A valid labelling for a preflow implies a “saturated cut”

Proof: Labels $d[i] \in \mathbb{Z}$ but $n = |V|$ nodes.

Exists $0 < k < n$ such that $d[i] \neq k$ for all $i \in V$.

$$R = \{i \in V : d[i] > k\} \quad \bar{R} = \{i \in V : d[i] < k\}$$



$$d[r] = n, \quad d[s] = 0, \quad \forall (i, j) \in E_x : d[i] \leq d[j] + 1$$

Preflow vs. maximum flow

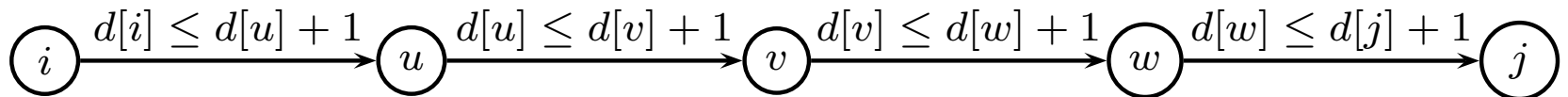
- **Corollary:** If a feasible flow x has a valid labelling, then x is a maximum flow.
- This gives us a possible termination condition for an algorithm.
- Therefore a push-relabel algorithm maintains a feasible preflow and a valid labelling (and a saturated cut) and terminates when the preflow becomes a flow.
- Note the duality with the augmenting path algorithm, which maintains a feasible flow and terminates when a cut becomes saturated.

Approx. to distances

- Let $d_x(i, j)$ denote the number of arcs in a shortest directed path from i to j in G_x .
- With the definition d we can prove that for any feasible preflow x and any valid labelling d for x we have

$$d_x(i, j) \geq d[i] - d[j]$$

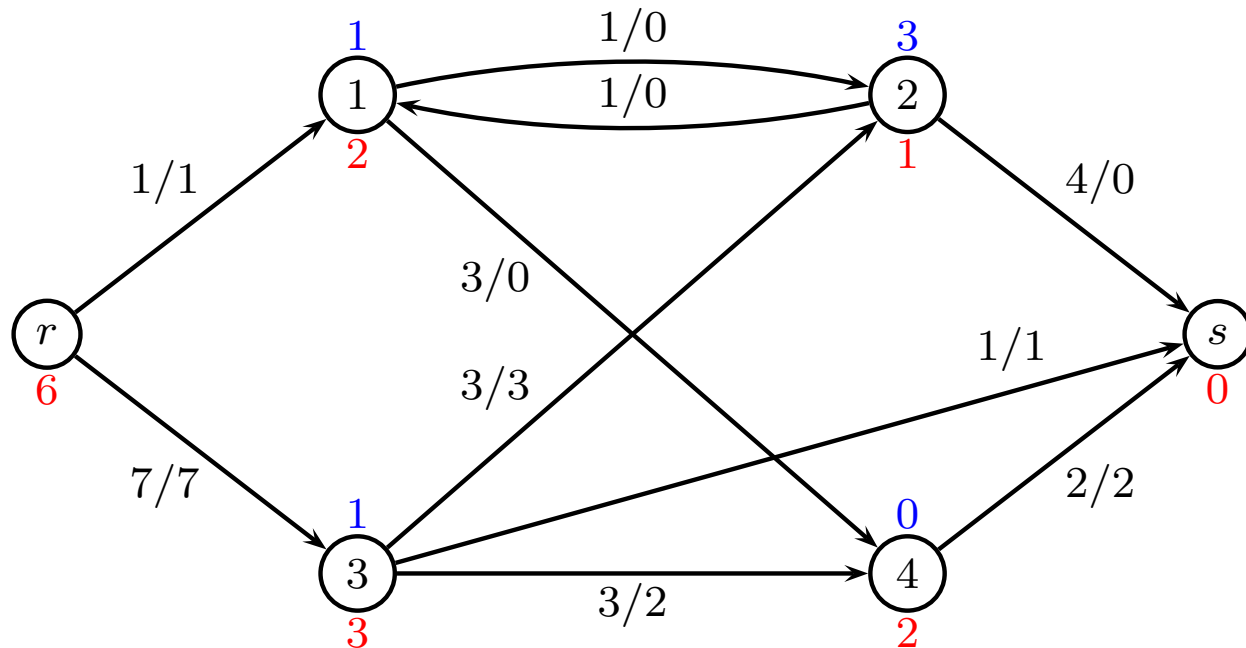
- We try to push flow towards nodes j having $d[j] < d[i]$, since such nodes are estimated to be closer to the ultimate destination.



$$d[i] \leq d[j] + d_x(i, j)$$

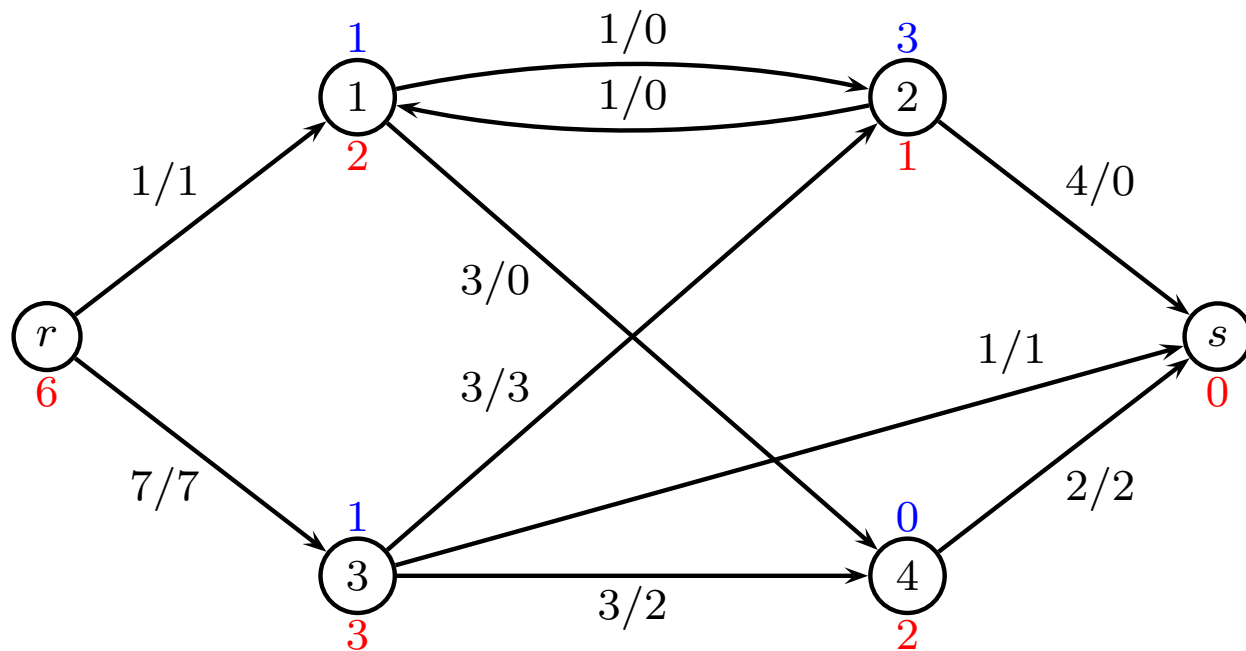
Approx. to distances: special cases

- $d[i]$ is a lower bound on the distance from i to s .
 $d_x(i, s) \geq d[i] - d[s] = d[i]$.
- $d[i] - n$ is a lower bound on the distance from i to r .
 $d_x(i, r) \geq d[i] - d[r] = d[i] - n$.
- If $d[i] \geq n$ this means that there is no path from i to s .
- Admissible edges can be used to push flow excess towards s (or, if s is not reachable, towards r).



Admissible edge

- An edge (i, j) is called **admissible** wrt. a preflow x and a valid labelling d , if $(i, j) \in E_x$, i is active and $d[i] = d[j] + 1$



- Edges $(1, 2)$, $(2, s)$, $(3, 4)$ are admissible

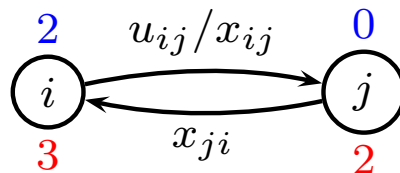
The Push operation

- Consider an **admissible** edge (i, j) .
- Calculate the amount of flow, which can be pushed to j

$$\min \{ f_x(i), \tilde{u}_{ij} \}$$

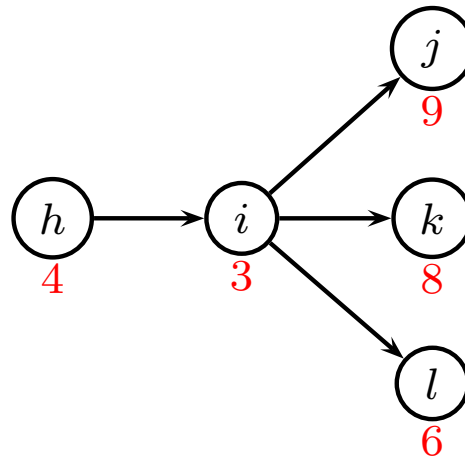
- Push this by first reducing x_{ji} as much as possible and then increasing x_{ij} as much as possible.

$$\tilde{u}_{ij} = u_{ij} - x_{ij} + x_{ji}$$



The Relabel operation

- No more admissible edges and a node i is still active?
- **Relabel**
 - Consider an active vertex i , for which **no** edge $(i, j) \in E_x$ with $d[i] = d[j] + 1$.
 - Increase $d[i]$ to $\min\{d[j] + 1 : (i, j) \in E_x\}$ - resulting in a new valid labelling.
- This will not violate the validity of the labelling.



$$d[r] = n, \quad d[s] = 0, \quad \forall (i, j) \in E_x : d[i] \leq d[j] + 1$$

The Preflow-Push Algorithm

Initialize x, d

while x is not a flow (i.e. there are active nodes)

if there exists an admissible arc (i, j)

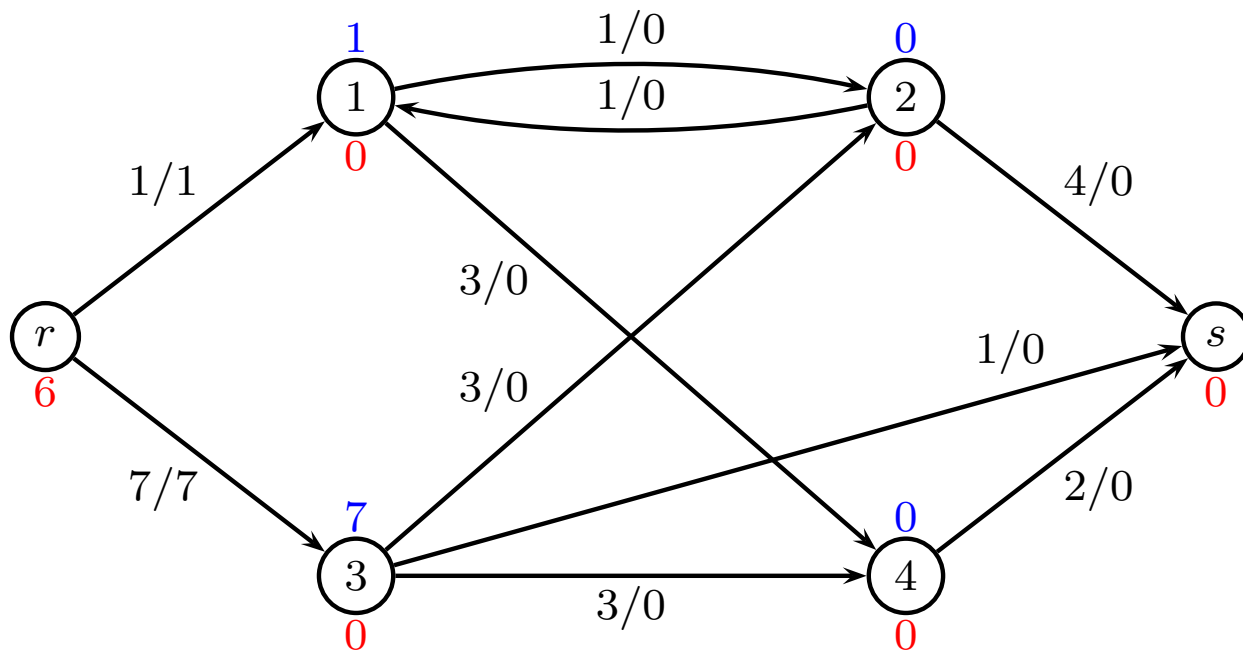
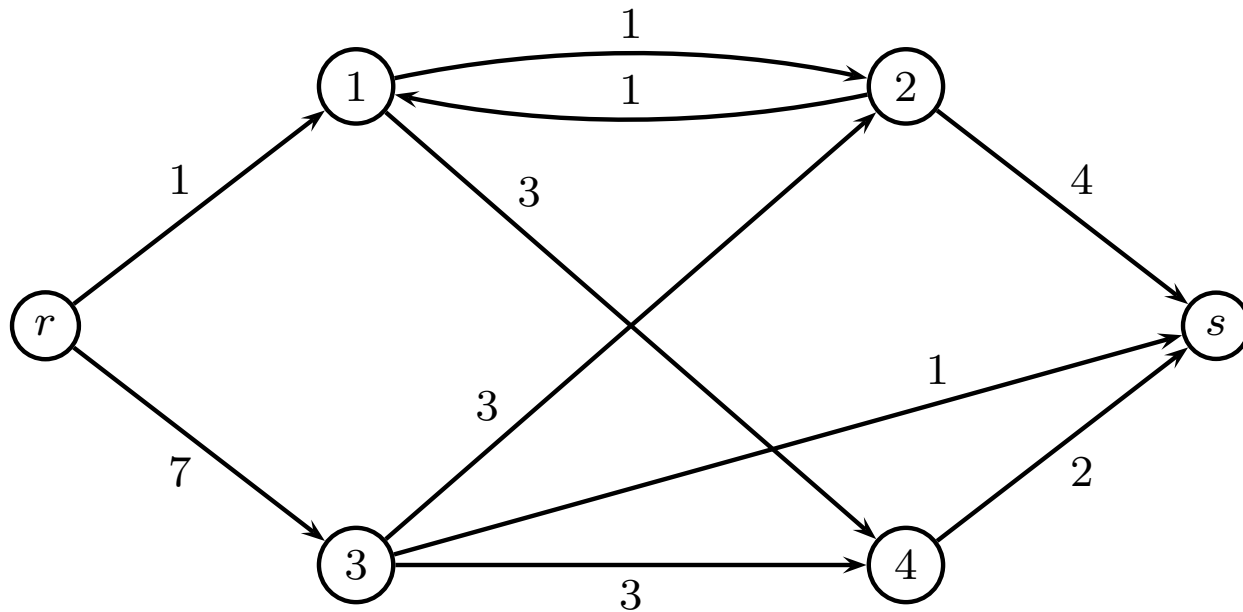
 push on (i, j)

if i is active and i can be relabelled

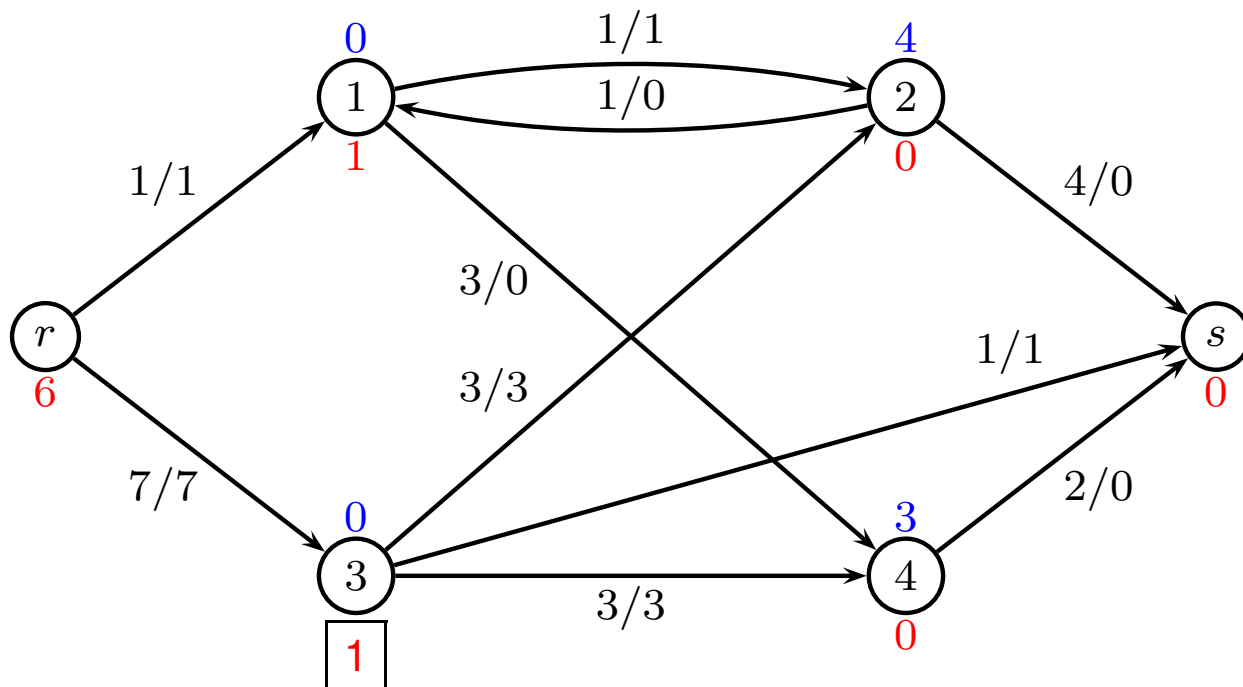
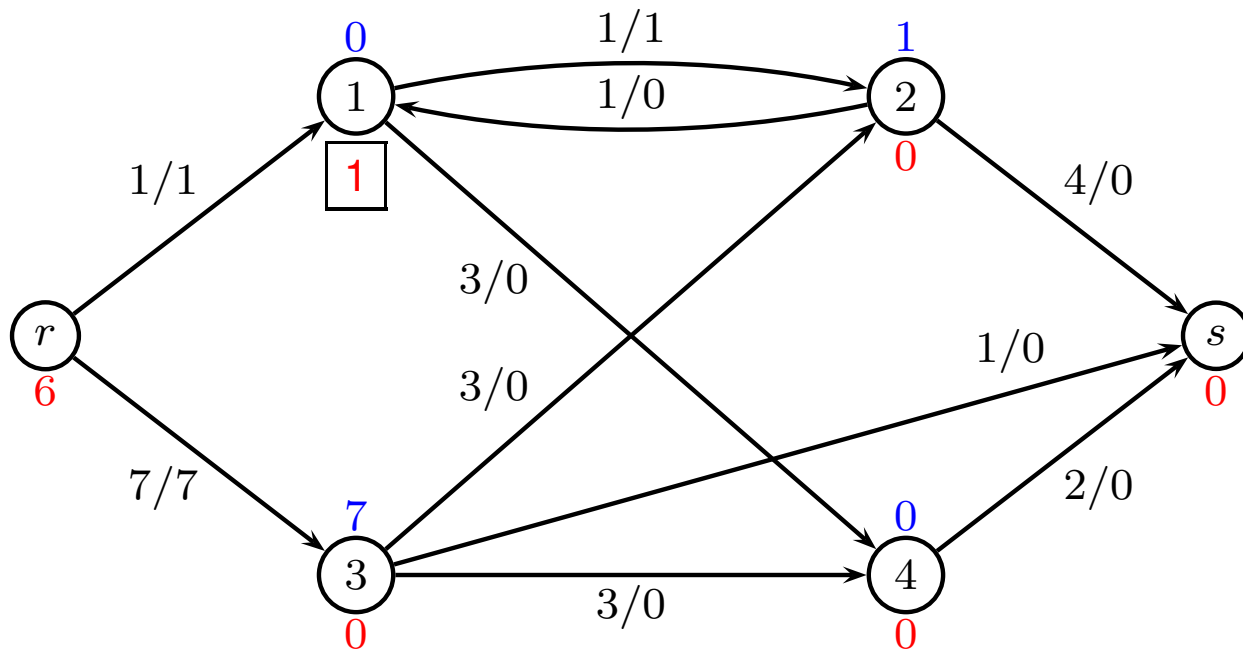
 relabel i

Push and relabel operations can be performed in any order

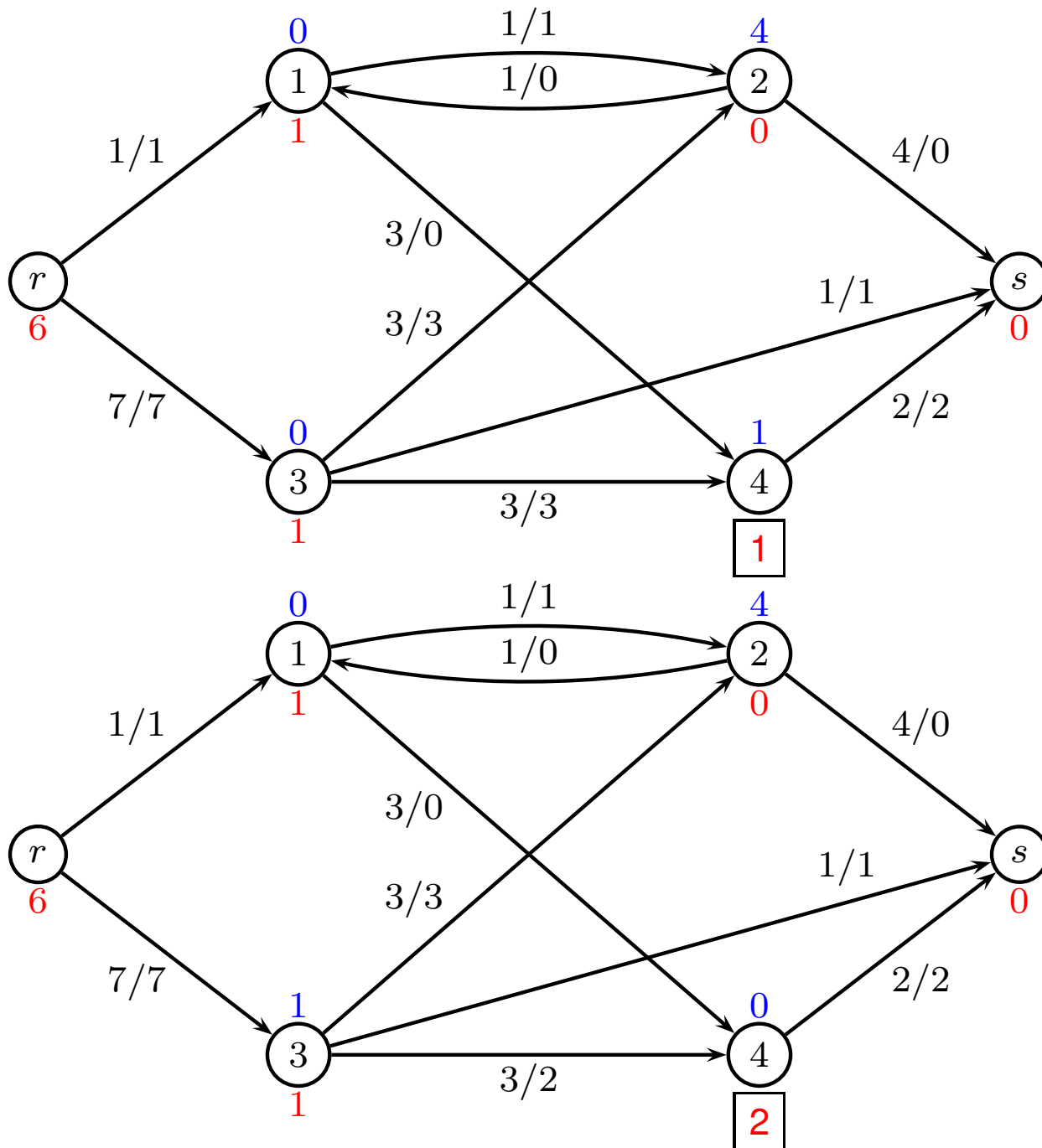
Example



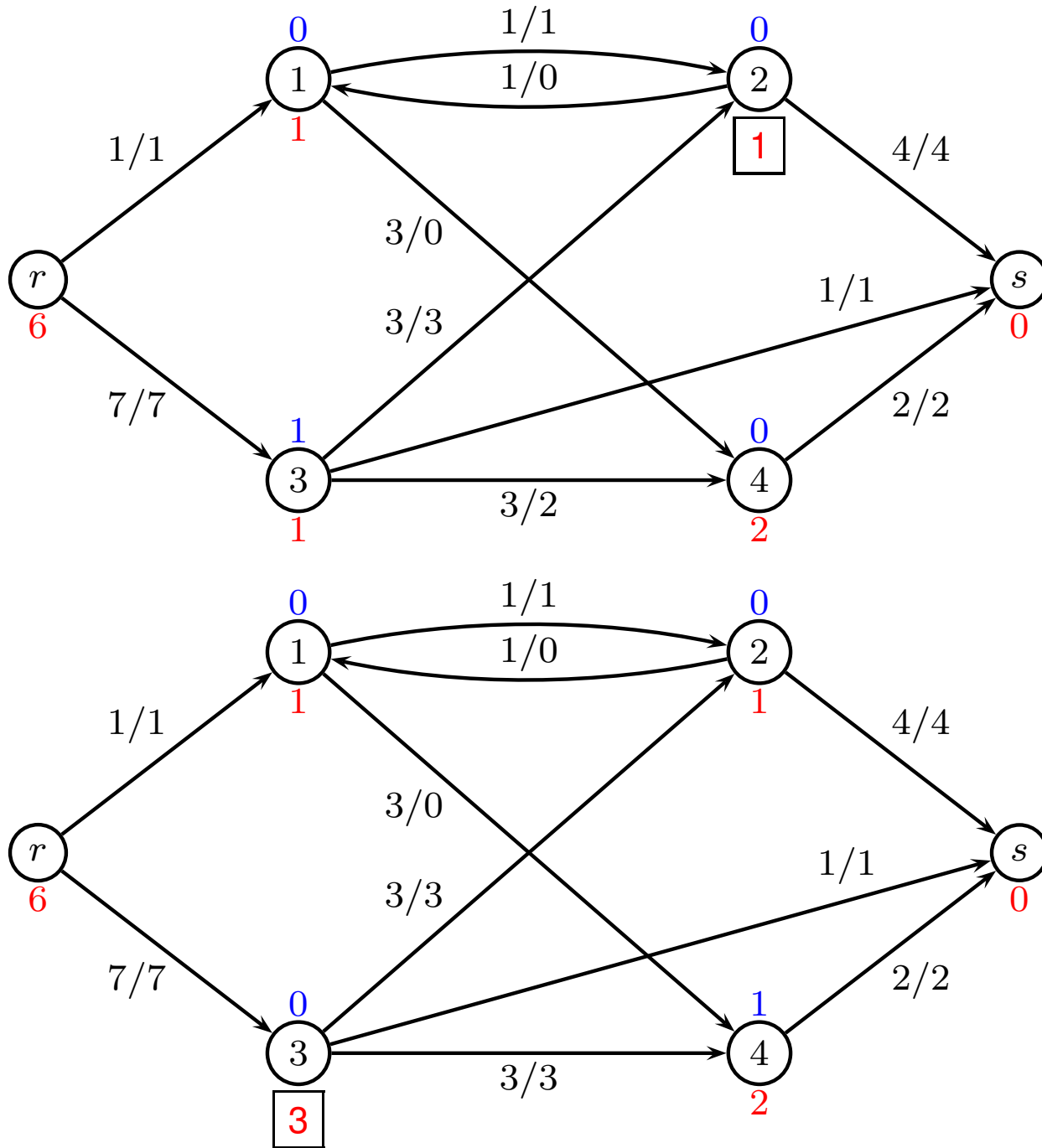
Example, iteration 1, 2



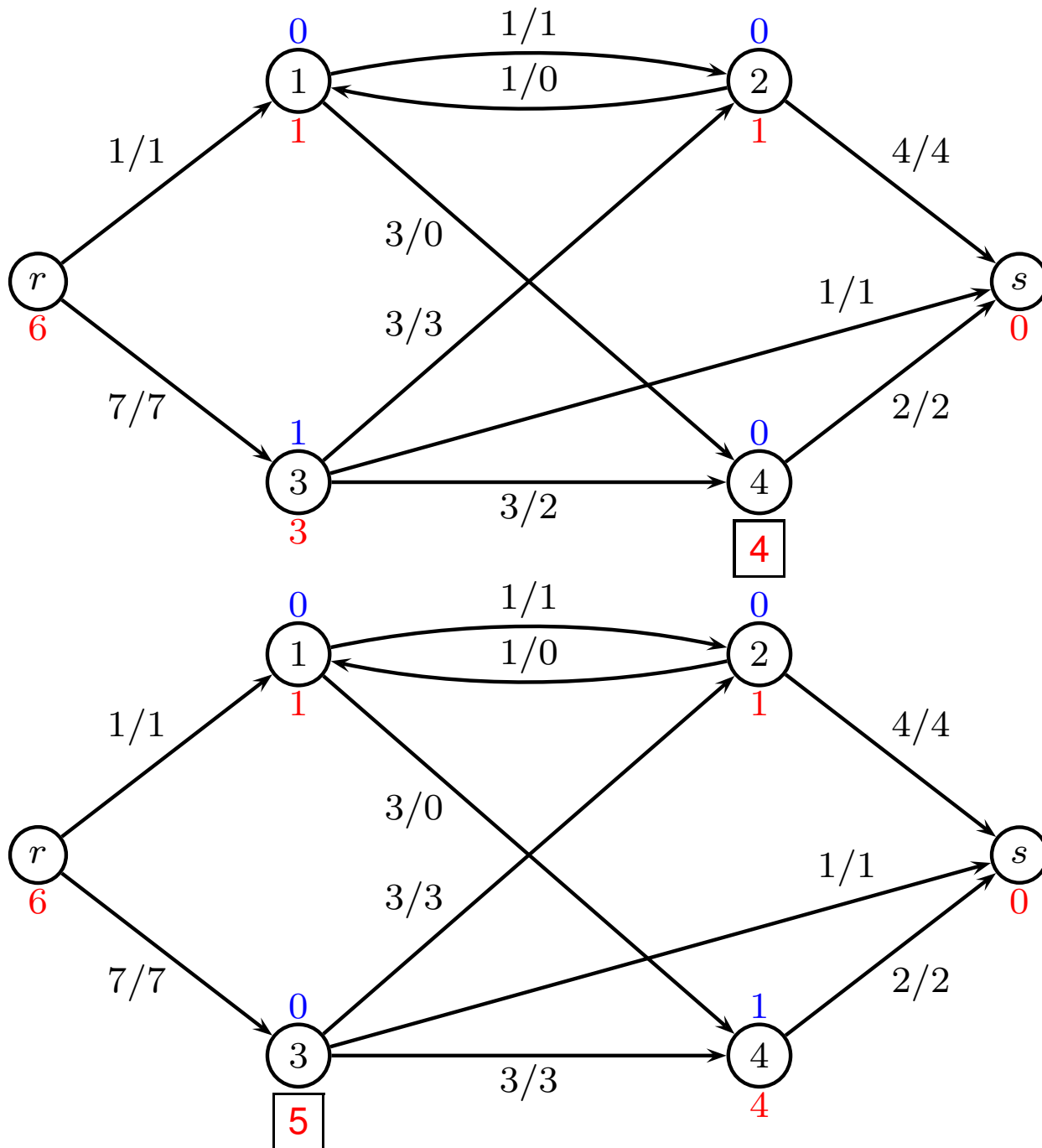
Example, iteration 3, 4



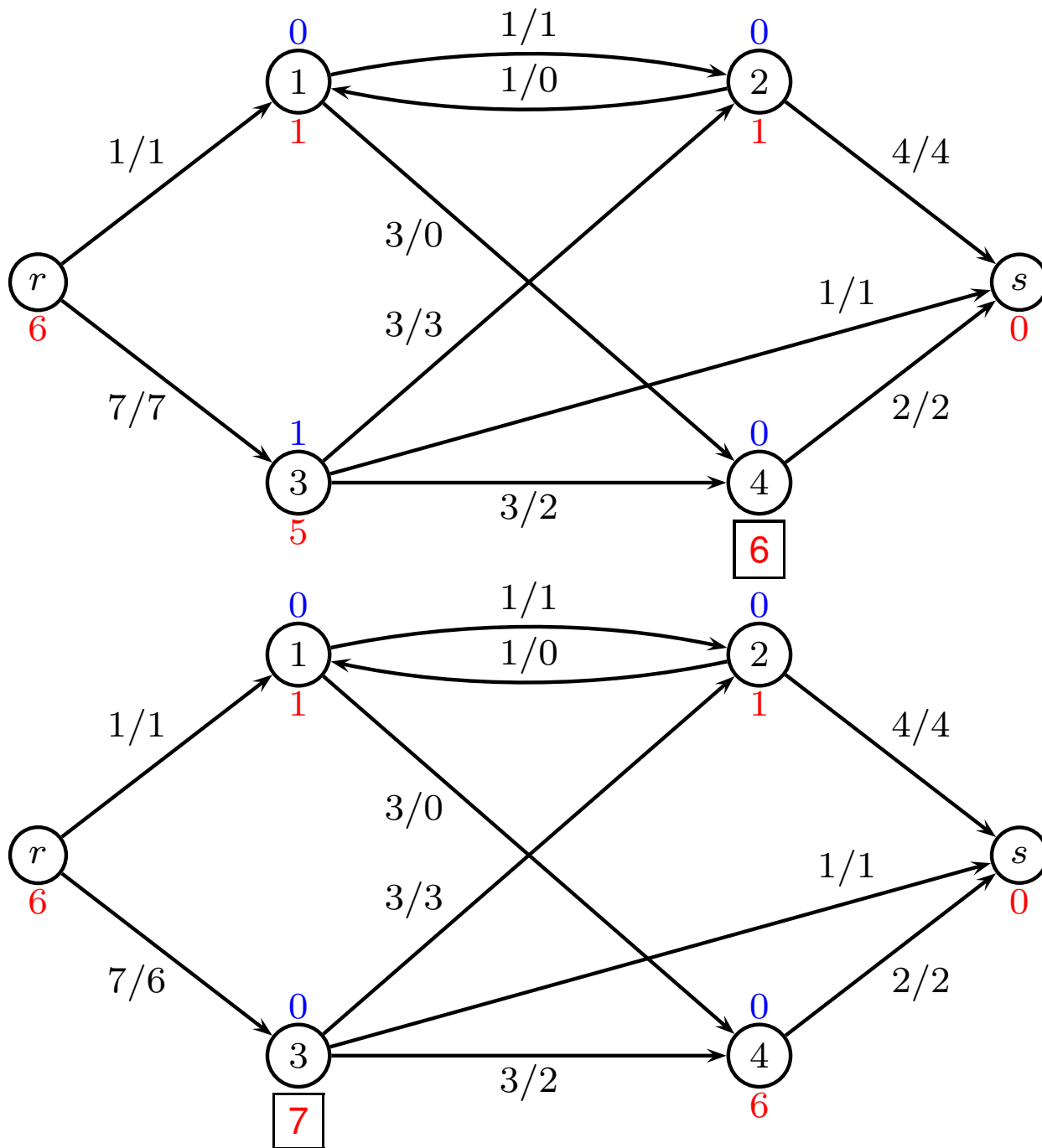
Example, iteration 5, 6



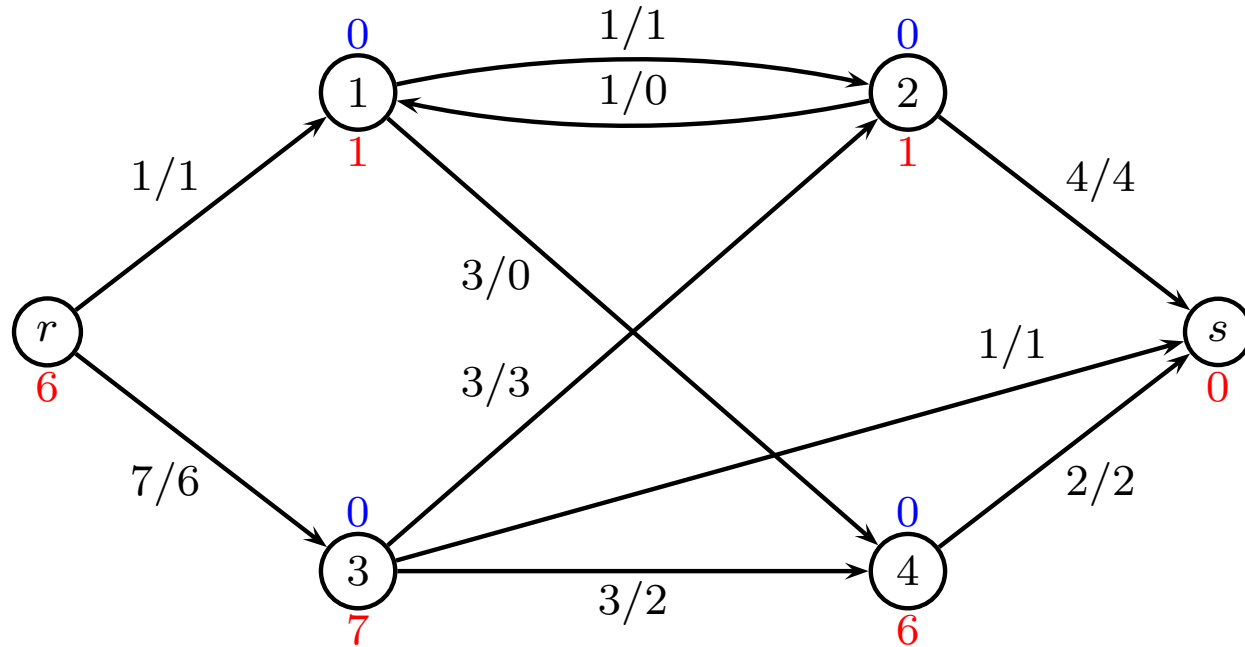
Example, iteration 7, 8



Example, iteration 9, 10



Max flow, min cut



- Choose, e.g. $k = 2$ giving $R = \{r, 3, 4\}$ and $\bar{R} = \{s, 1, 2\}$
- Although the many “ping-pong” iterations between node 3 and 4 were annoying, they are necessary to increase labels so we get correct cut

Running times

(Informally)

- Every node can have labels $0, \dots, 2n$ which is $O(n)$
(label $d[i] - n$ is lower bound on distance to r)
- We have n nodes to relabel
- After each relabel, we need to push. Roughly $O(m)$
- Total running time: $O(n^2m)$

Running times

- Notice, that every iteration provides a cut
- Analysis of the running times of the preflow-push algorithms is based on an analysis of the number of saturating pushes and non-saturating pushes.
- It can be shown that the push-relabel maximum flow algorithm can be implemented to run in time $O(n^2m)$ or $O(n^3)$.

Last time we showed that Edmonds-Karp has running time $O(nm^2)$.

The Min Cost Flow Problem

– *augmenting cycle method*

David Pisinger

`pisinger@man.dtu.dk`

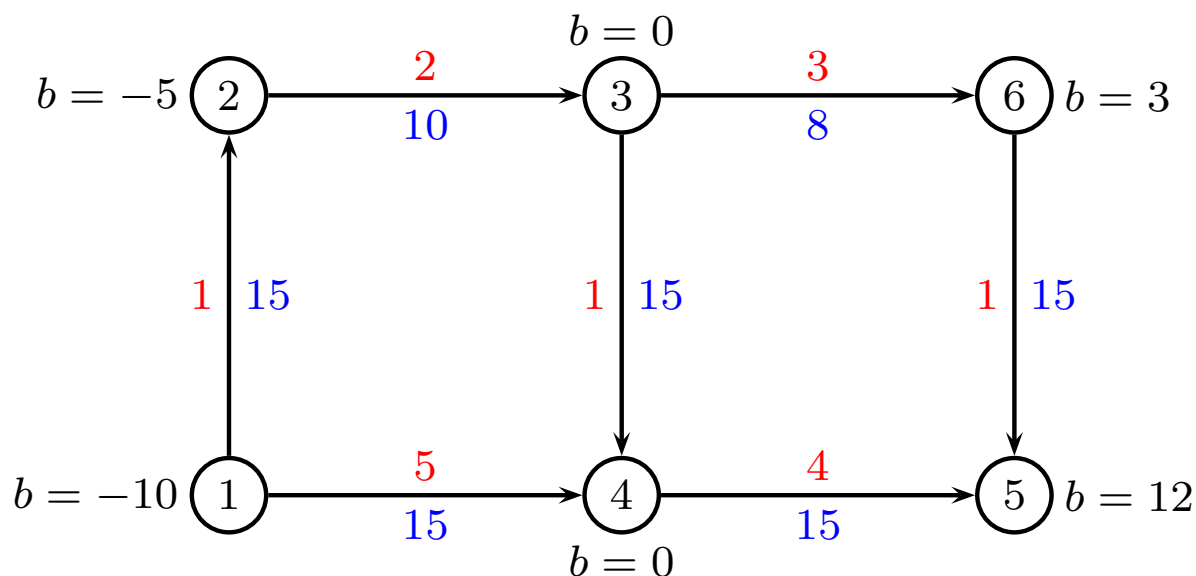
DTU Management

Technical University of Denmark

© Copyright David Pisinger. May not be distributed

Min Cost Flow - Terminology

- We consider a directed graph $G = (V, E)$ in which each edge e has a capacity $u_e \in \mathbb{R}_+$ and a unit transportation cost $w_e \in \mathbb{R}$.
- Each vertex i furthermore has a demand $b_i \in \mathbb{R}$. If $b_i \geq 0$ then i is called a **sink**, and if $b_i < 0$ then i is called a **source**.
- We assume that $b_V = \sum_{i \in V} b_i = 0$.



Min Cost Flow - Definition

The Min Cost Flow problem consists in supplying the sinks from the sources by a flow in the cheapest possible way:

$$\begin{array}{ll}\min & \sum_{(i,j) \in E} w_{ij} x_{ij} \\ \text{s.t.} & f_x(i) = b_i \quad i \in V \\ & 0 \leq x_{ij} \leq u_{ij} \quad (i,j) \in E\end{array}$$

where $f_x(i) = \sum_{(j,i) \in E} x_{ji} - \sum_{(i,j) \in E} x_{ij}$.

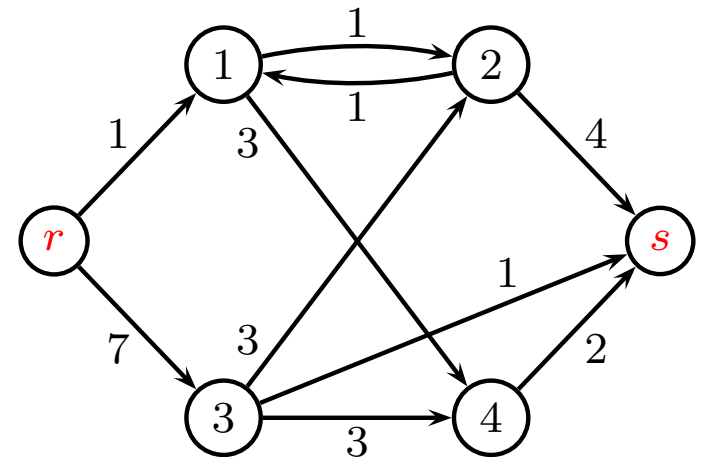
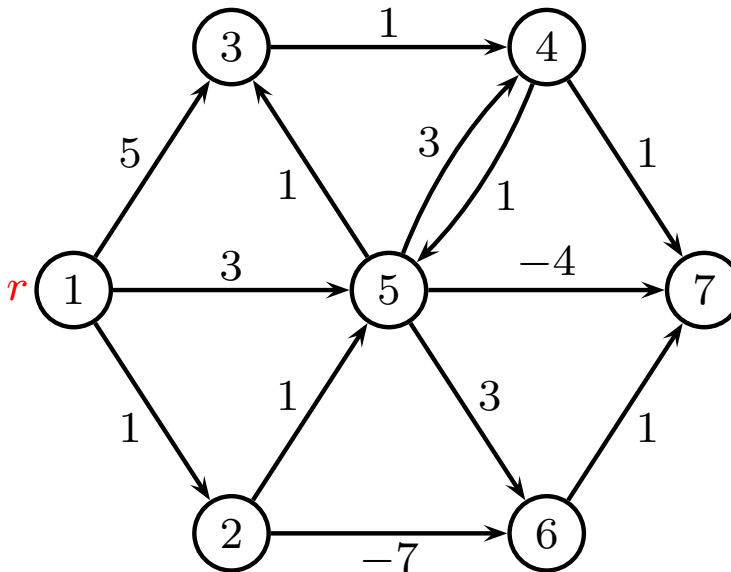
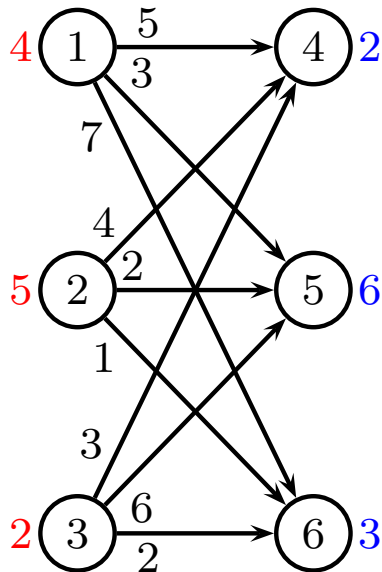
- Flow balance constraints
- Capacity constraints

Often integral flow: $x_{ij} \in \mathbb{Z}$

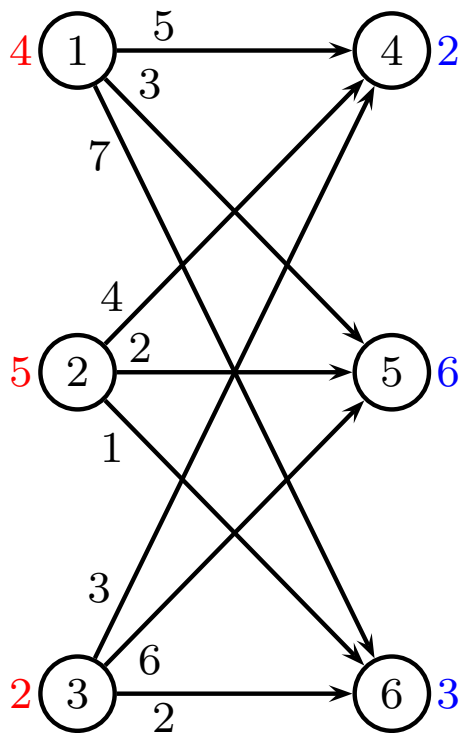
Special cases

Numerous flow problems can be stated as a Min Cost Flow problem:

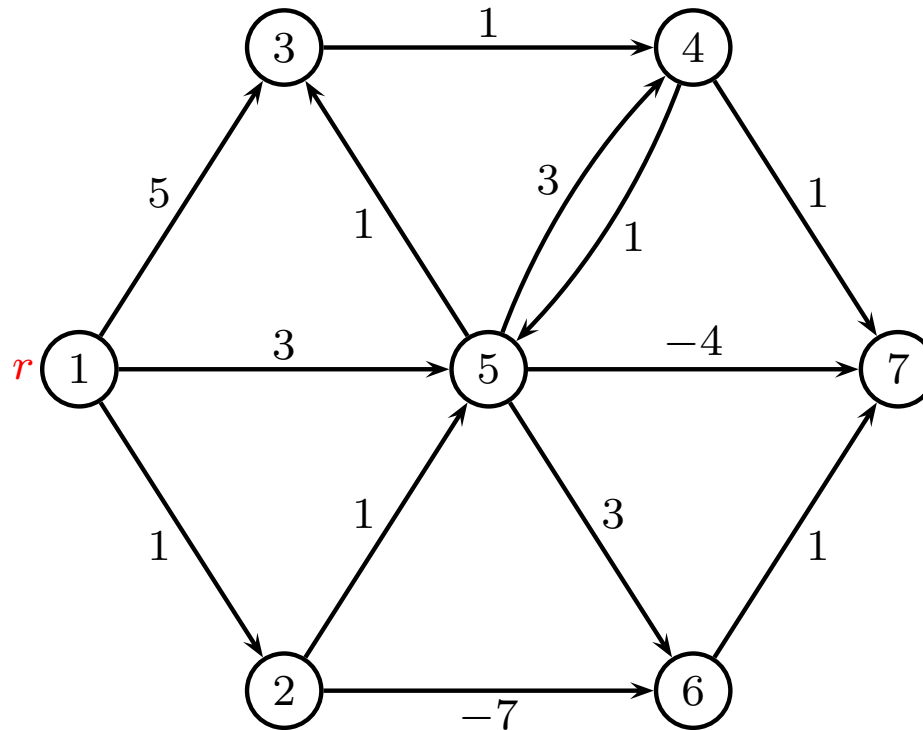
1. The Transportation Problem
2. The Shortest Path Problem
3. The Max Flow Problem



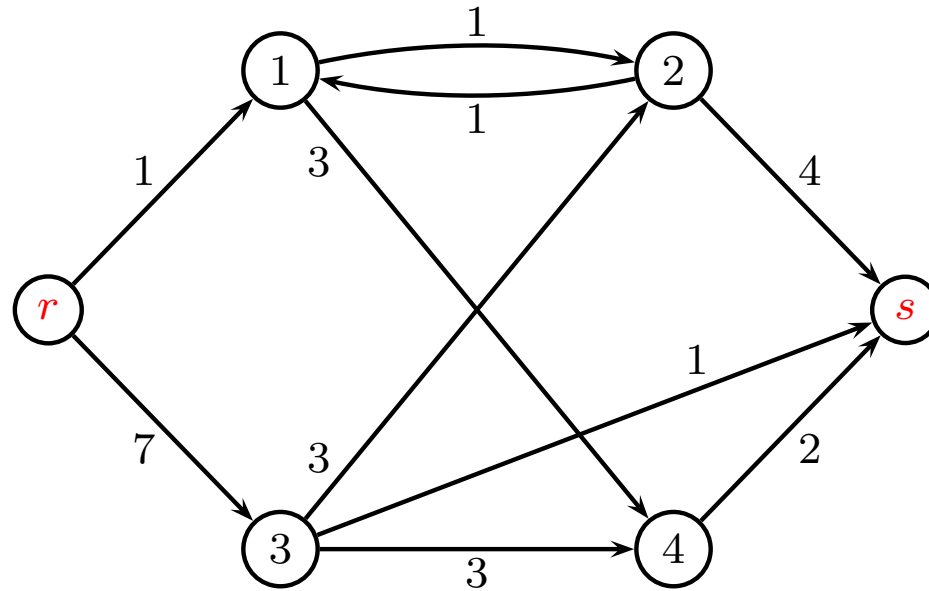
Transportation Problem



Shortest Path Problem



Max Flow Problem



TU and Min Cost Flow

- (Prop. 3.1): A matrix A is TU if and only if (i) the transpose matrix A^T is TU if and only if (ii) the matrix (A, I) is TU.
- (Prop. 3.2): (Sufficient condition). A matrix A is TU if
 1. $a_{ij} \in \{+1, -1, 0\}$ for all i, j .
 2. Each column contains at most two nonzero coefficients ($\sum_{i=1}^m |a_{ij}| \leq 2$).
 3. There exists a partition (M_1, M_2) of the set M of rows such that each column j containing two nonzeros satisfies $\sum_{i \in M_1} a_{ij} - \sum_{i \in M_2} a_{ij} = 0$.

TU and Min Cost Flow

	x_{e_1}	x_{e_2}	...	x_{ij}	...	x_{e_m}		
	w_{e_1}	w_{e_2}	...	w_{ij}	...	w_{e_m}		
1	-1	.	...		...		=	b_1
2	.	-1	...	.	...	.	=	b_2
.	.	.	...	.	...	.	=	.
i	1	1	...	-1	...	1	=	b_i (y_i)
.	.	.	...	.	...	.	=	
j	.	.	...	1	...	.	=	b_j (y_j)
.	.	.	...	.	...	-1	=	.
n	.	.	...	.	...	.	=	b_n
e_1	-1						$\geq$	$-u_1$
e_2		-1					$\geq$	$-u_2$
.	.	.	.	.	.	.	$\geq$	.
(i, j)				-1			$\geq$	$-u_{ij}$ (z_{ij})
.	.	.	.	.	.	.	$\geq$	.
e_m						-1	$\geq$	$-u_m$

Min Cost Flow - Dual LP

$$\begin{array}{ll}\min & \sum_{(i,j) \in E} w_{ij} x_{ij} \\ \text{s.t.} & \sum_{(j,i) \in E} x_{ji} - \sum_{(i,j) \in E} x_{ij} = b_i \quad i \in V \quad (y_i) \\ & -x_{ij} \geq -u_{ij} \quad (i,j) \in E \quad (z_{ij}) \\ & x_{ij} \geq 0 \quad (i,j) \in E\end{array}$$

Dual problem:

$$\begin{array}{ll}\max & \sum_{i \in V} b_i y_i - \sum_{(i,j) \in E} u_{ij} z_{ij} \\ \text{s.t.} & -w_{ij} - y_i + y_j \leq z_{ij} \quad (i,j) \in E \quad (x_{ij}) \\ & z_{ij} \geq 0 \quad (i,j) \in E\end{array}$$

Reduced cost $\bar{w}_{ij} = w_{ij} + y_i - y_j$. Last constraint becomes

$$-\bar{w}_{ij} \leq z_{ij} \quad (i,j) \in E \quad (x_{ij})$$

Optimality Conditions I

- If $u_{ij} = +\infty$ (i.e. no capacity constraints for e) then z_{ij} must be 0 and hence $\bar{w}_{ij} \geq 0$ just has to hold (primal optimality condition for the LP).
- If $u_{ij} \neq +\infty$ then $z_{ij} \geq 0$ and $z_{ij} \geq -\bar{w}_{ij}$ must hold. z has **negative** coefficient in the objective function – hence the best choice for z is as small as possible:

$$z_{ij} = \max\{0, -\bar{w}_{ij}\}$$

- Therefore, the optimal value of z_{ij} is uniquely determined from the other variables, and z_{ij} is “unnecessary” in the dual problem.

Optimality Conditions I - Comp. Slackness

Complementary slackness conditions

$$\begin{array}{ll} -\bar{w}_{ij} = z_{ij} & \vee \quad x_{ij} = 0 \\ x_{ij} = u_{ij} & \vee \quad z_{ij} = 0 \end{array}$$

This means

$$\begin{array}{l} x_{ij} > 0 \Rightarrow -\bar{w}_{ij} = z_{ij} = \max(0, -\bar{w}_{ij}) \\ \text{i.e. } (x_{ij} > 0 \Rightarrow -\bar{w}_{ij} \geq 0) \end{array}$$

and

$$\begin{array}{l} z_{ij} > 0 \Rightarrow x_{ij} = u_{ij} \\ \text{i.e. } (-\bar{w}_{ij} > 0 \Rightarrow x_{ij} = u_{ij}) \end{array}$$

$$\begin{array}{ll} x_{ij} > 0 & \Rightarrow \quad \bar{w}_{ij} \leq 0 \\ x_{ij} < u_{ij} & \Rightarrow \quad \bar{w}_{ij} \geq 0 \end{array}$$

Residual Graph I

Idea:

- Find initial feasible flow
 - Consider residual graph
 - Find augmenting path (cycle) in residual graph
 - Construct new feasible flow
-
- For a legal flow x in G , the residual graph is (like for Max Flow) a graph, in which the paths indicate **how flow excess can be moved** in G given that the flow x already is present. The only difference is that each edge has a cost assigned.

Residual Graph II

- The residual graph G_x for G wrt. x is defined by

$$\begin{aligned} V(G_x) &= V \\ E(G_x) &= E_x = \{(i, j) : (i, j) \in E \wedge x_{ij} < u_{ij}\} \\ &\quad \cup \{(i, j) : (j, i) \in E \wedge x_{ji} > 0\} \end{aligned}$$

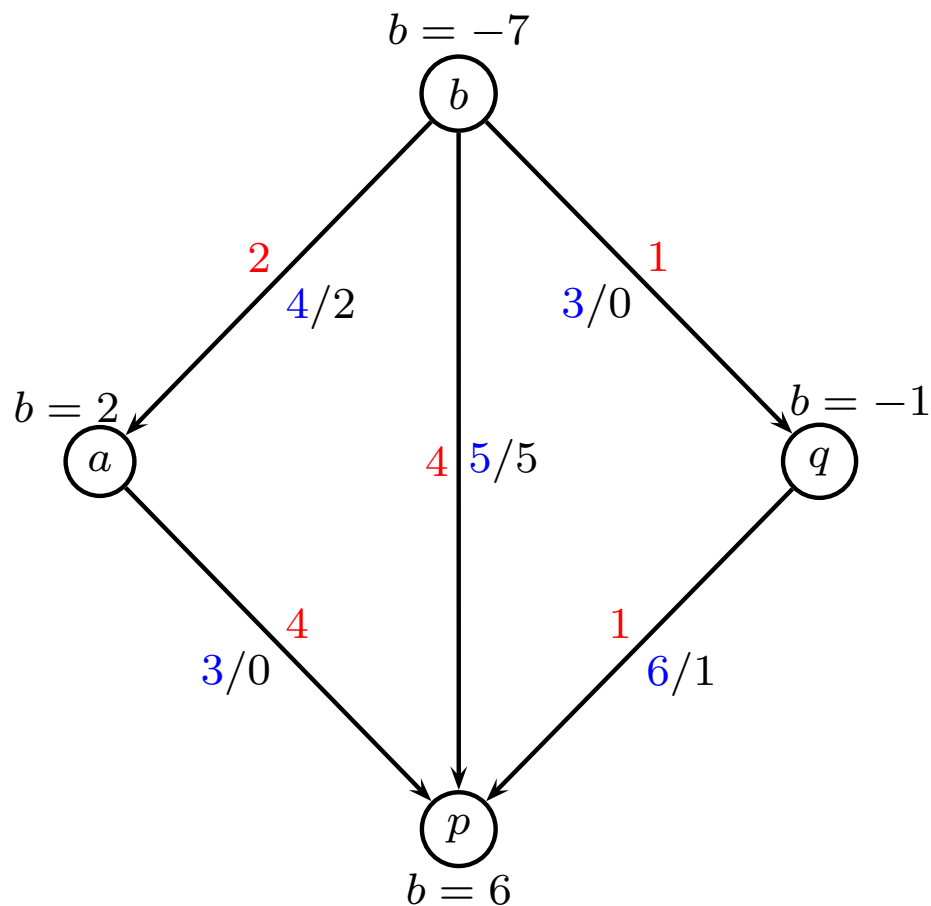
- The unit cost

$$w'_{ij} = \begin{cases} w_{ij} & \text{if } x_{ij} < u_{ij} \\ -w_{ji} & \text{if } x_{ji} > 0 \end{cases}$$

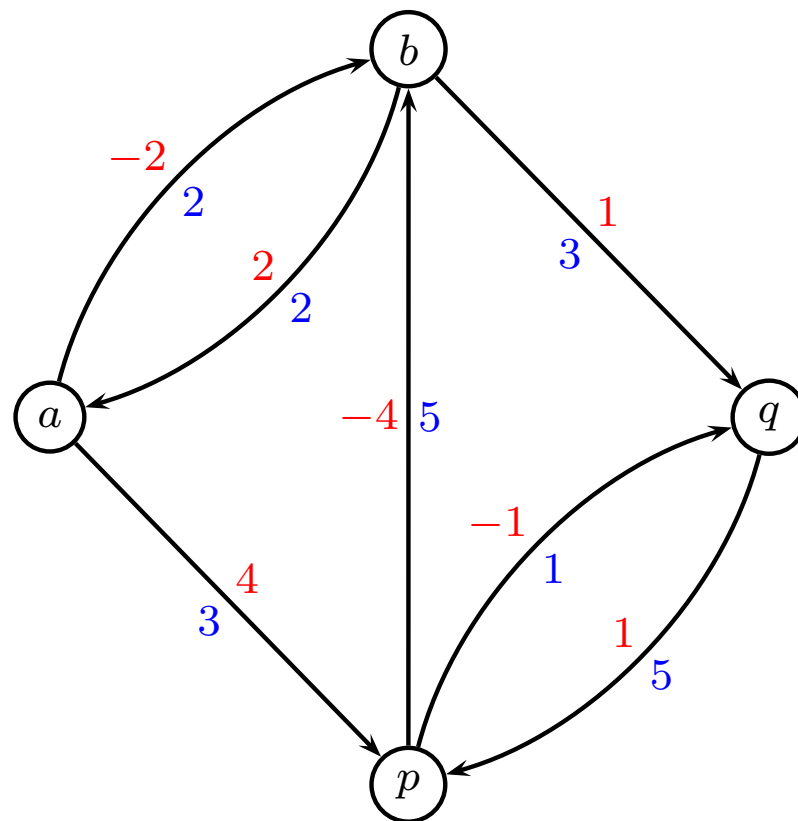
- The residual capacity

$$r_{ij} = \begin{cases} u_{ij} - x_{ij} & \text{if } x_{ij} < u_{ij} \\ x_{ji} & \text{if } x_{ji} > 0 \end{cases}$$

Residual Graph



Graph with flow



Residual graph with costs and capacities

Residual Graph III

- Note that a directed circuit with negative cost in G_x corresponds to a negative cost circuit in G , if cost are added for forward edges and subtracted for backward edges.
- Note that if a set of potentials $y_i, i \in V$ are given, and the cost of a circuit wrt. the reduced costs for the edges ($\bar{w}_{ij} = w_{ij} + y_i - y_j$) are calculated, the cost remains the same as the original costs – the potentials are “telescoped” to 0.

$$\sum_C \bar{w}_{ij} = \sum_C w_{ij} + \sum_C y_i - \sum_C y_j = \sum_C w_{ij}$$

- We normally use reduced costs to drive search (greedy principle) but same as using original costs

Min Cost Flow - Negative cost circuits

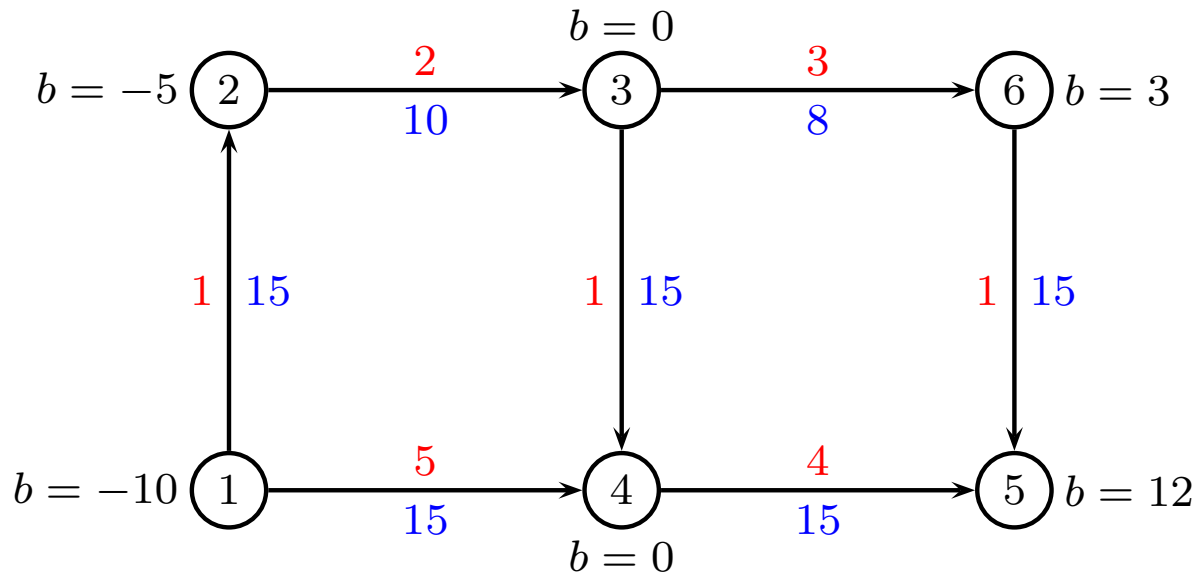
- A primal feasible flow satisfying sink demands from sources and respecting the capacity constraints is **optimal if and only if** an x -augmenting circuit with negative w -cost (or negative $\overline{w}$ -cost – there is no difference) does not exist.
(will be proved later)
- This is the idea behind the identification of optimal solutions in the **network simplex algorithm**.

Augmenting Circuit Algorithm

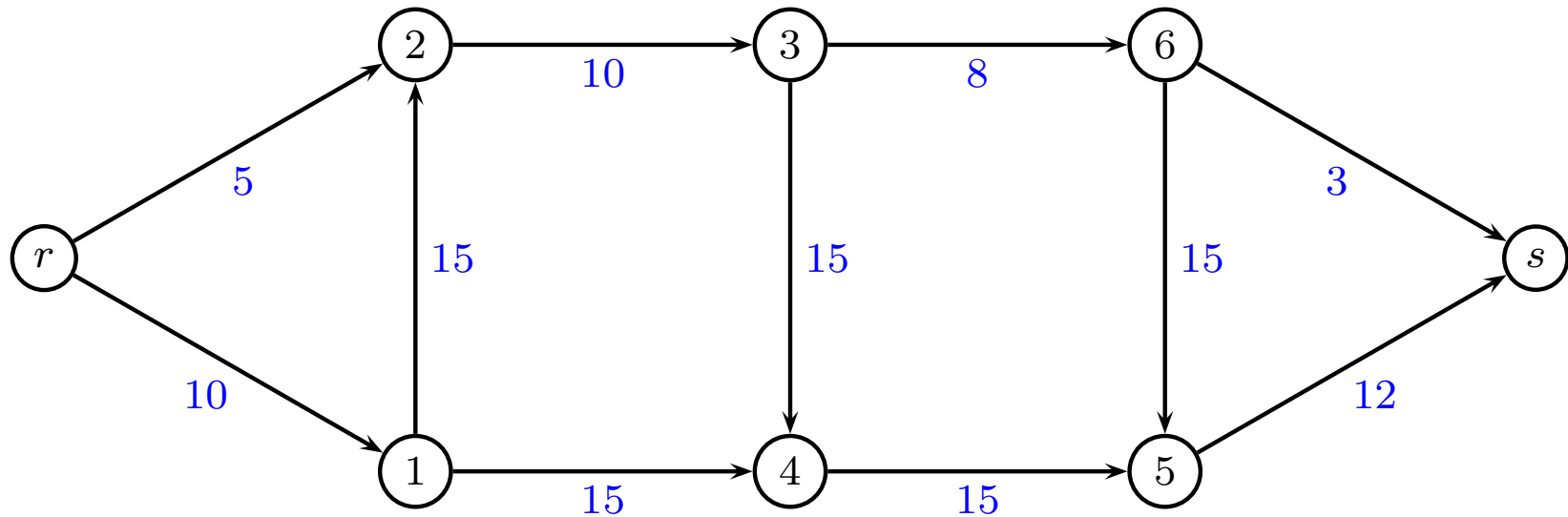
- 1 find a feasible solution x ;
- 2 **while** *there exists an augmenting circuit* **do**
- 3 find an augmenting circuit C ;
- 4 find “bottleneck” as minimum residual capacity in C
$$\theta = \min_{(i,j) \in C} \{r_{ij}\} ;$$
- 5 augment x on C with θ units ;

- initial flow is found by solving max-flow problem
- negative circuit is found by Bellman-Ford in $O(nm)$

Example

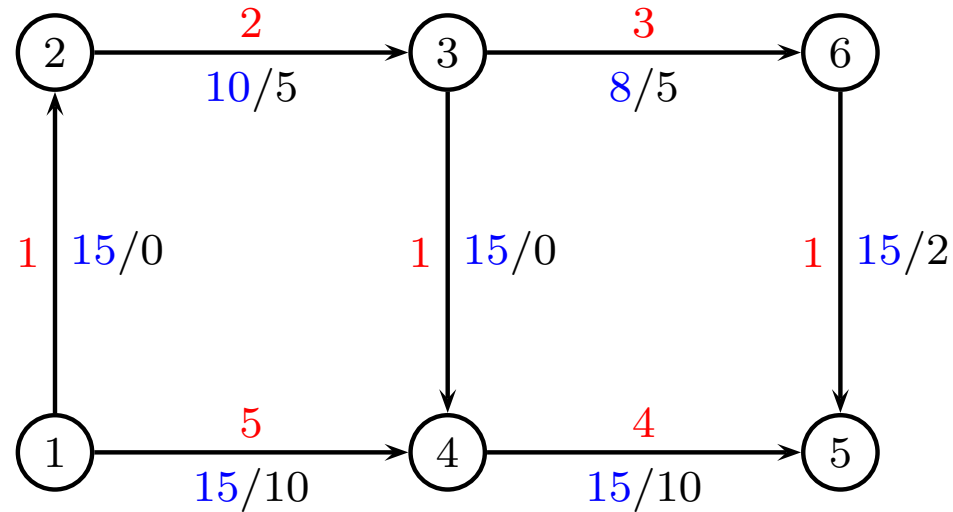


Initialize



Drop demands in the following

Example



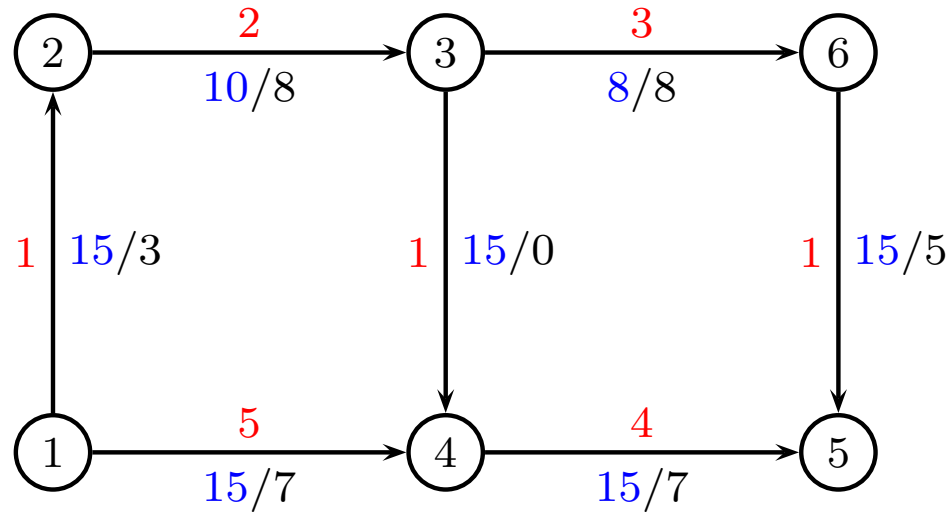
cost: $z = 2 \cdot 5 + 3 \cdot 5 + 1 \cdot 2 + 5 \cdot 10 + 4 \cdot 10 = 117$

cycle: $C = \{2, 3, 6, 5, 4, 1\}$

$w(C) = 2 + 3 + 1 - 4 - 5 + 1 = -2$

$\theta = \min\{5, 3, 13, 10, 10, 15\} = 3$

Example



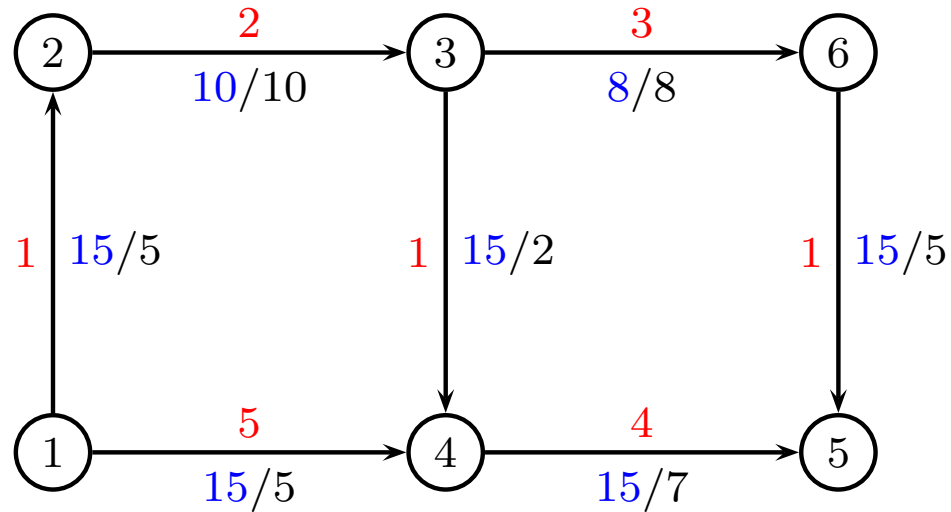
cost: $z = 2 \cdot 8 + 3 \cdot 8 + 1 \cdot 3 + 1 \cdot 5 + 5 \cdot 7 + 4 \cdot 7 = 111$

cycle: $C = \{2, 3, 4, 1\}$

$w(C) = 2 + 1 - 5 + 1 = -1$

$\theta = \min\{2, 15, 7, 12\} = 2$

Example



cost: $z = 2 \cdot 10 + 3 \cdot 8 + 1 \cdot 5 + 1 \cdot 2 + 1 \cdot 5 + 5 \cdot 5 + 4 \cdot 7 = 109$

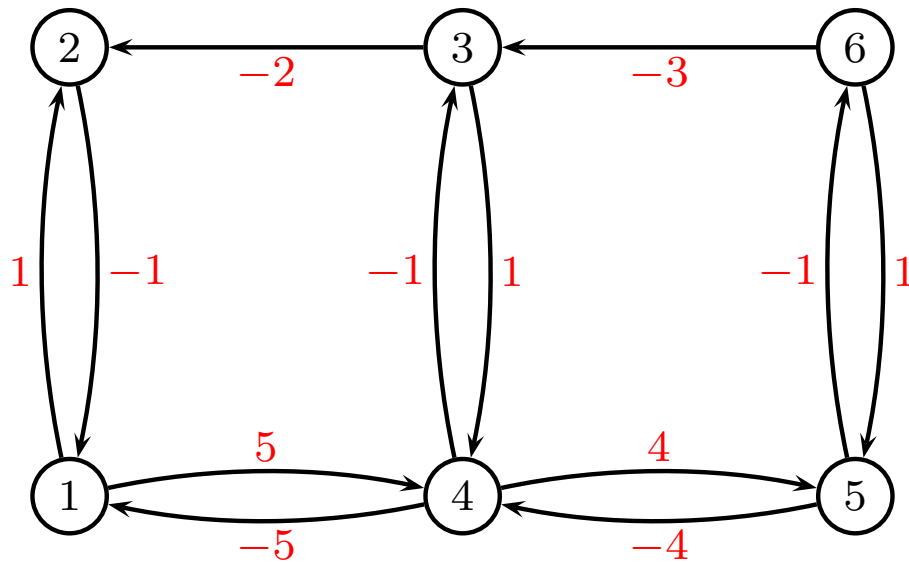
No negative cycles

stop

Proof of termination criteria

A flow x is optimal $\Leftrightarrow$ no augmenting circuit can be found

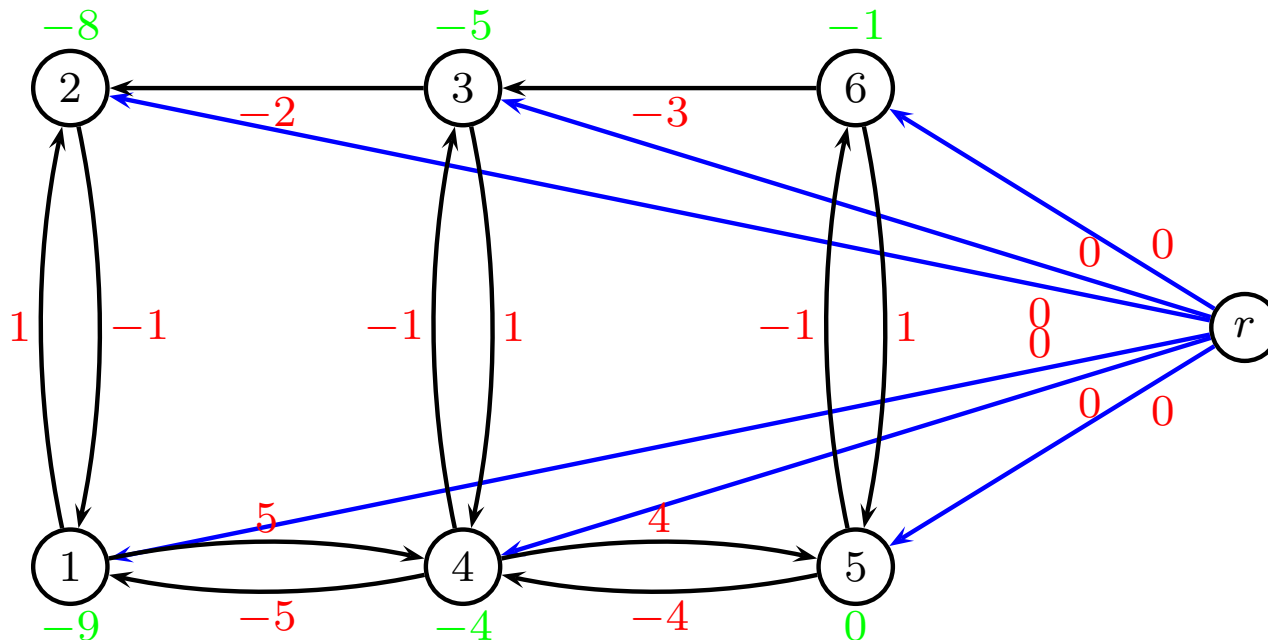
- Residual graph (only costs):



- if negative circuit can be found then construct better solution
- now, assume no negative circuit can be found in residual graph

○

Proof of termination criteria



Shortest path from r to i defines y_i . Satisfies $y_i + w'_{ij} \geq y_j$

$$y_i + w_{ij} \geq y_j \quad \text{if } x_{ij} < u_{ij}$$

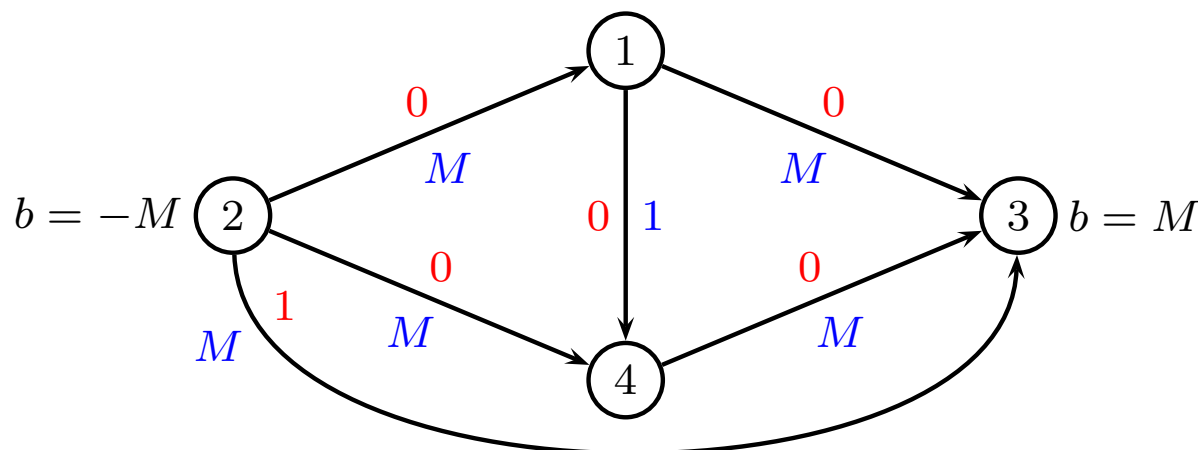
$$y_i - w_{ji} \geq y_j \quad \text{if } x_{ji} > 0$$

since $\bar{w}_{ij} = w_{ij} + y_i - y_j$ we have

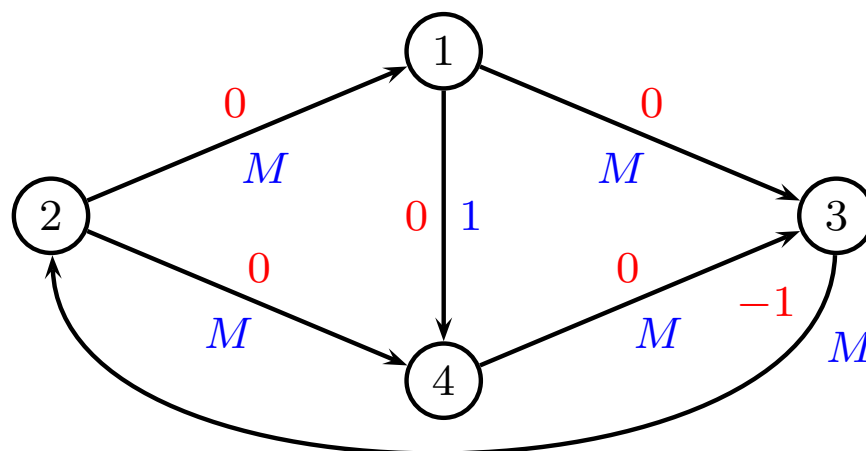
$$\bar{w}_{ij} \geq 0 \quad \text{if } x_{ij} < u_{ij}$$

$$\bar{w}_{ji} \leq 0 \quad \text{if } x_{ji} > 0$$

Time Complexity



Initialize: send M units along bottom edge. Residual graph



Negative cycle $C = \{2, 1, 4, 3\}$ then $C = \{2, 4, 1, 3\}$ then ...
 (Similar to proof for max-flow augmenting path method)

Time Complexity

- Finding negative cycle takes $O(nm)$ with Bellman-Ford
- Objective $z \leq mWU$ where $w_{ij} \leq W$ and $u_{ij} \leq U$
- Each iteration reduces z by at least 1
- At most mWU iterations
- Many strategies to limit the number of iterations
 - Most negative augmenting cycle at each step
 - Smallest average arc cost
- Expensive algorithm compared to network simplex

Other constraints

- Lower bounds on arcs
- Upper bounds on flow through node
- convex piecewise linear costs on arc flow
- Fixed cost to use a node
- Fixed cost to use an arc
- Proportionality of flow (e.g. $x_{i1} = x_{i2}$)
- Gains and losses of flow along arcs (e.g. power distribution)

Lower bounds on arcs

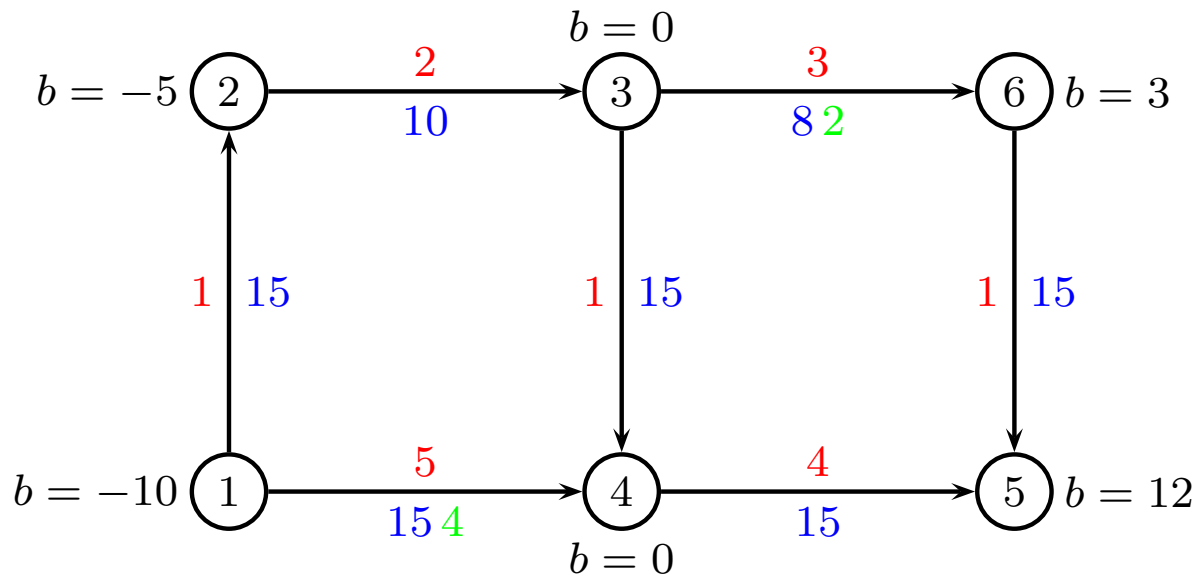
Assume that the flow x_{ij} on arc (i, j) has upper bound u_{ij} and lower bound ℓ_{ij} .

- Replace the upper bound with $u_{ij} := u_{ij} - \ell_{ij}$
- Set the demand at node i to $b_i := b_i + \ell_{ij}$
- Set the demand at node j to $b_j := b_j - \ell_{ij}$

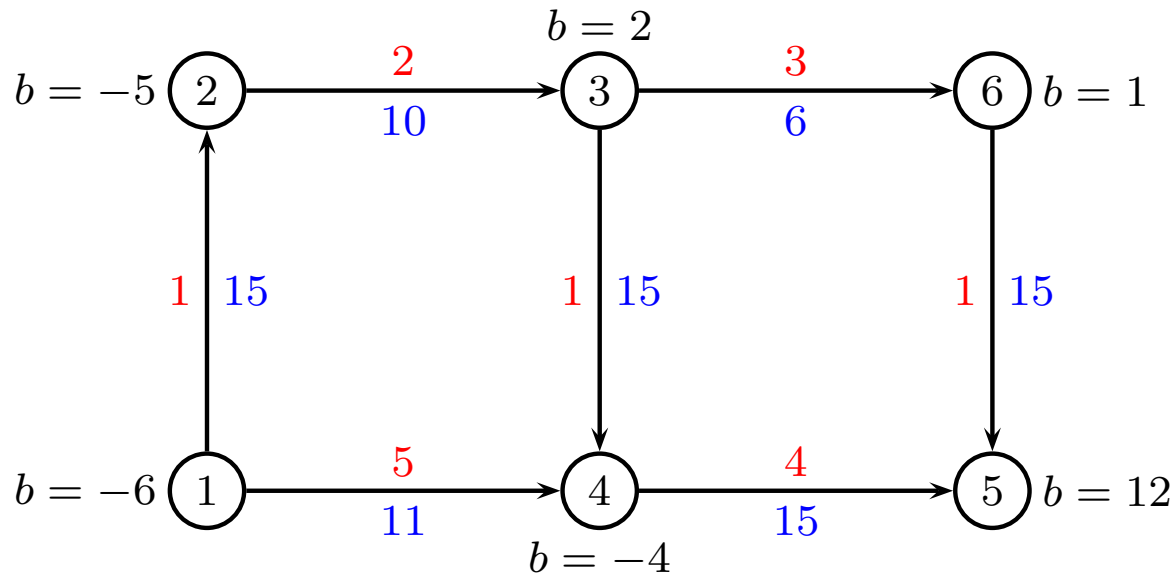
Now, the problem is a minimum cost flow problem with only upper bounds.

After having solved the problem, remember to add ℓ_{ij} to x_{ij}

Lower bounds on arcs



(green color is lower bound). Transformed graph:

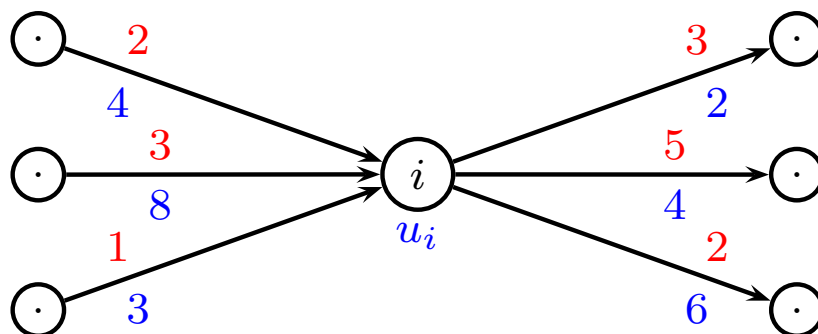


Upper bounds on flow through node

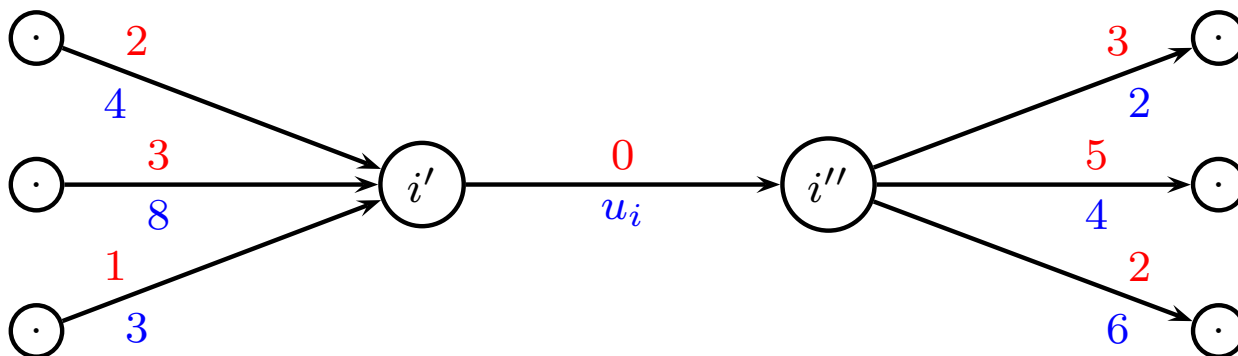
Assume that there is an upper bound u_i on node i

- Replace node i with two nodes i' and i''
- Create an arc from i' to i'' with capacity u_i
- Replace every ingoing arc (j, i) with an arc (j, i')
- Replace every outgoing arc (i, j) with an arc (i'', j)

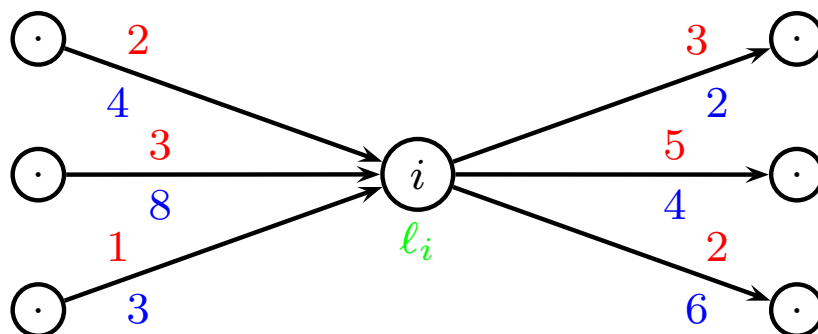
Upper bounds on flow through node



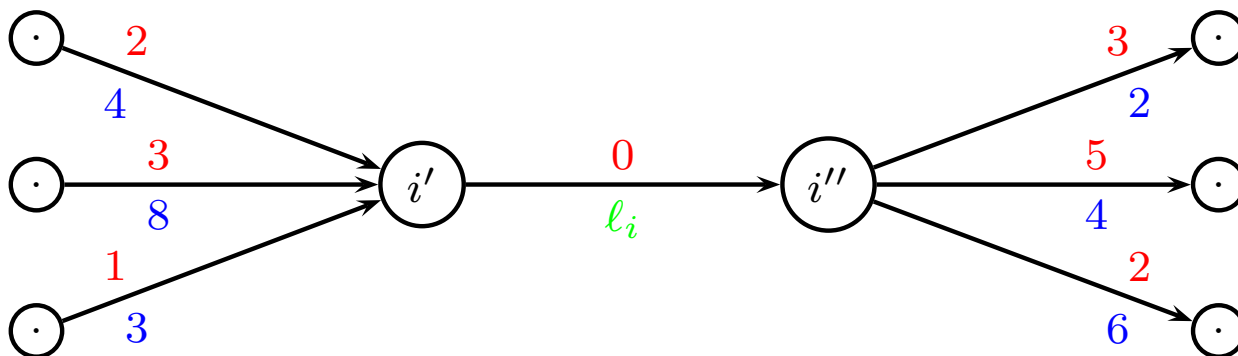
becomes



Lower bounds on flow through node



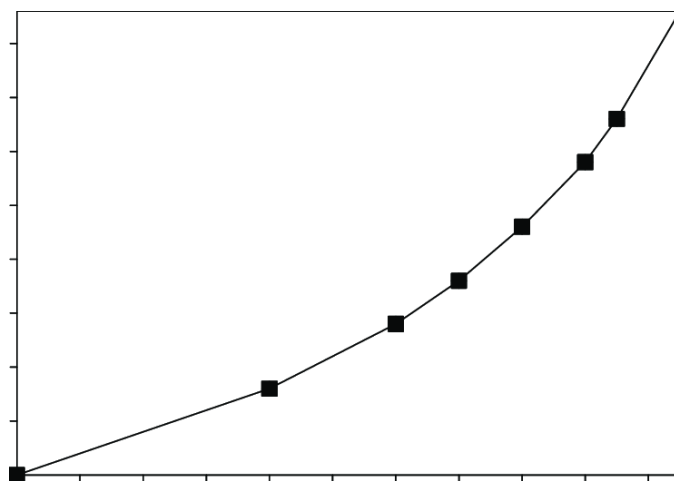
becomes



Convex, piecewise linear costs on arc flows

Assume that the cost on arc (i, j) is a convex piecewise linear function f_{ij} .

- Introduce multiple arcs, corresponding to each portion of the piecewise linear function. Use upper bounds to control the intervals.
- Convexity will assure that cheaper arcs are used before the more expensive arcs



Cannot handle

The following constraints “spoil” the structure of the problem

- Fixed cost to use a node
- Fixed cost to use an arc
- Proportionality of flow (e.g. $x_{i1} = x_{i2}$)
- Gains and losses of flow along arcs (e.g. power distribution)

However, the constraints can be handled using MIP models.
But we do not have constructive algorithms for MIP

Tips-and-tricks

Augmenting circuit method

- check that problem is balanced, otherwise add surplus node to make problem balanced
- When finding an initial solution with max-flow, remember to add source s and sink t
- if you cannot find a negative cycle, use Bellman-Ford
 - if Bellman-Ford continues decreasing distances, you have a negative cycle
 - if Bellman-Ford terminates with stable distances, there are no negative cycles

The Min Cost Flow Problem

– the network simplex algorithm

Jesper Larsen, David Pisinger

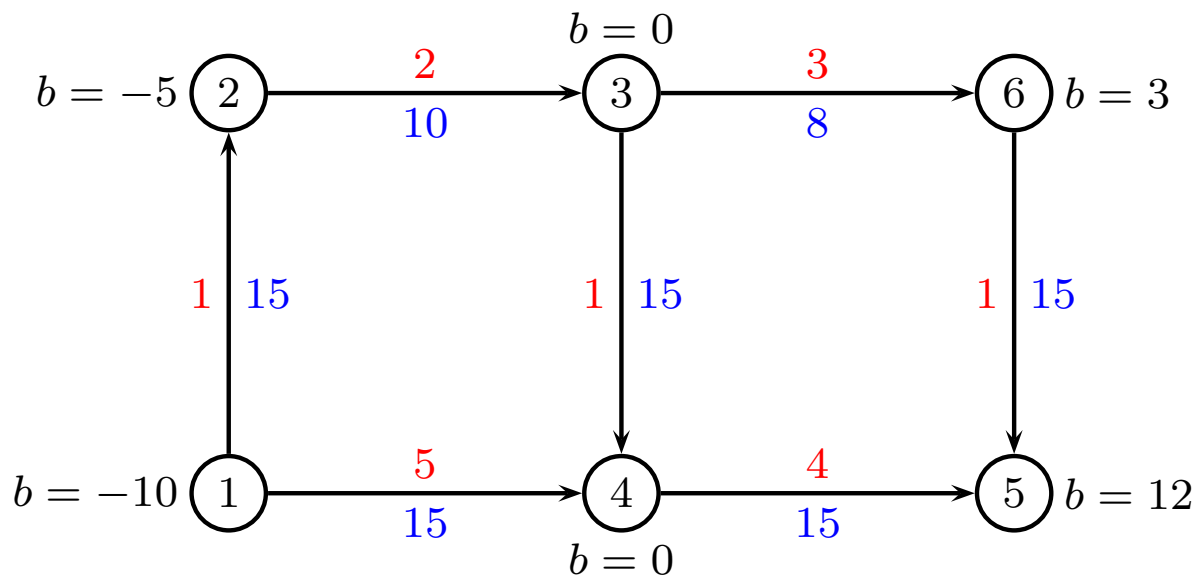
`jesla,pisinger@man.dtu.dk`

DTU Management
Technical University of Denmark

© Copyright David Pisinger. May not be distributed

Min Cost Flow - Terminology

- We consider a directed graph $G = (V, E)$ in which each edge e has a capacity $u_e \in \mathbb{R}_+$ and a unit transportation cost $w_e \in \mathbb{R}$.
- Each vertex i furthermore has a demand $b_i \in \mathbb{R}$. If $b_i \geq 0$ then i is called a **sink**, and if $b_i < 0$ then i is called a **source**.
- We assume that $b_V = \sum_{i \in V} b_i = 0$.



Min Cost Flow – Definition

The Min Cost Flow (MCF) problem consists in supplying the sinks from the sources by a flow in the cheapest possible way:

$$\begin{aligned} \min \quad & \sum_{e \in E} w_e x_e \\ & f_x(i) = b_i \quad i \in V \\ & 0 \leq x_e \leq u_e \quad e \in E \end{aligned}$$

where $f_x(i) = \sum_{(j,i) \in E} x_{ji} - \sum_{(i,j) \in E} x_{ij}$.

Assume graph G is connected
Otherwise solve separate problems

Min Cost Flow – Network Simplex

Interpretation of Simplex to min cost flow

- **Transshipment problem** (MCF with no capacities)
- Show that for every tree T exactly one tree solution
- Show that any solution can be transformed to equivalent tree solution
- Optimality criteria
- Network simplex for transshipment problem
- Generalize to **Min cost flow**
- Conclusion

Min Cost Flow – Characteristics

- If we look at the rows they are linearly dependent (sum them all together).
- But $n - 1$ of the rows are linearly independent, that is, there are $n - 1$ variables in a basic feasible solution.
- Moreover these $n - 1$ variables will correspond to a spanning tree! (in the minimum cost flow problem we just call it a tree).

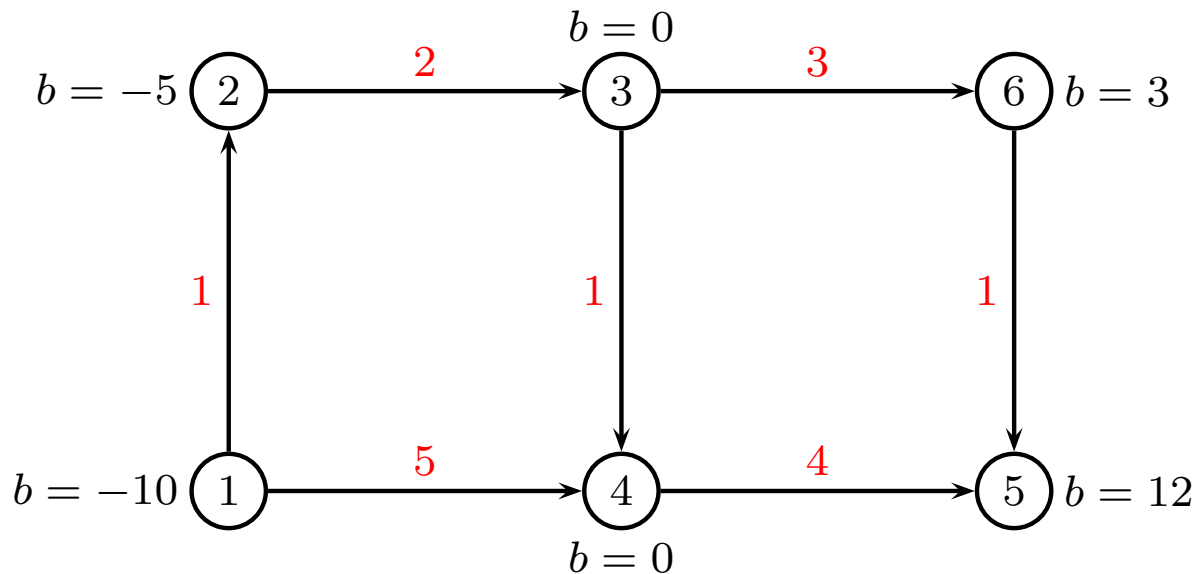
Basis is a tree T

Min Cost Flow – Model

	x_{e_1}	x_{e_2}	...	x_{ij}	...	x_{e_m}		
	w_{e_1}	w_{e_2}	...	w_{ij}	...	w_{e_m}		
1	-1	.	...		...		=	b_1
2	.	-1	...	.	...	.	=	b_2
.	.	.	...	.	...	.	=	.
i	1	1	...	-1	...	.	=	b_i
.	.	.	...	.	...	1	=	
j	.	.	...	1	...	.	=	b_j
.	.	.	...	.	...	-1	=	.
n	.	.	...	.	...	.	=	b_n
e_1	-1						$\geq$	$-u_1$
e_2		-1					$\geq$	$-u_2$
.	.	.	.	.	.	.	$\geq$	.
(i, j)				-1			$\geq$	$-u_{ij}$
.	.	.	.	.	.	.	$\geq$	.
e_m						-1	$\geq$	$-u_m$

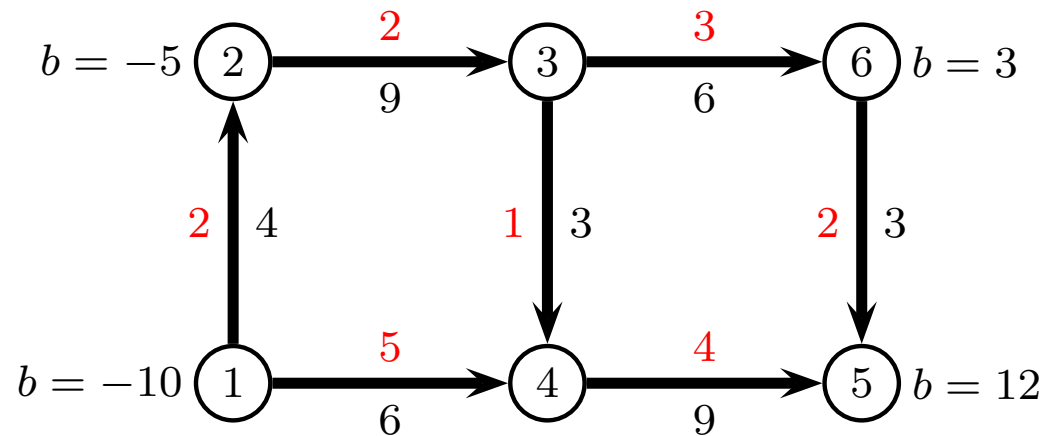
Transshipment problem

Min Cost Flow problem without capacities ($u_{ij} = \infty$)

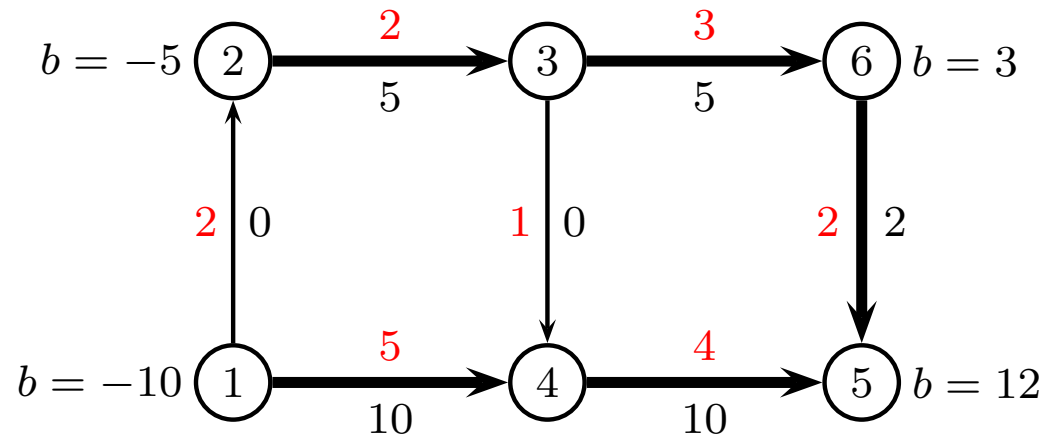


Solution vs. tree solution

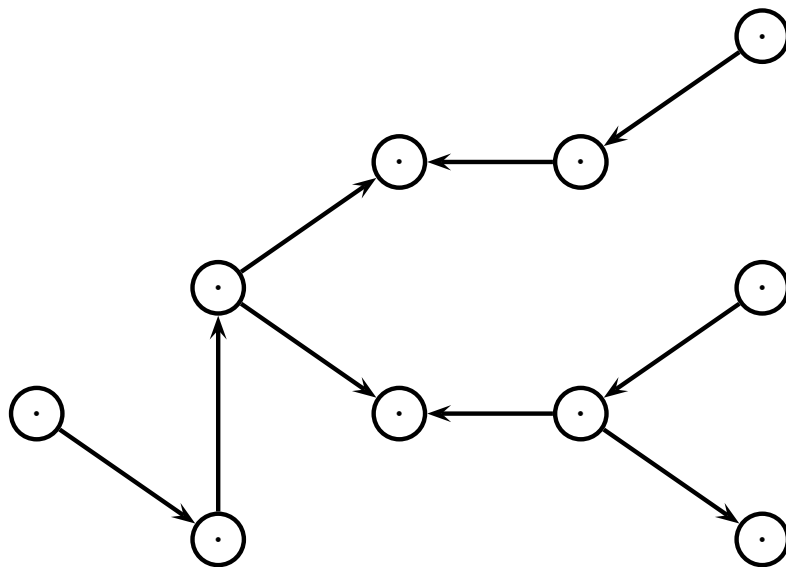
- If $G = (V, E, w, b)$ has an optimal solution, it also has an optimal tree solution.



cycle $\{1, 2, 3, 4\}$ cost 0, cycle $\{1, 2, 3, 6, 5, 4\}$ cost 0
(cycles have cost 0 or we could improve solution)



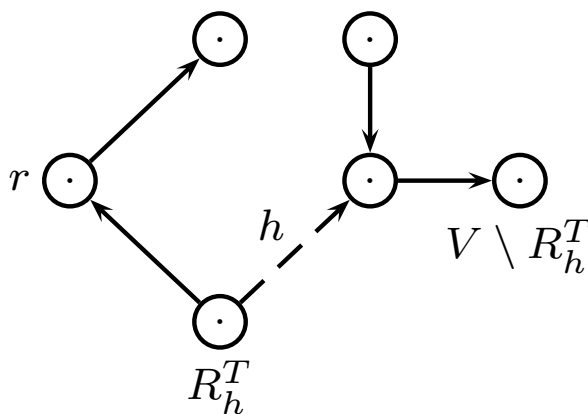
A tree solution



- A **tree solution** of the transshipment problem is a vector $x \in \mathbb{R}^E$ satisfying for some (unoriented) tree T :
 - $f_x(i) = b_i$, for all $i \in V$
 - $x_e = 0$ for all $e \notin T$
- A tree T uniquely identifies a tree solution x

Constructing a tree solution

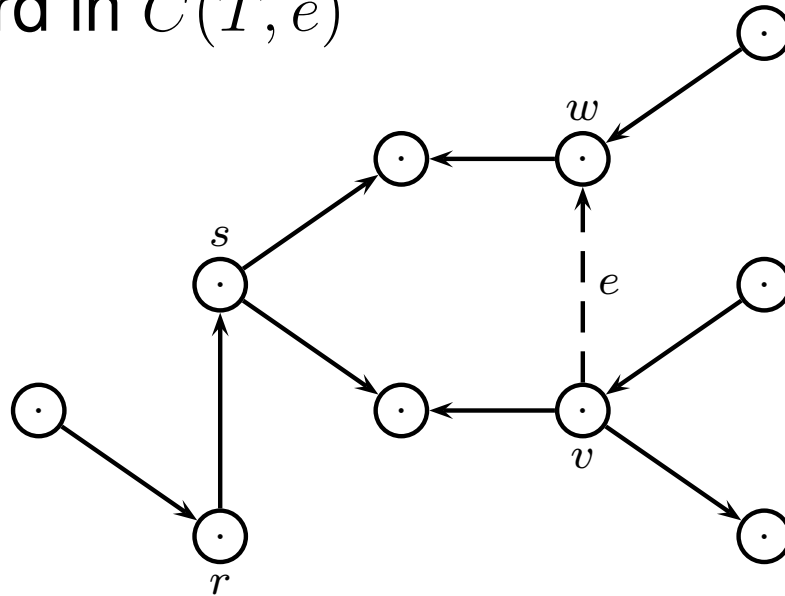
- Select a vertex r as **root** of T . Consider an edge h . This edge will partition V into two parts:
 - a part containing r denoted R_h^T , and
 - the remaining nodes $S_h^T = V \setminus R_h^T$.



- If h starts in R_h^T and ends in S_h^T then set $x_h = -\sum_{i \in R_h^T} b_i$
- If h starts in S_h^T and ends in R_h^T then set $x_h = \sum_{i \in R_h^T} b_i$

Negative cost circuit

- The Network Simplex Method moves from tree solution to tree solution using negative cost circuits $C(T, e)$ consisting of tree edges *and exactly one* non-tree edge e
- We have the following properties:
 - $C(T, e) \subseteq T \cup \{e\}$
 - e is forward in $C(T, e)$

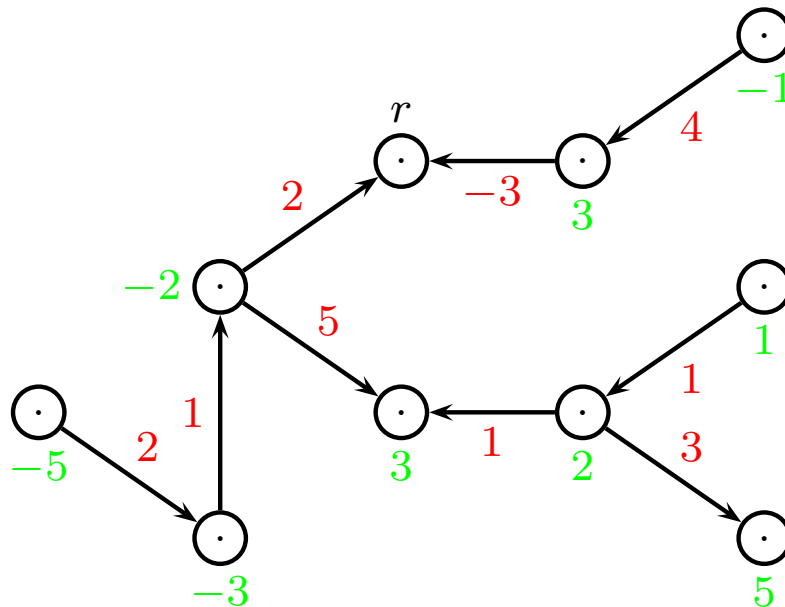


Set the potentials

- Set the vertex potential $y_i, i \in V$ equal to the cost of the path in T from r to i (counted with sign: plus for a forward edge, minus for a backward edge). It now holds:
- For all $i, j \in V$, the cost of the path from i to j in T equals $y_j - y_i$ (why ?). But then the reduced costs $\bar{w}_{ij}$ (defined by $w_{ij} + y_i - y_j$) satisfy:

$$\forall e \in T : \bar{w}_e = 0$$

$$\forall e \notin T : \bar{w}_e = \text{cost}(C(T, e))$$



Algorithm

find a tree T whose associated flow x is feasible

compute y

while there exists an arc $e = (u, v)$ such that $\bar{w}_e = w_e + y_u - y_v < 0$

find such an arc e

if $C(T, e)$ has no reverse arc, **stop**

compute $\theta = \min\{x_j : j \text{ reverse arc of } C(T, e)\}$

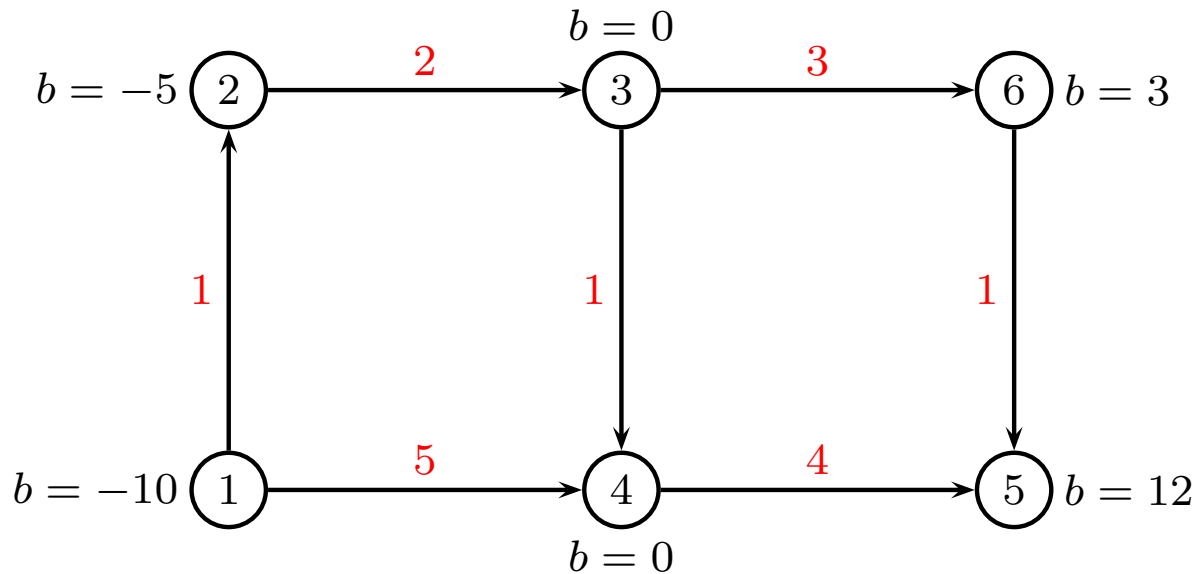
find a reverse arc h of $C(T, e)$ with $x_h = \theta$

augment x by θ on $C(T, e)$

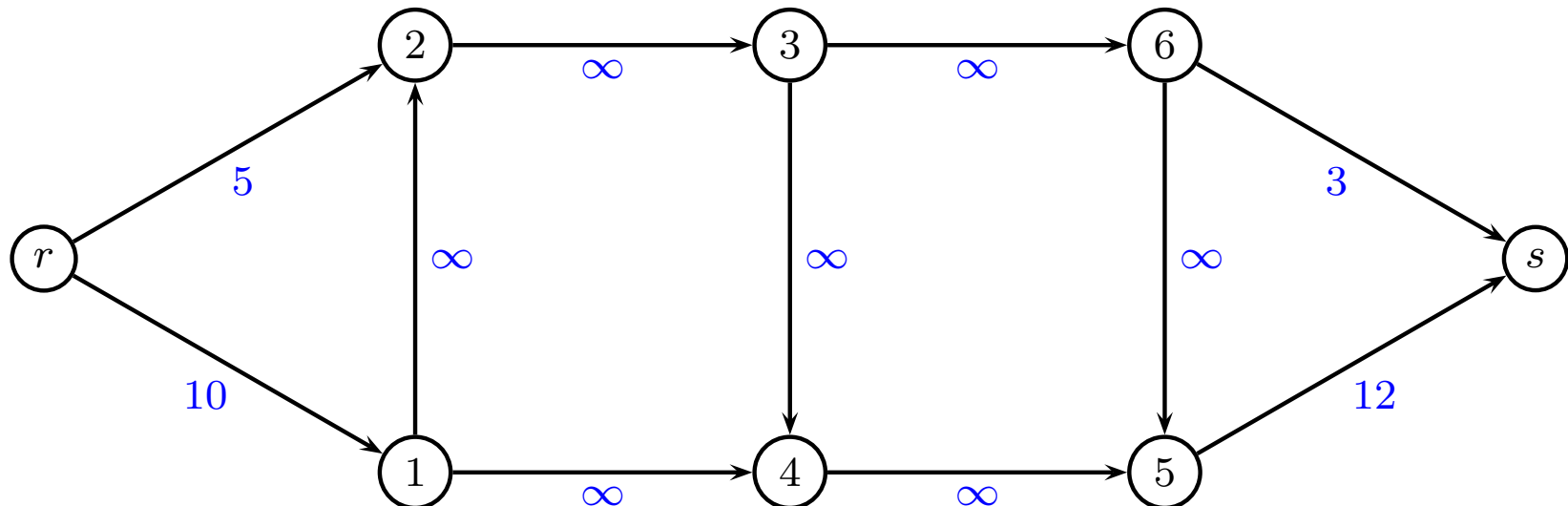
set $T := T \cup \{e\} \setminus \{h\}$

update y

Transshipment - Example



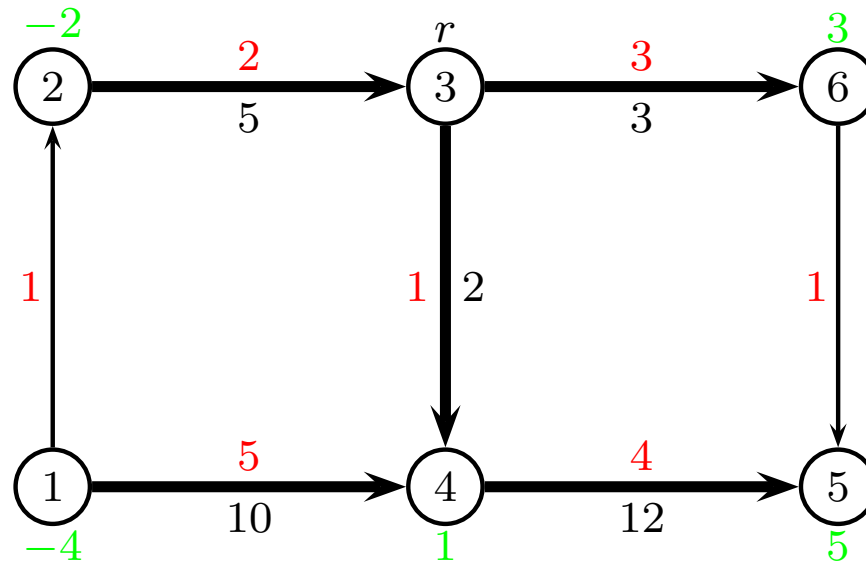
Solve max-flow to find initial solution:



We can drop demands in following

Transshipment - Example

Tree solution:



reduced cost $\bar{w}_{12} = 1 + (-4) - (-2) = -1$

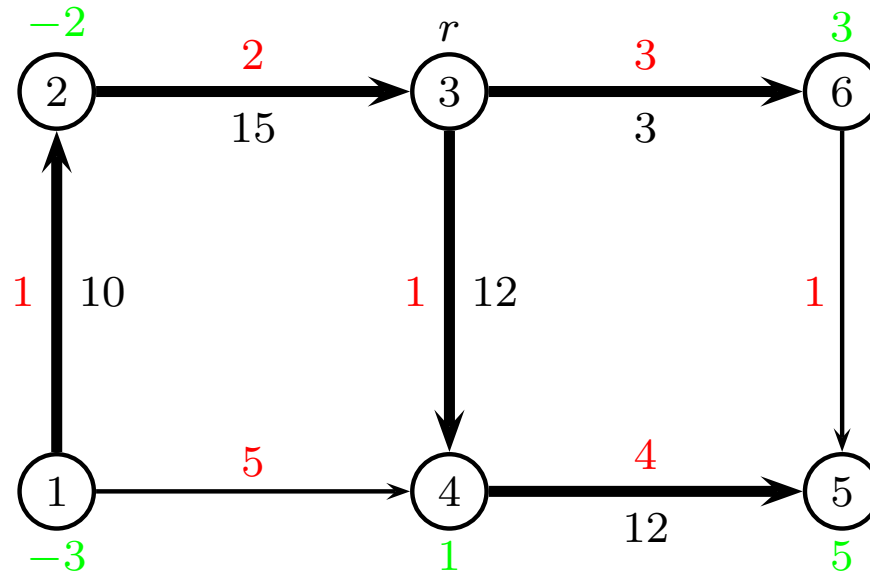
reduced cost $\bar{w}_{65} = 1 + 3 - 5 = -1$

imposed cycle $C = \{1, 2, 3, 4\}$

find $\theta = \min\{x_{14}\} = \min\{10\} = 10$

find edge $(1, 4) = \arg \min\{x_{14}\}$

Transshipment - Example



reduced cost $\bar{w}_{14} = 5 + (-3) - 1 = 1$

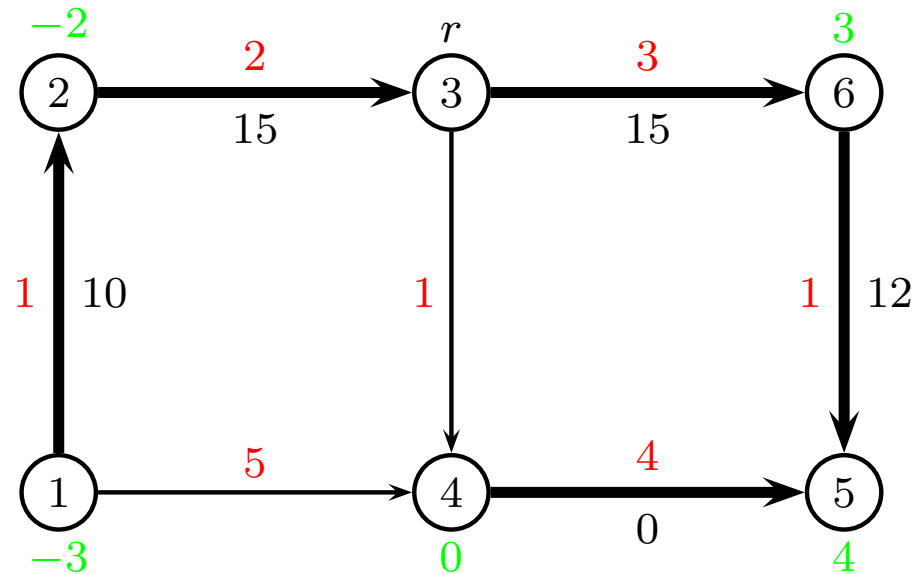
reduced cost $\bar{w}_{65} = 1 + 3 - 5 = -1$

imposed cycle $C = \{3, 6, 5, 4\}$

find $\theta = \min\{x_{34}, x_{45}\} = \min\{12, 12\} = 12$

find edge $(3, 4) = \arg \min\{x_{34}, x_{45}\}$

Transshipment - Example



reduced cost $\bar{w}_{14} = 5 + (-3) - 0 = 2$

reduced cost $\bar{w}_{34} = 1 + 0 - 0 = 1$

stop

Now: $\bar{w}_e = 0$ when $e \in T$, and $\bar{w}_e > 0$ when $e \notin T$.

Optimal solution

Min Cost Flow Problem

What if edge capacities are present?

- We consider the Min Cost flow - problem given by the network $G = (V, E)$ with demands b_i , $i \in V$, capacities u_e , $e \in E$ and costs w_e , $e \in E$.
- Remember the optimality conditions for a feasible flow $x \in \mathbb{R}^E$:

$$\bar{w}_e < 0 \Rightarrow x_e = u_e \ (\neq \infty)$$

$$\bar{w}_e > 0 \Rightarrow x_e = 0$$

Tree Solution Extended

- The concept of a tree solution is extended to capacity constrained problems – a non-tree edge e may have flow either 0 or u_e .
- $E \setminus T$ is hence split into two edge sets, L and U . Edges in L must have flow 0 - edges in U must be filled to their capacity.
- The tree solution x must satisfy:

$$\forall i \in V \quad : \quad f_x(i) = b_i$$

$$\forall e \in L \quad : \quad x_e = 0$$

$$\forall e \in U \quad : \quad x_e = u_e$$

A unique tree solution

- As before, the tree solution for T is unique:
- Select a vertex r called the *root* of T . Consider an edge h in T . h partitions V into two:
 - a part containing r denoted R_h^T , and
 - a remainder $S_h^T = V \setminus R_h^T$.
- Consider now the *modified demand* $B(S_h^T)$ in S_h^T given that a tree solution is sought:

$$B(S_h^T) = \sum_{i \in S_h^T} b_i - \sum_{\{(i,j) \in U : i \in R_h^T, j \in S_h^T\}} u_{ij} + \sum_{\{(i,j) \in U : i \in S_h^T, j \in R_h^T\}} u_{ij}$$

- if h starts in R_h^T and ends in S_h^T then set $x_h = B(S_h^T)$
- if h starts in S_h^T and ends in R_h^T then set $x_h = -B(S_h^T)$

Network Simplex for Min Cost Flow

- Idea: Start with a feasible tree solution - find it e.g using a Max Flow algorithm.
- We now search for *either* a non-tree edge e with $x_e = 0$ and negative reduced cost *or* a non-tree edges e with $x_e = u_e$ and positive reduced cost. Given e and (T, L, U) , the corresponding circuit is denoted $C(T, L, U, e)$.
- In this, the flow must be *increased* in forward edges respecting capacity constraints and *decreased* in backward edges respect the non-negativity constraints.

Algorithm

find a triple (T, L, U) whose associated flow x is feasible
compute y

while there exists an arc $e = (u, v) \in L$ such that $\bar{w}_e < 0$

... or $e \in U$ and $\bar{w}_e > 0$

find such an arc e

if $C(T, L, U, e)$ has no reverse arc

... and no forward arc of finite capacity, **stop**

compute $\theta_1 = \min\{x_j : j \text{ reverse arc of } C(T, L, U, e)\}$

compute $\theta_2 = \min\{u_j - x_j : j \text{ forward arc of } C(T, L, U, e)\}$

let $\theta = \min\{\theta_1, \theta_2\}$

find a reverse arc h of $C(T, L, U, e)$ with $x_h = \theta$

... or forward arc with $u_h - x_h = \theta$

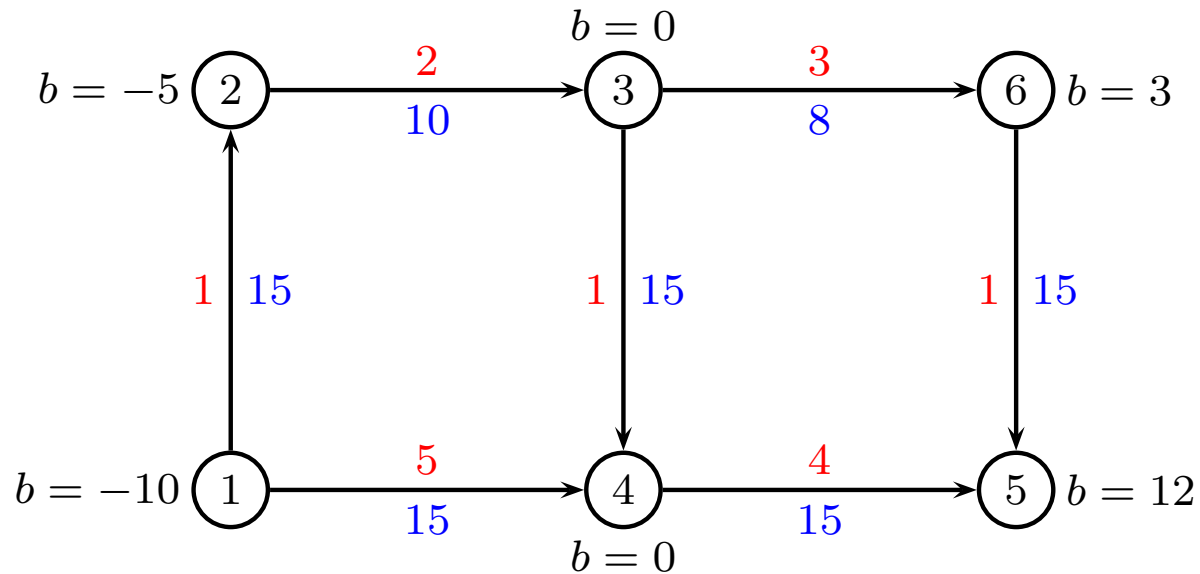
augment x by θ on $C(T, L, U, e)$

set $T := T \cup \{e\} \setminus \{h\}$

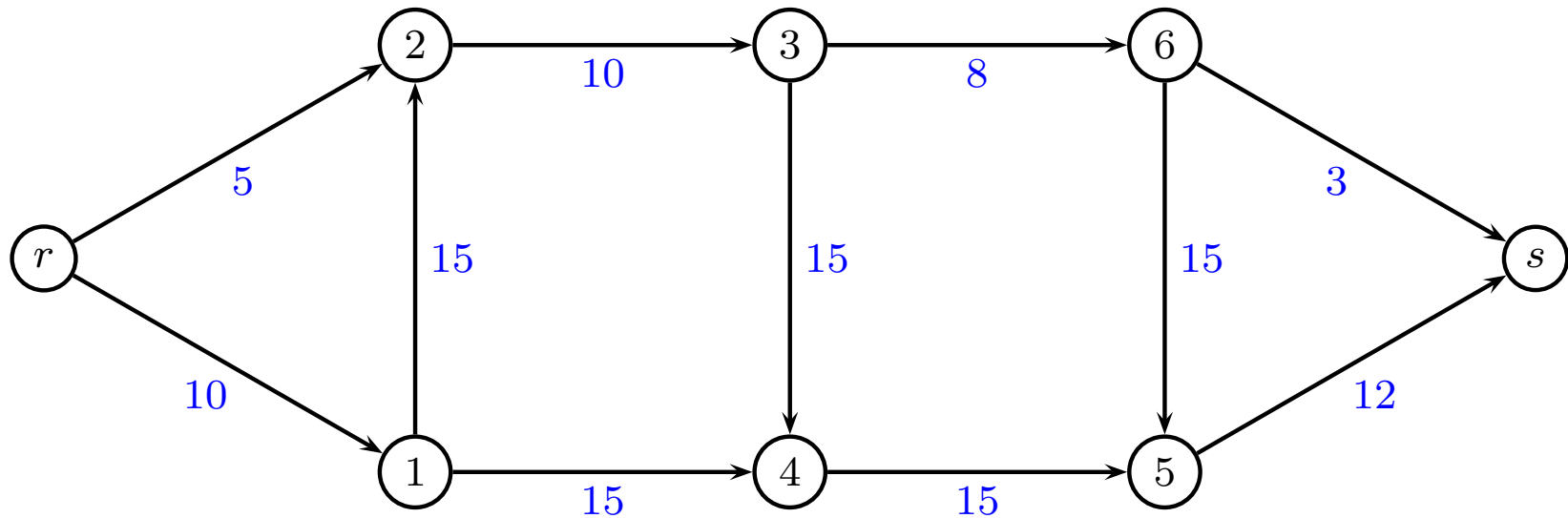
update L, U by deleting h and adding e as appropriate

update y

Min-cost flow - Example

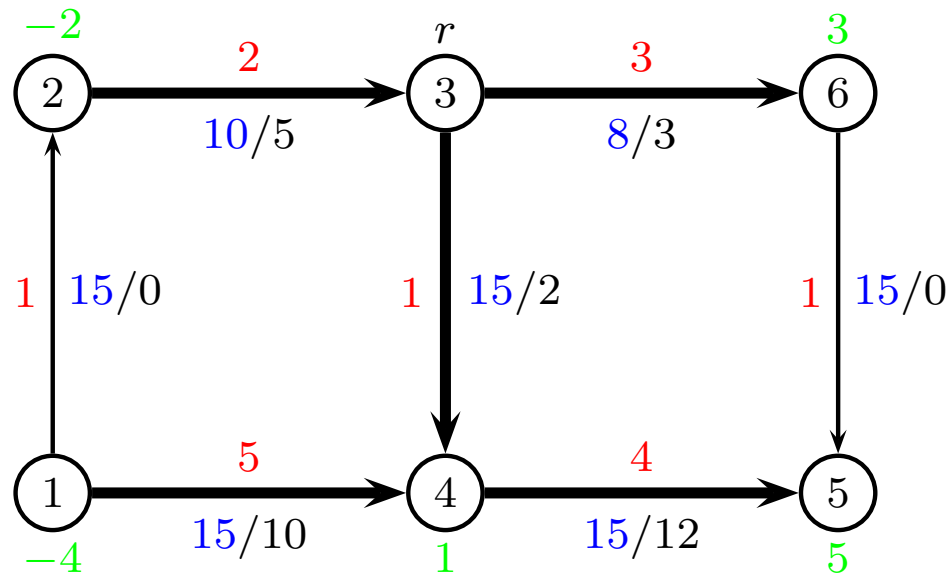


Solve max-flow to find initial solution:



Drop demands in following

Min-cost flow - Example



$$L = \{(1, 2), (6, 5)\}, U = \{\}$$

$$\text{reduced cost } \bar{w}_{12} = 1 + (-4) - (-2) = -1$$

(L edge)

$$\text{reduced cost } \bar{w}_{65} = 1 + 3 - 5 = -1$$

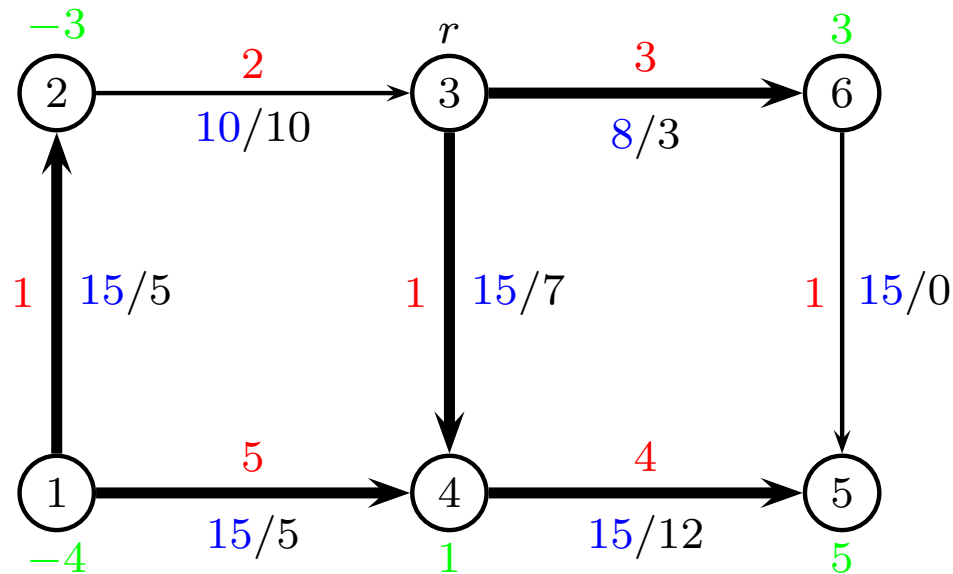
(L edge)

$$\text{cycle } C = \{1, 2, 3, 4\}$$

$$\text{find } \theta = \min\{15, 5, 13, 10\} = 5$$

Find: $\bar{w}_{ij} < 0$ if $(i, j) \in L$, or $\bar{w}_{ij} > 0$ if $(i, j) \in U$

Min-cost flow - Example



$$L = \{(6, 5)\}, U = \{(2, 3)\}$$

$$\text{reduced cost } \bar{w}_{23} = 2 + (-3) - 0 = -1$$

(U edge)

$$\text{reduced cost } \bar{w}_{65} = 1 + 3 - 5 = -1$$

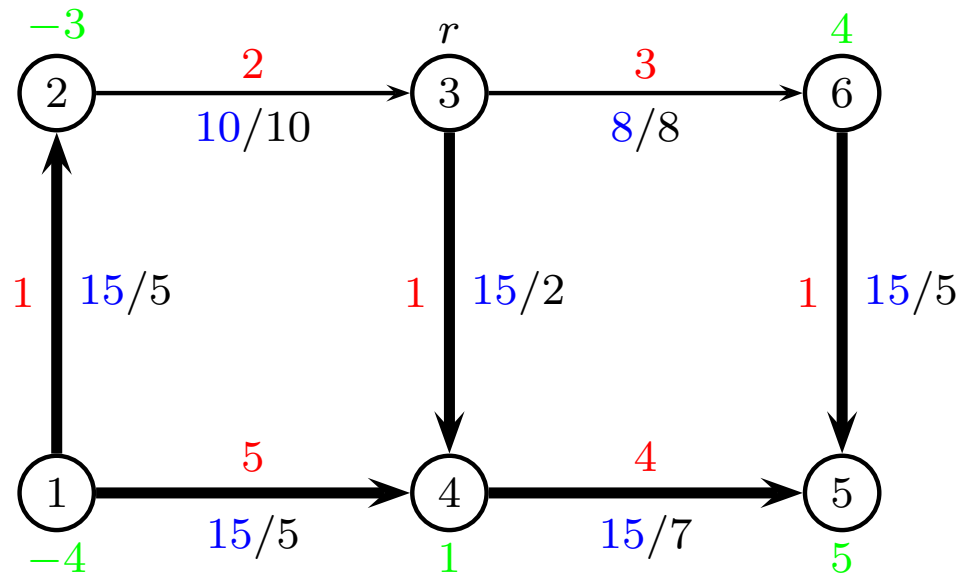
(L edge)

$$\text{cycle } C = \{3, 6, 5, 4\}$$

$$\text{find } \theta = \min\{5, 15, 12, 7\} = 5$$

Find: $\bar{w}_{ij} < 0$ if $(i, j) \in L$, or $\bar{w}_{ij} > 0$ if $(i, j) \in U$

Min-cost flow - Example



$$L = \{\}, U = \{(2, 3), (3, 6)\}$$

$$\text{reduced cost } \bar{w}_{23} = 2 + (-3) - 0 = -1$$

(U edge)

$$\text{reduced cost } \bar{w}_{36} = 3 + 0 - 4 = -1$$

(U edge)

stop

$$\text{cost: } z = 2 \cdot 10 + 3 \cdot 8 + 1 \cdot 5 + 1 \cdot 2 + 1 \cdot 5 + 5 \cdot 5 + 4 \cdot 7 = 109$$

Find: $\bar{w}_{ij} < 0$ if $(i, j) \in L$, or $\bar{w}_{ij} > 0$ if $(i, j) \in U$

Conclusion

Advantages over augmenting circuit

- Potentials y can be calculated in linear time
- Augmenting edge can be found in linear time
- Can be combined with selection criteria to reach polynomial running time

Relation to simplex

- basis of $n - 1$ edges
- most negative reduced cost enters basis
- bounds handled implicitly

CPLEX/Gurobi uses network simplex when appropriate structure

Tips-and-tricks

Network simplex

- if graph is not connected, split the problem into two or more problems
- check that problem is balanced, otherwise add surplus node(s) to make problem balanced
- if basis has less than $n - 1$ edges, add additional (thin) edges until basis is a spanning tree
- if $\theta = 0$ then just send zero items along the cycle, this will give a new basis
- the cycle C should move in the direction of the thin edge (i', j') with most negative reduced cost
- after an iteration, edge (i', j') becomes fat, and the bottleneck edge becomes thin

Multicommodity Flow Problem

David Pisinger

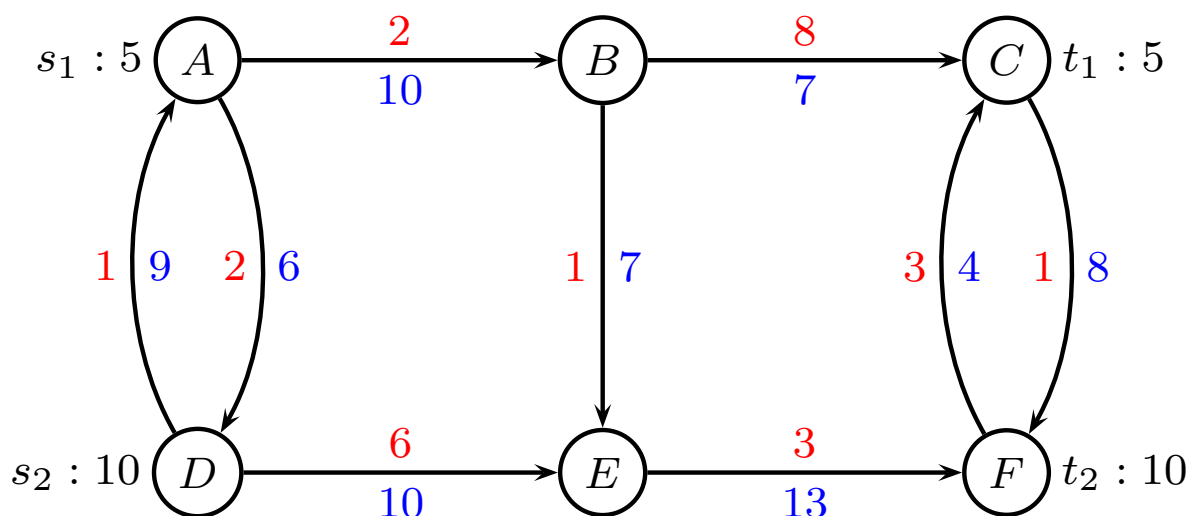
`pisinger@man.dtu.dk`

DTU Management
Technical University of Denmark

© Copyright David Pisinger. May not be distributed

Multicommodity Flow - Terminology

- We consider a directed graph $G = (V, E)$ in which each edge e has a capacity $u_e \in \mathbb{R}_+$ and a unit transportation cost $c_e \in \mathbb{R}$.
- A set K of commodities is given. Each commodity k has a source s_k , terminal t_k , and demand d_k .
- All commodities k should be transported from source to sink in the cheapest possible way, respecting capacities.



Applications

Direct network applications

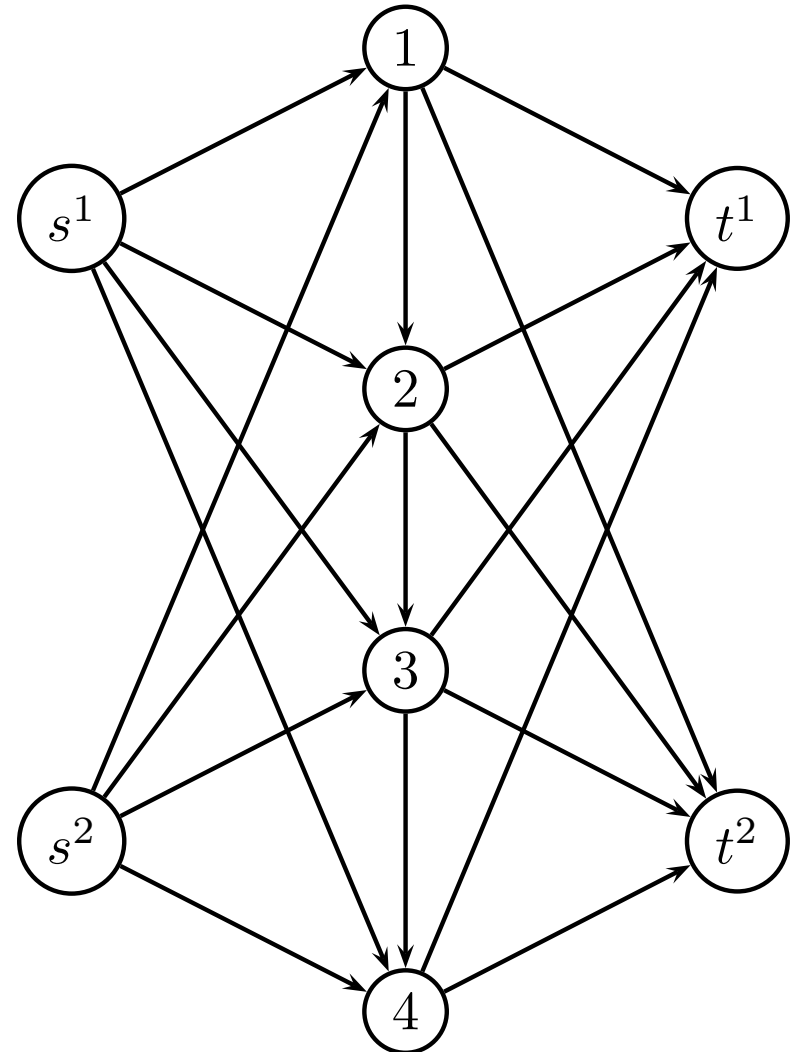
- Communication networks
- Computer networks
- Transportation network (busses, vehicles, trains)
- Foodgrain export-import network

Several commodities share capacity of network

Application: Warehousing

Seasonal production planning: four time periods i , two products k , shared inventory capacity u_i

- production cost p_i^k
- production capacity c_i^k
- storage cost h_i
- storage capacity u_i
- sales price z_i^k
- demand d_i^k



Multicommodity Flow, edge formulation

Let x_{ij}^k denote flow of commodity k on edge (i, j) .

$$\begin{aligned} \min \quad & \sum_{k \in K} \sum_{(i,j) \in E} c_{ij} x_{ij}^k \\ \text{s.t.} \quad & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = 0 & k \in K, j \in V \setminus \{s_k, t_k\} \\ & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = -d_k & k \in K, j = s_k \\ & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = d_k & k \in K, j = t_k \\ & \sum_{k \in K} x_{ij}^k \leq u_{ij} & (i, j) \in E \\ & 0 \leq x_{ij}^k \leq d_k & k \in K, (i, j) \in E \end{aligned}$$

Often integral flow: $x_{ij} \in \mathbb{Z}$

LP-problem hard to solve for large problems

IP-problem NP-hard to solve

MCF - Variants

- min total cost (normal formulation)
- variable cost for each k
- max flow (costs ignored)
- congestion
- splittable
- unsplittable
- k -splittable

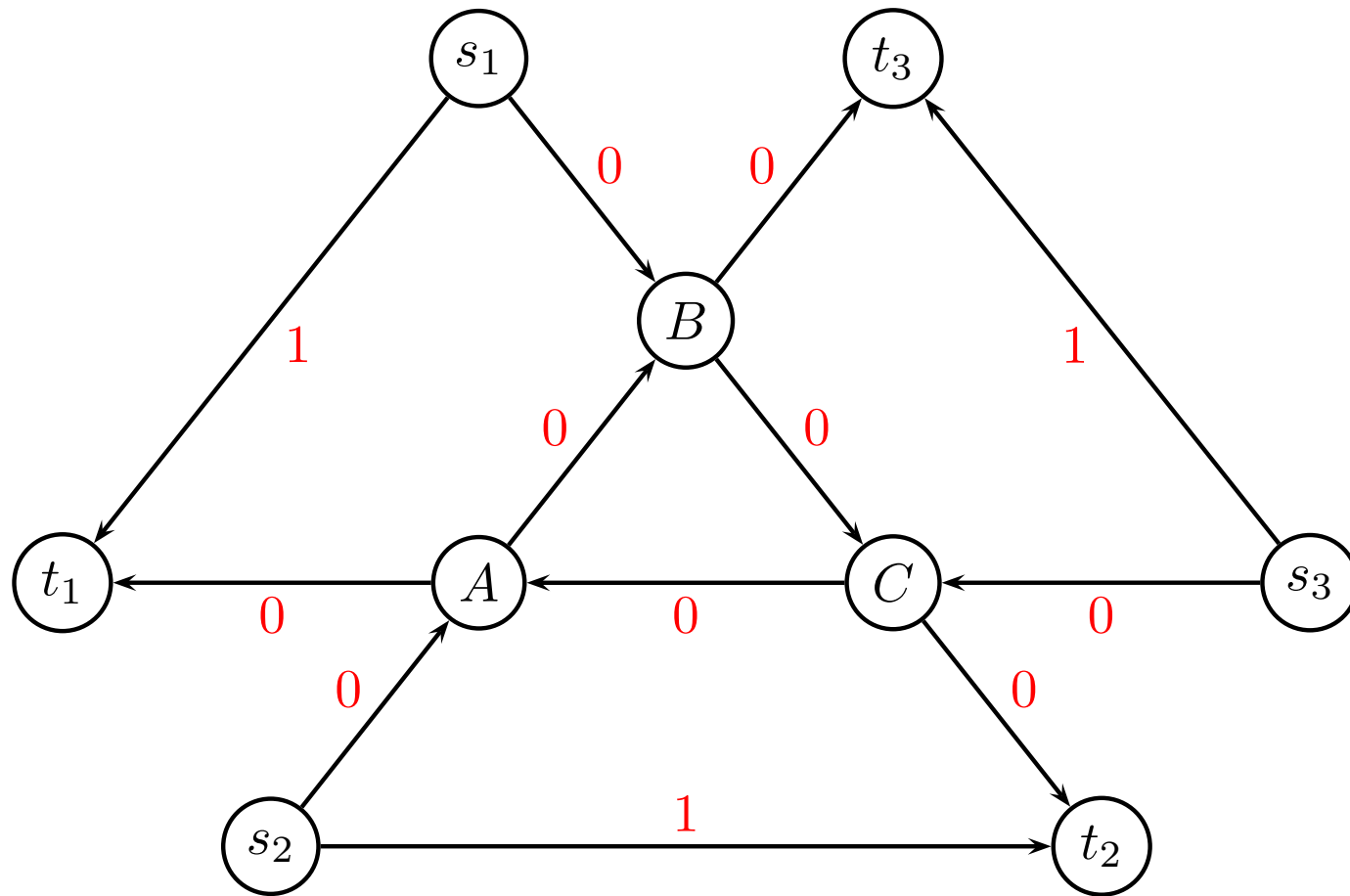
Congestion:

$$\min \sum_{(i,j) \in E} \frac{\sum_k x_{ij}}{u_{ij} - \sum_k x_{ij}}$$

Structure of LP model

	$x_{e_1}^1$	$x_{e_1}^2$	$x_{e_2}^1$	$x_{e_2}^2$	...	x_{ij}^k	...	$x_{e_m}^K$		
	$w_{e_1}^1$	$w_{e_1}^2$	$w_{e_2}^1$	$w_{e_2}^2$	...	w_{ij}^k	...	$w_{e_m}^K$		
1	-1	-1	1	-1	...	.	...	.	=	b_1
2	.	1	-1	.	...	1	...	.	=	b_2
.	.	.	.	.	...	.	...	.	=	.
.	.	.	.	.	...	.	...	.	=	.
n	.	.	.	1	...	-1	...	1	=	b_n
e_1	-1	.	-1	.	...	.	...	.	$\geq$	$-u_1$
e_2		-1	.	-1	...	-1	...	.	$\geq$	$-u_2$
.	.	.	.	.	...	.	...	.	$\geq$	.
e_m	.	.	.	.	...	.	...	-1	$\geq$	$-u_m$

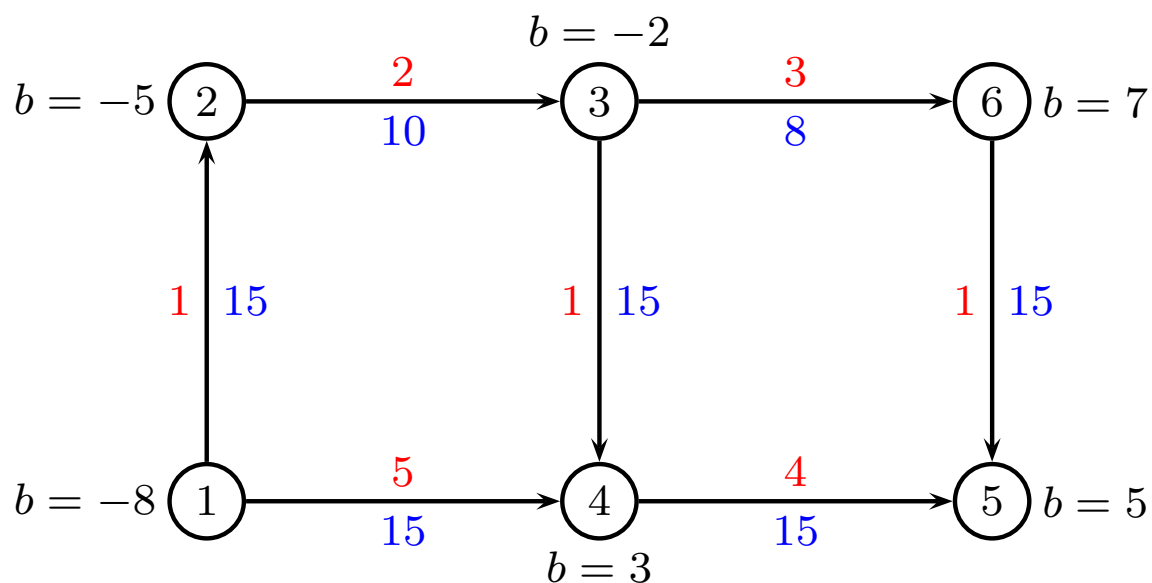
Fractional solutions



Demand $d_k = 1$ for all k . Capacity $u_e = 1$ for all e

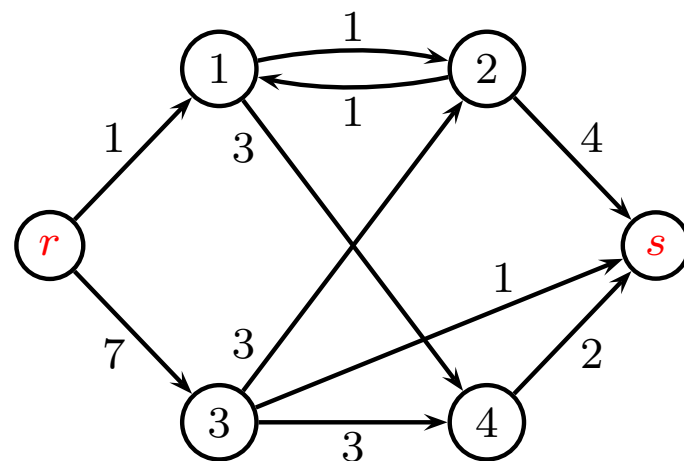
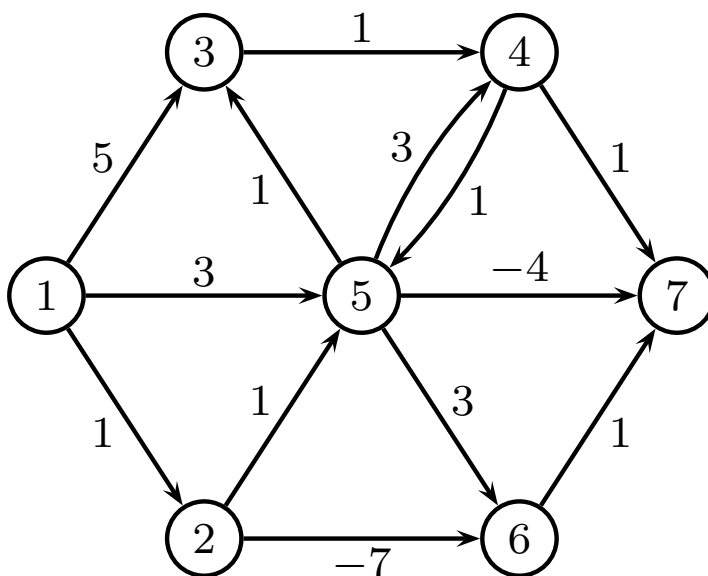
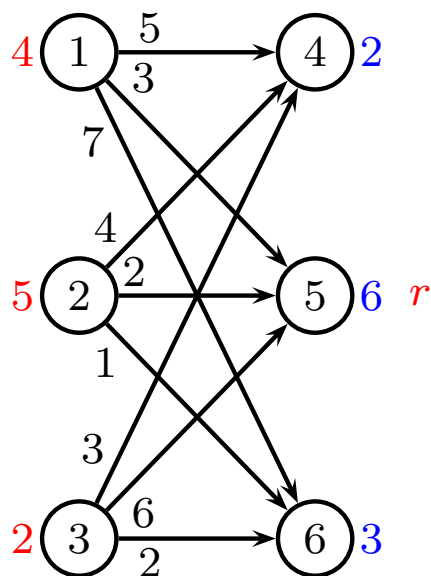
Special cases

MCF contains min cost flow as special case (one commodity)



Special cases

1. The Transportation Problem
2. The Shortest Path Problem
3. The Max Flow Problem



Solution methods

LP-based

- Simplex
- interior-point methods

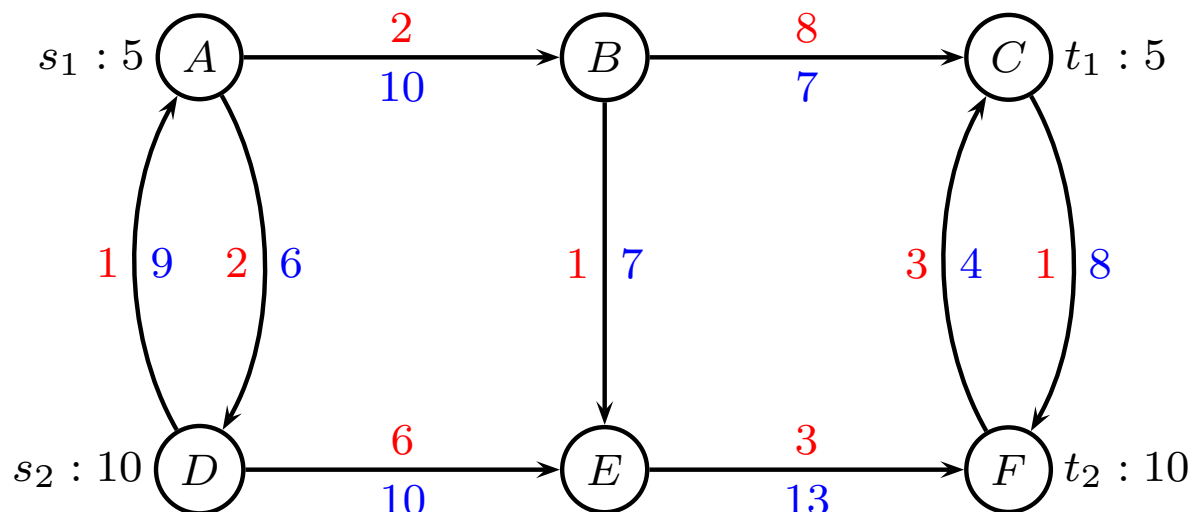
Column generation

- Dantzig Wolfe decomposition
- Master Problem
- Subproblem

Lagrangian methods

- Relax (delete) capacity constraints
- Punish costs if capacity violated

Multicommodity Flow - Example



All possible paths

path number	commodity k	path p	$\sum c_{ij}$
1	1	$A \rightarrow B \rightarrow C$	10
2	1	$A \rightarrow B \rightarrow E \rightarrow F \rightarrow C$	9
3	1	$A \rightarrow D \rightarrow E \rightarrow F \rightarrow C$	14
4	2	$D \rightarrow E \rightarrow F$	9
5	2	$D \rightarrow A \rightarrow B \rightarrow C \rightarrow F$	12
6	2	$D \rightarrow A \rightarrow B \rightarrow E \rightarrow F$	7

Path formulation

P_k set of possible paths from s_k to t_k

Binary $a_{p,ij}^k$ is 1 if path p for commodity k is using edge (i, j)

$\hat{c}_p^k$ denote the cost for path $p \in P_k$

x_p^k is flow on path p for commodity k

$$\begin{aligned} \min \quad & \sum_{k \in K} \sum_{p \in P_k} \hat{c}_p^k x_p^k \\ \text{s.t.} \quad & \sum_{k \in K} \sum_{p \in P_k} a_{p,ij}^k x_p^k \leq u_{ij} \quad (i, j) \in E \\ & \sum_{p \in P_k} x_p^k = d_k \quad k \in K \\ & 0 \leq x_p^k \leq d_k \quad k \in K, p \in P_k \end{aligned}$$

Dantzig-Wolfe reformulation

Generally easier to solve since fewer constraints

Path model

Set of paths: $P_1 = \{1, 2, 3\}$, $P_2 = \{4, 5, 6\}$.

minimize

$$10x_1 + 9x_2 + 14x_3 + 9x_4 + 12x_5 + 7x_6$$

subject to

$$\text{AB: } +x_1 + x_2 \quad \quad +x_5 + x_6 \leq 10 \quad (y_{\text{AB}})$$

$$\text{BC: } +x_1 \quad \quad +x_5 \leq 7 \quad (y_{\text{BC}})$$

$$\text{DE: } \quad \quad +x_3 + x_4 \leq 10 \quad (y_{\text{DE}})$$

$$\text{EF: } \quad +x_2 + x_3 + x_4 \quad +x_6 \leq 13 \quad (y_{\text{EF}})$$

$$\text{BE: } \quad +x_2 \quad \quad +x_6 \leq 7 \quad (y_{\text{BE}})$$

$$\text{AD: } \quad \quad +x_3 \leq 6 \quad (y_{\text{AD}})$$

$$\text{DA: } \quad \quad +x_5 + x_6 \leq 9 \quad (y_{\text{DA}})$$

$$\text{CF: } \quad \quad +x_5 \leq 8 \quad (y_{\text{CF}})$$

$$\text{FC: } \quad +x_2 + x_3 \leq 4 \quad (y_{\text{FC}})$$

$$\text{K1: } x_1 + x_2 + x_3 \geq 5 \quad (y^*_{\text{1}})$$

$$\text{K2: } x_4 + x_5 + x_6 \geq 10 \quad (y^*_{\text{2}})$$

Path model

P_k (paths from s_k to t_k) can be very large

Column generation (remember: fixed charge transportation problem)

- Master problem
- Restricted master problem $\overline{P}_k \subset P_k$
- Subproblem (generate a column — a path from s_k to t_k)

First iteration

Initial solution, paths 1 and 4. Restricted master problem:

minimize

$$10x_1 + 9x_4$$

subject to

$$\text{AB: } +x_1 \leq 10 \quad (y_{\text{AB}})$$

$$\text{BC: } +x_1 \leq 7 \quad (y_{\text{BC}})$$

$$\text{DE: } +x_4 \leq 10 \quad (y_{\text{DE}})$$

$$\text{EF: } +x_4 \leq 13 \quad (y_{\text{EF}})$$

$$\text{BE: } \leq 7 \quad (y_{\text{BE}})$$

$$\text{AD: } \leq 6 \quad (y_{\text{AD}})$$

$$\text{DA: } \leq 9 \quad (y_{\text{DA}})$$

$$\text{CF: } \leq 8 \quad (y_{\text{CF}})$$

$$\text{FC: } \leq 4 \quad (y_{\text{FC}})$$

$$\text{K1: } x_1 \geq 5 \quad (y^*_1)$$

$$\text{K2: } x_4 \geq 10 \quad (y^*_2)$$

Solution: $x_1 = 5, x_4 = 10$

Dual: $y_1^* = 10, y_2^* = 9$

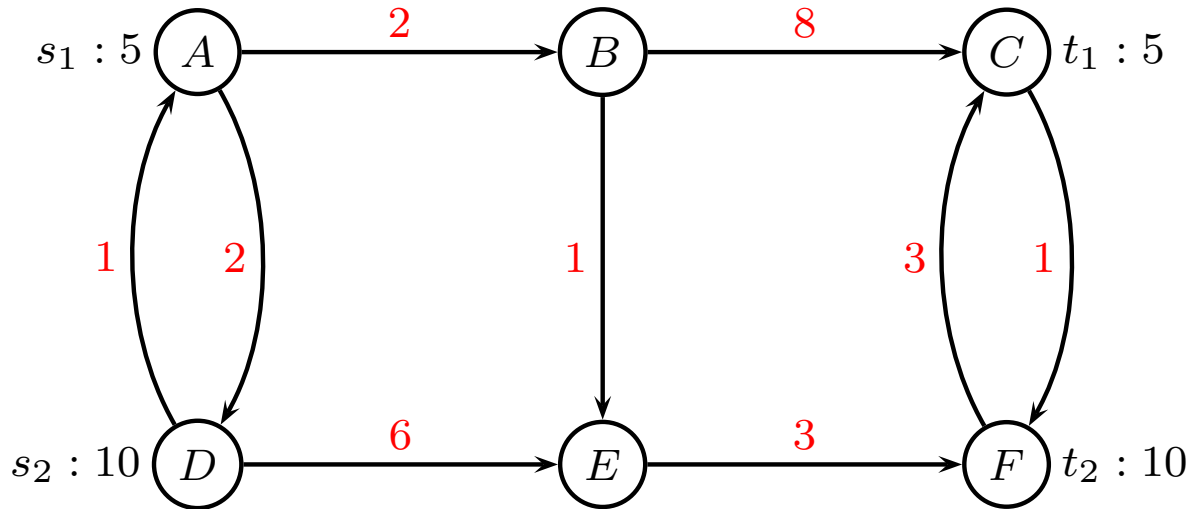
Calculate reduced costs

Reduced costs $\bar{c}_p = \sum_{(i,j) \in p} (c_{ij} - y_{ij}) - y_k^*$

path number	commodity k	path p	$\sum(c_{ij} - y_{ij})$	$\bar{c}_p$
1	1	A→B→C	10	0
2	1	A→B→E →F→C	9	-1
3	1	A→D→E →F→C	14	4
4	2	D→E→F	9	0
5	2	D→A→B→C→F	12	3
6	2	D→A→B→E→F	7	-2

Shortest path problem

Alternatively, find shortest path for each commodity



Commodity 1: $A \rightarrow B \rightarrow E \rightarrow F \rightarrow C$ cost 9

Commodity 2: $D \rightarrow A \rightarrow B \rightarrow E \rightarrow F$ cost 7

subtract $y_k^* = (10, 9)$ from path length

Second iteration

Add path 6. Restricted master problem:

minimize

$$10x_1 + 9x_4 + 7x_6$$

subject to

$$\text{AB: } +x_1 \quad +x_6 \leq 10 \quad (y_{AB})$$

$$\text{BC: } +x_1 \leq 7 \quad (y_{BC})$$

$$\text{DE: } \quad +x_4 \leq 10 \quad (y_{DE})$$

$$\text{EF: } \quad +x_4 + x_6 \leq 13 \quad (y_{EF})$$

$$\text{BE: } \quad +x_6 \leq 7 \quad (y_{BE})$$

$$\text{AD: } \leq 6 \quad (y_{AD})$$

$$\text{DA: } \quad +x_6 \leq 9 \quad (y_{DA})$$

$$\text{CF: } \leq 8 \quad (y_{CF})$$

$$\text{FC: } \leq 4 \quad (y_{FC})$$

$$\text{K1: } x_1 \geq 5 \quad (y^*_1)$$

$$\text{K2: } x_4 + x_6 \geq 10 \quad (y^*_2)$$

Solution: $x_1 = 5, x_4 = 5, x_6 = 5$

Dual: $y_1^* = 12, y_2^* = 9 \quad y_{AB} = -2$

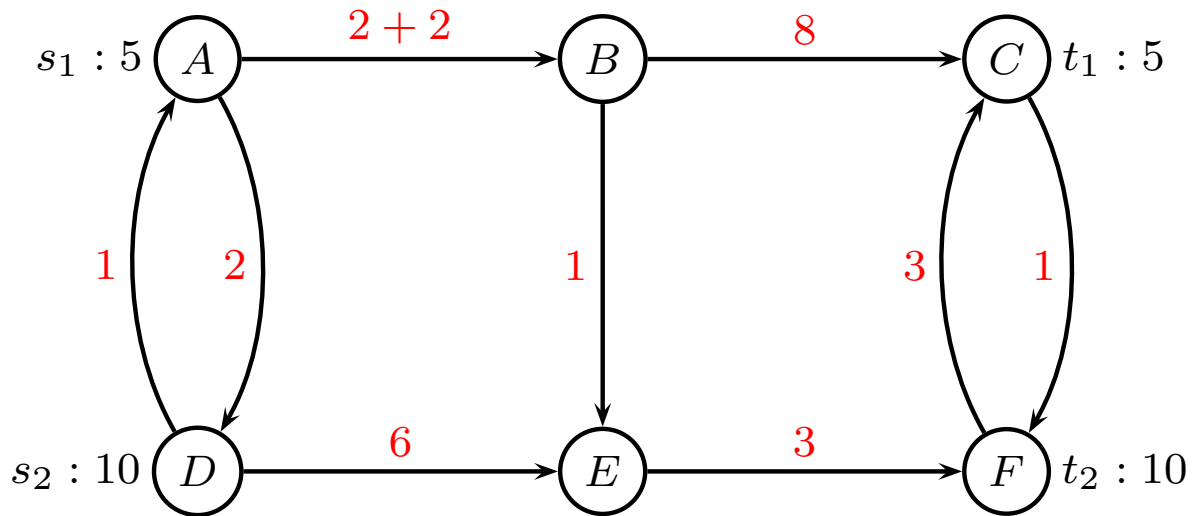
Calculate reduced costs

Reduced costs $\bar{c}_p = \sum_{(i,j) \in p} (c_{ij} - y_{ij}) - y_k^*$

path number	commodity k	path p	$\sum(c_{ij} - y_{ij})$	$\bar{c}_p$
1	1	A→B→C	12	0
2	1	A→B→E →F→C	11	-1
3	1	A→D→E →F→C	14	2
4	2	D→E→F	9	0
5	2	D→A→B→C→F	14	5
6	2	D→A→B→E→F	9	0

Shortest path problem

Alternatively, find shortest path for each commodity



Commodity 1: $A \rightarrow B \rightarrow E \rightarrow F \rightarrow C$ cost 11

Commodity 2: $D \rightarrow E \rightarrow F$ cost 9

subtract $y_k^* = (12, 9)$ from path length

Third iteration

Add path 2. Restricted master problem:

minimize

$$10x_1 + 9x_2 + 9x_4 + 7x_6$$

subject to

$$\text{AB: } +x_1 + x_2 + x_6 \leq 10 \quad (y_{AB})$$

$$\text{BC: } +x_1 \leq 7 \quad (y_{BC})$$

$$\text{DE: } +x_4 \leq 10 \quad (y_{DE})$$

$$\text{EF: } +x_2 + x_4 + x_6 \leq 13 \quad (y_{EF})$$

$$\text{BE: } +x_2 + x_6 \leq 7 \quad (y_{BE})$$

$$\text{AD: } \leq 6 \quad (y_{AD})$$

$$\text{DA: } +x_6 \leq 9 \quad (y_{DA})$$

$$\text{CF: } \leq 8 \quad (y_{CF})$$

$$\text{FC: } +x_2 \leq 4 \quad (y_{FC})$$

$$\text{K1: } x_1 + x_2 \geq 5 \quad (y^*_1)$$

$$\text{K2: } x_4 + x_6 \geq 10 \quad (y^*_2)$$

Solution: $x_1 = 3, x_2 = 2, x_4 = 5, x_6 = 5$

Dual: $y_1^* = 11, y_2^* = 9 \quad y_{AB} = -1, y_{BE} = -1$

Calculate reduced costs

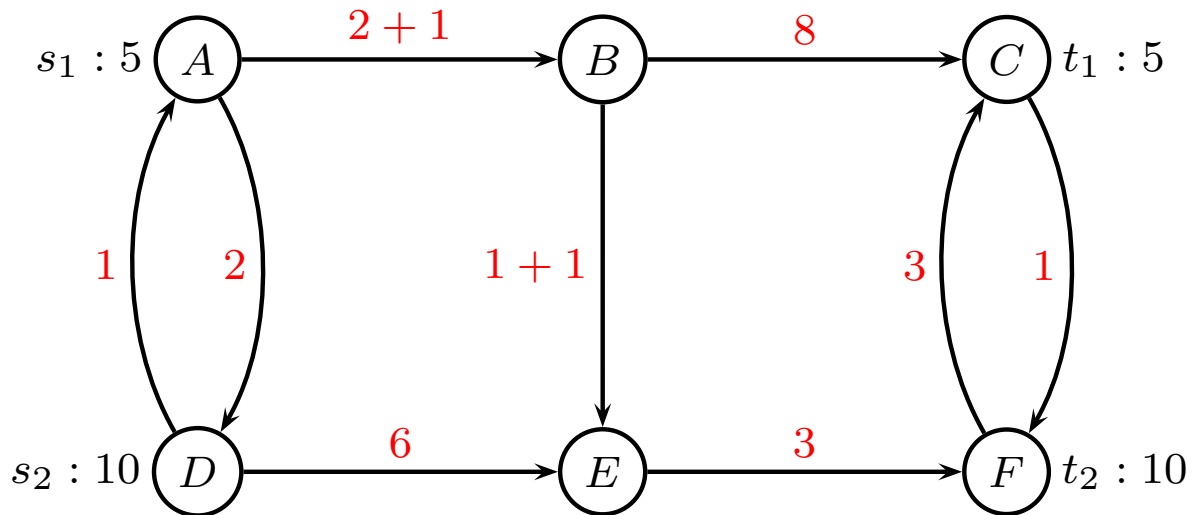
Reduced costs $\bar{c}_p = \sum_{(i,j) \in p} (c_{ij} - y_{ij}) - y_k^*$

path number	commodity k	path p	$\sum(c_{ij} - y_{ij})$	$\bar{c}_p$
1	1	A→B→C	11	0
2	1	A→B→E →F→C	11	0
3	1	A→D→E →F→C	14	3
4	2	D→E→F	9	0
5	2	D→A→B→C→F	13	4
6	2	D→A→B→E→F	9	0

No negative reduced costs, stop.

Shortest path problem

Alternatively, find shortest path for each commodity



Commodity 1: $A \rightarrow B \rightarrow C$ cost 11

Commodity 2: $D \rightarrow E \rightarrow F$ cost 9

subtract $y_k^* = (11, 9)$ from path length

No negative reduced cost, stop

Column Generation - summing up

Solution: $x_1 = 3, x_2 = 2, x_4 = 5, x_6 = 5$

Objective: $z = 10 \cdot 3 + 9 \cdot 2 + 9 \cdot 5 + 7 \cdot 5 = 128$

Generated only 4 out of 6 paths

In general:

- Interaction between master problem and subproblem
- Add all paths with negative reduced cost to master problem
- Stabilization: Dual variables may fluctuate in the beginning, add soft constraints to dual variables

Column Generation - advantages

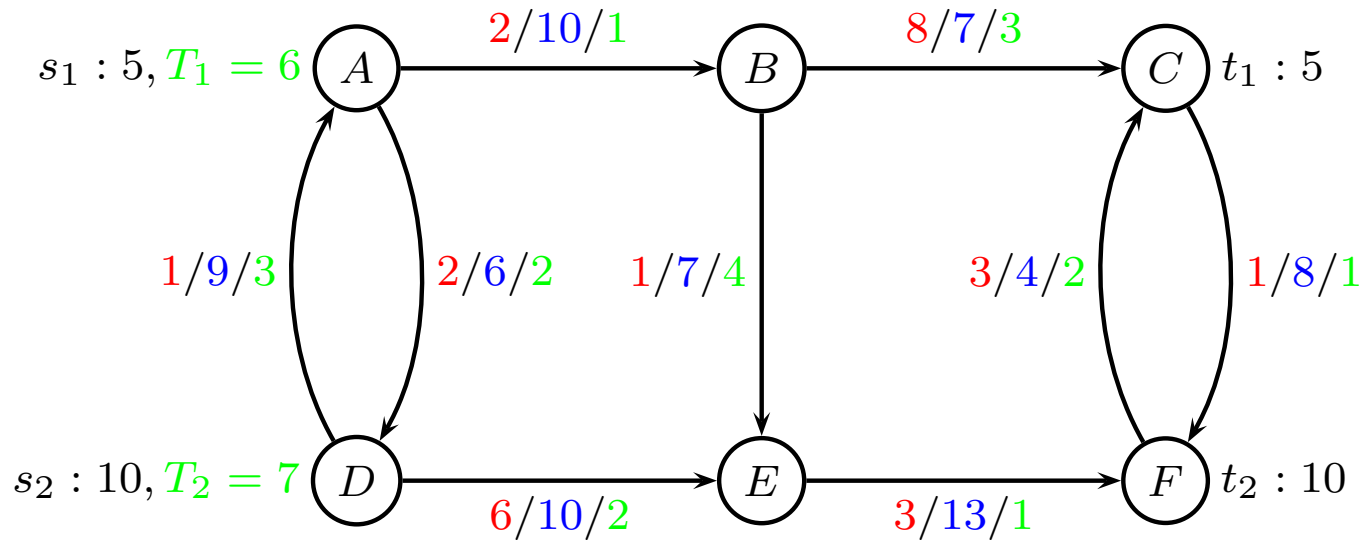
According to Ahuja, better than LP for large problems

Often easier to handle extra constraints

- min cost
- variable cost for each k
- max flow (costs ignored)
- splittable
- unsplittable
- k -splittable

Max travel time can be built into shortest-path algorithm

Time constrained MCFP



DEF is faster

DABEF is cheaper

Compact formulation ?

$$\begin{array}{ll}\min & \sum_{k \in K} \sum_{(i,j) \in E} c_{ij} x_{ij}^k \\ \text{s.t.} & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = 0 \quad k \in K, j \in V \setminus \{s_k, t_k\} \\ & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = -d_k \quad k \in K, j = s_k \\ & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = d_k \quad k \in K, j = t_k \\ & \sum_{k \in K} x_{ij}^k \leq u_{ij} \quad (i, j) \in E \\ & 0 \leq x_{ij}^k \leq d_k \quad k \in K, (i, j) \in E\end{array}$$

Difficult to handle aggregated time.

Need extra index on variables x_{ij}^{kt}

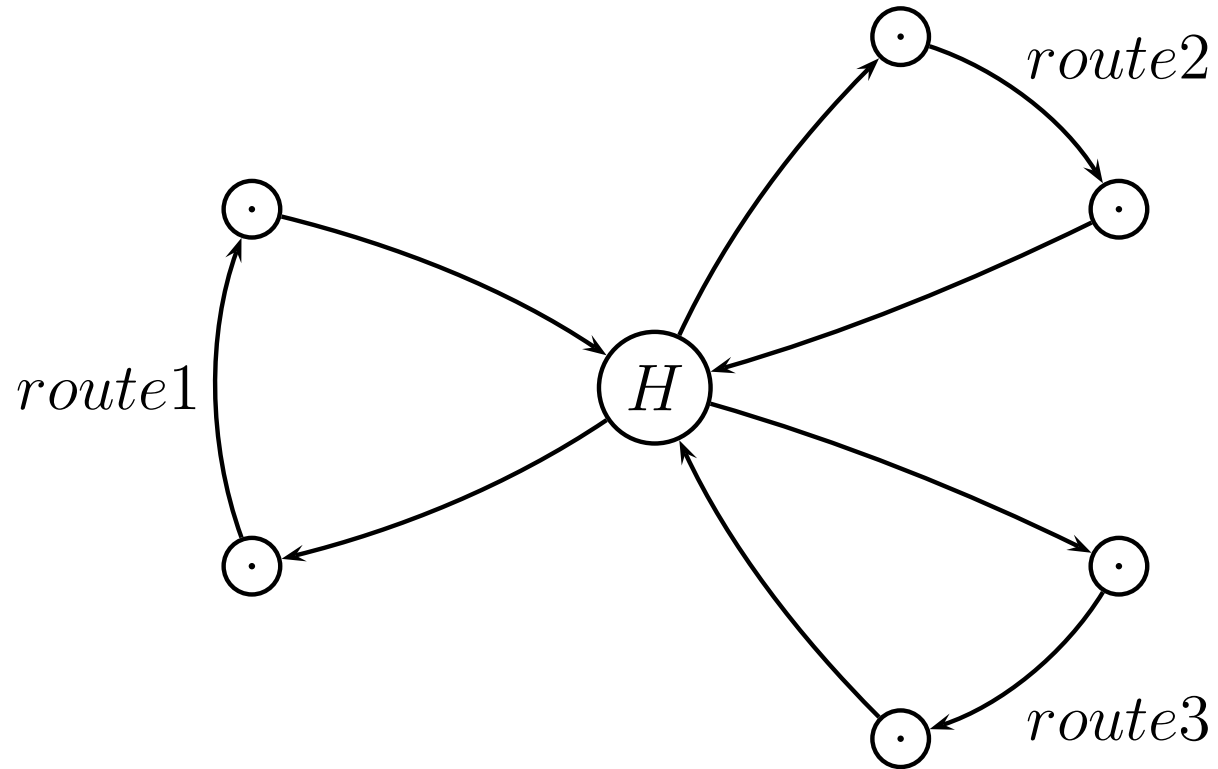
Path formulation

P_k set of possible paths from s_k to t_k **satisfying time constraint**

$$\begin{aligned} \min \quad & \sum_{k \in K} \sum_{p \in P_k} \hat{c}_p^k x_p^k \\ \text{s.t.} \quad & \sum_{k \in K} \sum_{p \in P_k} a_{p,ij}^k x_p^k \leq u_{ij} \quad (i, j) \in E \\ & \sum_{p \in P_k} x_p^k = d_k \quad k \in K \\ & 0 \leq x_p^k \leq d_k \quad k \in K, p \in P_k \end{aligned}$$

Finding paths is now time-constrained shortest path problem (NP-hard)

Modeling a hub port



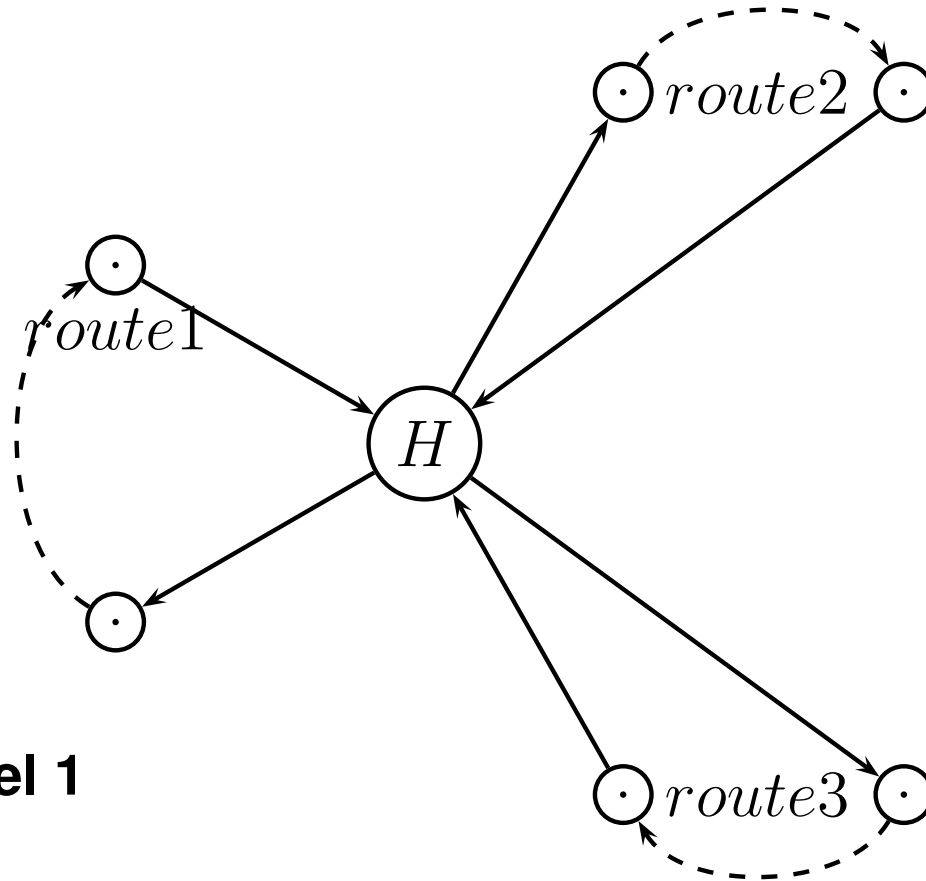
route	weekday
1	Saturday
2	Wednesday
3	Friday

Modeling a hub port



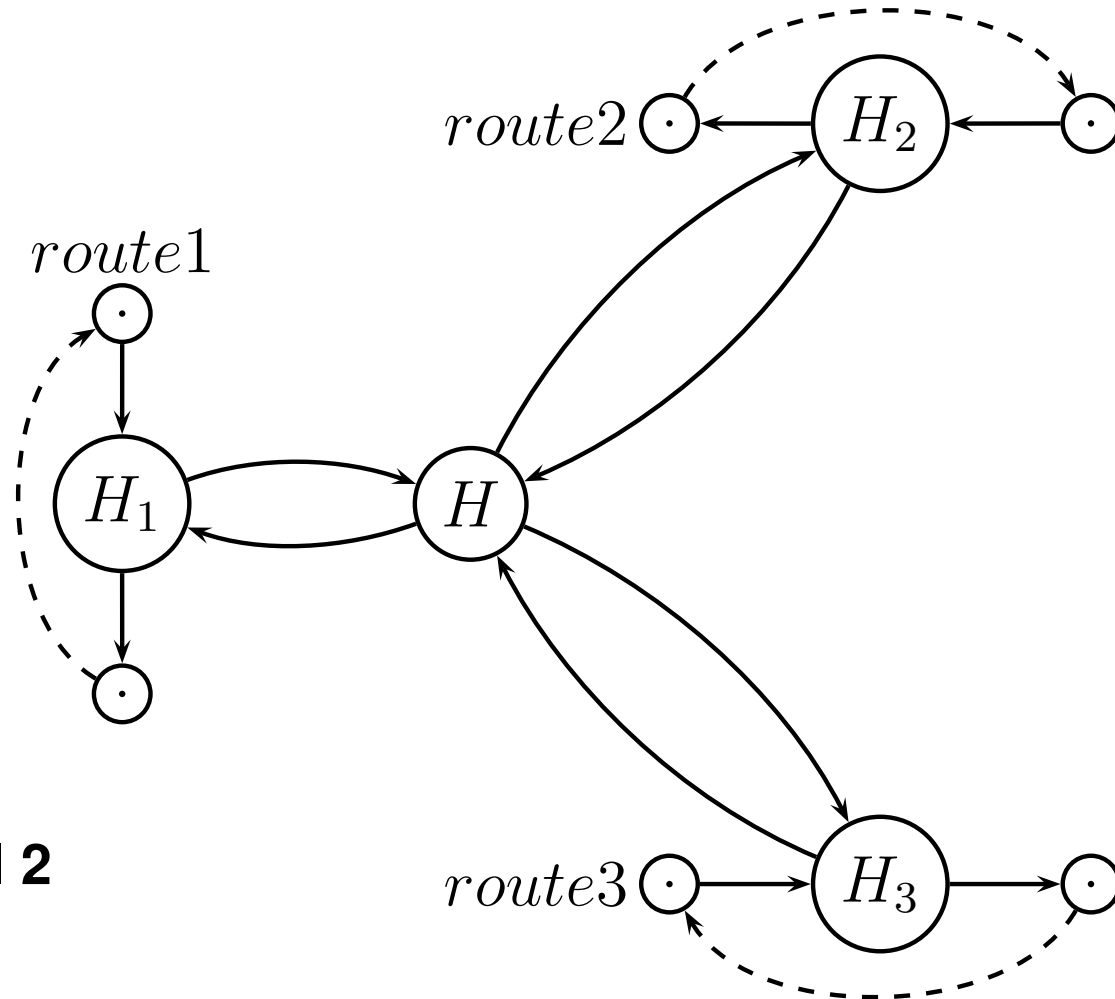
- Crane cost c (per container)
- Crane capacity u
- Transshipment time t_{ij}
- Storage cost $\bar{c}$ (per container)
- Storage capacity $\bar{u}$

Modeling a hub port



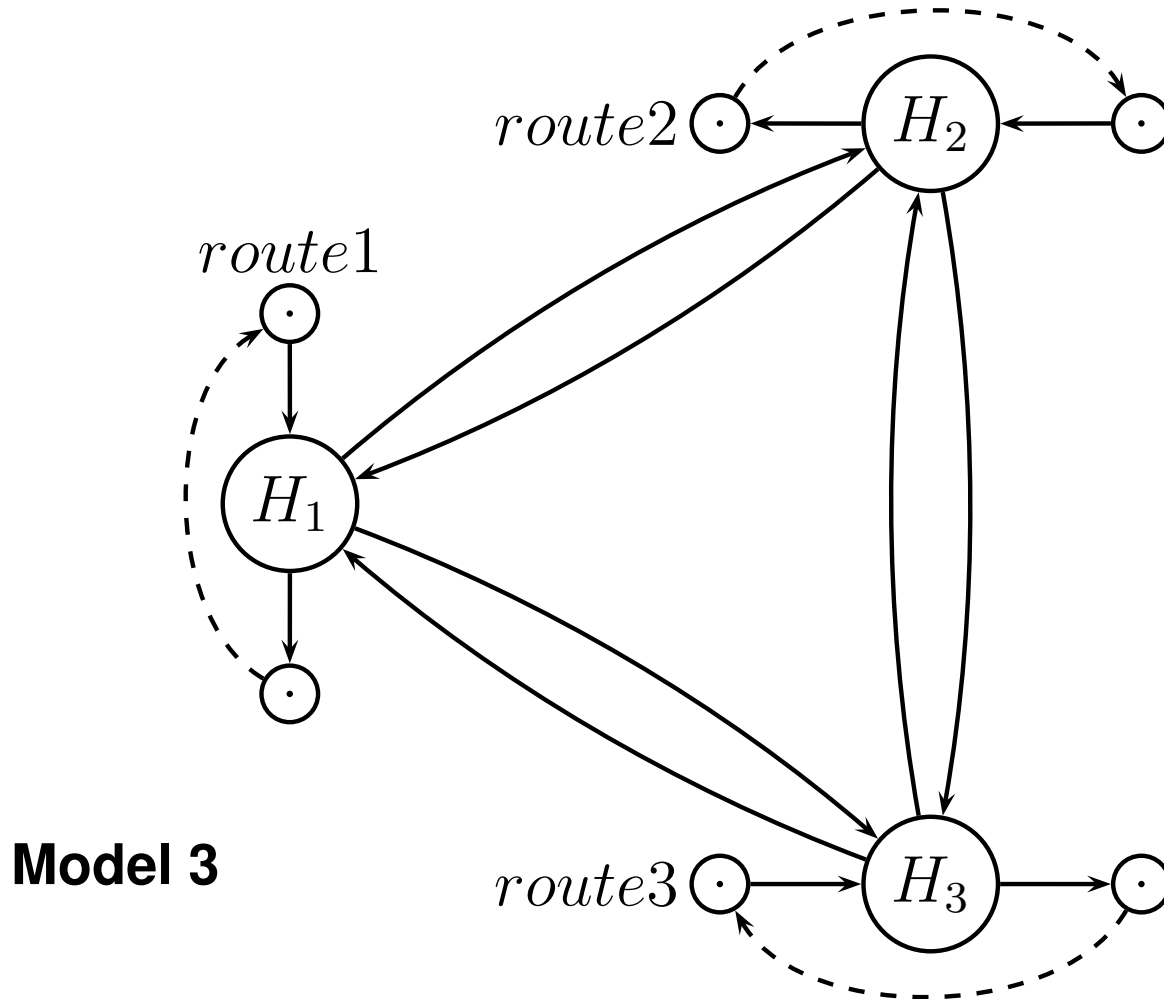
Model 1

Modeling a hub port

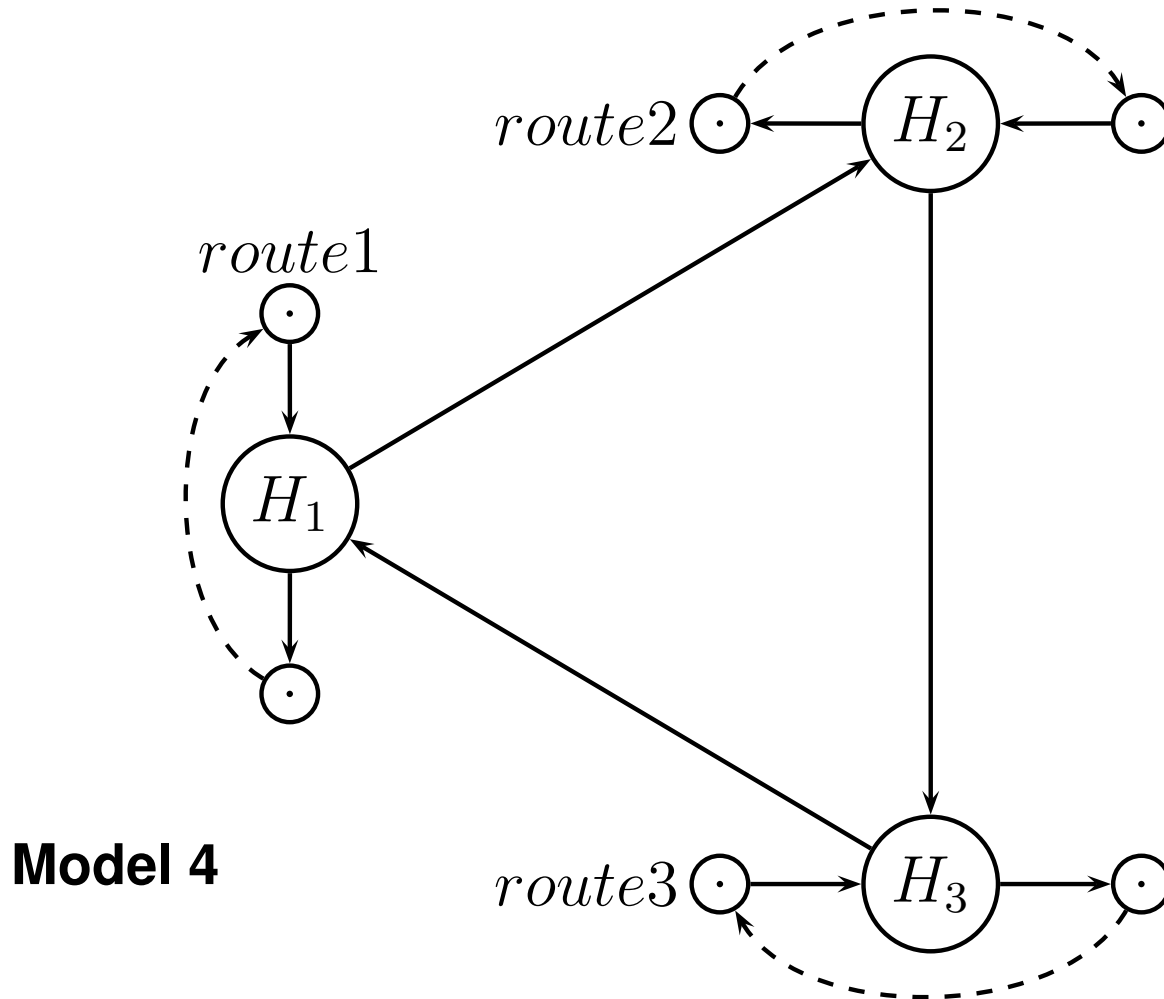


Model 2

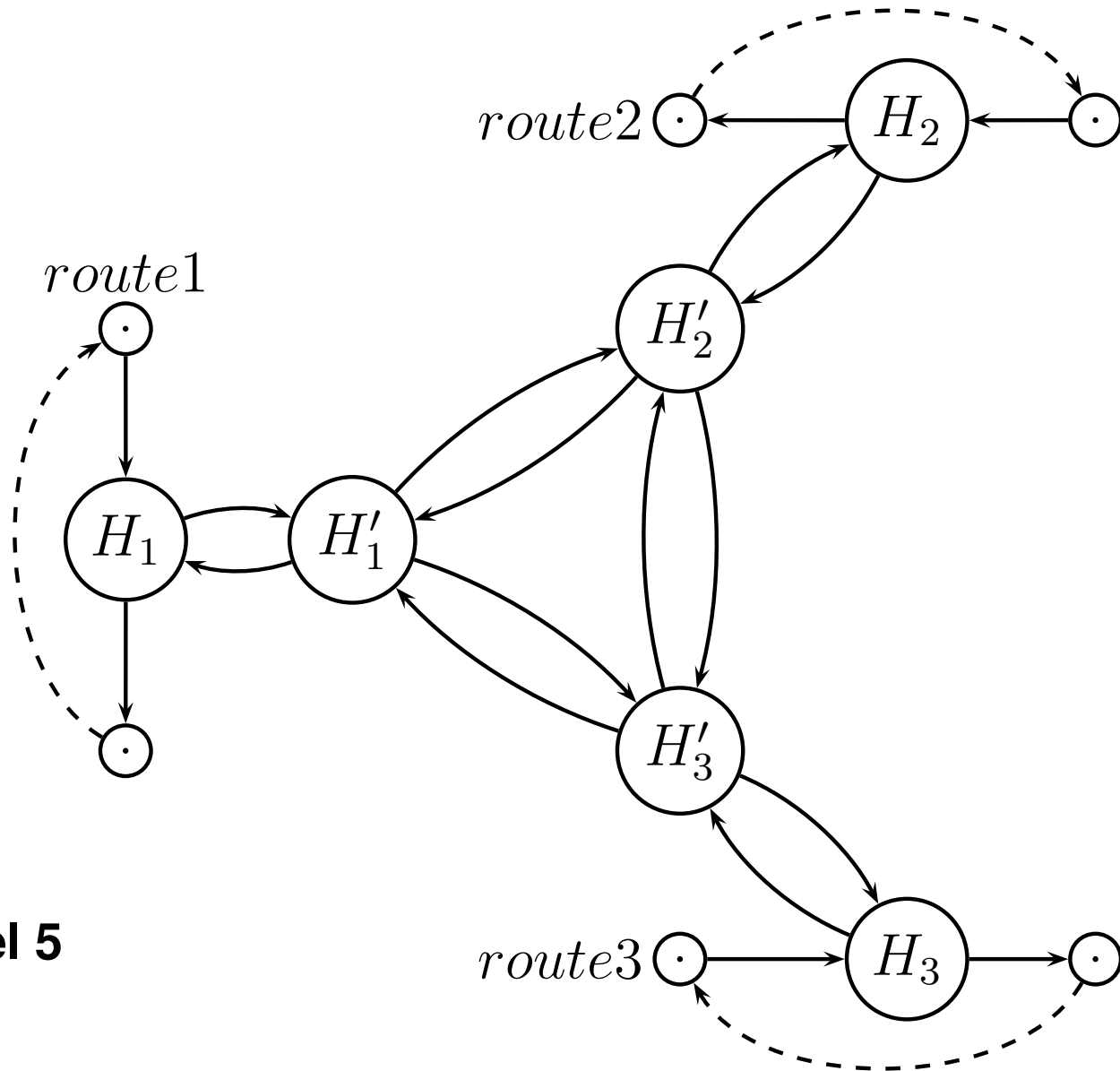
Modeling a hub port



Modeling a hub port

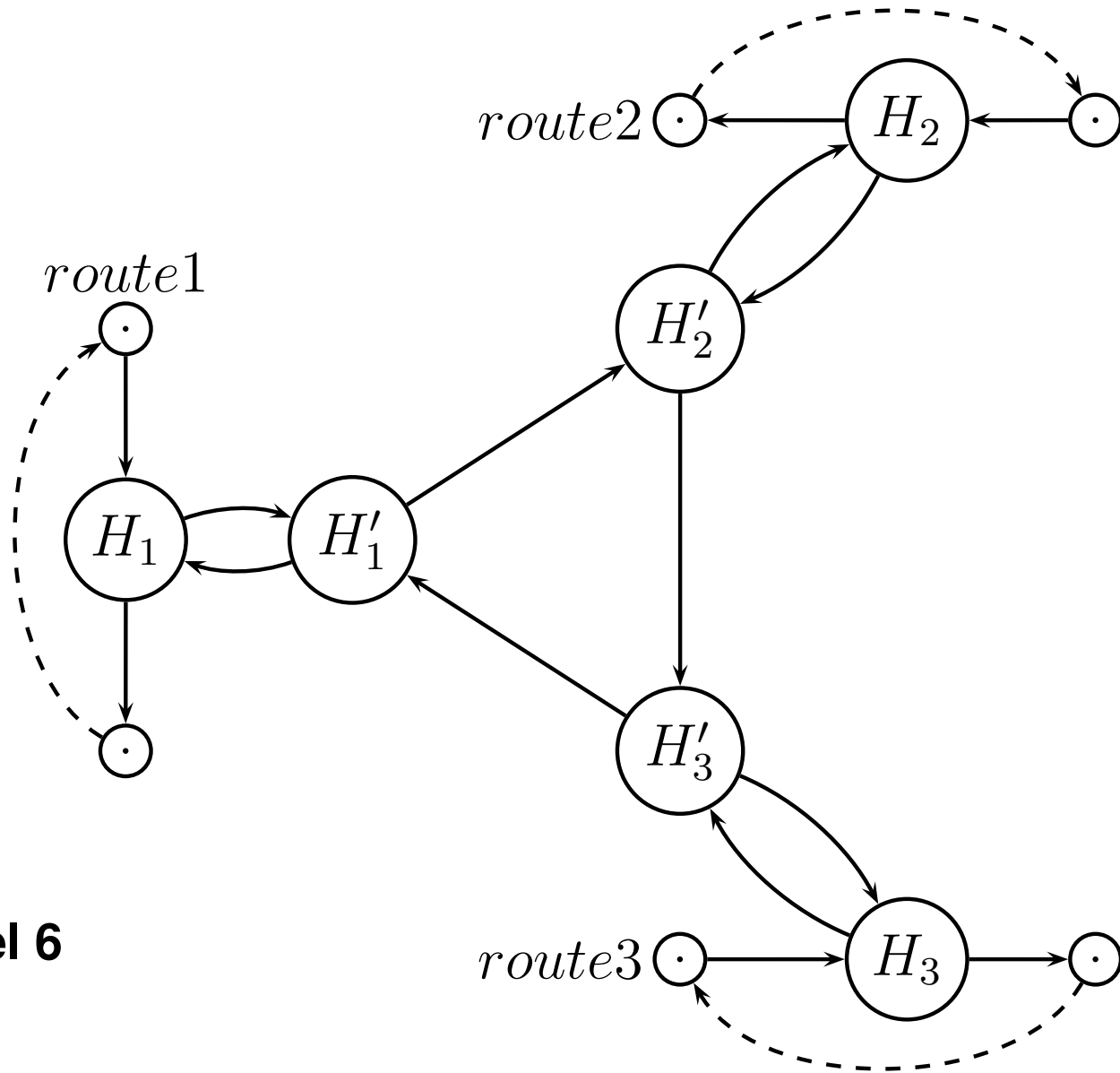


Modeling a hub port



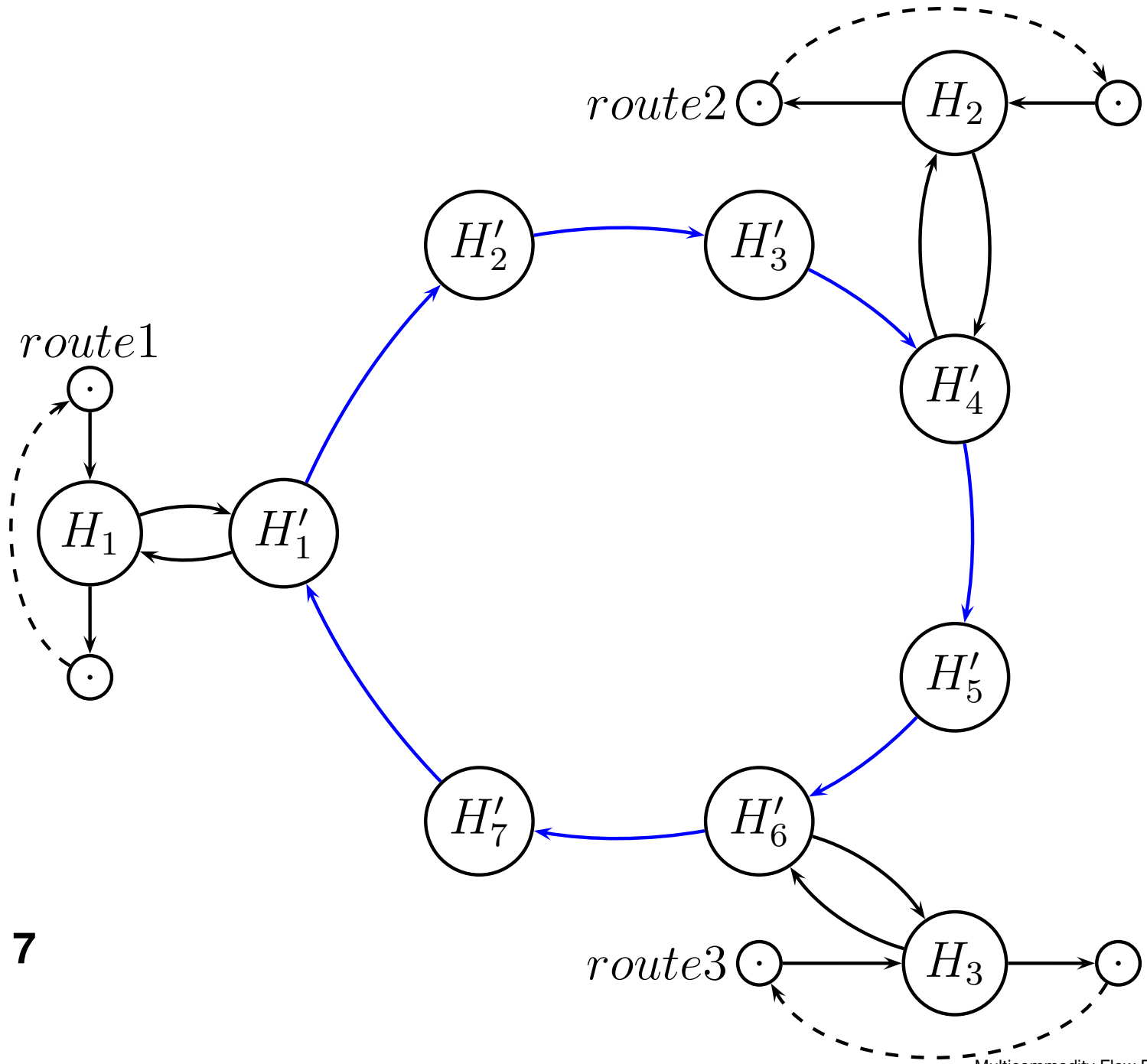
Model 5

Modeling a hub port



Model 6

Modeling a hub port



Model 7

Multicommodity Flow Problem

Lagrange methods

David Pisinger

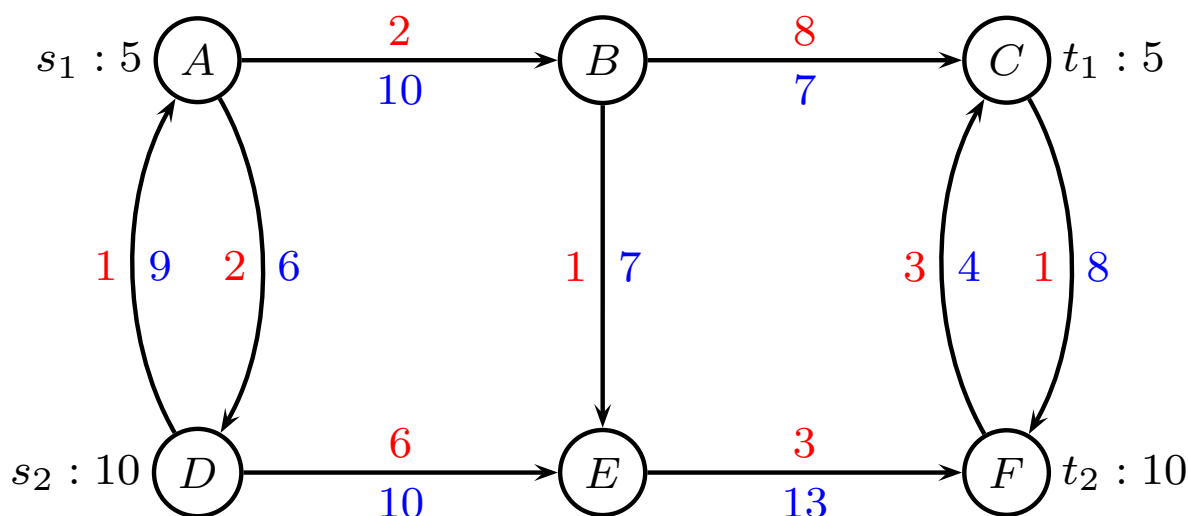
`pisinger@man.dtu.dk`

DTU Management
Technical University of Denmark

© Copyright David Pisinger. May not be distributed

Multicommodity Flow - Terminology

- We consider a directed graph $G = (V, E)$ in which each edge e has a capacity $u_e \in \mathbb{R}_+$ and a unit transportation cost $c_e \in \mathbb{R}$.
- A set K of commodities is given. Each commodity k has a source s_k , terminal t_k , and demand d_k .
- All commodities k should be transported from source to sink in the cheapest possible way, respecting capacities.



Multicommodity Flow - Definition

Let x_{ij}^k denote flow of commodity k on edge (i, j) .

$$\begin{aligned} \min \quad & \sum_{k \in K} \sum_{(i,j) \in E} c_{ij} x_{ij}^k \\ \text{s.t.} \quad & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = 0 & k \in K, j \in V \setminus \{s_k, t_k\} \\ & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = -d_k & k \in K, j = s_k \\ & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = d_k & k \in K, j = t_k \\ & \sum_{k \in K} x_{ij}^k \leq u_{ij} & (i, j) \in E \\ & 0 \leq x_{ij}^k \leq d_k & k \in K, (i, j) \in E \end{aligned}$$

Bounds $x_{ij}^k \leq d_k$ not strictly necessary.

MCF - compact formulation

Multicommodity flow problem

$$\begin{aligned} \min \quad & \sum_{k \in K} \sum_{(i,j) \in E} c_{ij} x_{ij}^k \\ \text{s.t.} \quad & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = b_j^k \quad k \in K, j \in V \quad (\pi_j^k) \\ & \sum_{k \in K} x_{ij}^k \leq u_{ij} \quad (i,j) \in E \quad (y_{ij}) \\ & x_{ij}^k \geq 0 \quad k \in K, (i,j) \in E \end{aligned}$$

where demand b_j^k in each node is

$$b_j^k = \begin{cases} 0 & \text{if } j \neq \{s_k, t_k\} \\ d_k & \text{if } j = t_k \\ -d_k & \text{if } j = s_k \end{cases}$$

Solution methods

LP-based

- Simplex
- interior-point methods

Column generation

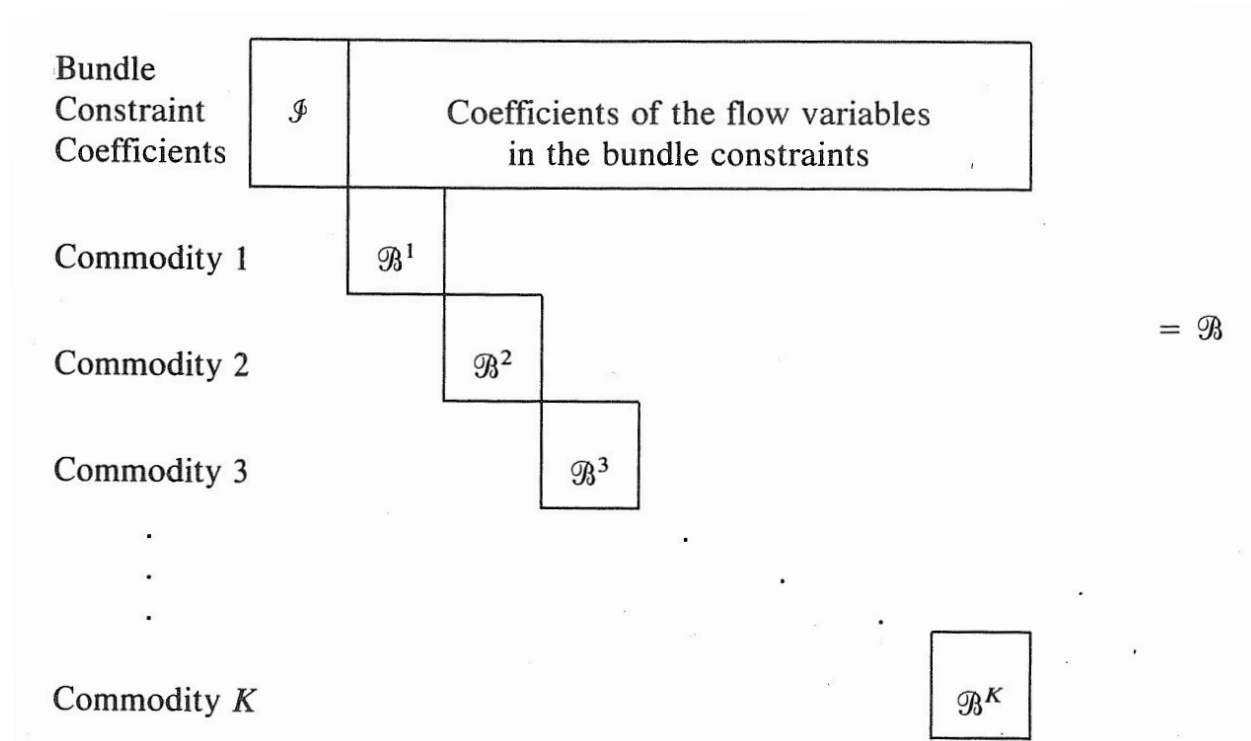
- Dantzig Wolfe decomposition
- Master Problem
- Subproblem

Lagrangian methods

- Relax (delete) capacity constraints
- Punish costs if capacity violated

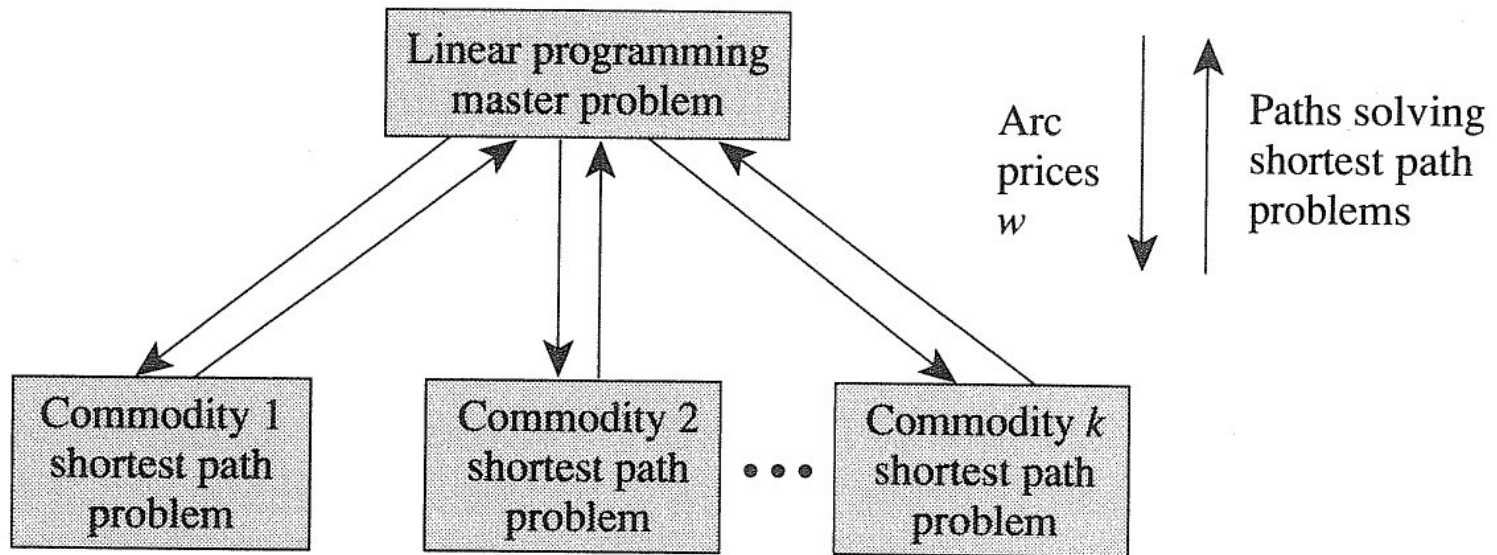
Dantzig Wolfe in general

The problem has block angular structure



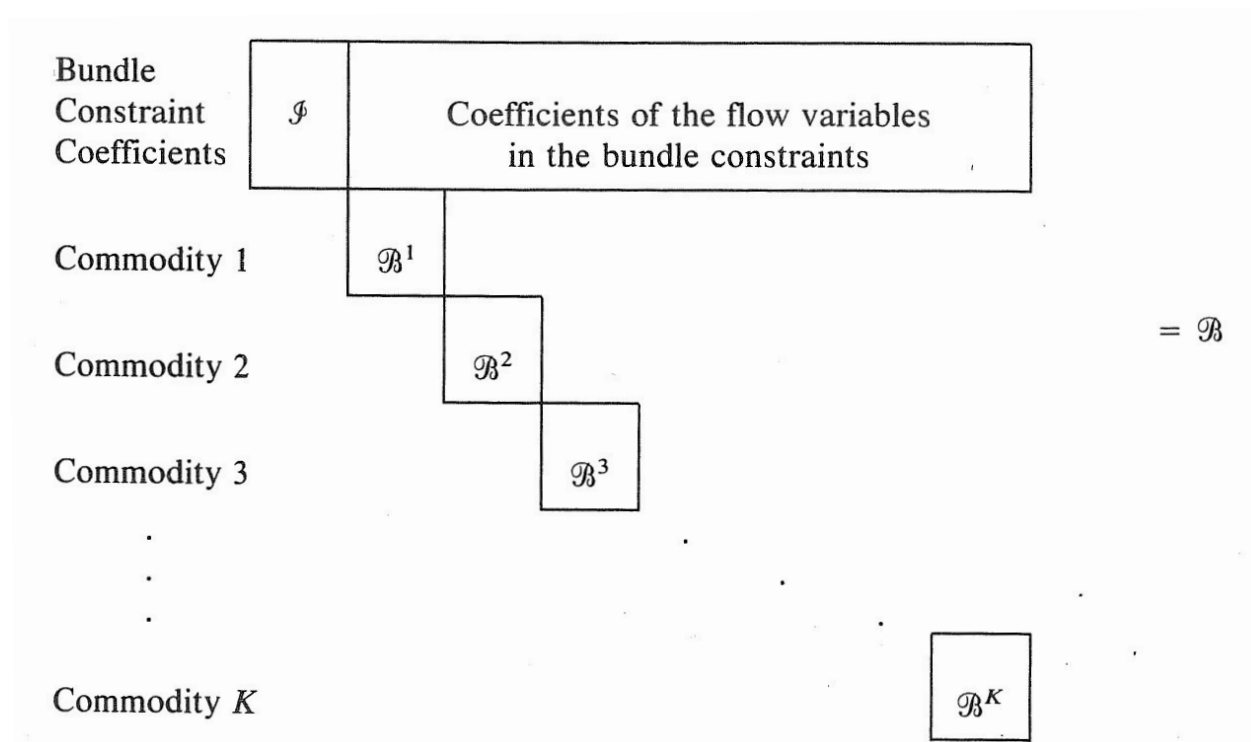
Reformulate by writing up all solutions to each block
Since reformulation may be big, write up as needed
Delayed column generation

Delayed column generation



Another idea

Get rid of bundle constraints (capacity constraints)



Problem becomes independent flow problem for each k

Some theory - Lagrangian relaxation

$$\begin{array}{ll}\text{minimize} & cx \\ \text{subject to} & Ax \leq b \\ & Dx \leq d \\ & x_j \in \mathbb{Z}_+, \quad j = 1, \dots, n\end{array}$$

Lagrange relax $Dx \leq d$, using multipliers $\lambda \geq 0$

$$\begin{array}{ll}\text{minimize} & z_{LR}(\lambda) = cx + \lambda(Dx - d) \\ \text{subject to} & Ax \leq b \\ & x_j \in \mathbb{Z}_+, \quad j = 1, \dots, n\end{array}$$

Some theory - Lagrangian relaxation

Proposition Optimal solution to relaxed problem is lower bound on original problem

Proof It is a relaxation

1. Set of feasible solutions is extended
2. Lagrangian relaxed objective function is smaller

multiplier λ_i “punishment”

If λ_i large $\Rightarrow$ constraint satisfied

If $\lambda_i = 0 \Rightarrow$ drop constraint

Some theory - Lagrangian relaxation

Lagrange relaxed problem as function of $\lambda \geq 0$

$$\begin{array}{ll}\text{minimize} & z_{LR}(\lambda) = cx + \lambda(Dx - d) \\ \text{subject to} & Ax \leq b \\ & x_j \in \mathbb{Z}_+, \quad j = 1, \dots, n\end{array}$$

Lagrangian Dual Problem

$$z_{LD} = \max_{\lambda \geq 0} z_{LR}(\lambda)$$

Find the λ that gives the largest lower bound

Lagrangian relaxation, MCFP

$$\begin{aligned} \min \quad & \sum_{k \in K} \sum_{(i,j) \in E} c_{ij} x_{ij}^k + \sum_{(i,j) \in E} \lambda_{ij} (\sum_{k \in K} x_{ij}^k - u_{ij}) \\ \text{s.t.} \quad & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = b_j^k \quad k \in K, j \in V \\ & x_{ij}^k \geq 0 \quad k \in K, (i,j) \in E \end{aligned}$$

Or equivalently

$$\begin{aligned} \min \quad & \sum_{k \in K} \sum_{(i,j) \in E} (c_{ij} + \lambda_{ij}) x_{ij}^k - \sum_{(i,j) \in E} \lambda_{ij} u_{ij} \\ \text{s.t.} \quad & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = b_j^k \quad k \in K, j \in V \\ & x_{ij}^k \geq 0 \quad k \in K, (i,j) \in E \end{aligned}$$

Last sum in objective is a "constant".

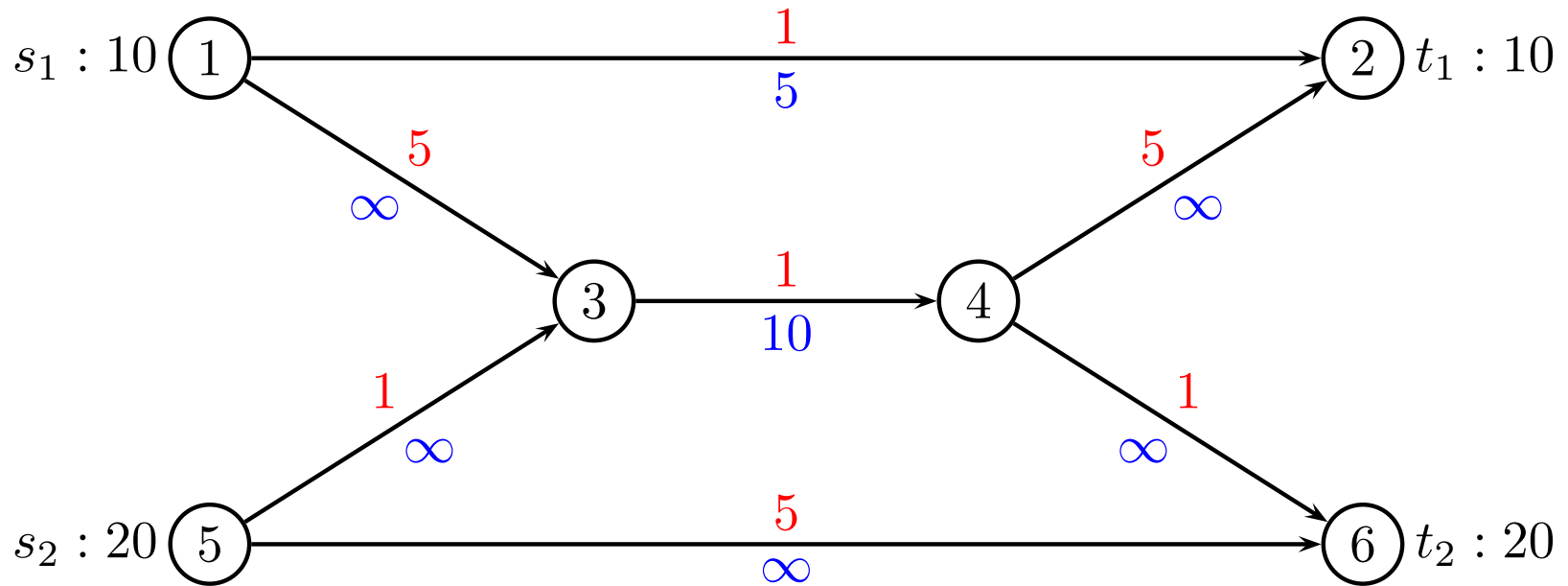
Lagrangian relaxation

Since no shared capacity, splits into separate problems for each $k \in K$

$$\begin{aligned} \min \quad & \sum_{(i,j) \in E} (c_{ij} + \lambda_{ij}) x_{ij}^k \\ \text{s.t.} \quad & \sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = b_j^k \quad j \in V \\ & x_{ij}^k \geq 0 \quad (i, j) \in E \end{aligned}$$

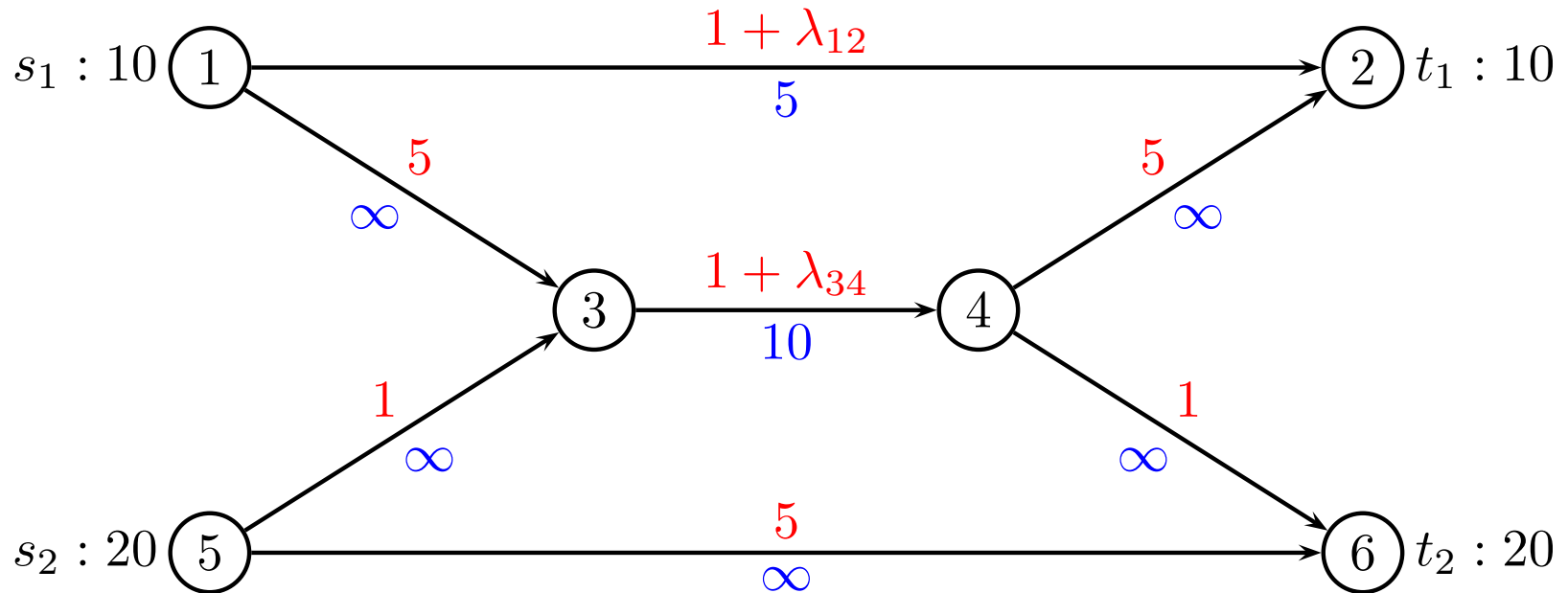
This is a minimum cost flow problem for each k
(actually a shortest path problem)

Multicommodity Flow - example



Multicommodity Flow - example

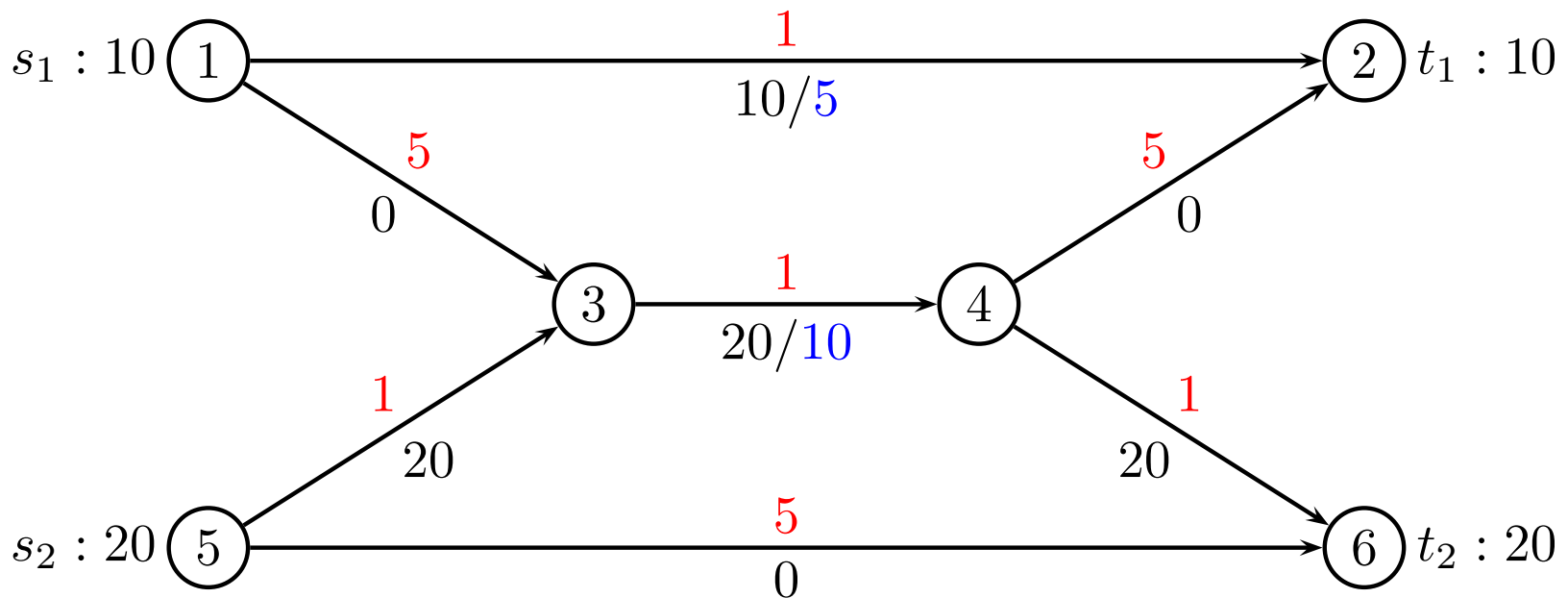
Lagrange relaxed



Iteration 0

Initially choose all Lagrange multipliers $\lambda = 0$

$$\lambda_{12} = 0, \lambda_{34} = 0$$



flow cost: $10 \cdot (1) + 20 \cdot (1 + 1 + 1) = 70$

constant: $5\lambda_{12} + 10\lambda_{34} = 0$

lower bound: 70

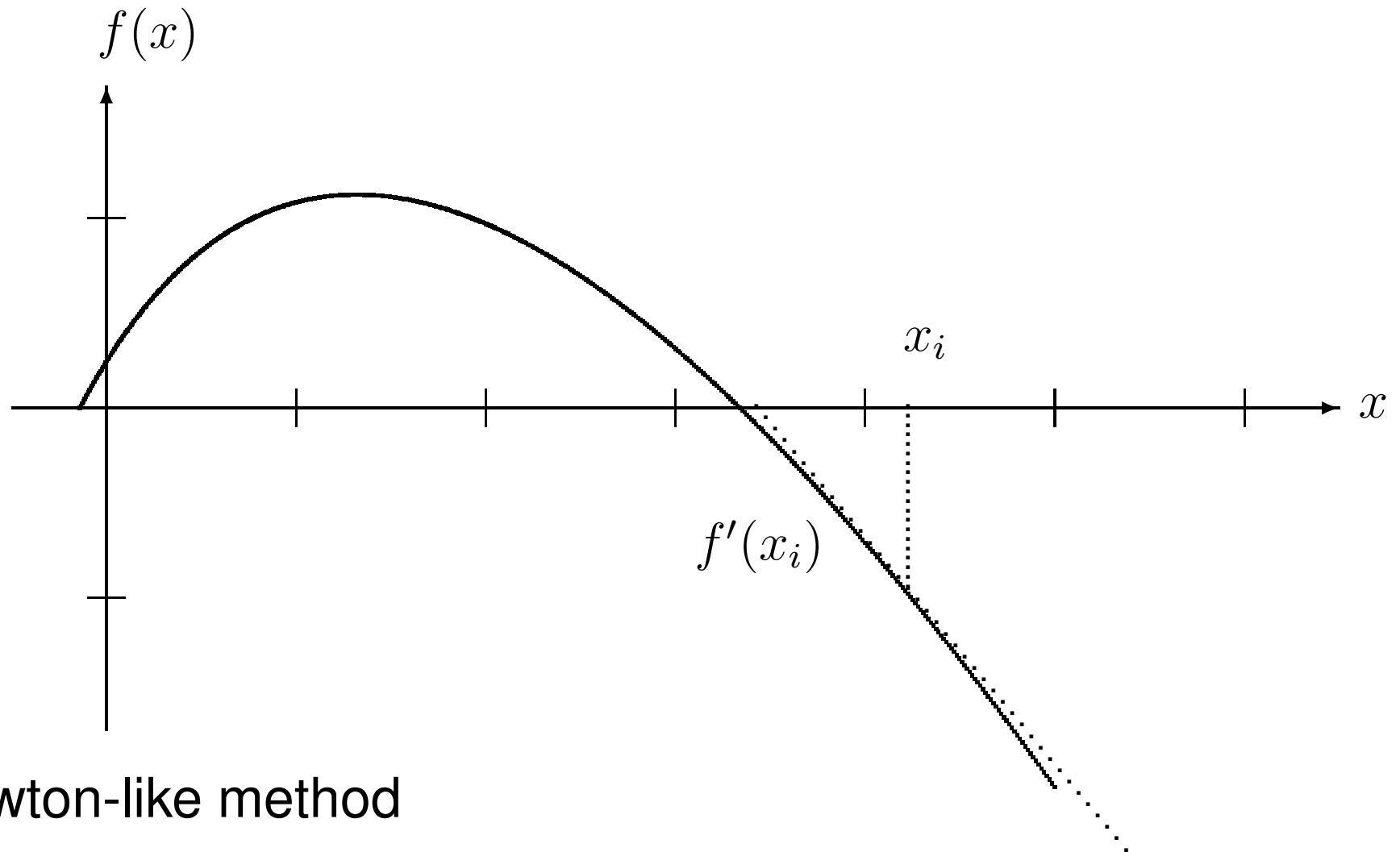
How can we improve this?

Find the λ that gives the largest lower bound
Lagrange Dual Problem

$$z_{LD} = \max_{\lambda \geq 0} z_{LR}(\lambda)$$

- Often we do not need best multipliers
- Subgradient optimization fast method
- Methods for minimizing a convex, possibly non-differentiable, function over a convex and closed set
- Roots in nonlinear programming, Held and Karp (1971)

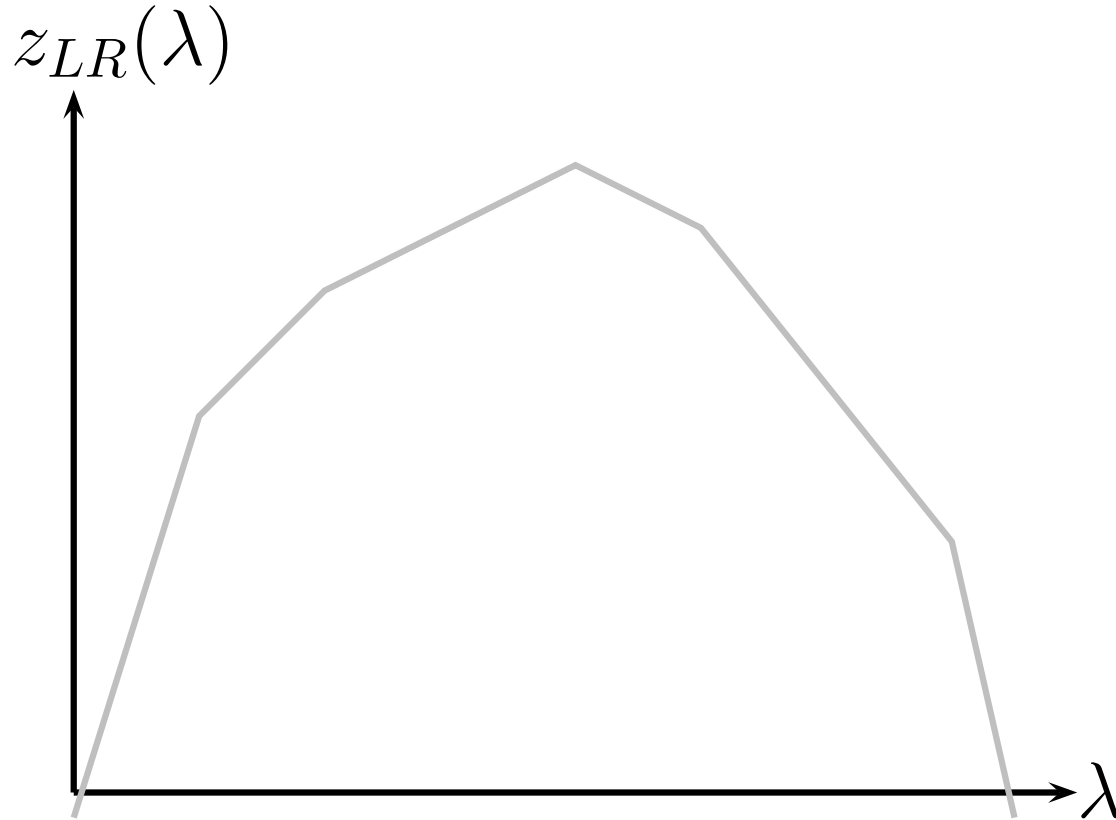
How can we improve this?



Newton-like method

How can we improve this?

Lagrange function $z_{LR}(\lambda)$ is piecewise linear and convex

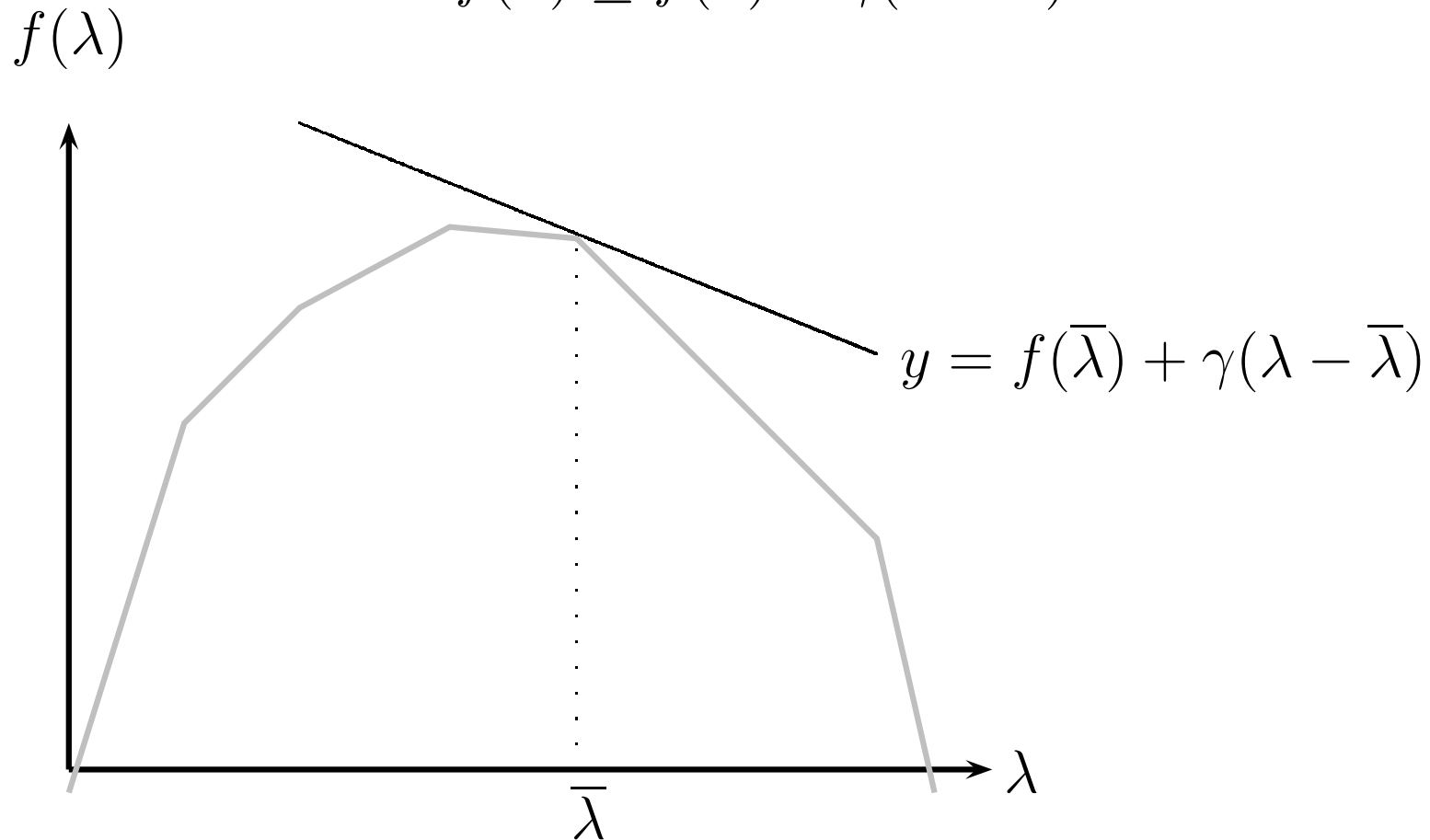


Subgradient

Generalization of gradients to non-differentiable functions.

Definition An m -vector γ is subgradient of $f(\lambda)$ at $\lambda = \bar{\lambda}$ if

$$f(\lambda) \leq f(\bar{\lambda}) + \gamma(\lambda - \bar{\lambda})$$



Subgradient

Given a choice of nonnegative multipliers $\bar{\lambda}$. If x' is an optimal solution to $z_{LR}(\bar{\lambda})$ then

$$\gamma = Dx' - d$$

is a subgradient of $z_{LR}(\lambda)$ at $\lambda = \bar{\lambda}$.

Intuition: Lagrange Relaxation

$$\begin{array}{ll}\text{minimize} & z_{LR}(\lambda) = cx + \lambda(Dx - d) \\ \text{subject to} & Ax \leq b \\ & x_j \in \mathbb{Z}_+, \quad j = 1, \dots, n\end{array}$$

want to solve

$$z_{LD} = \max_{\lambda \geq 0} z_{LR}(\lambda)$$

For small changes of λ we find same x' so γ is subgradient

Subgradient iteration

Recursion

$$\lambda^{(k+1)} = \max \left\{ \lambda^{(k)} + \theta \gamma^{(k)}, 0 \right\}$$

Where $\theta > 0$ is step-size.

If $\gamma > 0$ and θ is sufficiently small, lower bound $z_{LR}(\lambda)$ will increase.

- Small θ : slow convergence
- Large θ : unstable

Subgradient, Held and Karp method

Initially

$$\lambda^{(0)} = \{0, \dots, 0\}$$

compute the new multipliers by recursion

$$\lambda_i^{(k+1)} := \begin{cases} \lambda_i^{(k)} & \text{if } |\gamma_i| \leq \epsilon \\ \max(\lambda_i^{(k)} + \theta \gamma_i, 0) & \text{if } |\gamma_i| > \epsilon \end{cases}$$

where γ is subgradient.

The step size is $\theta = \mu \frac{\bar{z} - \underline{z}}{\sum_i \gamma_i^2}$ where μ is a constant (e.g. 1)
(μ is halved if lower bound not increased in 20 iterations)

Iteration 0

We use a **simplified scheme** $\theta = 1/k$ for iteration $k \geq 1$

Subgradient

$$\gamma = Dx' - d = \left\{ \sum_k x_{ij}^k - u_{ij} \right\} = \{10 - 5, 20 - 10\} = \{5, 10\}$$

Recursion

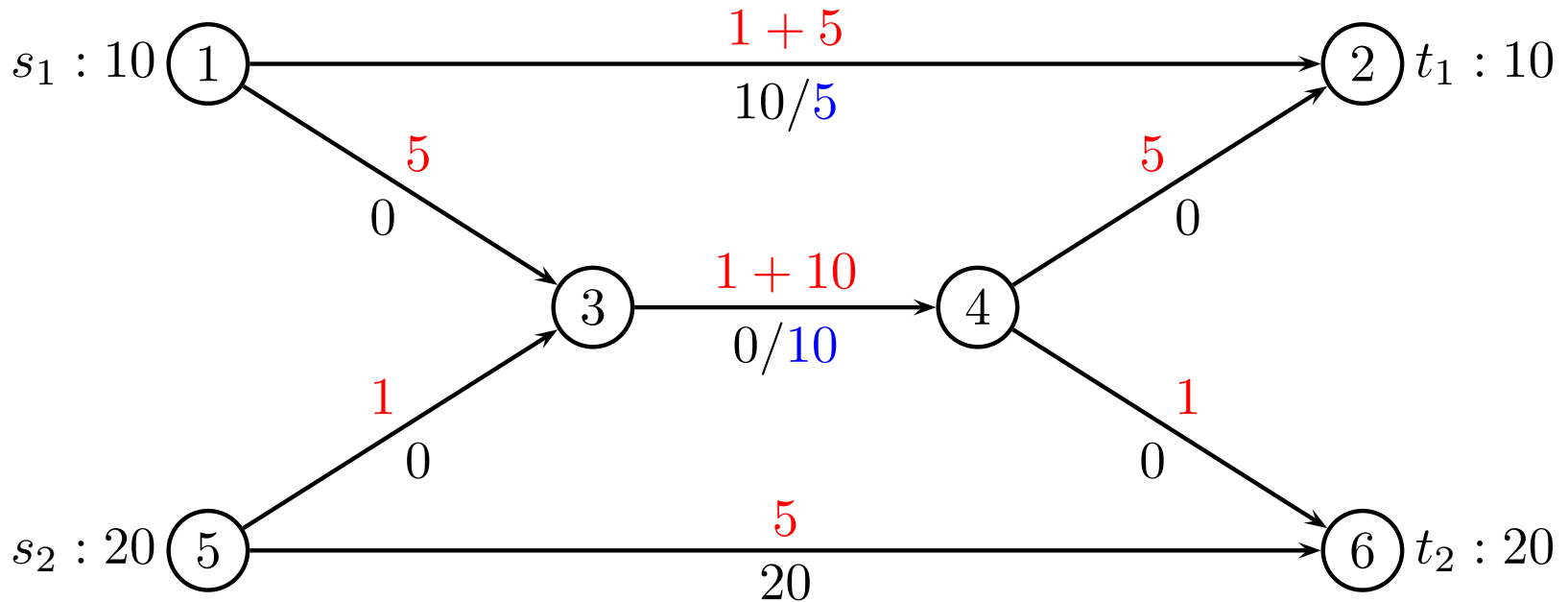
$$(\lambda_{12}, \lambda_{34}) = (\lambda_{12} + \theta \cdot 5, \lambda_{34} + \theta \cdot 10) = (0 + \theta \cdot 5, 0 + \theta \cdot 10)$$

Using $\theta = 1$ for $k = 0$ we get

$$(\lambda_{12}, \lambda_{34}) = (5, 10)$$

Iteration 1

$$\lambda_{12} = 5, \lambda_{34} = 10$$



flow cost: $10 \cdot (1 + 5) + 20 \cdot (5) = 160$

constant: $5\lambda_{12} + 10\lambda_{34} = 125$

lower bound: 35

Iteration 1

Subgradient

$$\gamma = \left\{ \sum_k x_{ij}^k - u_{ij} \right\} = \{10 - 5, 0 - 10\} = \{5, -10\}$$

Recursion

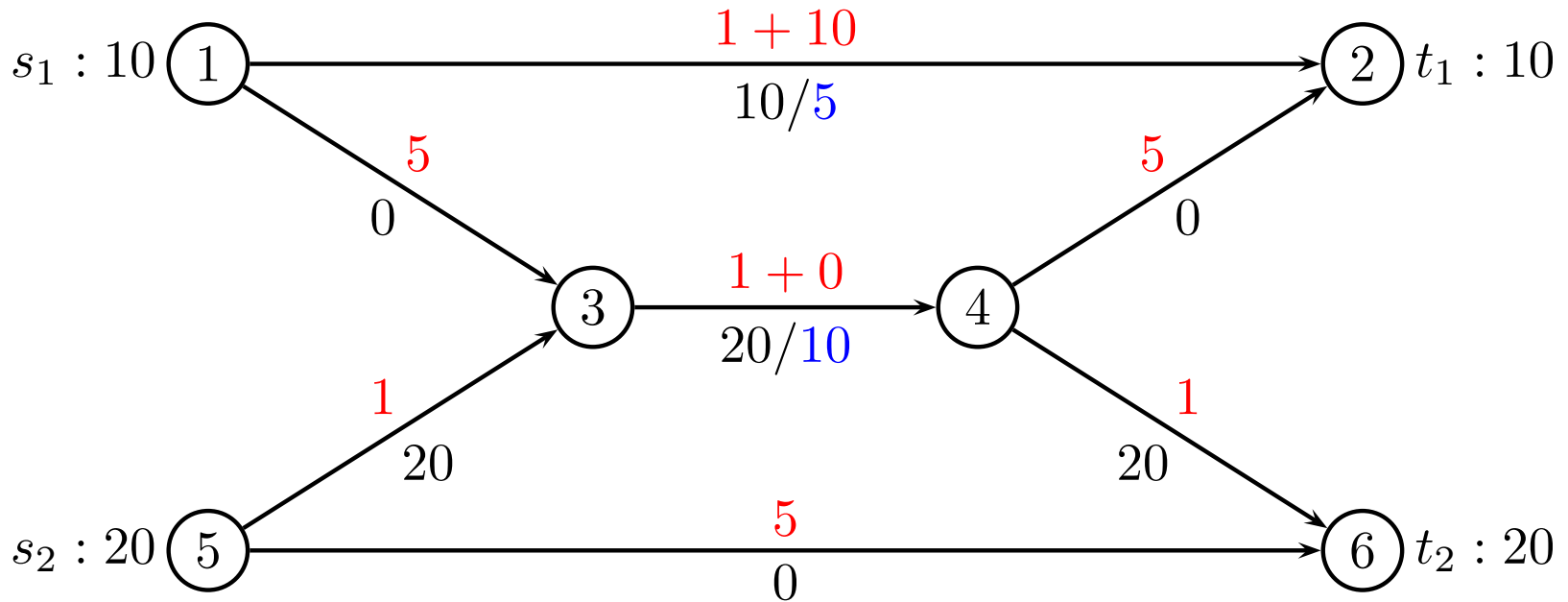
$$(\lambda_{12}, \lambda_{34}) = (5 + \theta \cdot 5, 10 - \theta \cdot 10)$$

Having $\theta = 1$ we get

$$(\lambda_{12}, \lambda_{34}) = (10, 0)$$

Iteration 2

$$\lambda_{12} = 10, \lambda_{34} = 0$$



flow cost: $10 \cdot (1 + 10) + 20 \cdot (1 + 1 + 1) = 170$

constant: $5\lambda_{12} + 10\lambda_{34} = 50$

lower bound: 120

Iteration 2

Subgradient

$$\gamma = \left\{ \sum_k x_{ij}^k - u_{ij} \right\} = \{10 - 5, 20 - 10\} = \{5, 10\}$$

Recursion

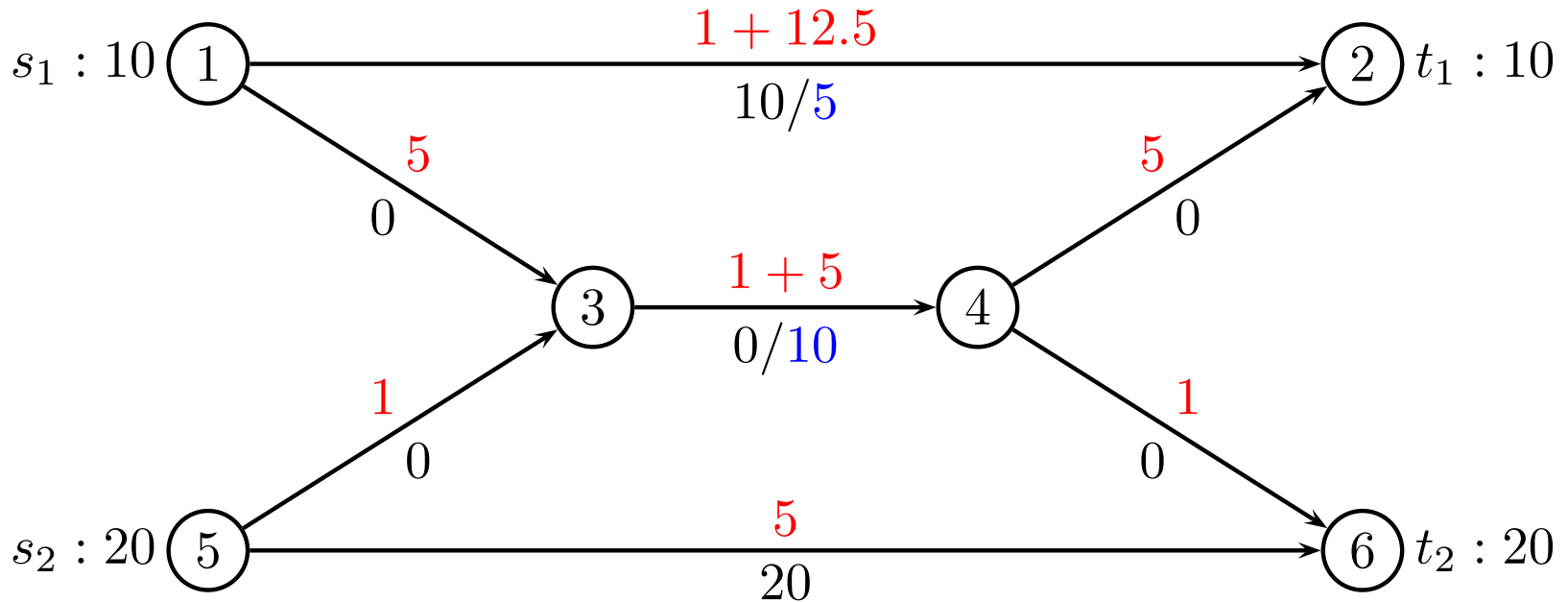
$$(\lambda_{12}, \lambda_{34}) = (10 + \theta \cdot 5, 0 + \theta \cdot 10)$$

Choosing $\theta = 1/2$ we get

$$(\lambda_{12}, \lambda_{34}) = (12.5, 5)$$

Iteration 3

$$\lambda_{12} = 12.5, \lambda_{34} = 5$$



flow cost: $10 \cdot (1 + 12.5) + 20 \cdot (5) = 235$

constant: $5\lambda_{12} + 10\lambda_{34} = 112.5$

lower bound: **122.5**

Iteration 3

Subgradient

$$\gamma = \left\{ \sum_k x_{ij}^k - u_{ij} \right\} = \{10 - 5, 0 - 10\} = \{5, -10\}$$

Recursion

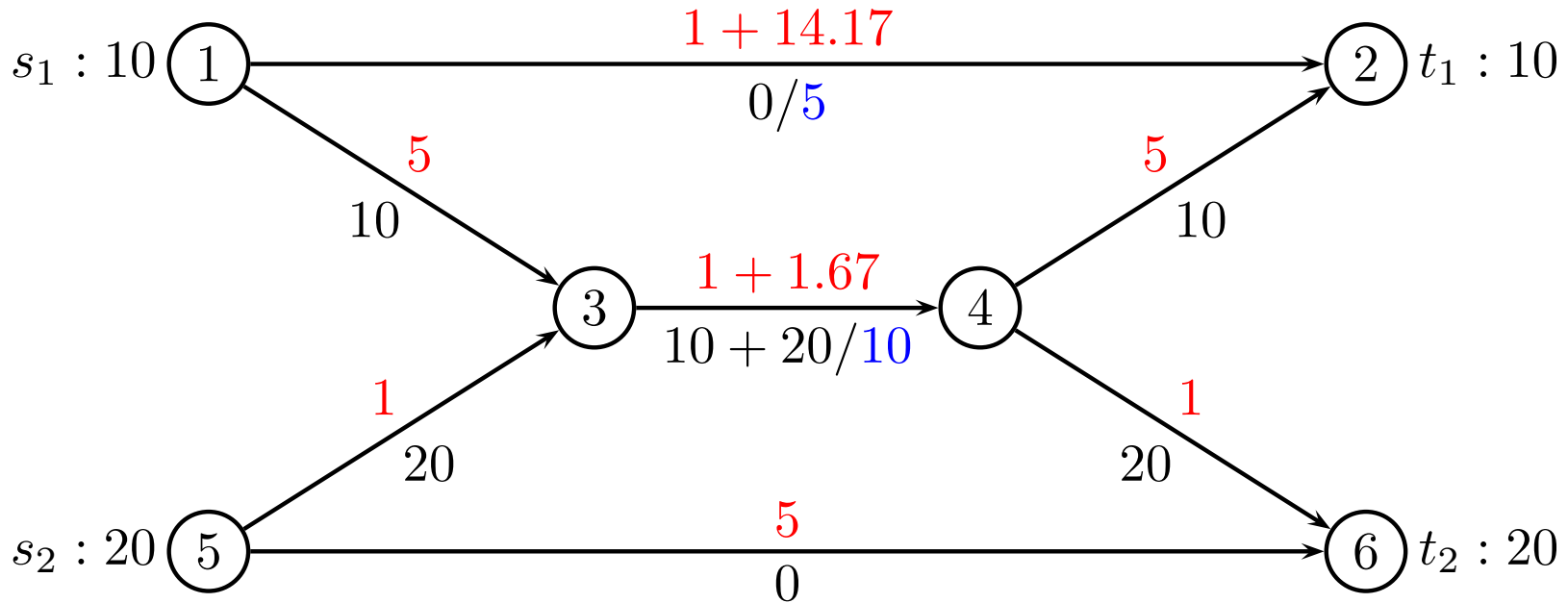
$$(\lambda_{12}, \lambda_{34}) = (12.5 + \theta \cdot 5, 5 - \theta \cdot 10)$$

Choosing $\theta = 1/3$ we get

$$(\lambda_{12}, \lambda_{34}) = (14.17, 1.67)$$

Iteration 4

$$\lambda_{12} = 14.17, \lambda_{34} = 1.67$$



flow cost: $10 \cdot (5 + 1 + 1.67 + 5) + 20 \cdot (1 + 1 + 1.67 + 1) = 220$

constant: $5\lambda_{12} + 10\lambda_{34} = 87.5$

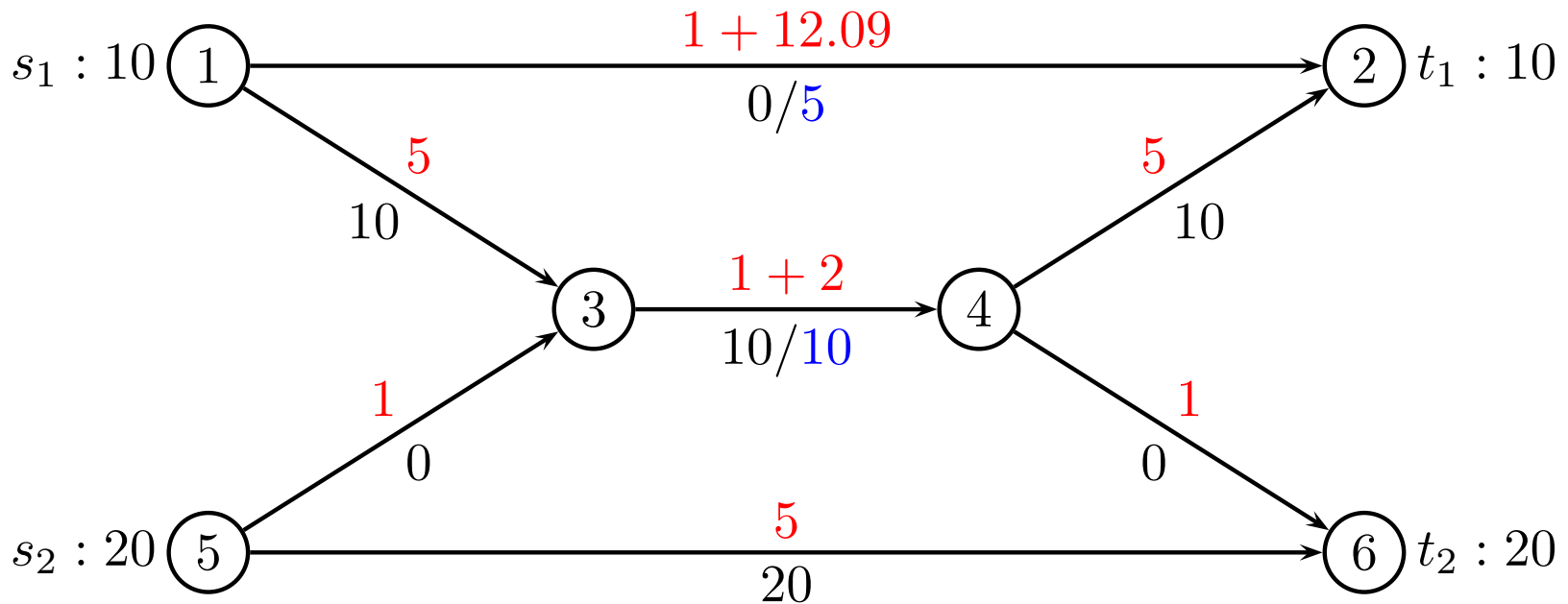
lower bound: 132.5

Iteration 5,...,13

calculating ...

Iteration 14

$$\lambda_{12} = 12.09, \lambda_{34} = 2$$



flow cost: $10 \cdot (5 + 3 + 5) + 20 \cdot (5) = 230$

constant: $5\lambda_{12} + 10\lambda_{34} = 80.45$

lower bound: 149.5

Greedy heuristic

For each iteration of λ run a greedy heuristic:

for each commodity $k \in K$

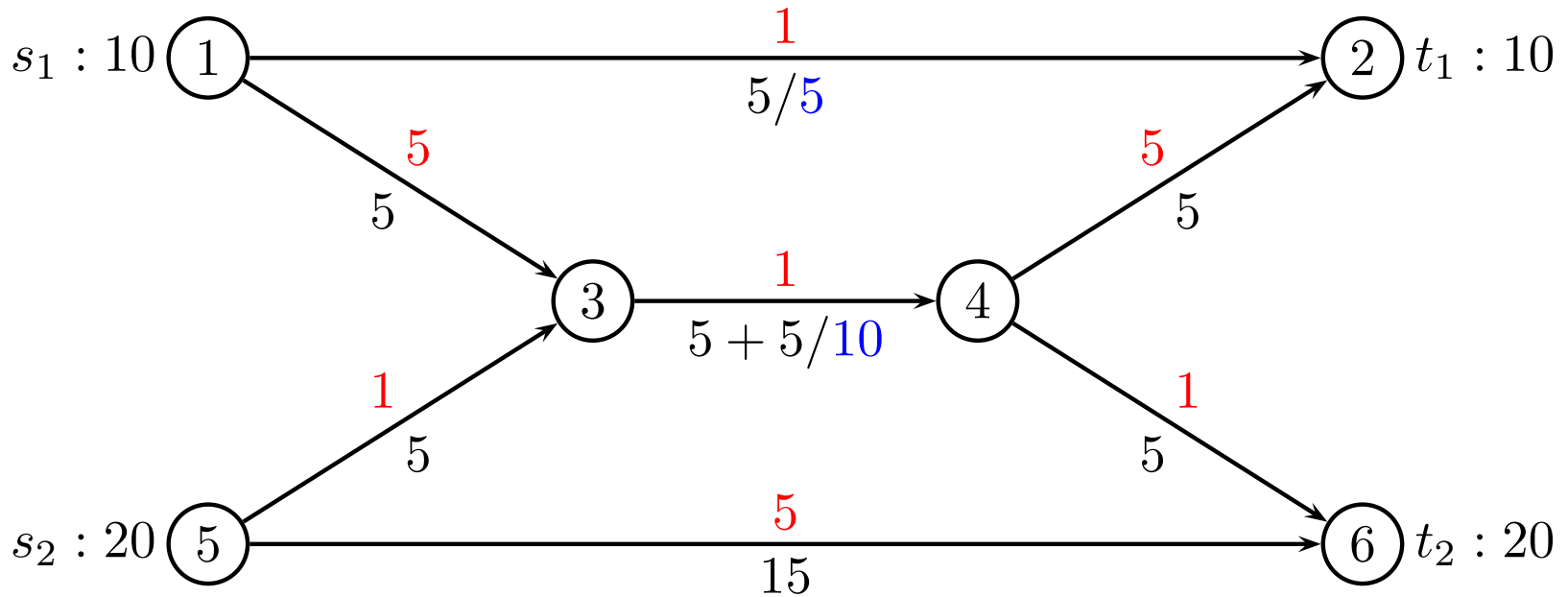
- 1 Find shortest path wrt. Lagrangian costs
- 2 Send as many units as residual capacity allows
- 3 If still demand left, remove filled edges, go to step 1

Greedy heuristic gives upper bound

if (upper bound = lower bound) **stop**

Heuristic solution

Iteration 0: $\lambda_{12} = 0$, $\lambda_{34} = 0$

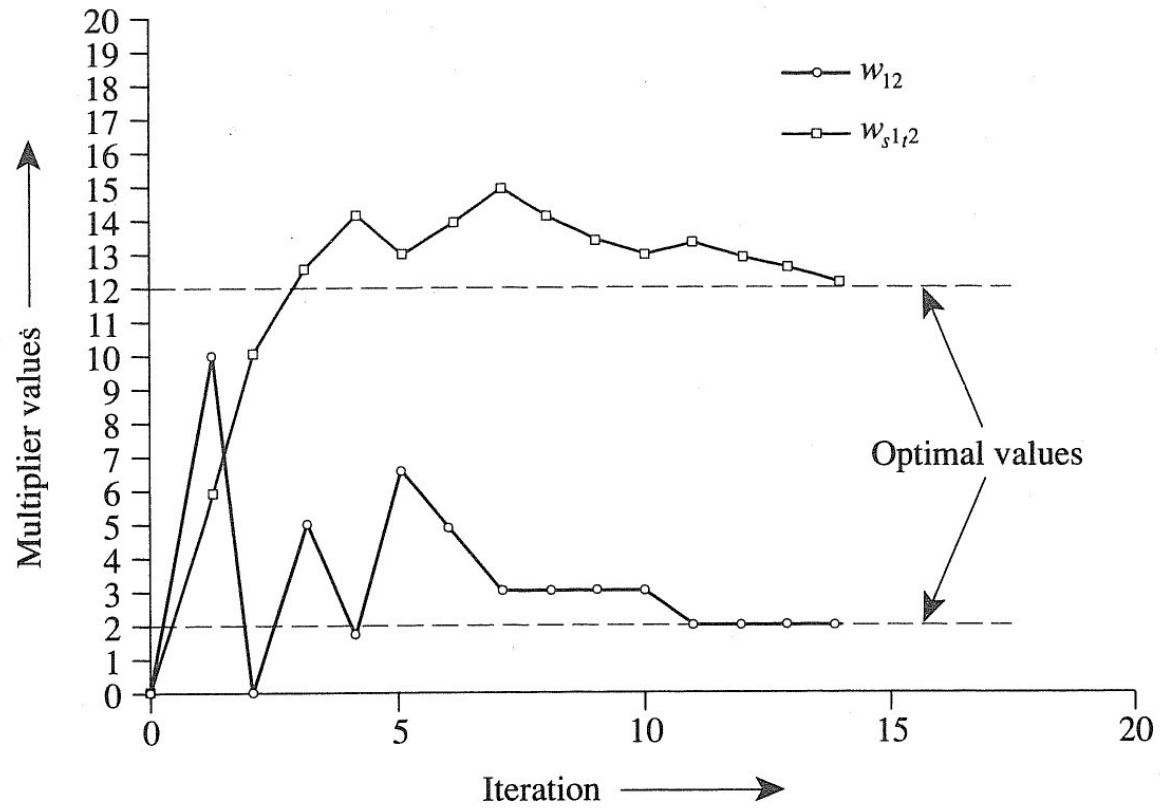


Cost: $5 \cdot 1 + 5 \cdot 5 + 10 \cdot 1 + 5 \cdot 5 + 5 \cdot 1 + 5 \cdot 1 + 15 \cdot 5 = 150$

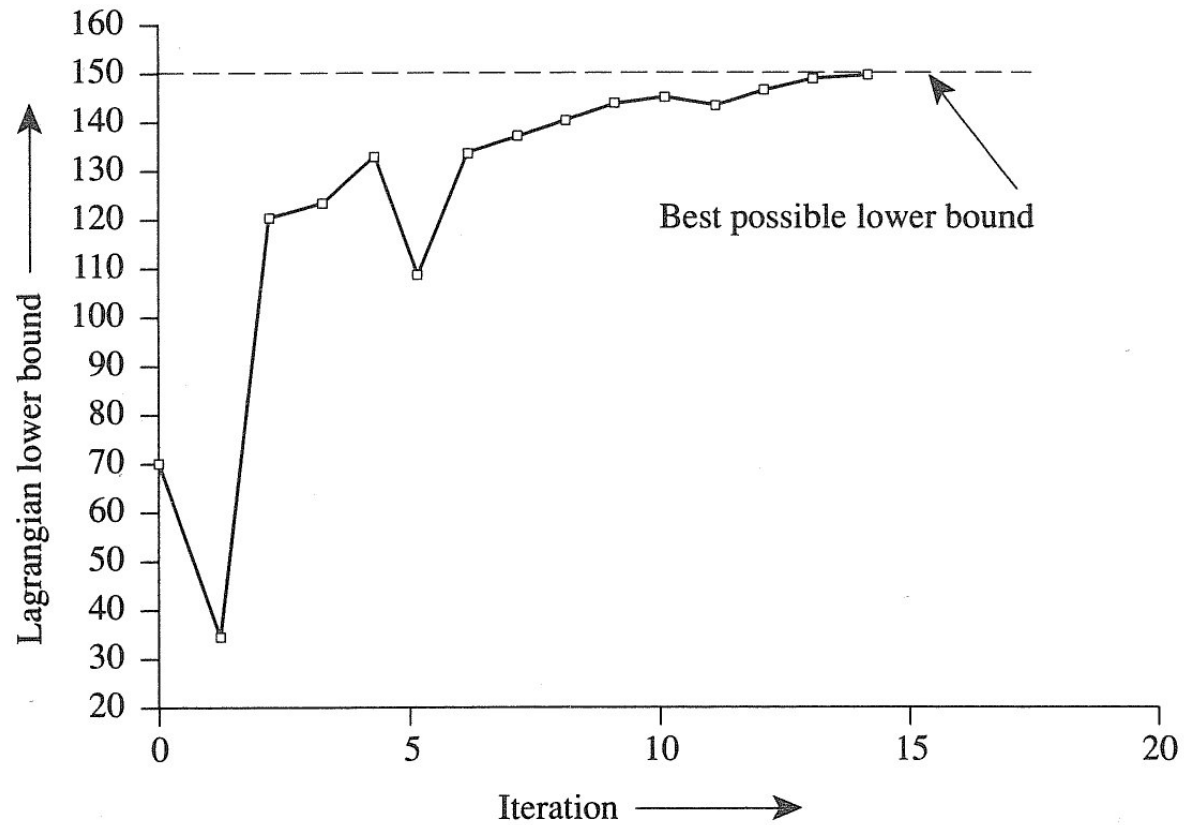
Lagrange iterations: Complete table

Iteration number q	w_{12}^q	$w_{s^1 t^1}^q$	Shortest paths	Shortest path costs	$10w_{12}^q + 5w_{s^1 t^1}^q$	Lower bound $L(w^q)$	θ_q
0	0	0	s^1-t^1 $s^2-1-2-t^2$	10(1) 20(1 + 1 + 1)	0	70	1
1	10	5	s^1-t^1 s^2-t^2	10(1 + 5) 20(5)	125	35	1
2	0	10	s^1-t^1 $s^2-1-2-t^2$	10(1 + 10) 20(1 + 1 + 1)	50	120	0.5
3	5	12.5	s^1-t^1 s^2-t^2	10(1 + 12.5) 20(5)	112.5	122.5	0.333
4	1.67	14.17	$s^1-1-2-t^1$ $s^2-1-2-t^2$	10(5 + 2.67 + 5) 20(1 + 2.67 + 1)	87.55	132.5	0.25
5	6.67	12.92	s^1-t^1 s^2-t^2	10(1 + 12.92) 20(5)	131.3	107.9	0.2
6	4.67	13.92	s^1-t^1 s^2-t^2	10(1 + 13.92) 20(5)	116.3	132.9	0.167
7	3	14.75	$s^1-1-2-t^1$ s^2-t^2	10(5 + 4 + 5) 20(5)	103.8	136.3	0.143
8	3	14.04	$s^1-1-2-t^1$ s^2-t^2	10(5 + 4 + 5) 20(5)	100.2	139.8	0.125
9	3	13.41	$s^1-1-2-t^1$ s^2-t^2	10(5 + 4 + 5) 20(5)	97.05	143.0	0.111
10	3	12.86	s^1-t^1 s^2-t^2	10(1 + 12.86) 20(5)	94.3	144.3	0.1
11	2	13.36	$s^1-1-2-t^1$ s^2-t^2	10(5 + 3 + 5) 20(5)	86.8	143.2	0.091
12	2	12.9	$s^1-1-2-t^1$ s^2-t^2	10(5 + 3 + 5) 20(5)	84.5	145.5	0.083
13	2	12.48	$s^1-1-2-t^1$ s^2-t^2	10(5 + 3 + 5) 20(5)	82.2	147.6	0.077
14	2	12.09	$s^1-1-2-t^1$ s^2-t^2	10(5 + 3 + 5) 20(5)	80.25	149.5	0.071

Lagrange multipliers



Lower bound



Lagrangian relaxation - wrapping up

Scheme for step size θ

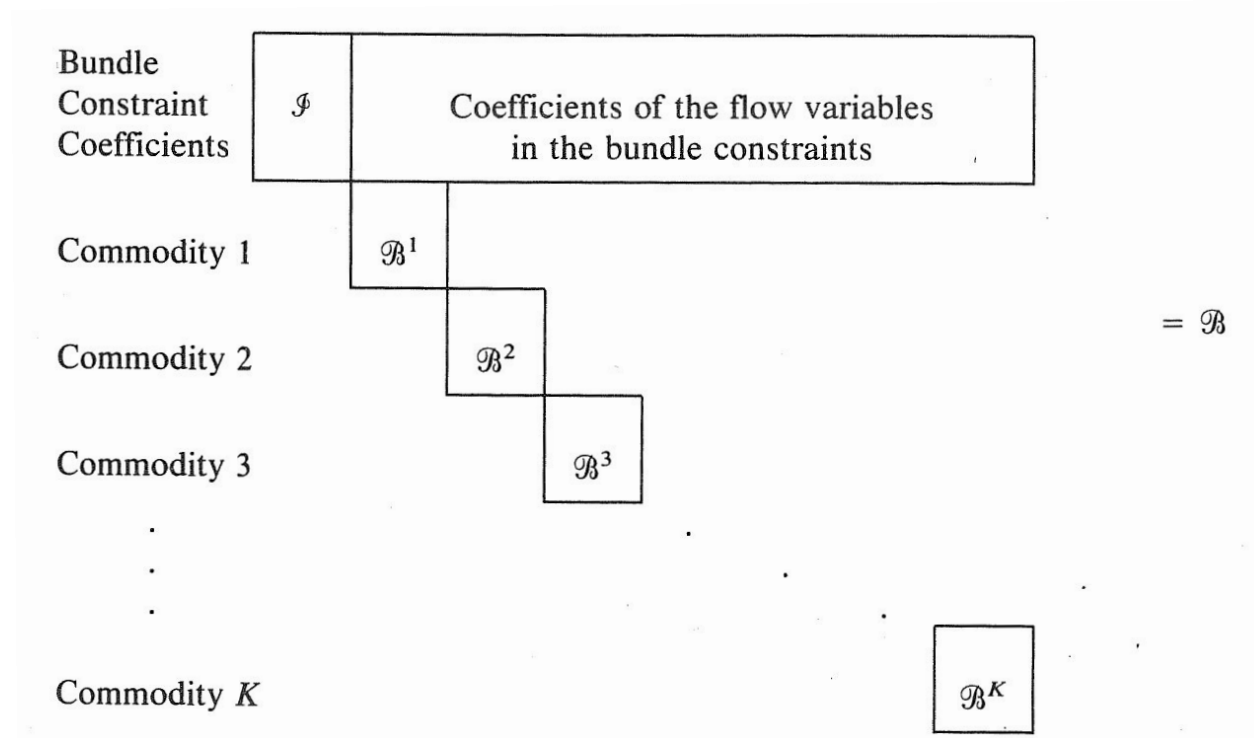
- Small problems, use $1, 1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \frac{1}{5}, \dots$
- Big problems, use Held and Karp

Stopping criteria

- In every iteration check if solution is feasible, and possibly update upper bound z
- If lower bound is equal to upper bound, stop
- If 20 iterations with no improvement in lower bound, decrease θ
- If 100 iterations with no improvement in lower bound stop

When stopped, you have a “guess” for dual variables.
Convert dual to primal.

Dantzig-Wolfe versus Lagrangian



Both split problem into easier subproblems
 Column generation: improves upper bound
 Subgradient optimization: improves lower bound

Lagrangian relaxation

- It can be shown that best Lagrangian multipliers λ are dual variables
- If we know λ we can find x by duality
- We could also solve LP-dual to find best λ
- As LP problems can have multiple solutions, so can Lagrangian dual

Generally

- We can Lagrangian relax all constraints and use subgradient to find dual variables, i.e. solve LP problem
- Alternative method to Simplex, not searching corner points
- Main idea in interior point methods for LP